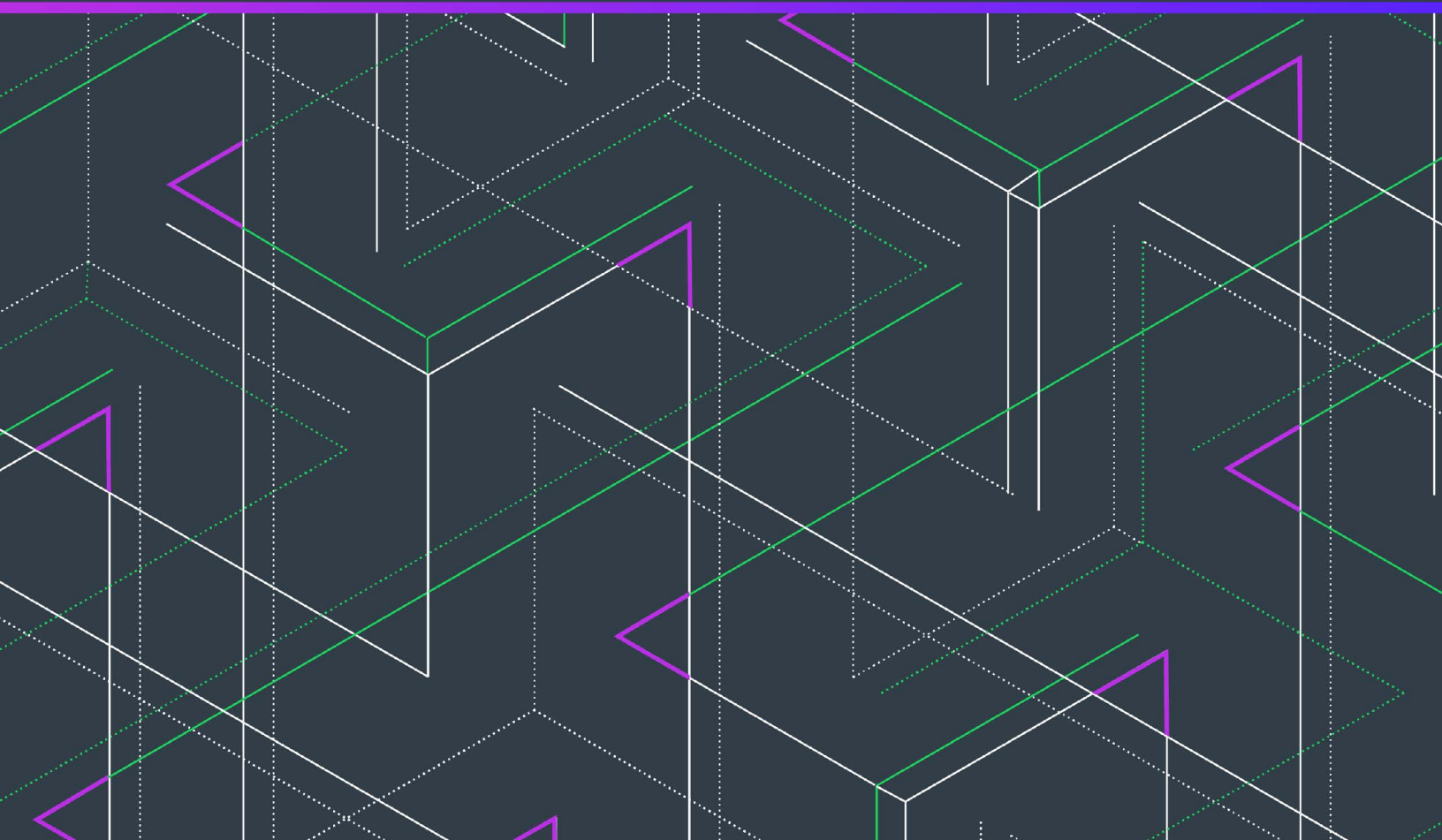


FlexNet Publisher 2023 R3 (11.19.5)

License Administration Guide



Legal Information

Book Name: FlexNet Publisher 2023 R3 (11.19.5) License Administration Guide
Part Number: FNP-11195-LAG00
Product Release Date: December 2023

Copyright Notice

Copyright © 2023 Flexera Software

This publication contains proprietary and confidential information and creative works owned by Flexera Software and its licensors, if any. Any use, copying, publication, distribution, display, modification, or transmission of such publication in whole or in part in any form or by any means without the prior express written permission of Flexera Software is strictly prohibited. Except where expressly provided by Flexera Software in writing, possession of this publication shall not be construed to confer any license or rights under any Flexera Software intellectual property rights, whether by estoppel, implication, or otherwise.

All copies of the technology and related information, if allowed by Flexera Software, must display this notice of copyright and ownership in full.

FlexNet Publisher incorporates software developed by others and redistributed according to license agreements. Copyright notices and licenses for these external libraries are provided in a supplementary document that accompanies this one.

Intellectual Property

For a list of trademarks and patents that are owned by Flexera Software, see <https://www.reverera.com/legal/intellectual-property.html>. All other brand and product names mentioned in Flexera Software products, product documentation, and marketing materials are the trademarks and registered trademarks of their respective owners.

Restricted Rights Legend

The Software is commercial computer software. If the user or licensee of the Software is an agency, department, or other entity of the United States Government, the use, duplication, reproduction, release, modification, disclosure, or transfer of the Software, or any related documentation of any kind, including technical data and manuals, is restricted by a license agreement or by the terms of this Agreement in accordance with Federal Acquisition Regulation 12.212 for civilian purposes and Defense Federal Acquisition Regulation Supplement 227.7202 for military purposes. The Software was developed fully at private expense. All other use is prohibited.

Contents

1	FlexNet Publisher License Administration Guide	11
	Product Support Resources	12
	Contact Us	14
2	Overview of Licensing	15
	License Server	16
	Using a License Server With License Files	17
3	Trusted Storage	19
	Overview of Trusted Storage	19
	Automated Delivery of Licenses to a License Server	19
	Using Licenses From Trusted Storage on a License Server	20
	Trusted Storage Components on a License Server	20
	Using a License Server With Trusted Storage	21
	Distribution of Node-Locked Licenses to Networked Machines	22
	Activation Transaction Types	23
	Comparison of Trusted Storage and License Files	24
	License Files and Fulfillment Records	24
	Locking of Licenses Using Hostid or Trusted Storage	26
	Single-User Access for Trusted Storage	26
	Licensing in Virtual Environments	26
	Binding in a Virtual Environment	26
	Best Practices	27
4	Reading a License File	29
	License File Format Overview	29
	License File Syntax	30
	SERVER Lines	30

VENDOR Lines	32
USE_SERVER Line	34
FEATURE and INCREMENT Lines	34
Publisher-Defined Required Keywords	35
Optional Publisher-Defined Keywords	36
Optional Keywords Defined by the License Administrator	39
Character Limitations in Keyword Values	39
Sort Rules	40
Changes in FEATURE and INCREMENT Line Format	40
PACKAGE Lines	41
UPGRADE Lines	43
Order of Lines in the License File	43
5 Locating Licenses	45
Determining the Location of the License File	45
Setting the License Search Path Using an Environment Variable	46
Order of Searching for a License	47
6 Hostids for Supported Platforms	49
Hostid Formats	49
Obtaining System Hostids	49
Special Hostids	54
Ethernet Hostids	55
TPM Hostid	56
Container ID	56
Hostids to Support Virtualization Policy	57
Hostids to Support Cloud Licensing	57
7 License Models	59
Floating (Concurrent) Licenses	59
Node-Locked Licenses Using Hostid	59
Mixed Node-Locked and Floating Licenses	60
Counted vs. Uncounted Licenses	61
Mobile Licensing	61
Node-Locked to a Laptop Computer	62
Node-locked to a FlexNet ID Dongle	62
Node-Locked to a FlexNet ID Dongle with FLOAT_OK	62
Using a FlexNet ID Dongle for Mobile Licensing Using a FLOAT_OK License	62
FLEXID with FLOAT_OK Example	63
License Borrowing with BORROW	64
Initiating License Borrowing	64
Application Interface	64
Running the Imborrow Utility	64
Setting the LM_BORROW Environment Variable Directly	65

Borrowing a License	65
Clearing the Borrow Period	66
Checking Borrow Status	66
Returning a Borrowed License Early	66
Support for License Borrowing	67
Node-locked to a User Name	67
Fulfilled from a Prepaid License Pool	67
License Transferring Using the Transferrable Counted Model—Windows Only	67
Initiating License Transferring	68
Application Interface	68
Running the Imtransfer Utility	68
Setting LM_BORROW Environment Variable Directly for Transferring	69
Transferring a License	69
Clearing the Transfer Period	70
Checking Transfer Status	70
Returning a Transferred License Early	70
Support for License Transferring	70
Limitation	70
8 Selecting a License Server Machine	73
License Server Sockets	73
License Server CPU Time	73
License Server Disk Space	74
License Server Memory	74
Network Bandwidth for License Server	74
License Server Locally Mounted Disks	75
License Server Port	75
Running the License Server in a Cloud	76
9 License Server Manager “Imadmin”	77
Installing Imadmin	78
System Requirements for Imadmin	78
Using the License Server Installer	78
License Server Directory Structure	82
Upgrading Imadmin	83
Upgrading OpenSSL Libraries	85
Uninstalling Imadmin	85
Using Imadmin	86
Specifying the “conf” Folder	86
Manually Starting the License Server Manager	87
Manually Stopping the License Server Manager	88
Accessing the License Server Management Interface	89
Viewing the Imadmin Log Files	90
Managing Imadmin from the Command Line	90

Adding a Vendor Daemon to Imadmin	91
Configuring the License File Upload Directory	92
Configuring Imadmin License Server Manager as a Windows Service with a Three-Server Configuration	92
Accessing Imadmin License Server Manager as a Windows Service with a Three-Server Configuration	93
Installing Imadmin License Server Manager as an Operating System Service	94
Running FlexNet Publisher License Server as a System Service With Non-Elevated Privileges	96
Imadmin Command-line Arguments	100
Extending Imadmin License Server Capability	110
Using the Imadmin Web Service Interface	110
Creating an Imadmin Alerter Service	110
10 License Server Manager “Imgrd”	111
Imgrd Command-Line Syntax	111
Starting the License Server Manager on UNIX Platforms	114
Manual Start	115
Automatic Start	115
Systemd Config File	115
SystemV Init Script	116
Starting the License Server Manager on Windows	117
Manual Start from the Command Line	118
Configuring the License Server Manager as a Windows Service	118
Configuring the License Server Manager Service for a Delayed Start	119
Manually Starting the License Server Using the Imtools Utility	120
Automatically Starting the License Server at System Startup	122
Three-Server Setup in Imtools	123
Maximum Client Connections to License Server	124
11 Migrating from Imgrd to Imadmin	127
A Fundamental Mode Change	127
Command Changes	128
Imadmin License Administration Functions	129
12 Using License Administration Tools	131
Command-Line Utilities	131
Common Arguments for Imutil	133
Imborrow	134
Initiating Borrowing	134
Clearing the Borrowed License Setting	135
Purging Expired Licenses	135
Determining Borrowed-License Status	135
Returning a Borrowed License Early	136
Imborrowl	137
Imdiag	138
Imdown	138
Imhostid	139

Iminstall	143
Imlicvalidator	144
Imnewlog	147
Impath	147
Imremove (in License File–Based Licensing)	148
Imremove (in Trusted Storage–Based Licensing)	150
Imreread	153
Imstat	154
Imswitch	157
Imswitchr	158
Imtpminfo	159
Imtransfer	160
Imver	163
Imvminfo	163
Other Important Utilities	165
Imobfslog	165
Imreporter	166
Setting Up the Imreporter Without Imadmin Installer	166
Accessing the Imreporter GUI	169
Log Analysis Page	170
Usage Over Time Page	173
Peak Usage Page	176
Denials Page	178
Download Reports	180
Imtools (Windows only)	180

13 Managing the Options File 183

Creating an Options File	184
Options File Syntax	184
AUTOMATIC_REREAD	188
ACTIVATION_LOWWATER	188
ACTIVATION_EXPIRY_DAYS	189
BORROW_LOWWATER	189
DAEMON_SELECT_TIMEOUT	190
DEBUGLOG	190
EXCLUDE	191
EXCLUDE_BORROW	193
EXCLUDE_ENTITLEMENT	193
EXCLUDEALL	194
EXCLUDEALL_ENTITLEMENT	195
FQDN_MATCHING	195
GROUP	197
GROUPCASEINSENSITIVE	198
HOST_GROUP	198
INCLUDE	198
INCLUDE_BORROW	200

INCLUDE_ENTITLEMENT	201
INCLUDEALL	203
INCLUDEALL_ENTITLEMENT	203
LINGER	204
MAX	205
MAX_BORROW_HOURS	206
MAX_CONNECTIONS	207
MAX_OVERDRAFT	207
NOLOG	208
REPORTLOG	208
Reporting on Projects with LM_PROJECT	209
RESERVE	209
TIMEOUT	210
TIMEOUTALL	211
How the Vendor Daemon Uses the Options File	211
Rules of Precedence in Options Files	212
Options File Examples	212
Simple Options File Example	212
Limiting Access for Multiple Users	213
EXCLUDE Example	213
EXCLUDE_ENTITLEMENT Example	214
INCLUDE Example	214
INCLUDE_ENTITLEMENT Example	214
Efficient Reservation	215
14 Ensuring License Availability	217
Redundancy Using the License Search Path	217
Limitations of Redundancy Using the License Search Path	218
Overview of Three-Server Redundancy	218
Configuring License Servers for Three-Server Redundancy	220
Managing License Servers in a Three-Server Redundant Configuration	221
Using Other Capabilities with Three-Server Redundancy	222
Troubleshooting Tips and Limitations for Three-Server Redundancy	224
15 Managing Virtualized License Servers for File-Based Licensing	227
Binding Solutions in a Virtual Environment	227
Setting Up a Virtual License Server on Microsoft Hyper-V	227
Using the UUID Hostid	228
Setting Up a Virtual License Server on VMware ESXi or XenServer	228
Using the UUID Hostid	228
Hypervisor COS Use	229
Virtualization Support	229
16 Licensing in a Cloud-Computing Environment	231

Licensing Challenges in a Cloud Environment	231
Scope of Support for Cloud Licensing	232
Hostids for Binding	232
Supported Hostid Types	232
Retrieving and Specifying Hostids	233
17 IPv6 Support	237
Capabilities that Support IPv6	237
Testing Considerations	240
18 Managing Licenses from Multiple Software Publishers	241
Overview of Multiple License Management Strategies	241
Multiple Systems	242
Starting the License Servers	242
Using Imadmin	242
Using Imgrd	244
One System with Multiple License Server Instances	244
Starting the License Servers	245
Using Imadmin	245
Using Imgrd	246
One System with One License Server and Multiple License Files	246
Starting the License Server	247
Using Imadmin	247
Using Imgrd	249
Managing Multiple License Files	249
Managing Multiple File in Imadmin	249
Managing Multiple Files in Imgrd	249
Defining the License File List	249
Additional Considerations	250
CVD support	250
Combining License Files	250
Criteria for Combining License Files	251
How to Combine License Files	252
Starting the License Server	252
Version Component Compatibility	253
19 Troubleshooting	255
General Troubleshooting Hints	255
FLEXLM_DIAGNOSTICS	256
Level 1 Content	256
Level 2 Content	256
Level 3 Content (Version 6.0 or Later Only)	257
20 Error Codes	259

Error Message Format	259
Format 1 (short)	260
Format 2 (long)	260
Error Code Descriptions	260
21 Report Log File	275
Managing Report Log Output	275
Enabling Report Log Output for a Vendor Daemon	276
Redirecting Report Log Output for a Vendor Daemon	276
22 Debug Log File	277
Managing Debug Log Output	277
Capturing Debug Log Output for a License Server	277
Capturing Debug Log Output for a Particular Vendor Daemon	278
Redirecting Debug Log Output for a Running Vendor Daemon	278
Limiting Debug Log Output for a Vendor Daemon	278
License Server Diagnostics in the Debug Log	278
Debug Log Messages	279
Informational Messages	280
Verbose Debug Log Messages	282
Configuration Problem Messages	284
Daemon Software Error Messages	285
23 Identifying FlexNet Publisher Versions	287
Version Compatibility Between Components	287
Determining the License File Version	288
24 Environment Variables	289
How to Set Environment Variables	289
Windows Registry	289
Precedence	289
Environment Variables	290
Index	293

FlexNet Publisher License Administration Guide

This guide describes FlexNet Publisher licensing for license administrators. It describes how to set up and administer FlexNet Publisher licensing for license models that require a license server.

In this document, “you” generally refers to the license administrator.

Table 1-1 ▪ *License Administration Guide* Chapter Overview

Section	Content
This section	An overview of the contents of this document.
Overview of Licensing	Overview of licensing and specifically licensing using license files.
Trusted Storage	An overview of licensing using license rights held in trusted storage.
Reading a License File	A description of the elements in a license file.
Locating Licenses	How to locate licenses so that they are available to FlexEnabled applications.
Hostids for Supported Platforms	Details of hostids available by platform and information about choosing an Ethernet address as hostid.
License Models	Overview of basic license models and methods of licensing for laptops that may be provided by your software publisher.
Selecting a License Server Machine	What to consider when selecting the machine on which to install the license server software.
License Server Manager “ladmin”	Description of how to install and use ladmin as your license server.

Table 1-1 ▪ *License Administration Guide* Chapter Overview

Section	Content
License Server Manager “lmgrd”	How to use lmgrd as your license server.
Migrating from lmgrd to lmadmin	A comparison of lmgrd and lmadmin.
Using License Administration Tools	How to use license administration tools to manage licenses and license servers.
Managing the Options File	Using the options file to control license utilization and the license server.
Ensuring License Availability	Methods of providing failover protection for license servers.
Managing Virtualized License Servers for File-Based Licensing	Virtualization of a license server.
Licensing in a Cloud-Computing Environment	Setup of licensing model in an Amazon EC2 environment.
IPv6 Support	Installing and configuring license servers in IPv6 and mixed IPv4 and IPv6 environments.
Managing Licenses from Multiple Software Publishers	Strategies for managing licenses from multiple software publishers.
Troubleshooting	Tips and information about generating additional diagnostic data.
Error Codes	A list of FlexNet Publisher error codes.
Report Log File	Enabling and managing report log output.
Debug Log File	Enabling and managing debug log output.
Identifying FlexNet Publisher Versions	Version compatibility between components and brief details of functional changes for each major version of FlexNet Publisher.
Environment Variables	Environmental variables that may be used with FlexNet Publisher.

Product Support Resources

The following resources are available to assist you:

- [Revenera Product Documentation](#)
- [Revenera Community](#)
- [Revenera Learning Center](#)

- Revenera Support

Revenera Product Documentation

You can find documentation for all Revenera products on the [Revenera Product Documentation](https://docs.revenera.com) site:

<https://docs.revenera.com>

Revenera Community

On the [Revenera Community](https://community.revenera.com) site, you can quickly find answers to your questions by searching content from other customers, product experts, and thought leaders. You can also post questions on discussion forums for experts to answer. For each of Revenera's product solutions, you can access forums, blog posts, and knowledge base articles.

<https://community.revenera.com>

Revenera Learning Center

The Revenera Learning Center offers free, self-guided, online videos to help you quickly get the most out of your Revenera products. You can find a complete list of these training videos in the Learning Center.

<https://learning.revenera.com>

Revenera Support

For customers who have purchased a maintenance contract for their product(s), you can submit a support case or check the status of an existing case by first logging into the [Revenera Community](https://community.revenera.com) and then making selections on the **Get Support** menu, including **Open New Case** and other options.

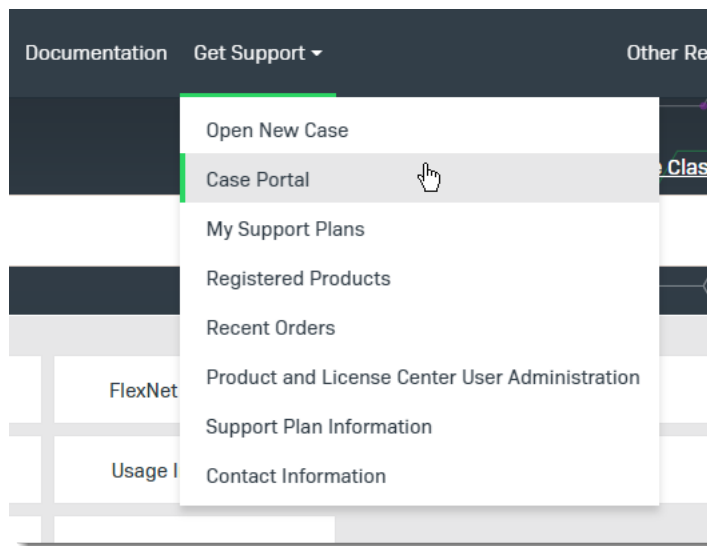


Figure 1-1: Get Support Menu of Revenera Community

Contact Us

Revenera is headquartered in Itasca, Illinois, and has offices worldwide. To contact us or to learn more about our products, visit our website at:

<http://www.revenera.com>

You can also follow us on social media:

- [Twitter](#)
- [Facebook](#)
- [LinkedIn](#)
- [YouTube](#)
- [Instagram](#)

Overview of Licensing

FlexNet Publisher is a method of providing software licensing that has two basic components:

- **FlexEnabled application**—The software application that requires a license.
- **A license**—Contains the license rights that define how the software application can be used.

Typically the license defines:

- What software functionality can be used. Functions provided by the software can be separately licensed. The licensed functions are referred to as *features*. When multiple features are defined, different versions of the product can be licensed by including different feature sets. For example, the license for the 'demo' version of the product could include the feature 'trial', the 'standard' version of the product the features 'trial' and 'basic' and the 'professional' version 'trial', 'basic' and 'extend' features.
- What versions of the software can be used.
- How many copies of the software can be running.
- The systems on which the software can be used.
- The period during which the software can be used.

These and other items in the license define how the software can be used and collectively are referred to as a *license model*.

The license can be stored:

- **In a license file**—A text file, *file_name.lic*, whose contents are protected by signatures that are authenticated by the FlexNet Publisher licensing components.
- **In trusted storage**—A secure location whose contents are encrypted. Licenses are stored as *fulfillment records*. Fulfillment records in trusted storage can be read only by FlexNet Publisher licensing components.

The FlexEnabled application can obtain a license directly, either from a license file or from *local* trusted storage on the same machine. Some license models, described as *served*, provide licenses that are held centrally by a *license server* and used by FlexEnabled applications connected to the license server across a TCP/IP network.

This document describes how to install and use a license server to provide licenses for FlexEnabled products that use served license models. The basic license model that requires a license server is referred to by several names depending on the context:

- Concurrent
- Floating

Concurrent licenses allow a fixed number of concurrent users to use licensed features at any one time. The license server controls the use of these licenses, which are not normally locked to a specific machine, and *float* on the network. FlexNet Publisher provides for many variations of this basic license model, for example the use of a set of concurrent licenses can be restricted to a group of users.

License Server

The basic components of a FlexNet Publisher license server are as illustrated in the following diagram:

- **License server manager**—lmadmin (or lmgrd) supplied by your software supplier or available from Revenera.
- **License file**—Created by your software supplier. In this document the supplier of a FlexEnabled application is referred to as the *publisher*.
- **Vendor daemon**—Created by the publisher. Each publisher has their own vendor daemon. If you have FlexEnabled applications from several publishers, you will need to install multiple vendor daemons.
- **Debug log**—Written by the license server manager.

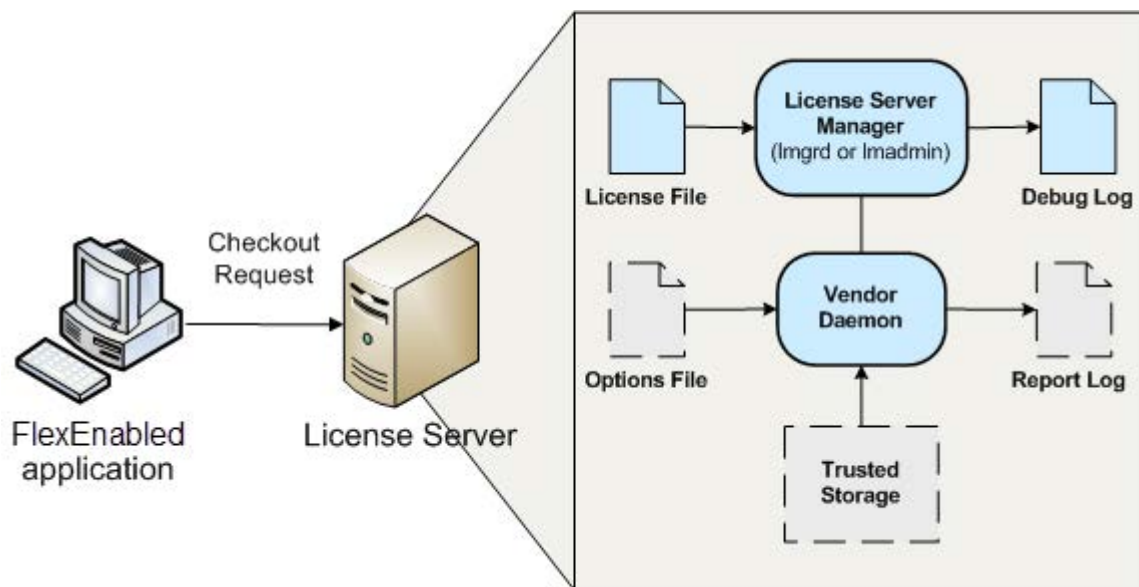


Figure 2-1: FlexNet Publisher license server

The following components may be present on a license server:

- **Options file**—Optional file that you create. Use it to limit license usage; for example, to allocate particular licenses to a user or group of users.

- **Report log**—Optional file that can be used by FlexNet Manager, Revenera's license management product. You enable report logging using the options file.
- **Trusted storage**—Some publishers use trusted storage to store licenses. When trusted storage is used, the publisher provides additional components (not shown on [Figure 2-1](#)) that create trusted storage and add licenses to it. See [Trusted Storage](#) for an overview.

Using a License Server With License Files

The following gives an outline of the steps in installing a license server and using it to serve licenses from license files. For further information about each of these steps, read the relevant sections of this document.

1. Choose the machine(s) on which the license server(s) will be installed.
 - Determine the number of licenses and machines on which FlexEnabled applications will be installed. See [Selecting a License Server Machine](#) for further information.
 - Consider what method, if any, you want to use to ensure that, whenever possible, licenses are available to your end users. See [Ensuring License Availability](#) for further information.

2. Install the license server components.

The publisher will supply a copy of their vendor daemon and instructions for installing it. The publisher also provides the license server manager (`lmadmin` or `lmgrd`).

3. Obtain details of the license server machine(s) and send them to the publisher.

Normally publishers supply concurrent licenses that are locked to a specific license server. When licenses are held in license files, they are locked to the license server using an identity obtained from the machine. This identity is called a *hostid* and is platform-specific. There are several different *hostids* available for each platform. The publisher will provide instructions on what *hostid* they are using for your licenses and platforms. They may supply an application that you can run to obtain the *hostid*; this might be the FlexNet Publisher utility, `lmhostid`, or a different utility. If you are using `lmadmin`, it displays the standard *hostids* for the machine on which it is running in **System Information**.

Depending on the license model, the publisher may require other details of your license server, the machine on which it is running, and details of your network.

4. Install licenses on the license server.

The publisher may specify a particular location for the license files on the license server. When no specific location is required, see information in [Locating Licenses](#) for instructions.

5. Install the FlexEnabled application on end user machines.

The publisher will supply installation instructions for installing the FlexEnabled application.

6. Set up end user machines to access the license server.

There are several methods of configuring the end user machine to access a single license server or multiple license servers. These depend on the contents of the license files supplied by the publisher and your license server(s) configuration. See information in [Locating Licenses](#) for instructions.

7. Optionally, create an options file.

If you want to limit license usage, configure logging, or turn off the automatic reread of licenses, create an options file and install it in the same directory as the vendor daemon. See instructions in [Managing the Options File](#).

8. Configure and start up the license server manager.

There is a fundamental difference between the configuration of `lmadmin` and `lmgrd`, so the processes required for each are separately outlined here:

lmadmin—The configuration settings are permanent and mainly set using the user interface. For details see the `lmadmin` Online help and [Using lmadmin](#).

lmgrd—The configuration settings are set when `lmgrd` is started. They are not persistent. For details see [License Server Manager “lmgrd”](#).

You can manage and monitor the operation of the license server using the license server manager. `lmadmin` provides direct management and monitoring of the license server through its user interface; `lmgrd` provides limited information as command-line output. Additional utilities are available for management and monitoring of the license server. For details, see [Using License Administration Tools](#). For more comprehensive monitoring and reporting of license usage, use FlexNet Manager. FlexNet Manager is a Web-based administration and reporting tool for FlexNet licenses and license servers.

Trusted Storage

Some publishers use trusted storage to store licenses. They might store all of their licenses in trusted storage or use a combination of licenses held in license files and in trusted storage. You can use a single license server to serve licenses from both license files and trusted storage.

Overview of Trusted Storage

Trusted storage is a secure location that is locked to the machine on which it is located using a combination of machine identities. The contents of trusted storage are encrypted and can only be accessed by FlexEnabled components. This method of storing licenses enables your publisher to provide additional license models and automate some licensing processes.

Automated Delivery of Licenses to a License Server

Trusted storage is maintained by licensing life-cycle operations, such as license activations, returns, repairs, and upgrades. These life-cycle operations are performed through transactions normally issued over a network connection between the license server and the publisher's activation server. However, when a network connection is not available, the messages that implement these transactions can be transmitted by other means.

Activation is the basic licensing life-cycle operation between the license server and the publisher's activation server. This operation configures your trusted storage for specific use by the publisher and writes a *fulfillment record* to it. The fulfillment record contains licenses defined using a similar format to that used for licenses held in license files.

The remaining life-cycle operations between a license server and the publisher's activation server maintain the trusted-storage licenses and are optional:

- **Return**—Returns a fulfillment record (and the licenses it contains) from trusted storage to the publisher server it was issued by.
- **Repair**—Repairs compromised fulfillment records in trusted storage.
- **Upgrade to a new version**—The old license is returned to the publisher's activation server so that entitlement to the upgrade can be checked and new licenses transmitted using an activation operation.

- **Rehost of license server**—When you need to move a license server to a different machine, a combination of return and then activation operations can provide a completely automated transfer.



Note ▪ *Not all publishers provide these facilities.*

Using Licenses From Trusted Storage on a License Server

Two types of licenses can be used in trusted storage on a license server. Your publisher may provide either or both of these types of license. They are used to provide different licensing models.

- **Concurrent**—Allows a fixed number of concurrent users to use licensed features at any one time. The license server controls the use of these licenses, which are not normally locked to a specific machine, and *float* on the network. FlexNet Publisher provides for many variations of this basic license model, for example the use of a set of concurrent licenses can be restricted to a group of users.
- **Activatable**—Licenses are distributed by the license server to network machines to provide local licenses for FlexEnabled applications. In this license model FlexEnabled components on the network machine request a license from the license server. License rights held in trusted storage on the license server are transferred to trusted storage on the network machine. This provides a license that is locked to the network machine. Depending on which license models your publisher is providing, these licenses may be of limited duration and automatically return to the license server when they expire on the network machine, or may be transferred to the network machine ‘permanently’.

Trusted Storage Components on a License Server

The basic components of a FlexNet Publisher license server that uses licenses held in trusted storage are as illustrated in the following diagram:

- **License server manager**—lmadmin (or lmgrd) supplied by your publisher or available from Revenera.
- **Bootstrap license file**—Created by your publisher. Required for starting the license server manager when the license server is using trusted storage to store all its licenses.
- **Vendor daemon**—Created by the publisher. This must be the publisher vendor daemon that can access trusted storage. Ensure that you always use the correct vendor daemon supplied by the publisher: an earlier version that is only able to use license files will not be able to use licenses held in trusted storage.
- **Trusted storage**—Contains licenses in fulfillment records.

- **Server activation utility**—A FlexEnabled component that manages the transactions with the publisher server and creates and manages the contents of trusted storage.

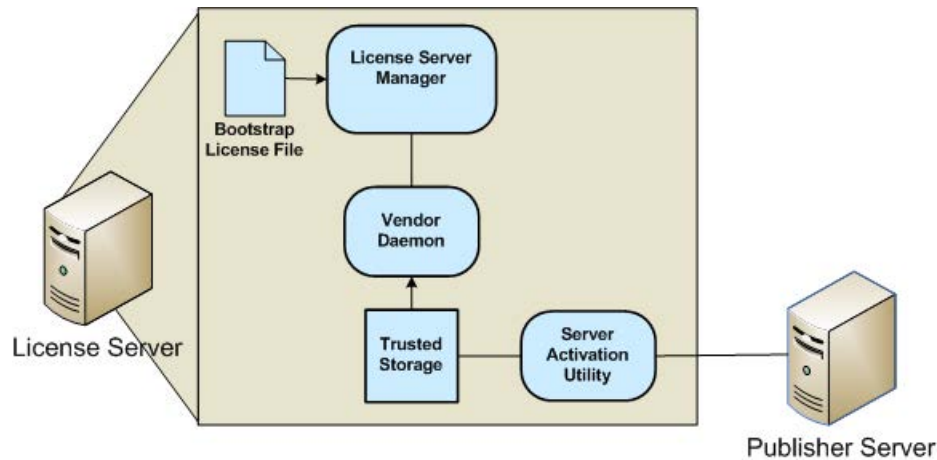


Figure 3-1: License Server Using Licenses in Trusted Storage

The following components not shown on the diagram may be present on the license server:

- **Debug log**—Written by the license server manager.
- **Options file**—Optional file that you create.
- **Report log**—Optional file used by FlexNet Manager.

Using a License Server With Trusted Storage

The following gives an outline of the steps in installing a license server and using it to serve licenses from trusted storage. For further information about each of these steps read the relevant sections of this document.

1. Choose the machine(s) on which the license server(s) will be installed.
 - Determine the number of licenses and machines on which FlexEnabled applications will be installed. See [Selecting a License Server Machine](#) for further information.
 - Consider what method, if any, you want to use to ensure that whenever possible licenses are available to your end users. See [Ensuring License Availability](#) for further information.

2. Install the license server components.

The publisher will supply a copy of his vendor daemon and instructions for installing it. The publisher also provides the license server manager (`lmadmin` or `lmgrd`). The latest license server manager, `lmadmin`, displays details of licenses held in trusted storage; `lmgrd` includes information about concurrent licenses held in trusted storage but does not display details of activatable licenses. Therefore, it is recommended that you install `lmadmin` as your license server manager.

3. Install licenses on the license server.

The publisher will supply instructions and software that requests licenses from the publisher server. This process may be completely transparent to you. The publisher provides the interface for the installation of licenses so publisher's licensing solutions may differ greatly. FlexNet Publisher is designed to allow publishers maximum flexibility in licensing models and processes.

4. Install the FlexEnabled application on end user machines.

The publisher will supply installation instructions for installing the FlexEnabled application and optionally any further FlexEnabled components.

5. Set up end user machines to access the license server to obtain concurrent licenses.

There are several methods of configuring the end-user machine to access a single license server or multiple license servers. These depend on the contents of any license files that may optionally be supplied by the publisher and your license server(s) configuration. See information in [Locating Licenses](#) for instructions.

6. Optionally, create an options file.

If you want to limit license usage, configure logging or turn off the automatic reread of licenses, create an options file and install it in the same directory as the vendor daemon. See instructions in [Managing the Options File](#).

7. Configure and start up the license server manager.

8. Optionally, install node-locked licenses on end user machines using activatable licenses from the license server.

The publisher will supply instructions for requesting licenses from the license server. Additional components may be installed on the end user machine for this licensing model, see [Distribution of Node-Locked Licenses to Networked Machines](#).

Distribution of Node-Locked Licenses to Networked Machines

The distribution of licenses from a license server to machines running FlexEnabled applications via a network is one license model that can be provided using activatable licenses held in trusted storage on a license server. FlexEnabled components on the network machine send a request for a license to the license server. The vendor daemon processes this request and if a suitable license is available transfers it to the network machine.

The FlexEnabled components on the network machine install the license in trusted storage. Trusted storage is locked to the network machine and thus licenses held in trusted storage are node-locked to that machine.

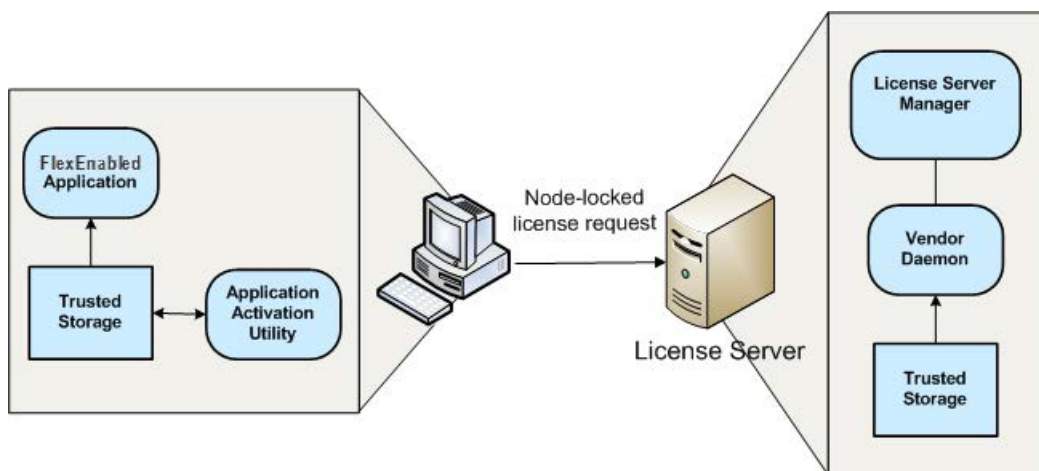


Figure 3-2: FlexEnabled components for trusted storage on a network machine

The FlexEnabled components required to implement the distribution of node-locked licenses to networked machines using trusted storage are:

- **License server manager**—Use `lmadmin` as the license server manager as it displays details of activatable licenses held in trusted storage.
- **Vendor daemon**—Created by the publisher. This must be the publisher vendor daemon that can access trusted storage. Ensure that you always use the correct vendor daemon supplied by the publisher: an earlier version that is only able to use license files will not be able to use licenses held in trusted storage.
- **Trusted storage on license server**—Contains activatable licenses that can be transferred to a networked machine. Concurrent licenses can only be used to implement floating license models.
- **Server activation utility** (not shown)—The FlexEnabled component on the license server that requests and loads licenses into the server's trusted storage from the publisher. This utility also manages the contents of the server's trusted storage through return, repair, and modify requests.
- **Application activation utility**—A FlexEnabled component that requests a license from the enterprise license server or the publisher's activation server and creates and manages the contents of trusted storage. The publisher can integrate this functionality into a component that provides other functions (for example, the utility could be integrated into the FlexEnabled application installer).
- **Trusted storage on the network machine**—Contains licenses locked to the machine.
- **FlexEnabled application**—The application that requires the license. Note that this component must have been built by your publisher so that it can access trusted storage. You must ensure that you use the correct version of the application.

You can use the options file to restrict the distribution of node-locked licenses to network machines. See [Managing the Options File](#) for details.

Activation Transaction Types

FlexNet Publisher supports two transaction types:

- **Composite Transactions**—From FlexNet Publisher 11.14.0, composite transactions are fully supported for all servers. From this version onwards, it is best practice to use composite transactions for all interactions with servers.
- **Single-Action Transactions**—Single-action transactions perform one activation action per transaction. For example, using single-action transactions to activate two fulfillment records from a license server to a FlexEnabled client involves issuing two separate transactions.

From FlexNet Publisher version 11.14.0, single-action transactions have been deprecated.

Composite Transactions

Composite transactions are supported for life-cycle operations between the publisher's activation server and the license server, as well as between the license server and FlexEnabled clients. A single composite transaction between the publisher's activation server and the license server uses one request from the license server and one response from the publisher's activation server to perform any number of activations, repairs, returns, and upgrades on the license. For example, a composite transaction can perform any of the following licensing life-cycle operations, providing greater flexibility in managing trusted-storage license rights:

- Return one or more fulfillment records
- Activate one or more fulfillment records
- Modify one or more fulfillment records (for example, to change the license count)
- Combinations of these actions to perform partial or full returns or upgrades

Additionally, composite transactions store more details about the activation than single-action transactions do, allowing a better chance to fully recover a trusted storage on the license server should errors occur.

Comparison of Trusted Storage and License Files

This section gives an overview of the significant differences between FlexEnabled products that use trusted storage and those that use license files. However, significant these changes might be, the methods for defining license rights in trusted storage are based on the methods for defining rights in license files. So if you have been using FlexEnabled products for years, the majority of your knowledge is directly applicable to licenses held in trusted storage.

License Files and Fulfillment Records

The license model is defined primarily in the feature definition lines (FEATURE and INCREMENT) in a license file. There are the same feature definition lines inside fulfillment records in trusted storage. See the following diagram that shows a typical `lmadmin` display for a fulfillment record.

Fulfillments

Product Name/Version:
PRprofessional PRprofessional

Export Data

Suite: No
Features: INCREMENT PRbasic demo 1.0 31-dec-2011 1 SIGN="0028 A68E 1647 8E4C 5636 9D25 F7F1 9B00 44CA B8E1 A0B2 BCDA D2E9 9D72 F829"/ INCREMENT PRadvanced demo 1.0 31-dec-2011 1 SIGN="0071 0C43 711B 259F 7ADD A962 98D1 CB00 6E39 7C21 3974 F052 4DFF 6222 B206"


Search for:

Search

Page 1 of 1

Fulfillment ID ▲	Quantity	Expiration	Type	Server Chain	Repair Count	
PR-589df128	10	31-DEC-2011	Activation	FIL-E0	10	Hosts

Figure 3-3: License server fulfillment record displayed by lmadmin

The fulfillment record PR-589df128 provides 10 activatable licenses for the product PRprofessional. Each activatable license licenses two features: PRbasic and PRadvanced. These two feature definition lines (in this example INCREMENT lines) are packaged together in a single fulfillment record.

Licenses held in trusted storage use all the mandatory fields and may contain most of the attributes described in [Reading a License File](#). The following are the exceptions:

- **BORROW**—Normally this feature definition line attribute is not used for licenses held in trusted storage, the publisher will provide this licensing model using the [Distribution of Node-Locked Licenses to Networked Machines](#) using trusted storage on the network machine.
- **HOSTID**—Normally this feature definition line attribute is not required for licenses held in trusted storage, see [Locking of Licenses Using Hostid or Trusted Storage](#) for details of how licenses are locked to a host machine.
- **SUPERSEDE**—This feature definition line attribute is not supported for licenses held in trusted storage. A combination of return and activation operations are used to remove the license for the old version of the application and replace it with a new license.

The following line types are not supported in trusted storage:

- UPGRADE (a combination of returns and activations are used for an upgrade)
- PACKAGE (a fulfillment record effectively packages multiple feature definition lines)



Note ▪ When other functions for package suites are required, a license file with a PACKAGE line can be provided.

- SERVER (not needed)
- VENDOR (lmadm provides direct vendor-daemon configuration)
- VM_PLATFORMS (not needed)
- USE_SERVER (not needed)

Locking of Licenses Using Hostid or Trusted Storage

When license files are used, licenses are locked to a machine using a hostid. This identifies either the machine or a FlexNet ID dongle that is attached to a machine. The hostid is incorporated into the licenses supplied by the publisher so you must supply details of hostids before the publisher can provide your licenses. This procedure is repeated when licenses need to be moved to another machine.

Trusted storage is locked to the machine on which it is created using machine identities retrieved automatically by the FlexEnabled components when trusted storage is created. The licenses held in trusted storage are locked to the machine because they are held securely within trusted storage.

Single-User Access for Trusted Storage

Trusted Storage has default full access to all the authenticated users. In Linux environment, the access to Trusted Storage can also be restricted to single user or a group. It results in limited access to the usage of routines, which requires access to the Trusted Storage, only to the nominated set. Restriction on Trusted Storage access can also impact the other producer application if internally Trusted Storage is used for License rights. Trusted Storage created for other producer application also share same space. It is recommended to restrict Trusted Storage access only if all the applications are required to be accessed in same mode. The access mode can be reshuffled easily without any impact on Trusted Storage. It is expected that Producer will provide the option to shuffle, if required.

Licensing in Virtual Environments

The server-side activation utility can run inside a virtual machine. See [Virtualization Support](#), for the list of hypervisors that FlexNet Publisher supports.

Binding in a Virtual Environment

FlexNet Publisher automatically uses the appropriate binding elements when running in a virtual environment, if they are available. Binding is to differentiate between physical and virtual environments and the legacy bind-to-VMID policy is deprecated. The binding elements are:

- **MAC address**—used on physical environments is now used in virtual environments as well and is a deal breaker.
- **UUID** (universally unique ID)—A binding identity used to configure trusted storage on the virtual machine, previously the only binding identity used in virtual environments; this is now a deal breaker.
- **Generation ID**—A property of a virtual machine available in some environments. Where it is available it is used as a binding identity; it is a deal breaker.

A binding item is a deal breaker if, when it changes, trust should be lost regardless of the number of binding items that still match. Since all binding identities used in virtual environments are deal breakers, trust will always be lost if any of them change.

Best Practices

To Avoid Trusted-Storage Breakage

If the publisher chooses to bind trusted storage to the VMID, this binding breaks should the UUID (from which the VMID is derived) ever change or is no longer available. The break prevents license leakage when virtual machine images are cloned. However, if you manage a virtualized environment where virtual machines are moved between different native (physical) systems, you do not want trusted storage to break each time you move a machine instance.

To prevent breakage, use these best practices:

- Do not change the MAC address of the virtual machine.
- Do not change the UUID of the virtual machine when it is moved. (Normally, UUIDs change only if the virtual machine is cloned.)
- Save the configuration file (or at least the UUID) of each virtual machine on which trusted-storage license activation is performed. This file ensures that the UUID value used at the time of activation is available should you need to revert to this value.
- Should trusted storage break, use the activation utility to issue a repair request; or reset the virtual machine's UUID to the value it had at activation time (powering the machine off and then on after the reset).

To Avoid Issues with the Licensing Life-Cycle Operations

Although both composite and single-action transactions are supported for licensing life-cycle operations between the publisher's activation server and the license server, best practice is always to use composite transactions. Issues can arise if you attempt to mix composite and single-action transactions, mainly because composite transactions support UMN3 and UMN5 while single-action transactions do not. For example, if a composite transaction is used initially to activate licenses on a virtual machine, the UMN3, if obtained, becomes the primary UMN for requester verification. If a single-action transaction is used later to return a license, the process fails because the return request contains no UMN3 to identify the requesting machine.

Reading a License File

A license file contains information required to manage licenses for a FlexEnabled application. This information includes:

- License server names and hostids
- VENDOR names and paths to vendor daemon executables
- Feature information

The license file must be accessible to systems that run the FlexEnabled application or a license server. For details see [Locating Licenses](#) and [Ensuring License Availability](#).

License File Format Overview

License files begin with either a single SERVER line or three SERVER lines (when configured for three-server redundancy) followed by one or more VENDOR lines, followed by one or more FEATURE or INCREMENT lines. In some cases, the license file requires no SERVER line and no VENDOR line.



Note - Eight-bit Latin-based characters are fully supported in license files, options files, log files, and FlexEnabled application environments.

See [Counted vs. Uncounted Licenses](#) for more information on SERVER and VENDOR line requirements.

You can modify these elements in the license file:

- On the SERVER line:
 - Host names on the SERVER lines
 - TCP/IP port numbers
 - HEARTBEAT_INTERVAL and PRIMARY_IS_MASTER properties
- On the VENDOR line:

- Paths to the vendor daemon.
- Options file paths
- TCP/IP port numbers (for firewall support only)
- The USE_SERVER line.
- On the feature definition lines:
 - The values in *keyword=value* pairs on FEATURE lines, if *keyword* is specified in lowercase (see [Optional Keywords Defined by the License Administrator](#))
 - You can use the \ line-continuation character to break up long lines.

See Also

[Ensuring License Availability](#)
[Counted vs. Uncounted Licenses](#)

License File Syntax

This section describes the contents of the license file, including SERVER lines and VENDOR lines. This is an example of a license file for a single VENDOR name with two features.

```
SERVER my_server 17007ea8 1700
VENDOR sampled
FEATURE f1 sampled 1.000 31-dec-2020 10 SIGN="<...>"
FEATURE f2 sampled 1.000 31-dec-2020 10 SIGN="<...>"
```

This example allows the license server, called **my_server** with the hostid **17007ea8**, to serve ten floating licenses for each feature, **f1** and **f2** to any user on the network.

SERVER Lines

The SERVER line specifies the host name and hostid of the license server and the TCP/IP port number of the license server manager (lmadmin or lmgrd). Normally a license file has one SERVER line. Three SERVER lines mean that you are using license servers configured for three-server redundancy. The absence of a SERVER line means that every feature definition line in the license file is uncounted.

The hostids from the SERVER lines are computed into the license key or signature on every feature definition line. For this reason, make sure you keep SERVER lines together with any feature definition lines as they were sent from the software publisher.

The format of the SERVER line is:

```
SERVER host hostid [port] [PRIMARY_IS_MASTER] [HEARTBEAT_INTERVAL=seconds]
```

For example:

```
SERVER my_server 17007ea8 21987
```

The following table describes the attributes on this line.

Table 4-1 ■ SERVER Line Format

Field	Description
<i>host</i>	The system host name or IP address. String returned by the UNIX <code>hostname</code> or <code>uname -n</code> command.
<i>hostid</i>	Usually the string returned by the <code>lmhostid</code> command. This is changed only by your publisher.
<i>port</i>	TCP/IP port number to use. You can edit this value either directly in the license file or using <code>lmadmin</code>. In the latter case, the port number specified in <code>lmadmin</code> takes precedence over any port set in the license file, either before or after the port is specified using <code>lmadmin</code>. Therefore, editing the port number using <code>lmadmin</code> is only recommended if no port number is specified in the license file. If a port number is already specified in the license file and the port is subsequently changed in <code>lmadmin</code>, during the next restart the vendor daemon will exit with a “port mismatch” error. A valid number is any unused port number in the range 1 to 65535. On UNIX, choose a port >1024, since those <1024 are privileged port numbers. If you specify a port number greater than 65535, the client fails to establish a connection with the license server manager (<code>lmgrd</code> or <code>lmadmin</code>). If no port number is specified, one of the default ports in the range 27000 to 27009 is used. You must specify a port number when the SERVER line defines license servers configured for three-server redundancy.  Note ■ Those producers concerned about potential DoS attacks based on knowledge of common license server ports may want to consider specifying a server port outside the range 27000 to 27009.

Table 4-1 ■ SERVER Line Format

Field	Description
PRIMARY_IS_MASTER	Used with license servers configured for three-server redundancy to indicate how master control is transferred between the primary and secondary servers. ● If this is set and the primary server goes down, when the primary server comes back up again, it will always become the master. ● If this is not set and the primary server goes down, the secondary server becomes the master and remains the master even when the primary server comes back up. The primary can only become the master again when the secondary license server fails. If both primary and secondary go down, licenses are no longer served. The tertiary server never becomes the master. This parameter is optional and is placed on the first SERVER line in the license file. You must be running a version 10.8 or later vendor daemon to use this parameter.
HEARTBEAT_INTERVAL= <i>seconds</i>	Used with license servers configured for three-server redundancy to indicate how long a license server waits to receive a heartbeat from another license server in the triad before shutting itself down. The <i>seconds</i> value is used in the following equation to calculate the timeout: ● $\text{timeout} = (3 \times \text{seconds}) + (\text{seconds} - 1)$ Valid timeout value is 0-120. If not specified, the default value for <i>seconds</i> is 20, equating to an actual timeout value of 79 seconds. Valid values for the <i>seconds</i> value are 0-30.  Note ■ When the <i>seconds</i> value exceeds 30, Imadmin displays the Heartbeat Interval value as -1 along with the an error message "Invalid three server redundancy configuration, valid timeout values are 0 - 120". This parameter is optional and is placed on the first SERVER line in the license file. You must be running a version 10.8 or later vendor daemon to use this parameter.

See Also

Ensuring License Availability

VENDOR Lines

The VENDOR line specifies the daemon name and path. The license server uses this line to start the vendor daemon, and the vendor daemon reads it to find its options file. The format of the VENDOR line is shown below.


```
VENDOR vendor [vendor_daemon_path]\
           [[OPTIONS=]options_file_path] [[PORT=]port]
```

where:

Table 4-2 • VENDOR Line Format

Field	Description
<i>vendor</i>	Name of the vendor daemon used to serve some features in the file. This name cannot be changed.
<i>vendor_daemon_path</i>	Optional path to the executable for this daemon. Generally, the license administrator is free to install the vendor daemon in any directory. It is recommended, however, that it be installed in a local directory on the license server. If omitted, the license server looks for the vendor daemon binary in: - the current directory (lmgd only) - the path specified in the license server's \$PATH environment variable - in the directory where the license server is located If <i>vendor_daemon_path</i> is blank, then any option or TCP/IP port number specifications require the OPTIONS= and PORT= strings.
<i>options_file_path</i>	Full path to the options file for this daemon. An options file is not required. If omitted, the vendor daemon, by default, looks for a file called <i>vendor.opt</i> (where <i>vendor</i> is the vendor daemon name) located in the same directory as the license file.
<i>port</i>	Vendor daemon TCP/IP port number. A valid number is any unused port number in the range 0 to 65535. On UNIX, choose a port >1024, since those <1024 are privileged port numbers. If <i>port</i> is not specified or if a port value of 0 is specified, an ephemeral port is chosen by the operating system at run-time. Sites with Internet firewalls need to specify the TCP/IP port number the daemon uses. If a TCP/IP port number is specified on the VENDOR line, there may be a delay restarting the vendor daemon.
<i>eport</i>	Vendor daemon TCP/IP port number for the secured communication. A valid number is any unused port number in the range 0 to 65535. On UNIX, choose a port >1024, since those <1024 are privileged port numbers. If <i>eport</i> is not specified or if a port value of 0 is specified, an ephemeral port is chosen by the operating system at run-time. Sites with Internet firewalls need to specify the TCP/IP port number the daemon uses. If a TCP/IP port number is specified on the VENDOR line, there may be a delay restarting the vendor daemon.

See Also

[Managing the Options File](#) for further information regarding options file contents.

USE_SERVER Line

The USE_SERVER line takes no arguments and has no impact on the license server. When the application sees the USE_SERVER line, it ignores everything in the license file except the preceding SERVER lines and transfers checkout validation to the vendor daemon.

USE_SERVER is recommended since it improves performance when a license server is used. For uncounted features, USE_SERVER is used to force logging of usage by the daemons.

FEATURE and INCREMENT Lines

FEATURE and INCREMENT lines describe the license model for a product. Only the first FEATURE line for a given feature name is processed by the vendor daemon. If you want to have additional copies of the same feature (for example, to have multiple node-locked, counted features), then you must use multiple INCREMENT lines. INCREMENT lines form license groups, or *pools*, based on the following fields:

- feature name
- version
- BORROW
- DUP_GROUP
- FLOAT_OK
- HOST_BASED
- HOSTID
- PLATFORM
- TZ
- USER_BASED
- VENDOR_STRING (if configured by the publisher as a pooling component)
- VM_PLATFORMS

If two lines differ by any of these fields, a new group of licenses, called a *license pool*, is created in the vendor daemon, and this group is counted independently from other license pools with the same feature name. A FEATURE line does not give an additional number of licenses, whereas an INCREMENT line always gives an additional number of licenses.

The basic feature definition line format is:

```
{FEATURE|INCREMENT} feature vendor feat_version exp_date num_lic [optional_attributes] SIGN="<...>"
```

Publisher-Defined Required Keywords

The six fields after the feature definition line keyword are required and have a fixed order. They are defined by the software publisher and cannot be changed. The following table presents these fields in the order they must appear.

Table 4-3 ■ Feature Definition Line Required Fields

Field	Description
<i>feature</i>	Name given to the feature by the software publisher.
<i>vendor</i>	Name of the vendor daemon; also found in the VENDOR line. The specified daemon serves this feature.
<i>feat_version</i>	Version of this feature that is supported by this license. When this field contains a date with the format <i>yyyy.mmdd</i> , this defines a date-based version that you can set as an Alert in the license server manager, <i>lmadmin</i> . In this case, <i>mmdd</i> must be in the range 101 to 1231 and valid (for example, <i>2020.1131</i> is not a valid date). For <i>dd</i> , both digits must be specified (for example, <i>2020.111</i> means January 11, 2020).
<i>exp_date</i>	Expiration date of license in the format <i>dd-mmm-yyyy</i> , for example, 31-dec-2020. 
	Note ■ If <i>exp_date</i> is the string “permanent” or the year is 0 (or 00, 000, 0000) then the license never expires.
<i>num_lic</i>	Number of concurrent licenses for this feature. If the <i>num_lic</i> is set to the string “uncounted” or 0, the licenses for this feature are uncounted and no license server is required but a <i>hostid</i> on the FEATURE line is required. See Counted vs. Uncounted Licenses .
SIGN= <i>sign</i> or AUTH= ...	SIGN= signature to authenticate this FEATURE line. If your publisher has deployed his vendor daemon using the common vendor daemon technology, signatures are embedded within the AUTH= keyword. Contact your publisher for further details.

Optional Publisher-Defined Keywords

Table 4-4 lists attributes that may appear in a FEATURE or INCREMENT line. They are supplied at the discretion of the software publisher to define the license model. If present in the FEATURE or INCREMENT line, they must remain there and cannot be altered by the end user. These attributes have a *keyword=value* syntax where *keyword* is in uppercase.

Table 4-4 ■ Attributes Set by the Software Publisher

Attribute	Description
BORROW [=n]	Enables license borrowing for a particular feature definition line. n is the number of hours that the license is borrowed. The default borrow period is 168 hours, or one week.
DUP_GROUP=...	The syntax is: <pre>DUP_GROUP=NONE SITE ([UHDV]</pre> U = DUP_USER H = DUP_HOST D = DUP_DISPLAY V = DUP_VENDOR_DEF Any combination of UHDV is allowed, and the DUP_MASK is the OR of the combination. For example, DUP_GROUP=UHD means the duplicate grouping is (DUP_USER DUP_HOST DUP_DISPLAY), so for a user on the same host and display, additional uses of a feature do not consume additional licenses.
FLOAT_OK [=server_hostid]	Enables mobile licensing via FLEXID with FLOAT_OK for a particular feature definition line. This feature definition line must also be node-locked to a FLEXID. When FLOAT_OK=server_hostid is specified on a FEATURE line: The server_hostid must refer to the same host that appears on the SERVER line of the license file. The license server runs only on the system with the hostid that lmhostid returns equal to the server_hostid specified with FLOAT_OK.
HOSTID= "hostid1 [hostid2 ... hostidn]"	Id of the host to which the FEATURE line is bound. <i>hostid</i> is determined with the lmhostid utility. This field is required for uncounted licenses; but can be used for counted licenses as well. See Hostids for Supported Platforms for more information. Note ■ Host names generated dynamically will change the composite hostid value.
HOST_BASED[=n]	Host names must be specified in INCLUDE statements in the options file, and the number of hosts is limited to <i>num_Lic</i> , or the number specified in <i>=n</i> .
ISSUED=dd-mm-yyyy	Date issued.

Table 4-4 ■ Attributes Set by the Software Publisher

Attribute	Description
ISSUER="..."	Issuer of the license.
NOTICE="..."	A field for intellectual property notices.
ONE_TS_OK	Detects when a node-locked uncounted license is used by an application running under remote desktop. The following are the two scenarios where you can expect 178 and 179 error codes: • -178: Internal error, please report to Revenera LLC. This error occurs when an administrator performs more than one remote desktop checkout on more than one terminal server remote client. • -179: Only one terminal server remote client checkout is allowed for this feature. This error occurs when a normal user performs more than one remote desktop checkout on more than one terminal server remote client.  Note - <i>ONE_TS_OK attribute is only used with uncounted licenses and is not supported with counted (served) licenses.</i>
OVERDRAFT=<i>n</i>	The overdraft policy allows a software publisher to specify a number of additional licenses which users are allowed to use, in addition to the licenses they have purchased. This allows your users to not be denied service when in a “temporary overdraft” state. Usage above the license limit is reported by the FlexNet Manager reporting tool.
SN=<i>serial_num</i>	Serial number, used to identify FEATURE or INCREMENT lines.
START=<i>dd-mm-yyyy</i>	Start date.
SUITE_DUP_GROUP=...	Similar to DUP_GROUP, but affects only the enabling FEATURE line for a package suite. It limits the total number of users of the package to the number of licenses, and allows the package to be shared among the users that have the SUITE checked out.
SUPERSEDE= "f1 f2 ..."	If this appears, all INCREMENT or FEATURE lines for the specified feature that were issued earlier than the ISSUED date for the SUPERSEDE are <i>superseded</i> by this line and become ineffective. (SUPERSEDE also overrides those INCREMENT and FEATURE lines for the feature that have no ISSUED date.)

Table 4-4 ■ Attributes Set by the Software Publisher

Attribute	Description
SUPERSEDE_SIGN= {f1:xxxx, f2:xxxx}	Overrides the license models of all feature definition lines or package lines defined as the value.
SUPERSEDE_SIGN= {p1:xxxx, p2:xxxx}	
TS_OK	FlexNet Publisher detects when a node-locked uncounted license is running under Windows Terminal Server. To run the application via a Terminal Server client window, TS_OK must be added to the FEATURE line. Without TS_OK, a user running on a Terminal Server client is denied a license.  Note ■ TS_OK attribute is only used with uncounted licenses and has no effect on counted (served) licenses.
TZ= [SERVERTZ/ <[+-]hh<.30 .45> <:[+-]hh<.30 .45>>>]	Enforces license usage for a feature relative to a time zone; where the time zone is specified and measured relative to Greenwich Mean Time (GMT). The computer system on which the FlexEnabled application is running must be within the specified time zone or range of time zones, or in the same time zone as the license server (using the value SERVERTZ).
USER_BASED[=n]	Users must be specified in INCLUDE statements in the options file, and the number of users are limited to <i>num_Lic</i> , or the number specified in <i>=n</i> .
VENDOR_STRING="..."	This is a custom value defined by the software publisher and enclosed in double quotes.
VM_PLATFORMS= [PHYSICAL VM_ONLY VM_ALL]	Restricts feature usage to virtual machines (VM_ONLY) or to physical machines (PHYSICAL). VM_ALL allows running the FlexEnabled application on both physical and virtual machines (default behavior). This is equivalent to not specifying the VM_PLATFORMS keyword.

Examples

```
FEATURE sample_app sampled 2.300 31-dec-2020 20          SIGN="<...>"
INCREMENT f1 sampled 1.000 permanent 5          HOSTID=INTERNET=195.186.*.* NOTICE="Licensed to \
Sample corp" SIGN="<...>"
```

Optional Keywords Defined by the License Administrator

The following attributes listed in [Table 4-5](#) are optional and are under control of the license administrator. These attributes have a *keyword=value* syntax where *keyword* is in lowercase.

Table 4-5 • Optional Feature Line Attributes

Attribute	Description
asset_info="..."	Additional information provided by the license administrator for asset management. This keyword can be used in combination with the options file as a feature name modifier to denote a specific group of licenses (see Specifying Features). In addition, this information is stored inside the CONFIG structure and can be retrieved later (for example by calling lc_auth_data).
dist_info="..."	Additional information provided by the software distributor. This keyword can be used in combination with the options file as a feature name modifier to denote a specific group of licenses (see Specifying Features). In addition, this information is stored inside the CONFIG structure and can be retrieved later (for example by calling lc_auth_data).
sort=nnn	Specifies sort order of license file lines. See Sort Rules.
user_info="..."	Additional information provided by the license administrator. This keyword can be used in combination with the options file as a feature name modifier to denote a specific group of licenses (see Specifying Features). In addition, this information is stored inside the CONFIG structure and can be retrieved later (for example by calling lc_auth_data).
vendor_info="..."	Additional information provided by the software publisher.

Character Limitations in Keyword Values

In places where a keyword value in a FEATURE or INCREMENT line is a string surrounded with double quotes ("..."), the string can contain any characters except the following:

- Single or double quotes (within the surrounding double quotes)
- The backslash character sequence: \ (space+backslash+space)
- The double-backslash character sequence: \\ (space+backslash+backslash+space)

Sort Rules

Feature definition lines are automatically sorted when they are read from the license file. The default sorting rules are as follows:

1. License file. Automatic sorting does not occur across files in a license search path.
2. Feature name.
3. VENDOR_STRING entry.
4. FEATURE before INCREMENT.
5. Uncounted before counted.
6. Version, earlier versions before later versions.
7. Issued date, in reverse order, newest first. The date is taken from ISSUED= or START=.
8. Original order is otherwise maintained.

You can override the automatic ordering by adding the sort attribute for each FEATURE and INCREMENT line. The value defined for the sort attribute identifies the relative position of that feature in the sort order. For example, features with a sort value of 25 will be sorted before those with a sort value of 50. Unless explicitly defined, the sort value for each feature is set to 100. All lines with the same number are sorted as they appear in the file. The value range for the sort attribute is 0 through 255. If you specify a value higher than 255, the encryption process converts it to 255.

Overriding the sort order might be useful to change the order in which licenses are consumed. Consider the following example: A company has multiple pools of licenses with different expiration dates. The company wants their users to first consume the licenses from the pool that expire first. To achieve this, the license administrator can add the sort=1 keyword to the feature line that expires first.

Changes in FEATURE and INCREMENT Line Format

The following lists the significant changes in the format of feature definition lines and when additional keywords were introduced.

- Version 7.1 and earlier feature definition line format uses *license_key*:

```
{FEATURE|INCREMENT} feature vendor feat_version exp_date \  
    num_lic [optional_attributes] SIGN="<...>"
```

The version 7.1 and earlier format is understood by the current release.

- The SIGN= keyword introduced in the version 7.1.
- For version 7.1 through version 8.0 client libraries and vendor daemons, the feature definition line must have a SIGN= signature and, for backward compatibility with version 8.1 and earlier, can contain a *license_key*:

```
{FEATURE|INCREMENT} feature vendor feat_version exp_date \  
    num_lic [license_key] [optional_attributes] SIGN="<...>"
```

- *license_key* obsoleted in version 8.1 client library and vendor daemon
- The keyword “permanent” for *exp_date* introduced in version 6 client library.
- The keyword “uncounted” for *num_lic* introduced in version 6 client library.

- BORROW keyword introduced in version 8.0 client library and vendor daemon.
- FLOAT_OK keyword introduced in version 8.0 client library and vendor daemon.
- TS_OK keyword introduced in version 8.0 client library and vendor daemon.
- AUTH keyword introduced in version 10.8 client library and vendor daemon.

PACKAGE Lines

The purpose of the PACKAGE line is to support two different needs:

- To license a product SUITE, or
- To provide a more efficient way of distributing a license file that has a large number of features, which largely share the same FEATURE line arguments.

A PACKAGE line, by itself, does not license anything—it requires a matching feature definition line to license the whole package. A PACKAGE line is shipped by your software publisher with a product, independent of any licenses. Later, when you purchase a license for that package, one or more corresponding feature definition lines enable the PACKAGE line.

Example

```
PACKAGE package vendor [pkg_version] COMPONENTS=pkg_List \  
    [OPTIONS=SUITE] [SUPERSEDE=["p1 p2 ..."] ISSUED=date]  
    SIGN="<...>"
```

The following lists the PACKAGE line fields. They must appear in the order listed.

Table 4-6 ■ PACKAGE Line Fields

Field	Description
<i>package</i>	Name of the package. The corresponding feature definition line must have the same name.
<i>vendor</i>	Name of the vendor daemon that supports this package.
<i>pkg_version</i>	Provide the version of the package. The corresponding feature definition line must have the same version.

Table 4-6 ■ PACKAGE Line Fields

Field	Description
COMPONENTS= <i>pkg_List</i>	List of package components. The format is: <pre><i>feature[:version[:num_Lic]]</i></pre> Packages must consist of at least one component. Version and count are optional, and if left out, their values come from the corresponding feature definition line. <i>num_Lic</i> is only legal if OPTIONS=SUITE is not set—in this case the resulting number of licenses is <i>num_Lic</i> on the COMPONENTS line multiplied by the number of licenses in the feature definition line. Examples: <pre>COMPONENTS="comp1 comp2 comp3 comp4" COMPONENTS="comp1:1.5 comp2 comp3:2.0:4"</pre>
OPTIONS=SUITE	Optional field. Used to denote a package suite. If set, the corresponding feature of the same name as the package is checked out in addition to the component feature being checked out. If not set, then the corresponding feature of the same name as the package is removed once the package is enabled; it is not checked out when a component feature is checked out.
OPTIONS=SUITE_RESERVED	Optional field. If set, reserves a set of package components. Once one package component is checked out, all the other components are reserved for that same user.
SUPERSEDE [="p1 p2 ..."]	Optional field. Used in conjunction with ISSUED date. Replaces all PACKAGE lines for the same package name with previous or no ISSUED dates.
ISSUED= <i>dd-mm-yyyy</i>	Optional field. Used in conjunction with SUPERSEDE in the PACKAGE line. Replaces all PACKAGE lines for the same package name with previous or no ISSUED dates.
SIGN= <i>sign</i> or AUTH=...	SIGN= signature to authenticate this FEATURE line. If your publisher has deployed his vendor daemon using the common vendor daemon technology, signatures are embedded within the AUTH= keyword. Contact your publisher for further details.

Examples

```
PACKAGE suite sampled 1.0 SIGN="<...>" \
    COMPONENTS="comp1 comp2" OPTIONS=SUITE
FEATURE suite sampled 1.0 31-dec-2020 5 SIGN="<...>"
```

This is a typical OPTIONS=SUITE example. There are two features, “comp1” and “comp2,” which are each version 1.0, each with five licenses available. When “comp1” or “comp2” is checked out, “suite” is also checked out.

```
PACKAGE suite sampled 1.0 SIGN="<...>"\
  COMPONENTS="apple:1.5:2 orange:3.0:4"
FEATURE suite sampled 1.0 31-dec-2020 3 SN=123 SIGN="<...>"
```

In this example, the component version overrides the feature version, and the number of licenses available for any component is the product of the three licenses for “suite” and the number of licenses for that component. The result is equivalent to:

```
FEATURE apple sampled 1.5 31-dec-2020 6 SN=123 SIGN="<...>"
FEATURE orange sampled 3.0 31-dec-2020 12 SN=123 SIGN="<...>"
```



Note • The following lists changes to PACKAGE lines:

- Ability to store **PACKAGE** lines in separate files introduced in version 6 client library.
- *pkg_version* field required is mandatory.
- *AUTH* keyword introduced in version 10.8 client library and vendor daemon.

UPGRADE Lines

```
UPGRADE feature vendor from_feat_version to_feat_version \
exp_date num_lic [options ... ] SIGN="<...>"
```

All the data is the same as for a FEATURE or INCREMENT line, with the addition of the *from_feat_version* field. An UPGRADE line removes up to the number of licenses specified from any old version ($\geq$ *from_feat_version*) and creates a new version with that same number of licenses.

For example, the two lines provide three version 1.0 licenses of **f1** and two version 2.0 licenses of **f1**.

```
INCREMENT f1 sampled 1.000 31-dec-2020 5 SIGN="<...>"
UPGRADE f1 sampled 1.000 2.000 31-dec-2020 2 SIGN="<...>"
```

An UPGRADE line operates on the closest preceding FEATURE or INCREMENT line with a version number that is $\geq$ *from_feat_version*, and $<$ *to_feat_version*.

Order of Lines in the License File

The order of the lines in a license file is not critical. They are sorted when they are processed so that in most cases the optimal result is achieved. However, version 7.0 and earlier versions of FlexEnabled applications and license servers implicitly impose an ordering to license file lines. Note the following suggestions for ordering lines in the license file:

- Place FEATURE lines before INCREMENT lines for the same feature.

The rules regarding FEATURE lines include the following: 1) only the first counted FEATURE line is observed by the license server, and 2) if both a FEATURE line and INCREMENT lines exist, the FEATURE line must appear first.

- Where multiple counted FEATURE lines exist for the same feature, make sure the desired FEATURE line appears first.

All but the first is ignored.

- Place node-locked, uncounted lines before floating lines for the same FEATURE. Otherwise, it is possible the floating license is consumed instead of the node-locked license, resulting in denial for other users.
- The placement of a USE_SERVER line affects behavior. A USE_SERVER line is recommended. Normally, the USE_SERVER line is placed immediately after the SERVER line. However, place any uncounted licenses not served by SERVER before the USE_SERVER line. Make sure each user that needs the uncounted license has direct access to a current copy of the file. The advantage to placing USE_SERVER right after the SERVER line is users don't need up-to-date copies of the license file.

See Also
[Sort Rules](#)

Locating Licenses

This section covers various topics that are related to the ability of FlexEnabled applications to locate licenses. The following are described:

- Determining a location for license files on a license server
- Configuring the machine where the FlexEnabled application is running to access licenses.

Determining the Location of the License File

Software publishers often recommend a specific location for your license file. You have the following options for making your licenses available to all systems:

- Place the license file in a partition which is available to all systems in the network that need it.
- Copy the license file to each of the individual systems.
- Set the `LM_LICENSE_FILE` or `VENDOR_LICENSE_FILE` (where `VENDOR` is the vendor daemon name) environment variable on the machines where the FlexEnabled applications are running to access license files or license servers. For details see [Setting the License Search Path Using an Environment Variable](#).

Do not choose a location for a license file where the path to the license file contains the `@` symbol. The `@` symbol is used to identify a license server as illustrated in [Table 5-1](#).



Note - A directory path that contains an `@` symbol results in an error.

Since the vendor daemon keeps track of license usage, and since the license file contains encrypted data to protect it against modification, you may move and copy the license file as much as necessary.

On Windows, if the application cannot find the license file, the user is presented with a dialog that asks the user to specify the license file location, the license server, or license fulfillment from the internet.

For counted licenses, no matter which option you choose, you must first copy `lmadmin` or `lmgrd` and the vendor daemon to a location that the FlexEnabled application can access on the network.

Setting the License Search Path Using an Environment Variable

Most applications specify a location where they expect to find the license file and install it automatically. However, you can change the license file location by setting the `LM_LICENSE_FILE` environment variable to a license search path. Wherever a license search path is specified, it can consist of one or more of the following entries. On UNIX, the license search path entries are separated by colons ':' and on Windows, the entries are separated by semicolons ';'.

- The full path to the license file
- A directory containing one or more license files with a `.lic` extension
- One of the following port settings:
 - The port setting `port@host` setting, where `port` and `host` are the TCP/IP port number and host name from the SERVER line in the license file.
 - The shortcut specification, `@host`, if the license file SERVER line uses a default TCP/IP port or specifies a port in the default port range (27000–27009).
 - A three-server redundant triad. The triad is a single entry on the license search path and is specified using a comma-separated list of three `port@hosts` (for example, `port1@host1,port2@host2,port3@host3`).

The following table shows some examples of `LM_LICENSE_FILE` and `VENDOR_LICENSE_FILE` environment variable settings.

Table 5-1 ■ Environment Variable Specification Examples

LM_LICENSE_FILE or VENDOR_LICENSE_FILE Setting	Description
40000@myserver	Used where the SERVER line in the license file is the following: SERVER myserver 17007ea8 40000 • host = myserver • port = 40000
@myserver	Used where the SERVER line in the license file is the following: SERVER myserver 17007ea8 • host = myserver • port = None specified. A default TCP/IP port number in the range of 27000-27009 is used.
C:\licenses; 40000@host1,40000@host2,40000@host3	License search path on a Windows system. Unserved licenses are stored in C:\licenses; served licenses are obtained from the three-server redundant triad 40000@host1,40000@host2,40000@host3.

Table 5-1 ■ Environment Variable Specification Examples

LM_LICENSE_FILE or VENDOR_LICENSE_FILE Setting	Description
licenses:40000@myserver:40000@mybackupserver	License search path on a UNIX system. Unserved licenses are stored in the local directory licenses; served licenses are obtained from either myserver or mybackupserver. In the first instance, a license is requested from myserver; if this fails, mybackupserver is tried.

Applications accept an environment variable (or Windows Registry) named `VENDOR_LICENSE_FILE`, where `VENDOR` is the vendor daemon name, for example, `DEMO_LICENSE_FILE`. This environment variable's scope is limited to just those applications from software publisher using the `VENDOR` name.

With `lmgrd` and `lmutil` (`lmstat`, `lmdown`, and so on), the `-c` option overrides the setting of the `LM_LICENSE_FILE` environment variable.



Note ■ Some applications do not recognize the `LM_LICENSE_FILE` environment variable. FlexEnabled Java applications, in particular, do not recognize it.

Order of Searching for a License

A FlexEnabled application looks for a license file as follows:

1. When the `VENDOR_LICENSE_FILE` environment variable has been set for the publisher of the application, then items in the license search path set in this environment variable are searched in order.
2. The items in a license search path set in the `LM_LICENSE_FILE` environment variable are searched in order.
3. When any license file specified in a license search path contains a `USE_SERVER` line, then a license is requested from the license server specified in the `SERVER` line. Any `FEATURE` and `INCREMENT` lines entries after the `USE_SERVER` line in the license file are ignored.
4. When the environment variables are not set and the FlexEnabled application does not specify the location of the license, then the following default locations are searched:
 - On UNIX—`/usr/local/flexlm/licenses/license.dat`
 - On Windows—`C:\flexlm\license.dat`

When licenses are held in trusted storage on the same machine as the FlexEnabled application, normally the publisher will have configured the application to search local trusted storage first and then look for license files as previously described.

See Also

[Managing Multiple License Files](#) for more information about `LM_LICENSE_FILE`.
[Environment Variables](#)
[Ensuring License Availability](#)

Hostids for Supported Platforms

FlexNet Publisher uses system identifiers, called *hostids*, to node-lock licenses to a machine. The system identifiers may be system specific. For example, all Sun Microsystems systems have a unique hostid.

Hostid Formats

Numeric, 32-bit hostids are normally used in hexadecimal format. On some systems, the system command returns the ID in decimal format. Use a # character before the hostid to indicate a decimal number. For example, if the system command returns 2005771344, FlexNet Publisher accepts **#2005771344**. Alternatively, convert the decimal value to hexadecimal.

Obtaining System Hostids

The `lmhostid` utility prints the exact hostid that FlexNet Publisher requires on any given system. If your hostid contains characters other than the ASCII A through Z, a through z, or 0 through 9, use the `-utf8` option with `lmhostid`. To view a correct representation of the resulting hostid, use a utility, such as Notepad, that can display UTF-8 encoded strings.

`lmadmin` displays hostids available for the license server on the System Information tab.

Because `lmadmin` is available only as a 32-bit process, the 32-bit FlexNet Licensing Service must be installed to display TPM details in `lmadmin`'s System Information tab. The 32-bit FlexNet Licensing Service is available from the i86_n3 kit. The TPM hostid is currently supported on Windows platforms only.



Note ▪ For the following cases, do not use the System Information tab in the `lmadmin` user interface to obtain hostids. Instead, use the methods described in the [Hostid Procurement Methods](#) table.

- **When the license server is operating on a virtual machine but bound to the physical hardware**—A limitation in `lmadmin` causes the System Information tab to show virtual machine values for Host Name, Host Domain Name, IPv4 Address, IPv6 Address, Ethernet Address, and Volume Serial Number rather than the physical machine values.

- **When running license clients or a license server in an Amazon EC2 environment**—At this time, the System Information page is unable to show hostids specific to the Amazon EC2 environment.

The following table lists some sample `lmhostid` and alternate methods to obtain the required hostid for each system architecture. FlexNet Publisher also supports a group of special hostids and vendor-defined hostids.

Table 6-1 ■ Hostid Procurement Methods

Hardware Platform	Hostid	Type this command	Example hostid
AIX (RS/6000, PPC)	32-bit hostid	<code>lmhostid</code>	02765131
		As best practice it is recommended to use <code>lmhostid</code> and that most of the time, the <code>uname</code> rule applies. <code>uname -m</code> For example, returns 000276513100. Remove last two digits and use remaining last eight digits.	
HP-UX (64-bit Itanium)	machine identification	<code>lmhostid</code> or <code>getconf CS_PARTITION_IDENT Prefix</code> returned value with "ID_STRING="	ID_STRING=9c766319-db72-d411-af62-0060b05e4c05
OS X	Ethernet address	<code>lmhostid -ether</code>	000A277EA17E
			
		Note ■ This command lists the available in built Ethernet addresses, active or inactive. It does not currently list Ethernet addresses from removable devices such as USB Ethernet adapters.	
		<code>/sbin/ifconfig enx</code> where enx is the Ethernet interface name (with x a value from 0 to 9); remove colons from returned value.	000A277EA17E
	FlexNet ID USB port dongle	<code>lmhostid -flexid</code> <code>lmhostid -flexid -long</code>	FLEXID=9-b28520b9

Table 6-1 ▪ Hostid Procurement Methods

Hardware Platform	Hostid	Type this command	Example hostid
Linux	Ethernet address	<code>lmhostid -ether</code>	00400516E525
		 Note ▪ This command lists all available Ethernet addresses (active or inactive), including those for team-bonding virtual adaptors.	
		<code>/sbin/ifconfig <ethernet_name></code> where <ethernet_name> is one of these Ethernet interface names: • <code>ethx</code> • Other name type (for example, <code>p3p4</code>) • <code>bond0</code> (for a team-bonding virtual adaptor) Remove colons from returned HWaddr.	00400516E525
FlexNet ID USB port dongle		<code>lmhostid -flexid</code> <code>lmhostid -flexid -long</code>	FLEXID=9-b28520b9
		 Note ▪ The commands <code>lmhostid -flexid</code> is to fetch the dongle id and <code>lmhostid -flexid -long</code> command is used to get the error or log information along with the dongle id.	
UUID (Universally Unique Identifier) for virtual machines		<code>lmhostid -ptype VM -uuid</code> Supported for Xen, VMware, Oracle VirtualBox or Hyper-V (recommended), Parallels, QEMU-KVM, Google Compute, Microsoft Azure, and Nutanix.	VM_UUID=DF440538-8EB7-11DC-BBDA-FE7FE89E000F
		 Note ▪ Obtaining this hostid requires that the FlexNet Licensing Service be installed. To do this, certificate-only customers need to ensure “<code>install_fnp.sh --cert</code>” is run from the publisher directory with root-privilege at install time.	

Table 6-1 ■ Hostid Procurement Methods

Hardware Platform	Hostid	Type this command	Example hostid
	AMZN_EIP in Amazon EC2 environment	<code>lmhostid -ptype AMZN -eip</code>	AMZN_EIP=184.72.45.35
		 Note ■ Obtaining this hostid requires that the FlexNet Licensing Service be installed.	
	AMZN_AMI in Amazon EC2 environment	<code>lmhostid -ptype AMZN -ami</code>	AMZN_AMI=ami-6a807503
		 Note ■ Obtaining this hostid requires that the FlexNet Licensing Service be installed.	
	VM_UUID in Amazon EC2 environment	<code>lmhostid -ptype VM -uuid</code>	VM_UUID=i-7d409db1 (Server only)
		 Note ■ Obtaining this hostid requires that the FlexNet Licensing Service be installed.	
Sun	32-bit hostid	<code>lmhostid</code> or <code>hostid</code>	170a3472
	Ethernet address	<code>lmhostid -ether</code>	00400516E525
Windows	Ethernet address	<code>lmhostid</code>	00B0A9DF9A32
		 Note ■ This command lists all available Ethernet addresses (active or inactive), including those for team-bonding virtual adaptors.	
	Disk serial number	<code>lmhostid -vsn</code>	DISK_SERIAL_NUM= 3e2e17fd

Table 6-1 ▪ Hostid Procurement Methods

Hardware Platform	Hostid	Type this command	Example hostid
	FlexNet ID parallel or USB port dongle	<code>lmhostid -flexid</code> <code>lmhostid -flexid -long</code>	FLEXID=9-b28520b9
		 Note ▪ For parallel port dongles, the parallel port must be configured in bi-directional mode. The commands <code>lmhostid -flexid</code> is to fetch the dongle id and <code>lmhostid -flexid -long</code> command is used to get the error or log information along with the dongle id.	
	VM_UUID (Universally Unique Identifier) for virtual machines	<code>lmhostid -ptype VM -uuid</code> where <virtual machine type> is one of these: Supported for Xen, VMware, Oracle VirtualBox or Hyper-V (recommended), Parallels, QEMU- KVM, Google Compute, Microsoft Azure, and Nutanix.	VM_UUID=DF440538-8EB7- 11DC-BBDA-FE7FE89E000F (Server only)
		 Note ▪ Obtaining this hostid requires that the FlexNet Licensing Service be installed.	
	AMZN_EIP in Amazon EC2 environment	<code>lmhostid -ptype AMZN -eip</code>	AMZN_EIP=184.72.45.35
		 Note ▪ Obtaining this hostid requires that the FlexNet Licensing Service be installed.	
	AMZN_AMI in Amazon EC2 environment	<code>lmhostid -ptype AMZN -ami</code>	AMZN_AMI=ami-6a807503
		 Note ▪ Obtaining this hostid requires that the FlexNet Licensing Service be installed.	
	TPM (Trusted Platform Module)	<code>lmhostid -tpm_id1</code>	TPM_ID1=MKG5S-LU9TR- 4LT7M-NICDK
		 Note ▪ Obtaining this hostid requires that the FlexNet Licensing Service be installed.	

Special Hostids

FlexNet Publisher contains a number of special hostid types that apply to all platforms. These hostid types are valid to use in both SERVER lines and FEATURE lines, wherever a hostid is required.

Table 6-2 • Special Hostid Types

Hostid	Description
ANY	Locks the software to any system (meaning that it does not lock anything).
DEMO	Similar to ANY , but only for use with uncounted FEATURE lines.
COMPOSITE= <composite_ hostid>	Locks the software to a composite hostid. The composite hostid can be made up of any number of the simple hostids supported by FNP. These are aggregated together and the resultant buffer hashed down to a 12-byte hexadecimal value. The process is one-way hashing, hence makes it impossible to reverse the constituent input simple hostids. Note that composite hostids are not returned by <code>lmhostid</code>, <code>LMTTOOLS</code>, or <code>lmadmin</code>: when composite hostids are used, the software publisher will provide a utility that determines the publisher's composite hostid. On some systems multiple composite hostids may be provided, any of which may be used to identify the system that the software is locked to.
DISPLAY= <display>	Locks the software to a display. On UNIX, <display> is <code>/dev/ttyxx</code> (which is always <code>/dev/tty</code> when an application is run in the background) or the X-Display name. On Windows, it is the system name or, in the case of a terminal server environment, the terminal server client name (version 8 or later FlexEnabled applications only)
HOSTNAME= <host>	Locks the software to computer host name <host>.
ID=<n>	Functionally equivalent to the ANY hostid—it runs on any system. The difference is that the license is unique and is used to identify the end user. This hostid is used to lock the license server (on the SERVER line) or the FlexEnabled application (on the feature definition line). The number can have dashes included for readability—the dashes are ignored. Examples: ● ID=12345678 is the same as ● ID=1234-5678 is the same as ● ID=1-2-3-4-5-6-7-8

Table 6-2 ▪ Special Hostid Types

Hostid	Description
INTERNET= <IP_address(es)>	Locks the software to an Internet IP address, or group of IP addresses. Wildcards are allowed. For example, 198.156.*.* means any host with a matching INTERNET IP address. The main use is to limit usage access by subnet, implying geographic area. For this purpose, it is used on the feature definition line as a hostid lock. For more information about obtaining this hostid, see the <code>-internet</code> option for lmhostid.  Note ▪ (Windows only) If a Windows machine is connected to a VPN, <code>lmhostid -internet</code> returns the virtual adapter IP address. To ensure that this command returns the physical adaptor IP address during checkouts, the user needs to perform this workaround: Run <code>regedit</code>, navigate to <code>HKEY_LOCAL_MACHINE\SYSTEM\CurrentControlSet\services\Tcpip\Linkage</code>, locate the <code>Bind</code> parameter, and edit the adapter order so that the physical adapter is listed first. Run <code>ipconfig/registerdns</code> to put the edit into effect.
USER=<user>	Locks the software to user name <user>. User names specified in license files cannot contain spaces.

Examples

```
FEATURE f1 demo 1.0 31-dec-2020 uncounted \
    HOSTID=FLEXID=9-a6300015f SIGN="<...>"
```

or

```
FEATURE f1 demo 1.0 31-dec-2020 uncounted \
    HOSTID=INTERNET=10.10.10.* SIGN="<...>"
```

Ethernet Hostids

The Ethernet address is used on some system architectures as the hostid. An ethernet address is a 6-byte quantity, with each byte specified as two hexadecimal digits. Specify all twelve hex digits when using an Ethernet address as a hostid. For example, if the ethernet address is 8:0:20:0:5:ac, specify **0800200005ac** as the hostid.

Several devices with an ethernet address can be attached to a machine; some of these might be virtual devices that generate a different ethernet address each time they are activated. An example of a virtual device that might generate an ethernet address is VPN (virtual private network) software.

Some devices that have an ethernet address can be detachable from the machine. For example, a laptop plugged into a docking station uses the ethernet address of the docking station; however, when it is disconnected from the docking station, the ethernet address is no longer available. A wireless adapter also has an ethernet address and this address is not available when either the wireless adapter is removed from the machine or when the wireless adapter is disabled, but still physically attached to the machine.

When `lmhostid` returns multiple ethernet hostids, ensure that you choose a permanent or stable hostid to identify your machine.



Note - FlexNet Publisher considers the address of any team-bonding virtual adaptor (for teamed Ethernet interfaces) as a stable identifier for use as a permanent hostid.

TPM Hostid

FlexNet Publisher supports the TPM (Trusted Platform Module) hostid for to uniquely identify a computer, specified as TPM_ID1 in license files. The TPM hostid is currently supported on Windows platforms only.

As a prerequisite to obtaining and using the TPM hostid, a TPM version 2.0 device must be available and enabled. In addition, the FlexNet Licensing Service must be installed.

The TPM hostid is only supported on the SERVER line. It is supported as a served node-locked hostid from FlexNet Publisher version 11.15.0 onwards. The TPM hostid is not supported for trusted storage-based licensing.

You can identify the TPM status using the utilities `lmtpminfo` and `lmhostid`, or the API `lc_tpmstatusget`, or natively in Windows by using the Microsoft Management Console with `tpm.msc` specified, or by using WMI and the `Win32_Tpm` class.

For TPM status values as output by `lmtpminfo`, see [lmtpminfo](#) on page 159.

Troubleshooting

If you encounter problems when trying to obtain the TPM hostid, check that the following requirements have been met:

- The FlexNet Licensing Service must be installed. Contact your software publisher for information on how to install the FlexNet Licensing Service.
- The TPM must be turned on and enabled in the machine's BIOS. To enable it, do the following:
 1. Restart the machine. During the restart, follow the instructions on the screen that explain how to interrupt normal startup and enter the BIOS setup utility. Locate the Security Settings and enable the TPM. Reboot the system. Run `lmtpminfo` or `lmhostid` to identify the TPM status. If the status cannot be obtained, continue with step 2.
 2. Open the TPM management console (`tpm.msc`). If the status is TPM not fully enabled, it may be necessary to enable UEFI mode in the machine's BIOS.

Container ID

FlexNet Publisher supports the Container ID hostid to uniquely identify a docker container. The Container ID hostid is currently supported on Windows and Linux platforms.

The Container ID hostid is supported on the SERVER and FEATURE line. It is supported as a served node-locked hostid. The Container ID hostid is supported for certificate based licensing.

You can identify the Container ID status using the `lmhostid` utility, or the API `lc_dockerstatusget`.

Hostids to Support Virtualization Policy

Your software publisher may choose to enforce a Virtualization support policy using the special hostid constructs in the license file. Three hostids are available for use in license file–based license servers running inside virtual machine: VM_UUID, ETHER, and VM_GENID.

VM_GENID is the Generation ID of the virtual machine and has the following restrictions:

- *Windows guests only*
- *Not used as a traditional hostid—for example, may be provided in a VENDOR_STRING field.*

Refer to the Virtualization white paper for the example usage of VM_GENID.



Note ▪ *The FlexNet Licensing Service must be installed on machines that use virtualization features.*

Hostids to Support Cloud Licensing

The hostids listed in this chapter for the Amazon EC2 environment support typical use cases for licensing software in a cloud. For a description of cloud hostids, see [Licensing in a Cloud-Computing Environment](#).

License Models

License rights are created by the software publisher. License rights specify floating (concurrent) usage, node-locked (both counted and uncounted), or any combination of floating, counted, and uncounted.

Floating (Concurrent) Licenses

A *floating license* means anyone on the network can use the FlexEnabled application, up to the limit specified in the license file or fulfillment record (also referred to as *concurrent usage* or *network licensing*). Floating licenses have no hostids on the individual FEATURE lines. Floating licenses requires a license server manager and a vendor daemon to be running to count the concurrent usage of the licenses.

An example of a license file that provides floating licenses is:

```
SERVER lulu 17007ea8
VENDOR sampled
FEATURE f1 sampled 1.00 31-dec-2020 2 SIGN="<...>"
FEATURE f2 sampled 1.00 31-dec-2020 6 SIGN="<...>"
FEATURE f3 sampled 1.00 31-dec-2020 1 SIGN="<...>"
```

This license file specifies that two licenses for feature **f1**, six licenses for feature **f2**, and one license for feature **f3** are available anywhere on the network that can access the license server, called **lulu**. The license server manager uses one of the default TCP/IP ports.

The equivalent floating licenses are held in trusted storage as a fulfillment record that contains the same FEATURE lines as in the license file without any SERVER or VENDOR lines.

Node-Locked Licenses Using Hostid

This section describes node-locked licenses using a hostid. Licenses held in trusted storage are node-locked because trusted storage is locked to a machine, see [Locking of Licenses Using Hostid or Trusted Storage](#) for an explanation.

Node-locking means the FlexEnabled application can be used on one system or a set of systems only. A node-locked license has a hostid on the FEATURE line that identifies a specific host. There are two types of node-locked licenses: uncounted and counted.

If the number of licenses value is set to either zero (0) or uncounted, then the license will not be counted which allows the license to be used an unlimited number of times. This configuration does not require a license server because it is not necessary to count the concurrent usage of the features.

The license server can be used to serve uncounted licenses. However, served uncounted INCREMENT line signatures are not linked to the hostid on the SERVER line, therefore using served uncounted licenses with HOSTID=ANY is an exploit risk. Instead, Revenera recommends served counted licenses with a large count.

The following license file allows unlimited usage of feature **f1** on the systems with hostids of **17007ea8** and **1700ab12**:

```
FEATURE f1 sampled 1.000 31-dec-2020 uncounted HOSTID=17007ea8 SIGN="<...>"
FEATURE f1 sampled 1.000 31-dec-2020 uncounted HOSTID=1700ab12 SIGN="<...>"
```

Alternately, these two FEATURE lines could have been issued by your software publisher with a *hostid list*:

```
FEATURE f1 sampled 1.000 31-dec-2020 uncounted HOSTID="17007ea8 1700ab12" SIGN="<...>"
```

If these were the only FEATURE lines in this license file, neither the license server manager or vendor daemon are necessary and you do not need to start one.

The following license file provides three licenses for feature **f1**, locked to the system with hostid **1300ab43**. Since the license server and licenses are locked to the same system, the daemons run on the same system that runs the FlexEnabled application.

```
SERVER lulu 1300ab43 1700
VENDOR sampled /etc/sampled
FEATURE f1 sampled 1.00 31-dec-2020 3 HOSTID=1300ab43 SIGN="<...>"
```

Mixed Node-Locked and Floating Licenses

Uncounted node-locked and concurrent usage licenses can be mixed in the same license file.

The following license file allows unlimited use of feature **f1** on systems **17007ea8** and **1700ab12**, while allowing two other licenses for feature **f1** to be used anywhere else on the network:

```
SERVER lulu 17001234 1700
VENDOR sampled C:\flexlm\sampled.exe
FEATURE f1 sampled 1.00 31-dec-2020 uncounted HOSTID=17007ea8 SIGN="<...>"
FEATURE f1 sampled 1.00 31-dec-2020 uncounted HOSTID=1700ab12 SIGN="<...>"
FEATURE f1 sampled 1.00 31-dec-2020 2 SIGN="<...>"
```

This configuration requires a license server manager and vendor daemon because the licenses on the third FEATURE line are counted.

Counted vs. Uncounted Licenses

The license model (as defined in the license file on the end user machine) determines whether a license server is needed. If all feature definition lines have a license count set to either zero (0) or uncounted, then the customer does not need a license server. This type of license is called uncounted. Alternatively, if any features have a non-zero license count, then the customer needs a license server to count those licenses. If a software publisher wants to use FlexNet Publisher without a license server, they must issue uncounted licenses.

The license server can serve uncounted licenses also. This is often done so that:

- Transactions can be logged into the report log for all license requests, which can then be reported on by FlexNet Manager.
- Options file constraints can be applied to the licenses.

When serving uncounted licenses from a license server, note the following:

- Include the SERVER and VENDOR lines in the license file.
- The user application must point to the license server with `port@host` (recommended).
- Served uncounted INCREMENT line signatures are not linked to the hostid on the SERVER line, therefore using served uncounted licenses with `HOSTID=ANY` is an exploit risk. Instead, Revenera recommends served counted licenses with a large count.

Mobile Licensing

End users often want to use applications on computers that do not have a continuous connection to a license server. These situations include:

- Working on a laptop
- Using a computer both at work and at home
- Working from several different computers not connected to a license server

FlexNet Publisher supports licenses that allow one of several kinds of mobile licensing:

- [Node-Locked to a Laptop Computer](#)
- [Node-locked to a FlexNet ID Dongle](#)
- [Node-Locked to a FlexNet ID Dongle with FLOAT_OK](#)
- [License Borrowing with BORROW](#)
- [Node-locked to a User Name](#)
- [Fulfilled from a Prepaid License Pool](#)
- [License Transferring Using the Transferrable Counted Model—Windows Only](#)
- Optionally when provided by the publisher, [Distribution of Node-Locked Licenses to Networked Machines](#) using trusted storage can be used.

You should use license rehosting if an enterprise wants to move a license without using one of these methods. The software publisher must generate a new node-locked license file for each new client computer. Rehosting requires administrative overhead because the software publisher must be involved with each move.

Node-Locked to a Laptop Computer

To use a license exclusively on one laptop computer, the license should be node-locked to that computer. When the license is held in a license file, it resides on the laptop computer. Any license held in trusted storage on a laptop computer is node-locked to the laptop.

Node-locked to a FlexNet ID Dongle

To move a license between different systems, it can be locked to a FlexNet ID dongle (a dongle that connects to a parallel or USB port). You can move this license between systems by installing a copy of the license file with a hostid set to the FLEXID of the dongle on each system and moving the dongle from one system to another. Since the license is tied to the dongle, only the system with the dongle can use the license.

FlexNet ID dongles are made available by your software publisher. Your software publisher can also provide you with an installer that installs drivers for all FlexNet ID dongles.

Node-Locked to a FlexNet ID Dongle with FLOAT_OK

A dongle can be used to provide floating licenses via a license server by including the key word `FLOAT_OK`. This method has an advantage over simply using a license locked to a `FLEXID`, because the dongle can now be attached to a license server so that the licenses can be checked out by any network machine.

The software publisher issues you a dongle and a license file that contains a `FEATURE` line `node-locked` to the `FLEXID` containing the `FLOAT_OK` keyword. One dongle and `FEATURE` line containing the `FLOAT_OK` keyword is needed for each instance of a license that is mobile. When the dongle is attached to a license server, the license floats on the network. When the dongle is removed from the license server, the license is available only on the standalone computer.

This method supports parallel or USB dongles. Because it is simpler to attach multiple USB dongles to a computer, they may be preferable.

Using a FlexNet ID Dongle for Mobile Licensing Using a `FLOAT_OK` License

The software publisher provides a dongle, a dongle driver installer, and a license file that contains a `FEATURE` line `node-locked` to the `FLEXID` containing the `FLOAT_OK` keyword. A license administrator then:

1. Installs the license file on the license server.
2. Installs the FlexNet ID dongle driver on the license server.
3. Attaches the dongle to the license server.
4. Starts the license server or rereads the license file

While the dongle is attached to the license server, the node-locked license associated with it floats on the network.



Task

To transfer a license from the pool of floating licenses to a disconnected computer:

1. Copy the license file containing the FLOAT_OK node-locked FEATURE line from the license file on the license server to a license file on the client in the location where the FlexEnabled application expects to find its license file.
2. Install the dongle driver on the client computer, if it is not already installed.
3. Move the dongle matching the node-locked FEATURE line from the license server to the client. When the dongle is removed from the license server, this license is unavailable on the network.
4. Disconnect the client computer from the network. Now the license is available on the computer with the dongle, even though that computer is disconnected from the network.



Task

To return the license to the license server so it floats on the network again:

1. Remove the dongle from the client and replace it on the license server.
2. Reread the license file for the license server that serves the floating version of the license by running `lmreread`.

When the dongle is returned to the license server, the FLOAT_OK license does not float on the network again until `lmreread` is run.

FLEXID with FLOAT_OK Example

The following is a sample license file. It is shipped with two dongles: FLEXID=9-34efc1d8 and FLEXID=9-30eb7ff6.

```
SERVER myhost ANY
VENDOR sampled
FEATURE f1 sampled 1.0 permanent uncounted FLOAT_OK \
    HOSTID=FLEXID=9-34efc1d8 SIGN="<...>"
FEATURE f1 sampled 1.0 permanent uncounted FLOAT_OK \
    HOSTID=FLEXID=9-30eb7ff6 SIGN="<...>"
```

The user installs the license file and the two dongles on the license server. When attached to the license server, each uncounted FLOAT_OK license floats on the network and allows a single use. Therefore, up to two users can use **f1** on the end user's network, except on the license server itself, where the license use is disallowed.

If a user wants to work at home, the user installs a license file that contains the FEATURE line node-locked to FLEXID=9-34efc1d8 (this only needs to be done once), transfers the dongle with FLEXID=9-34efc1d8 from the license server to the client, and installs the dongle driver on the client computer (this also only needs to be done once). The user disconnects the client computer from the network and uses the transferred FLOAT_OK license on the client computer. The license server allows only the single remaining FLOAT_OK license to float on the network.

After returning the dongle to the license server, the license administrator runs `lmreread` so the returned license can float again.



Note • *FLOAT_OK keyword introduced in version 8.0 client library, license server manager, and vendor daemon. All components must be version 8.0 or later in order to use FLOAT_OK.*

License Borrowing with BORROW

This method of implementing mobile licensing is used only when license rights are held in license files.

If a license is to be used on a computer that is intermittently connected to a license server, that license can be issued as a floating license with the BORROW keyword. A BORROW license can be borrowed from a license server via a special checkout and used later to run an application on a computer that is no longer connected to the license server. License borrowing must be enabled by a software publisher before a user can borrow licenses.

With license borrowing, a software publisher issues a floating license with a FEATURE line that contains the BORROW keyword. A user specifies the expiration date a borrowed license is to be returned and runs the application while connected to the network which writes borrowing information on the client computer. The license server keeps the borrowed license checked out. The FlexEnabled application automatically uses the local borrowing data to do checkouts during the borrow period. If enabled by the software publisher, borrowed licenses can be returned early, that is, before the borrow period expires. Upon the earlier of either the expiration of the borrow period or the early return of a borrowed license, the local borrowing data no longer authorizes checkouts and the license server returns the borrowed license to the pool of available licenses. No clock synchronization is required between the license server and the system running the FlexEnabled application.

Initiating License Borrowing

If a software publisher has enabled license borrowing by issuing a license file that contains a FEATURE line with the BORROW keyword, an user initiates license borrowing in one of three ways:

- Using the borrowing interface in application, if provided in the application
- Running the `lmborrow` utility to set `LM_BORROW`
- Setting the `LM_BORROW` environment variable directly.

Application Interface

The user initiates license borrowing this way only if the application provides a borrowing interface. Information about this is supplied by the software publisher.

Running the lmborrow Utility

`lmborrow` is one of the `lmutil/lmtools` utilities. To initiate borrowing, the user runs `lmborrow` from the command line or through `lmtools`:


```
lmborrow {vendor|all} enddate [time]
```

where *vendor* is the vendor daemon that serves the licenses to be borrowed, or *all* specifies all vendor daemons in the license server. *enddate* is the date the license is to be returned in *dd-mmm-yyyy* format. *time* is optional and is specified in 24-hour format (*hh:mm*) in the FlexEnabled application's local time. If *time* is unspecified, the checkout lasts until the end of the given end date.

For example:

```
lmborrow sampled 20-aug-2017 13:00
```

Setting the LM_BORROW Environment Variable Directly

The `lmborrow` utility is a user interface to set `LM_BORROW` in either the registry (Windows) or in `$HOME/.flexlmborrow` (UNIX). `LM_BORROW` can also be set directly as an environment variable:

```
today:{vendor|all}:enddate[:time]
```

where:

Table 7-1 ■ LM_BORROW Environment Variable Arguments

Argument	Description
<i>today</i>	Today's date in <i>dd-mmm-yyyy</i> format. Any checkouts done on this date create local borrow information. If a checkout is done on a different date than this date, no local borrowing information is created.
<i>vendor</i>	Vendor daemon that serves the licenses to be borrowed, or <i>all</i> specifies all vendor daemons in the license server.
<i>enddate</i>	Date the license is to be returned in <i>dd-mmm-yyyy</i> format.
<i>time</i>	Optional. <i>time</i> is specified in 24-hour format (<i>hh:mm</i>) in the FlexEnabled application's local time. If <i>time</i> is unspecified, the checkout lasts until the end of the given end date.

For example:

```
LM_BORROW=15-aug-2017:sampled:20-aug-2017:13:00
```

In this example, one or more licenses served by the `sampled` vendor daemon are borrowed on August 15, 2017, and are scheduled to be returned at 1 P.M. on August 20, 2017.

Borrowing a License

To borrow a license for a desired feature, *on the same day and the same system* that the user runs `lmborrow` or sets `LM_BORROW` (and while still connected to the network), the user runs the application to check out and borrow the license. If the user runs the application more than once that day, no duplicate license is borrowed. No license is borrowed if the application is run on a day different than the date borrowing was set to be initiated.

For example, say that today you want to borrow a license for the PageWizard feature for a week. The PageWizard feature is served by the `sampld` vendor daemon. Today, while you are connected to the network, run `lmborrow` or set `LM_BORROW` directly. For example:

```
lmborrow sampled enddate
```

Today, after you run `lmborrow`, while you are connected to the network, run the application that checks out a license for the PageWizard feature. After the license is checked out, close the application and disconnect your system from the network. The license that you just checked out stays checked out from the license server until the borrow period expires—that license now is used on your disconnected system until the borrow period expires. Once checked out, it remains checked out for the full borrow period. The borrow period cannot be renewed until the period has expired.

Clearing the Borrow Period

Once you have borrowed all the licenses that you need for the current borrow period (defined by the `LM_BORROW` environment variable), prevent licenses for any additional features from being borrowed by running `lmborrow -clear`. This clears the `LM_BORROW` setting in the registry (Windows) or `$HOME/.flexlmborrow` (UNIX). `lmborrow -clear` does *not* clear the local information about licenses you have already borrowed.

Checking Borrow Status



Task **To print information about borrowed features:**

1. Issue the following command on the system from which they are borrowed:

```
lmborrow -status
```

The system that borrowed the features does not have to be connected to the network to determine the status.

Returning a Borrowed License Early



Task **To return a borrowed license before the borrow period expires:**

1. Reconnect the borrowing system back to the network.
2. From the same system that initiated the borrowing, issue the command:

```
lmborrow -return [-c licfile] [-d display_name] [-u username] [-h hostname] [-fqdn] [-vendor name]  
feature [-bv version]
```

This option may or may not be allowed by your software publisher. Check directly with your software publisher to determine if they support borrowed licenses being returned early.

The option `-bv[version]` is used to return a particular version of a feature.

Support for License Borrowing

See the following sections for more information about the utilities and keywords in the options file that support license borrowing:

- `lmborrow` utility
- `lmdown` utility
- `lmstat` utility
- `BORROW_LOWWATER` keyword
- `EXCLUDE_BORROW` keyword
- `INCLUDE_BORROW` keyword

Node-locked to a User Name

This method of implementing mobile licensing is used only when license rights are held in license files.

If a license is to be used exclusively by one user on different systems, that license can be node-locked to the user's user name. The license file is copied to the different systems on which the user might work; the user's user name must be identical on each system. For this method to be useful, individual user names in an organization must be unique. Note that a user name, when used in a license file in this way, cannot contain spaces.

Fulfilled from a Prepaid License Pool

In this method, the user buys a prepaid number of license-days from the software publisher. The user can then fulfill a license using a partial amount of the total license-days for the given borrow period, node-locked to a particular system. For example, in preparation for a business trip (or even during a business trip), the user fulfills a license that expires in five days that is node-locked to their laptop. Each fulfillment can be node-locked to a different system (or even multiple times to the same system), thus allowing mobility of license usage within the pre-paid number of license-days.

This model is like pay-per-use because each fulfillment is made from a decreasing number of license-days. It is different than other pay-per-use models because, once node-locked to a system, that system is allowed unlimited use of the application until the license expires. This short-term license cannot be returned early; once fulfilled, those license-days cannot be refunded. Other pay-per-use models charge based on the number of times the application is used.

License Transferring Using the Transferrable Counted Model—Windows Only

The Transferrable Counted Model (TCM), a mobile licensing approach, introduced to transfer licenses from a parent licensing server to a remote licensing machine.

A license can be issued as a transferrable license using the `BORROW` keyword in a `FEATURE` line of a license file, if it is intended to be served on a machine that is not connected to a license server, such a license may be distributed to a remote machine via. a parent license server. A transferred license may subsequently be served on a machine which is not connected to the parent license server.

When transferring licenses, a user initializes the transfer and specifies an expiration date for returning the transferred license to the floating licensing pool. On the Windows platforms, a transferred license is cached in the system registry. On the Linux platforms, a transferred license is cached in a local secured data file. The system can now be disconnected from network after transfer. The parent license server keeps the transferred license checked out via the linger mechanism.

Running a local license server on a remote machine enables to serve the cached transferred licenses and also counted on the server. These licenses are checked out by the FlexEnabled application.

If the vendor daemon in primary machine is enabled via the `ls_borrow_return_early` setting in the machine/`lsvendor.c`, transferred licenses can be returned early i.e. before the transfer period expired.

The licensing server returns the transferred license to the pool of accessible licenses when one of the following occurs: the transfer period expired, the license is returned early, or the transfer cache no longer authorizing licenses. No clock synchronization is required between the parent license server and remote machine.



Note ▪ While checking out a transferred license from the transfer cache through a local license server, the requested license should be in the same case as that of the license transferred from the license server. Usage from the transfer cache through a local license server is case-sensitive.

Initiating License Transferring

If a software publisher has enabled license transferring by issuing a license file that contains a FEATURE line with the BORROW keyword, an user initiates license transferring in one of these ways:

- Using the transferring interface in application, if provided in the application.
- Running the `lmtransfer` utility.
- Setting the `LM_BORROW` environment variable directly.

Application Interface

The user initiates license transferring this way only if the application provides a transferring interface. Information about this is supplied by the software publisher.

Running the lmtransfer Utility

`lmtransfer` is one of the `lmutil/lmtools` utilities. To initiate transferring, the user runs `lmtransfer` from the command line or through `lmtools`:

```
lmtransfer vendorname -set dd-mm-yyyy [hh:mm]
```

where `vendorname` is the vendor daemon that serves the licenses to be transferred. `dd-mm-yyyy` is the Date of license to be expired. `hh:mm` is the time which is optional and specified in 24-hour format in the FlexEnabled application's local time. If time is unspecified, the checkout lasts until the end of the given end date.

For example:

```
lmtransfer sampled -set 20-aug-2023 [14:00]
```

Setting LM_BORROW Environment Variable Directly for Transferring

The `lmtransfer` utility is a user interface to set `LM_BORROW` in either the registry (Windows) or in `$HOME/.flexlmtransfer` (UNIX). `LM_BORROW` can also be set directly as an environment variable:

```
today:{vendor|all}:enddate[:time]
```

where:

Table 7-2 • `LM_BORROW` Environment Variable Arguments

Argument	Description
<i>today</i>	Today's date in <i>dd-mmm-yyyy</i> format. Any checkouts done on this date create local transfer information. If a checkout is done on a different date than this date, no local transferring information is created.
<i>vendor</i>	Vendor daemon that serves the licenses to be transferred, or <i>all</i> specifies all vendor daemons in the license server.
<i>enddate</i>	Date the license is to be returned in <i>dd-mmm-yyyy</i> format.
<i>time</i>	Optional. <i>time</i> is specified in 24-hour format (<i>hh:mm</i>) in the FlexEnabled application's local time. If <i>time</i> is unspecified, the checkout lasts until the end of the given end date.

For example:

```
LM_BORROW=22-dec-2023:sampled:26-dec-2023:14:00
```

In this example, one or more licenses served by the `sampled` vendor daemon are transferred on December 22, 2023, and are scheduled to be returned at 2 P.M. on December 26, 2023.



Note • The `LM_BORROW` environment variable can be used to set expiry for both the transfer and borrow licenses.

Transferring a License

To transfer a license for a desired feature, *on the same day and the same system* that the user runs the `lmtransfer` while still connected to the network, the user runs the application to check out and transfer the license. User can run the application more than once to do incremental license transfer. No licenses are transferred if the application is run on a day different than the transferring date was set to be initiated.

For example, say that today you want to transfer a license for the PageWizard feature for a week. The PageWizard feature is served by the `sampled` vendor daemon. Today, while you are connected to the network, run the `lmtransfer` directly. For example:

```
lmtransfer sampled -set dd-mm-yyyy
```

Today, after you run `lmtransfer`, while you are connected to the network, run the application that checks out a license for the PageWizard feature. After the license is checked out, close the application and disconnect your system from the network. The license that you just checked out stays checked out from the license server until the transfer period expires—that license now is used on your disconnected system until the transfer period expires. Once checked out, it remains checked out for the full transfer period. The transfer period cannot be renewed until the period has expired.

Clearing the Transfer Period

Once you have transferred all the licenses that you need for the current transfer period, prevent licenses for any additional features from being transferred by running the `lmtransfer vendorname -clear` command. This does *not* clear the local information about licenses you have already transferred.

Checking Transfer Status



Task *To print information about transferred features:*

1. Issue the following command on the system from which they are transferred:

```
lmtransfer vendorname -status
```

The system that transferred the features need not to be connected to the network to determine the status.

Returning a Transferred License Early



Task *To return a transferred license before the transfer period expires:*

1. Reconnect the transferring system back to the network.
2. From the same system that initiated the transferring, issue the command:

```
lmtransfer -return [-vendor name] feature
```

This option may or may not be allowed by your software publisher. Check directly with your software publisher to determine if they support transferred licenses being returned early. Here, `feature` is used to specify the name of the transferred feature to be returned early.

Support for License Transferring

Refer the [lmtransfer](#) section for more information that support license transferring.

Limitation

The Transferrable Counted Model (TCM) is not supported for the following list:

- Package and Suite
- UPGRADE keyword in options file
- Multiple vendors
- Transferring multiple features at a time
- Imadmin
- Imgrd started in service mode
- Java and C# clients

The Transferrable Counted Model (TCM) will be supported for each of the above lists in future releases.

Selecting a License Server Machine

When selecting a machine on which to install a license server, select a stable system; do not choose systems that are frequently rebooted or shut down. Normally, it is not required that each system be the same architecture or operating system as other license servers or the client machines on which the FlexEnabled applications are running.

The following sections discuss the resources used by the license server. When you select a machine on which to install a license server, you may need to consider whether it has sufficient resources. For small numbers of licenses (under about 100), most of these system limits are not a problem on any workstation.

License Server Sockets

When using TCP/IP ports, each FlexEnabled application connected to a license server uses one or more sockets. Depending on how the publisher implemented licensing, the FlexEnabled application may need one or more sockets. Ask the publisher for this information. The per-process system limit for file descriptors determines the number of sockets available to the license server. The total number of sockets that the license server uses is slightly larger than the total number needed by the FlexEnabled applications that connect to it.

If the number of sockets required by the license server on a single system becomes excessive, then one solution is to run multiple license servers and split the licenses between them. This reduces the networking traffic to each license server. See [Redundancy Using the License Search Path](#) for instructions and information about this configuration. Your publisher will need to agree to issue new license files, if you want to move licenses from an existing license server. If the licenses are held in trusted storage, the publisher may provide an automated process for returning them and activating them on another license server.

License Server CPU Time

For small numbers of clients, the license servers use very little CPU time. The servers might have consumed only a few seconds of CPU time after many days.

For a large number of clients (where each are exchanging heartbeat messages with the license server), or for high checkout and checkin activity levels (hundreds per second), the amount of CPU time consumed by the server may start to become significant; although, even here, CPU usage is normally not high. In this case, you may need to ensure that the system you select has enough CPU cycles to spare.

License Server Disk Space

The only output files created by the license servers are the debug and report log files. FlexNet Manager, Revenera's Web-based software license management system, uses the report log files to generate accurate usage reports. If there is a lot of license activity, these log files grow very large. You need to consider where to put these files and how often to rotate and archive them. You have the option to suppress log file output if disk space is at a premium.

It is recommended that the log files are local files on the server systems to avoid networking dependencies.

See Also

[Report Log File](#)

[Debug Log File](#)

License Server Memory

The license server uses little memory. The vendor daemons use approximately 2 MB each, although memory usage increases in the vendor daemon with the number of concurrent licenses, size of the options file, and the number of concurrent users. `lmadmin`, uses between 7 and 10 MB of memory during typical usage. Typically, the command-line license server manager, `lmgrd`, uses approximately 2 MB.

Network Bandwidth for License Server

FlexNet Publisher sends relatively small amounts of data across the network. Each transaction, such as a checkout or checkin of a license, generally transfers less than 1 KB of data. This means that FlexNet Publisher can be effectively run over slow networks (such as dial-up SLIP lines) for small numbers of clients.

For a large number of FlexEnabled applications (hundreds), each of which exchange heartbeat messages with the vendor daemon, the network bandwidth used may become significant. In this case, run the FlexEnabled application and server on the same local area network, and run multiple license servers if required. Users can use a license search path in the `LM_LICENSE_FILE` environment variable to have effective access to both servers. Enterprises can experience a performance issue when there is slow network communication or if FlexEnabled clients are using a dial-up link to connect to the network.

When you are using `lmadmin`, which uses HTTP, you need to consider the clients that connect to the `lmadmin` user interface. Depending on the number of clients and the frequency of the page refresh, they can impose a significant burden on network traffic.

License Server Locally Mounted Disks

It is recommended that you do not use remote mounted disks when you run the license server. In other words, it is recommended that `lmadmin` or `lmgrd`, the vendor daemons, the license file, and the debug and report log files are all on locally mounted disks. If any of these files are on a remote mounted disk, this doubles the points of failure, which could lead to a temporary loss of all of your licenses. When all files are mounted locally, the licenses are available as long as the server is running. When the files are on a different system, licenses may become unavailable if the license server or file server fails.

License Server Port

It is recommended that a specific port is designated on the license server machine to be used only by license server components. The benefits of this are that it is:

- Easy to track processes by the port that they are run on.
- Easier to configure FlexEnabled clients to access the license server.
- Easier to manage license server components in an environment where a firewall and/ or antivirus software is in use.
- Useful in preventing port conflicts and the hijacking of the port by other processes.

Configuring a Port Using `lmgrd`

Specify the license server port in the license file used to start `lmgrd`.



Task

To configure the port using `lmgrd`

1. Add the port number to the SERVER line as illustrated in the following example SERVER line:

```
SERVER pat 17003456 2837
```

where `pat` is the host name of the license server machine, `17003456` is the hostid of the license server machine and `2837` is the TCP/IP port number used by the license server.

2. Use the license file that contains the SERVER line that includes the port number to start `lmgrd`.

Configuring a Port Using `lmadmin`

Configure the license server manager port either:

- Using the `lmadmin` user interface. See the online help for information.
- Using the `lmadmin -licport` command. For information about the `-licport` argument, see [lmadmin Command-line Arguments](#).

Running the License Server in a Cloud

Operating in a public or virtual private cloud in an Amazon EC2 environment, you can run the license server on an AMI instance and then deploy instances of the FlexEnabled application as one or more license clients in the cloud, in your enterprise network, or in both, with all clients pointing to the license server in the cloud.

In a public cloud, you can run `lmhostid` directly on the AMI instance containing the license server to obtain the hostid needed to bind the license. For more information about these use cases and the required hostids, see [Licensing in a Cloud-Computing Environment](#).

License Server Manager

“ladmin”

The *license server manager* is one of the components that makes up a license server (the other being the vendor daemon). It handles the initial contact with FlexEnabled applications, passing the connection on to the appropriate vendor daemon. The purpose of the license server manager is to:

- Start and maintain vendor daemons as required for serving license rights from different software publishers.
- Refer application checkout (or other) requests to the correct vendor daemon.

There are two versions of the license server manager:

- **ladmin**—The Web-based license server manager
- **lgrd**—The original license server manager with a command-line interface

This section describes ladmin. For information about lgrd, see [License Server Manager “lgrd”](#).

ladmin provides improved methods of managing the license server and vendor daemons. A brief description of the improved capabilities follows. For a more detailed comparison of lgrd and ladmin, see [Migrating from lgrd to ladmin](#).

ladmin Capabilities

- **Direct configuration of the vendor daemons and license server manager**—License-server port number, vendor-daemon path and port, and three-server redundant port can be configured without any edits to the license files.
- **Configurable alerts**—You can set up ladmin to issue alerts to warn you of potential problems (for example, license expiry, no available licenses, or vendor daemon status).
- **License rights status display**—The ladmin user interface provides a display of all available and in-use license rights. This display can include all concurrent (floating) licenses both from license files and from trusted storage. It can also include activatable and hybrid licenses (held in trusted storage) when these are available on the license server.
- **Command-line functions accessed by option buttons**—For example, the **Stop Server** and **Reread License Files** buttons perform the same actions as the lmdown and lmreread functions, respectively. For a list of

license administration functions that are available directly from ladmin, see [ladmin License Administration Functions](#).

- **Minimal editing of license files**—Option file specification requires editing.

This release of ladmin is available for use on a limited number of platforms. For full details, contact your software publisher or see the *FlexNet Publisher Release Notes*. ladmin is compatible with licensing components from version 9.2 or later. See [Version Compatibility Between Components](#) for detailed information on how to determine what versions of the licensing components are provided in your licensed applications.

Installing ladmin

This section contains instructions for installing the ladmin license server.

System Requirements for ladmin

For information on supported platforms and Web browsers of ladmin see the latest *Release Notes*.

To use ladmin on Windows platforms, the relevant Microsoft Visual C++ Redistributable Package must be installed. For the required minimum version, refer to the *Release Notes*, section *System Requirements for ladmin*. You have an option to install this package during the ladmin installer process.

The ladmin installer requires that JRE 11 or later (for OS X: JRE 11 or later) be installed. If the JRE is not already present on the machine, it must be installed separately, because it is not bundled with the ladmin installer.

The JDK 17.0.8, JDK 11.0.20, and OpenJDK 20.0.2 are tested Java Standard Edition systems for installing the ladmin.

Do not run the 32-bit ladmin and the 64-bit ladmin on the same system. If you are upgrading from the 32-bit ladmin to the 64-bit ladmin, you must stop and deinstall the 32-bit ladmin first.



Note • If you want to use a different JRE version while installing/uninstalling you can use the LAX_VM command, as follows:

```
C:\Users\fnpauto\Desktop>ladmin-i86_n3-11_14_1_1.exe LAX_VM "C:\Program Files\Java\jre1.8.0_112\bin\java.exe"
```

Using the License Server Installer

This section describes how to install the license server for the first time. If you have an existing installation of the ladmin license server, see [Upgrading ladmin](#) on page 83 for instructions.

When running the ladmin installer, accept the default settings whenever possible. If you are given the option to modify installation settings, keep the following information in mind.

- **Choose Install Folder window**—Do not install the ladmin license server in the same folder as an existing FlexNet Publisher installation. Choose a new or an empty folder as the installation directory.

Under Windows, the default path is C:\Program Files\FlexNet Publisher 64-bit License Server Manager for the 64-bit ladmin or C:\Program Files (x86)\FlexNet Publisher License Server Manager for the 32-bit ladmin.

- **Choose Data Folder window**—(Windows only) The installer creates the ladmin service of the type "Local Service", with start-up type "Automatic". If the service is started with a non-admin account, it does not have sufficient privileges to modify the contents of the Program Files (x86) directory. It is therefore strongly recommended installing the folders that are updated during run time (conf, logs, cache, licenses) under ProgramData. The default path is C:\ProgramData\FLEXlm\ladmin.
- **Import Files from Previous Installation window**—(Windows only) If you want to import license files, log files, configuration data, and vendor daemons from a previous installation of the ladmin installer, provide the previous installation paths. For more information, see [Upgrading ladmin](#) on page 83.
- **lmreporter tool**—(Windows and Linux Only) The lmreporter tool is an optional user interface that allows you to view feature usage, peaks, and denials. This functionality must be explicitly enabled by a vendor. User can check the checkbox to enable the lmreporter tool. The lmreporter tool is supported for the ladmin installers only. For more information on usage of the lmreporter tool, see the [lmreporter](#).
- **Service Configuration window**—While it is possible to manually start and stop the ladmin license server manager, it is recommended that you install it as an operating system service so that it will automatically start whenever the operating system restarts. The installer will set up the service for you on Windows. For UNIX or Mac, see [Installing ladmin License Server Manager as an Operating System Service](#) for more details.
- **Windows Active Directory domain user or group prompt**—During installation on a Windows machine, the installer might prompt you to pre-designate an Active Directory domain user or group name to be used to sign into the ladmin user interface. (Providing this information is optional.) If you specify a domain user name, administrators can sign in using this user name and its associated password as defined in Active Directory. If you specify a domain group name, administrators can sign in using the name and password of any domain user belonging to the specified group. (Designating a domain group is helpful in a large enterprise where you might have several license administrators who need access to the interface.) Once ladmin is installed, you can use its interface to set up additional domain users or groups for sign-in.



Note ■ The ladmin installer of certain software publishers might provide this option to pre-designate an Active Directory user or group for sign-in.

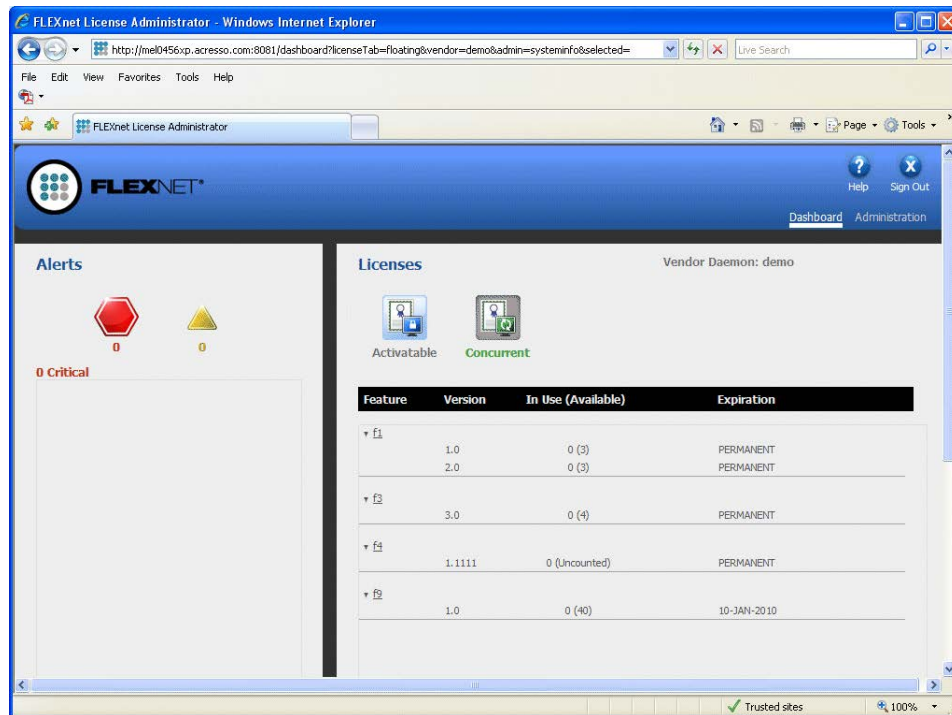


Figure 9-1: ladmin UI

After installing ladmin with the default configuration, you can configure the location where it stores the license files it uses. See [Configuring the License File Upload Directory](#) for details.

- **LDAP user support by ladmin on Linux**—Linux platforms support LDAP users. License administrators can create an LDAP user account as an ladmin domain-administrator-type or domain-user-type user.

The License Administrator can then log in to the ladmin user interface using the just-added LDAP user ID (and its associated password). To identify this account, use the format *domain\username*, where *domain* is a valid LDAP domain to where the machine running ladmin can connect, and *username* identifies a valid account within that domain. (Also, ladmin should have been already configured to connect to that LDAP server). This value specified for account is not case-sensitive and can include up to 64 characters. Using the ladmin user interface, you can change this account's role to domain-administrator or domain-user or remove the account from the ladmin user-interface user list. However, you cannot manage the account as an LDAP user.



Task To configure LDAP for ladmin on Linux:

1. Configure the LDAP server using the following command:

```
./ladmin -ldapHost <ActiveDirectoryServerHost> -ldapPort <ActiveDirectoryServerHost> -ldapUser
<domain\user> -ldapPassword <ldapUserPassword> -ldapBaseDN <ActiveDirectoryServerHostBaseDN> -noWeb
-nolic
```

The above command is used to configure ladmin running on the Linux server to connect and authenticate to an LDAP server.

Table 9-1 ■ Parameters used by ladmin on Linux to connect to LDAP

Option	Description
ldapHost	Specify the server host name or IP address of the LDAP server
ldapPort	Specify the server port of the LDAP
ldapUser	Specify the domain user name
ldapPassword	Password of the specified domain user.
ldapBaseDN	Base Distinguished name to be used while searching for users on the LDAP server. For example: If the Active Directory server is testads.revenera.com, the BaseDN value would usually be DC=TESTADS,DC=REVENERA,DC=COM

The “-ldapUser” and “-ldapPassword” options should be used to specify a user-account and password combination that can be used to bind to the AD server. This account/password info would be used for binding while checking for the existence of a particular domain user before adding it to the configuration file.

If 'ladmin' is installed through the 'ladmin' installer on a non-English version of OS, then the permission to 'server.xml' needs to be manually provided through the below command:

```
icacls server.xml /grant <'Users' equivalent on non-English version of OS>:M
```

2. Check the configuration file (conf\server.xml) and start the ladmin server.
3. Log on to the ladmin web GUI as an administrator. The ladmin dashboard is displayed.
4. To add domain users, select the **Administration** tab, open the **User Configuration** tab and click **New**. The **Create User** page is displayed.
5. For **Role**, select **Domain Administrator**.
6. Enter the user name in the format *domain\user*. For example: *DOMAIN1\USER1*.
7. The new domain users can now log on to the **Administration** pages using the domain account and password.

Running the Installer in Console Mode

On AIX, Linux, Solaris, or Windows platforms, you can run the ladmin installer in console mode. This mode enables you to install ladmin from the command line, providing an alternative to the user interface-based method.

Use the following command to run the installer in console mode:

```
<ladmin_installer> -i console
```

where <ladmin_installer> is the name of the ladmin installation program.

The console installation process prompts you for information needed to complete the installation.



Note ■ When uninstalling ladmin, it is recommended that the same mode used for installation is also used for uninstallation. Otherwise, you might not be prompted to save the logs and server.xml file or to stop and uninstall the ladmin service (the ladmin service is only available on Windows platforms).

License Server Directory Structure

After installing the ladmin license server, you see that files have been installed in the following directories:

- **Program Files\FlexNet Publisher 64-bit License Server Manager** (64-bit ladmin) or **Program Files (x86)\FlexNet Publisher License Server Manager** (32-bit ladmin)—(Windows only) The installation folder that you specified on the panel **Choose Install Folder** when installing ladmin. This directory contains folders and files that are not modified during run time; as well as ladmin.exe, the license server executable. This directory is often referred to as the installation root directory (<ladmin_install_dir>).
- **ProgramData\FLEXlm\ladmin**—(Windows only) The installation folder that you specified on the panel **Choose Data Folder** when installing ladmin. This directory contains folders with files that are updated during run time. This directory is also referred to as the runtime data directory (<ladmin_runtimedata_dir>).
- **...\FNPLicenseServerManager**—(non-Windows platforms) The installation folder that you specified when installing ladmin; often referred to in this documentation as the installation root directory (<ladmin_install_dir>). Configuration paths are usually specified relative to this directory location. This directory contains ladmin, the license server executable.

Do *not* edit the contents of any file or directory except where explicitly instructed to by this *License Administration Guide*, or by other supplied licensing documentation.

Table 9-2 ▪ Directories used by the License Server Manager

Directory	Description of Contents
\demo	Contains sample license files and the <i>demo</i> vendor daemon. (On Windows systems, this folder is located in the installation root directory.)
\eventlog	Exists on Window systems only and includes the files needed to allow the license server to record messages to the Windows event log. (On Windows systems, this folder is located in the installation root directory.)
\examples	Contains code samples that show how to build capabilities using the Web services. (On Windows systems, this folder is located in the installation root directory.)
\uninstall	Contains the files required to uninstall ladmin. (On Windows systems, this folder is located in the installation root directory.)
\web	Contains the license server management interface. (On Windows systems, this folder is located in the installation root directory.)
\wsdl	Contains the WSDL file that you can use to generate a client proxy for the Web services. (On Windows systems, this folder is located in the installation root directory.)

Table 9-2 ▪ Directories used by the License Server Manager

Directory	Description of Contents
\cache	System directory that is created after you start any vendor daemon using the license server management interface. (On Windows systems, this folder is located in the runtime data directory.)
\conf	Contains server.xml and other system files that define the license server configuration. (On Windows systems, this folder is located in the runtime data directory.)
\licenses	Contains the license files served by ladmin. (On Windows systems, this folder is located in the runtime data directory.)
\logs	Contains the application log files. This directory is created after the license server is started for the first time. (On Windows systems, this folder is located in the runtime data directory.)

Upgrading ladmin

Before installing a new version of ladmin:

- If you have configured ladmin as a system service, shut down the service.
- Shut down any ladmin processes running on the system.
- Do not run the 32-bit ladmin and the 64-bit ladmin on the same system. If you are upgrading from the 32-bit ladmin to the 64-bit ladmin, you must stop and uninstall the 32-bit ladmin first.

The ladmin installer provides the option **Import Files from Previous Installation**. This option enables you to upgrade ladmin while retaining a previous ladmin configuration.

The following files and folders are imported from an existing ladmin installation:

- **ladmin configuration data**—Permanent ladmin settings that were configured either via the license server management interface or via the command-line. For example, these settings include the license server port number, vendor daemon path and port, three-server redundant port, and the maximum size of a file import. These and other configuration data are held in the file server.xml which is imported to the conf directory.
- **Vendor daemon files and license files**—The vendor daemon executable, the license file used to start the vendor daemon, and additional license files imported via the license server management interface after the initial import of a vendor daemon. A copy of the directory structure is created and these files are imported.
- **Log files**—The log file (<vendor>.log) for each vendor daemon.

This section describes the upgrade procedure when you are installing the latest version of ladmin and want to import files from a previous ladmin installation. To determine which version of ladmin you are using, see the **Release Version** value displayed on the **System Information** page.



Task

To upgrade an existing lmadmin installation:

1. Run the lmadmin installer.
2. In **Choose Install Folder**, set the installation root directory. Ensure that the installation root directory is not a sub-directory of the existing installation. (On Windows, the default path is C:\Program Files\FlexNet Publisher 64-bit License Server Manager for the 64-bit lmadmin or C:\Program Files (x86)\FlexNet Publisher License Server Manager for the 32-bit lmadmin.)
3. (Windows only) In **Choose Data Folder**, set the runtime data directory. The default path is C:\ProgramData\FLEXlm\lmadmin.
4. In **Import Files from Previous Installation**, you can select to import data from a previous installation. To do so, select the **Import** check box and provide the path to the installation root directory of the relevant lmadmin installation.

Provide the following information:

- **Previous Installation Path**—Specify the path to the installation root directory of the lmadmin installation that you want to import.
- **Previous Data Folder Path**—(Windows only) Specify the path to the runtime data folder of the lmadmin installation that you want to import. This enables you to import data files from a previous installation.



Note - If you have uninstalled the previous lmadmin installation and saved the configuration files in a separate directory (for example, C:\temp\<previous_installer_data>\conf\server.xml and C:\temp\<previous_installer_data>\logs\lmadmin.log), specify the path to the directory that contains the conf and logs folders (C:\temp\previous_installer_data).

If your previous installation did not distinguish between an installation root directory and runtime data directory, specify the path to the installation root directory in both fields.

5. If you have configured the license server manager port, in **Launch Configuration** enter the license server port number. (See [License Server Manager Not Starting](#).)
6. If you have configured the TCP/IP port that the Web server uses to listen for communication with clients connecting to the license server management interface, in **Launch Configuration** enter the HTTP port number. (See [License Server Manager Not Starting](#).)
7. Complete the remaining installation dialogs.



Note - Note the following when upgrading lmadmin:

- The lmadmin installer imports an existing vendor daemon and its associated files only when the license file used to import the vendor daemon contains the license file path as a relative path on its VENDOR line.
- Any existing demo vendor daemon and associated license files and log files are not imported. The installer always installs an up-to-date version of the demo vendor daemon and the files required to run it.

Upgrading OpenSSL Libraries

To enable customers to quickly upgrade to a higher version of OpenSSL libraries, lmadmin is dynamically linked against the OpenSSL libraries. Only the following type of OpenSSL upgrades is supported by Revenera: Upgrade from X.Y.Za to X.Y.Zb—Low risk (e.g. 1.0.2h to 1.0.2k version of OpenSSL)

More significant upgrades are unsupported due to possible API incompatibility issues; for example, an upgrade from 1.0.2 to 1.0.3 or 1.1.0 is unsupported.



Task *To upgrade to a higher version of OpenSSL libraries on Linux and Solaris platforms:*

After installing lmadmin and before running lmadmin, add the path to the replacement OpenSSL dynamic libraries to the environment variable LD_LIBRARY_PATH.

Example: For Linux and Solaris platforms, use the LD_LIBRARY_PATH environment variable:
export LD_LIBRARY_PATH=\$LD_LIBRARY_PATH:<OpenSSL dynamic libraries path>



Task *To upgrade to a higher version of OpenSSL libraries on OS X platforms:*

After installing lmadmin and before running lmadmin for the first time, change the rpath to point to the location of the replacement OpenSSL dynamic libraries. Use a command similar to the following:

```
install_name_tool -add_rpath /usr/local/lib <lmadmin_install_dir>/lmadmin
```



Note ▪ The actual rpath entry (/usr/Local/Lib in the example above) may depend on the way OpenSSL libraries have been built.



Task *To upgrade to a higher version of OpenSSL libraries on Windows platforms:*

After installing lmadmin and before running lmadmin for the first time, replace the OpenSSL dynamic libraries in the lmadmin install directory with the updated versions.



Note ▪ The lmadmin provided for AIX platforms does not support OpenSSL decoupling. Therefore, the AIX lmadmin is linked against the static libraries of OpenSSL.

Uninstalling lmadmin

The lmadmin uninstaller removes all data folders (conf, cache, logs, and conf) and all executables, DLLs and license files. When you execute the uninstaller, you can choose to save log files and the server.xml file.

When uninstalling lmadmin, it is recommended that the same mode used for installation is also used for uninstallation. Otherwise, you might not be prompted to save the log and server.xml files or the lmadmin service.



Task

To uninstall lmadmin using a user-interface-based process:

1. In folder <lmadmin_install_dir>\uninstall (on Windows, the default is C:\Program Files\FlexNet Publisher 64-bit License Server Manager\uninstall for the 64-bit lmadmin or C:\Program Files (x86)\FlexNet Publisher License Server Manager\uninstall for the 32-bit lmadmin), execute Uninstall FlexNet Publisher License Server Manager to start the uninstaller.
2. In the panel **Get User Input for saving log and server.xml files**, select the check box **Yes save the files** if you want to save log files and server.xml files. Click **Next**. If you selected to save log files, on the next panel, specify a path. Click **Next**.
3. In the panel **Get User Input**, select **Yes** if you want to stop and remove the service.
4. Click **Uninstall**.



Task

To uninstall lmadmin in console mode:

At a command prompt from the folder <lmadmin_install_dir>\uninstall (on Windows, the default is C:\Program Files\FlexNet Publisher 64-bit License Server Manager\uninstall for the 64-bit lmadmin or C:\Program Files (x86)\FlexNet Publisher License Server Manager\uninstall for the 32-bit lmadmin), enter the following command to run the uninstaller in console mode:

```
<lmadmin_uninstaller> -i console
```

where <lmadmin_uninstaller> is the name of the lmadmin uninstallation program.

The console uninstallation process prompts you for information needed to complete the uninstallation.

Using lmadmin

Specifying the “conf” Folder

In a default lmadmin installation on a Windows platform, the conf folder is installed in the runtime data directory under ProgramData (that is, not in the same location where lmadmin.exe is installed). Therefore, for all operations that involve updates to files located in the conf folder (for example, server.xml), you need to specify the location of conf using the -configDir command:

```
lmadmin.exe -configDir <conf_path>
```

where <conf_path> is the path to the conf folder (for example, C:\ProgramData\FLEXlm\lmadmin\conf).

For command-line examples listed within this chapter, it is assumed that the conf folder is not installed in the same location as lmadmin.exe.

In installations on non-Windows platforms, and in Windows installations where the conf folder is installed in the same location as lmadmin.exe, using the -configDir command is not required.

Manually Starting the License Server Manager

You can start the license server using one of the following methods:

- Windows platforms: If the conf folder has been installed in the same location as lmadmin.exe, open the installation directory in Windows Explorer and double-click the lmadmin.exe file. This mechanism does not allow you to specify non-default command-line arguments.
- Execute the lmadmin command from the root installation directory. To see a list of available command-line arguments, execute the command:

```
lmadmin -help
```

The help display identifies the default arguments and which arguments are *persistent*, options that will remain in effect for later instances of lmadmin.

- Create a shell script file (UNIX) or a batch file (Windows) that will run the lmadmin command with your desired command-line arguments and then execute that file. For a default installation on a Windows platform, specify the location of the conf folder.



Note ▪ If either the default license server port or the HTTP port for the user interface is in use, the license server manager will not start. For instructions see [License Server Manager Not Starting](#).



Important ▪ In addition, ensure that the http/https ports and the licensing ports (required by lmadmin) and the vendor daemon ports are opened on the firewall, so that remote clients can checkout licenses and the HTTP clients/browsers can connect to the lmadmin web GUI.

License Server Manager Not Starting

Incorrect Port

The license server manager will not start if either of the following ports are in use:

- Default license server port (no ports in range 27000 to 27009 available)
- Default HTTP port for the license server manager user interface (port 8090)



Task

To check for this error and correct it:

1. Run lmadmin from the command line using the -foreground argument:

```
lmadmin -configDir <conf_path> -foreground
```

2. Examine the output at the command prompt. The following shows typical output when there is a clash on the HTTP port:

- Non-Windows systems:

```
<OS 10048>Only one usage of each socket address <protocol/network address/port> is normally
permitted. : make_sock: could not bind to address 0.0.0.0:8090
no listening sockets available, shutting down
Unable to open logs
```

- Windows systems:

```
<OS 10048>Only one usage of each socket address <protocol/network address/port> is normally permitted. : make_sock: could not bind to address 0.0.0.0:8090
```

Note that on Windows, the error message is displayed despite the fact that it is possible to run two instances of lmadmin on the same port (using an IPv4 and an IPv6 socket). In this case, no remedial action is required.

However, if a third instance of lmadmin is started, the following error message is displayed:

```
<OS 10048>Only one usage of each socket address <protocol/network address/port> is normally permitted. : make_sock: could not bind to address 0.0.0.0:8090
no listening sockets available, shutting down
Unable to open logs
```

In this case, proceed with [Step 3](#).

3. Reconfigure any port where there is a clash:

- Use the `-licPort` argument for the license server port.
- Use the `-webPort` argument for the HTTP port.

For example, the following command reconfigures the HTTP port to 8091:

```
lmadmin -configDir <conf_path> -webPort 8091
```

When you have reconfigured the HTTP port, you access the license server management interface using the new port number. For example, `lmadmin -configDir <conf_path> -webPort 8091` connects to the URL `http://<server>:8091`.

Insufficient Privileges (Windows only)

If the lmadmin service does not start due to an “Access denied” error, ensure that it has sufficient privileges.

The lmadmin folders that are updated during run time should be installed under ProgramData, to allow the service, which has Local Service privileges, to modify these files.

Manually Stopping the License Server Manager

The `allowStopServer` command-line argument toggles the presence of the **Stop Server** button in the lmadmin user interface. The default is the **Stop Server** button is present. Click the **Stop Server** button in the Administration section of the license server management interface to shut down the license server manager (lmadmin) and all vendor daemons.

If lmadmin is started with the command-line argument `-allowStopServer no`, or if `-allowStopServer no` was the most recent use of the `-allowStopServer` argument, you cannot stop the license server using the license server management interface. In this situation, to stop the license server you must stop the lmadmin process.

On UNIX systems, you can use the `ps` utility to identify the process and the `kill` command to terminate it.



Caution ▪ Do not use `kill -9`, use only `kill` with its default signal; otherwise, the license server will not shut down cleanly.

On Windows systems, you can use the Task Manager to identify the ladmin.exe process and stop it.

You cannot restart the license server from the management interface. You must restart the license server as described in the previous section.

Accessing the License Server Management Interface

The license server management interface has two modes of operation—Standard mode and Section 508 mode. Either mode is accessible from a supported Web browser. See [System Requirements for ladmin](#) for a list of supported Web browsers.

1. Make sure that you have started the license server.
2. Open the Web browser and browse to one of the following URLs:
 - **Standard mode**—This is the standard license server management interface:
`http://<server>:8090`
 where <server> is the system name where the license server is running.
 - **Section 508 mode**—Section 508 mode provides access to people with disabilities and has the same capabilities that are available in Standard mode:
`http://<server>:8090/login508`
 where <server> is the system name where the license server is running. (The term "Section 508 mode" comes from Section 508, 36 CFR 1194.21, "Requirements for Software Applications and Operating Systems" of the 1998 amendment to the Federal Rehabilitation Act.)

Signing in to ladmin as an Administrator

To use the following pages of the license server management interface, you must sign in as an administrator:

- System Information
- User Configuration
- Alert Configuration
- Server Configuration
- Vendor Daemon Configuration

When ladmin is first installed, the administrator user name and password are both set to **admin**. Use this information when you first sign in to the interface as an administrator.



Note ■ If ladmin is running on a Windows machine, you can also sign in to the interface using the Active Directory user ID specified as an ladmin Administrator (either directly or via the specified Active Directory group) during ladmin installation. However, this type of sign-in is available only if your ladmin installation prompted for an Active Directory domain user or group name and if this information was provided.

Viewing the lmadmin Log Files

Application log files (except the report.log file) are written to the <lmadmin_install_dir>/logs directory (non-Windows platforms) or <lmadmin_runtimedata_dir>\logs (default installations on Windows platforms).

Table 9-3 • Log files

Log File Name	Description
access.log	Contains information recorded about access to the license server management interface.
lmadmin.log	Contains information recorded by the license server.
web.log	Contains information recorded by the license server management interface. This file does not contain information about login events. See the access.log file for that information.
<vendor>.log	These files contain information recorded by the corresponding vendor daemons (where <vendor> is the vendor daemon name). Each vendor daemon has its own log file, called the debug log file. In the installation package, you should see the demo.log file as the debug log file for the default demo vendor daemon.
report.log	A sample report log for the demo vendor daemon. Each vendor daemon can maintain a separate report log to record information about features that have been checked out by users. By default, a vendor daemon does not maintain report logs. This capability (in addition to the location of the report log file) must be enabled using the Options file.
FLEXnet_Publisher_License_Server_Manager_Install_MM_dd_yyyy_hh_mm_ss.log	A log recording events of the lmadmin installation. This file is usually created in the uninstall\Logs directory located under the installation root directory. However, the lmadmin installer might create it under a different name and location. If necessary, contact the publisher for the log file's name and location.  Note • Prior to lmadmin 11.10.1, the installation log file was created directly under the installation root directory.

Managing lmadmin from the Command Line

This section describes some of the common tasks that can be performed using the lmadmin command line and the command-line arguments.

Adding a Vendor Daemon to ladmin

The ladmin license server must be configured with data about vendor daemons and license files. To add a vendor daemon, you must import it using a license file. You can do this from the command line (or from the ladmin user interface.)



Note • FlexNet Publisher provides a sample vendor daemon (demo or demo.exe) that you can use with ladmin. To use this vendor daemon, first copy it and its associated library from the <platform_dir> directory to the <platform_dir>\ladmin\demo directory. (The associated library is one of the following: demo_libFNP.so for UNIX, demo_libFNP.dll for Windows, demo_server_libFNP_notr.so for AIX, or demo_libFNP.dylib for Mac.) Then, issuing the command described in the following procedure, import the vendor daemon, using the sample license file ladmin\demo\demo.lic.



Task

To add a vendor daemon from the command line:

1. Create or locate a valid license file (for example, mylicense.lic) with the appropriate SERVER lines, VENDOR lines, and feature definition lines.
2. Make sure that the vendor daemon executable is in the correct location relative to ladmin. This location is defined in the VENDOR line.
3. Import the license file by executing the following command:

```
ladmin -configDir <conf_path> -import <mylicense.lic>
```

where <myLicense.Lic> is the path and name of the license file, and <conf_path> is the path to the conf folder. This command imports the license file, but does not start the license server.

When you import a license file, the license server configuration file (Windows:

<ladmin_runtimedata_dir>\conf\server.xml; non-Windows: <ladmin_install_dir>/conf/server.xml) is populated with the vendor daemon information (vendor name, vendor daemon path, port number, etc.).

4. Start the license server. See [Manually Starting the License Server Manager](#).



Note • For information about how to import a license file using the license server management interface, see the Online help available from within the license server management interface.



Note • The following behavioral modifications to ladmin as part of the RCE vulnerability fix are:

- The vendor daemon executable, which needs to be started by ladmin, must be located by default within the ladmin root directory or its child directories.
- Remote sites, including Windows network share paths, are not permitted by default.
- If a vendor daemon location, other than ones permitted by default is necessary, the License administrator can use the newly-added -vdSearchPaths command-line option of ladmin to specify absolute paths of the required vendor daemon location. This will result in the specified path(s) being whitelisted as an allowed path.

Configuring the License File Upload Directory

The license file upload directory is the location where copies of license files used by lmadmin are stored when license files are imported using the **Import License** button on the **Vendor Daemon Configuration** tab or the `-import` command-line argument.

When this location is not configured, the default settings create the following directory structure into which license files are uploaded:

```
...\<lmadmin_runtimedata_dir>\licenses\<vendor daemon name>\
```

For example, on Windows when lmadmin is installed in the default location and two vendor daemons, demo and publisherA, are being managed by lmadmin, the following directories are created when license files for these vendor daemons are imported:

```
C:\ProgramData\FLEXlm\lmadmin\licenses\demo
```

```
C:\ProgramData\FLEXlm\lmadmin\licenses\publisherA
```

The license file that contains license rights for the vendor daemon demo is copied to the `..\demo` directory. The license file that contains license rights for the vendor daemon publisherA is copied to the `..\publisherA` directory. When additional license files are imported for either of these vendor daemons, they are uploaded to the appropriate vendor daemon-specific directory.

You can replace this default configuration as described in the following instructions. Typically the license file upload directory is configured when lmadmin is installed for the first time and then not altered. This ensures that license files, once imported, are available to lmadmin and the vendor daemons it manages.



Task

To configure the license file upload directory:

1. If lmadmin is running, shut it down ([Manually Stopping the License Server Manager](#)).
2. From the command line, execute an lmadmin command using the `-uploadDir` argument.

The upload directory can be specified either as a relative or absolute path. When a relative path is used, it is relative to the current directory. Additionally a special string, `%v`, can be used to include the vendor daemon name in the directory path. Thus the following example specifies that the upload directory will be located at `C:\ProgramData\FLEXlm\lmadmin\<vendor>\licenses`:

```
lmadmin -configDir C:\ProgramData\FLEXlm\lmadmin\conf -uploadDir  
C:\ProgramData\FLEXlm\lmadmin\%v\licenses
```

Configuring lmadmin License Server Manager as a Windows Service with a Three-Server Configuration

Configure and maintain a set of three license server systems specifically for three-server redundancy. This provides fail over protection only. You manage only one version of the license file and vendor daemon on all three license servers. This configuration option is only available when licenses are held in license files. Using the three-server redundancy capability in FlexNet Publisher, all three license servers operate to form a triad.



Task

Procedure to run Imadmin as a service on a three-server configuration:

1. Go to the `i86_n3\lmadmin` directory and import the license file on each of the three servers, using the following command:

`lmadmin.exe -configDir <conf_path> -import <ThreeServerLicFile> [-force]`
2. Install the service on each of the three servers, using the following command:

`lmadmin.exe -configDir <conf_path> -installService <Servicename>`
3. From the **Services** window, start the installed service on each of the three servers. A quorum should get established.

Example

The three-server license file can have a format similar to the one shown below:

```
SERVER 172.18.24.193 00155d18e600 27000
SERVER 172.18.24.112 000c2954c827 27000
SERVER 172.18.24.184 000c29020077 27000
VENDOR demo demo/demo
USE_SERVER
FEATURE TestFeature1 qavend9 1.0 01-feb-2020 1 ISSUED=01-feb-2020 \
SIGN="005C 1651 50F5 7FAE 7035 7118 52AB 9300 18C5 E454 7F52 \
8291 F4F7 E6CF D3E5"
FEATURE TestFeature2 qavend9 1.0 30-sep-2020 1 ISSUED=01-feb-2020 \
SIGN="00B5 CD90 7A6C 5D22 FABD 22CA 221B 7B00 7483 9D07 9BCD \
57F1 317B 89B1 192C"
FEATURE TestFeature3 qavend9 1.0 30-sep-2020 3 ISSUED=01-feb-2020 \
SIGN="00EF 8163 4EBD 1038 5D04 D17B 5815 1500 ED87 4CF6 0843 \
2E1F 17DD 2691 8B5B"
```

Accessing Imadmin License Server Manager as a Windows Service with a Three-Server Configuration

The license server management interface has two modes of operation—Standard mode and Section 508 mode. Either mode is accessible from a supported Web browser. See [System Requirements for Imadmin](#) for a list of supported Web browsers.

1. Make sure that you have started the license server.
2. Open the Web browser and browse to one of the following URLs:
 - **Standard mode**—This is the standard license server management interface:

`http://<Master node>:8090`

where <master node> is the IP address or host name of the current master node.
 - **Section 508 mode**—Section 508 mode provides access to people with disabilities and has the same capabilities that are available in Standard mode:

`http://<Master node>:8090/login508`

where <Master node> is the IP address or host name of the current master node.

For more information on Three-Server Redundancy, refer to the section [Overview of Three-Server Redundancy](#) on page 218.

Below is a sample output of the lmadmin GUI for Three-Server Redundancy:

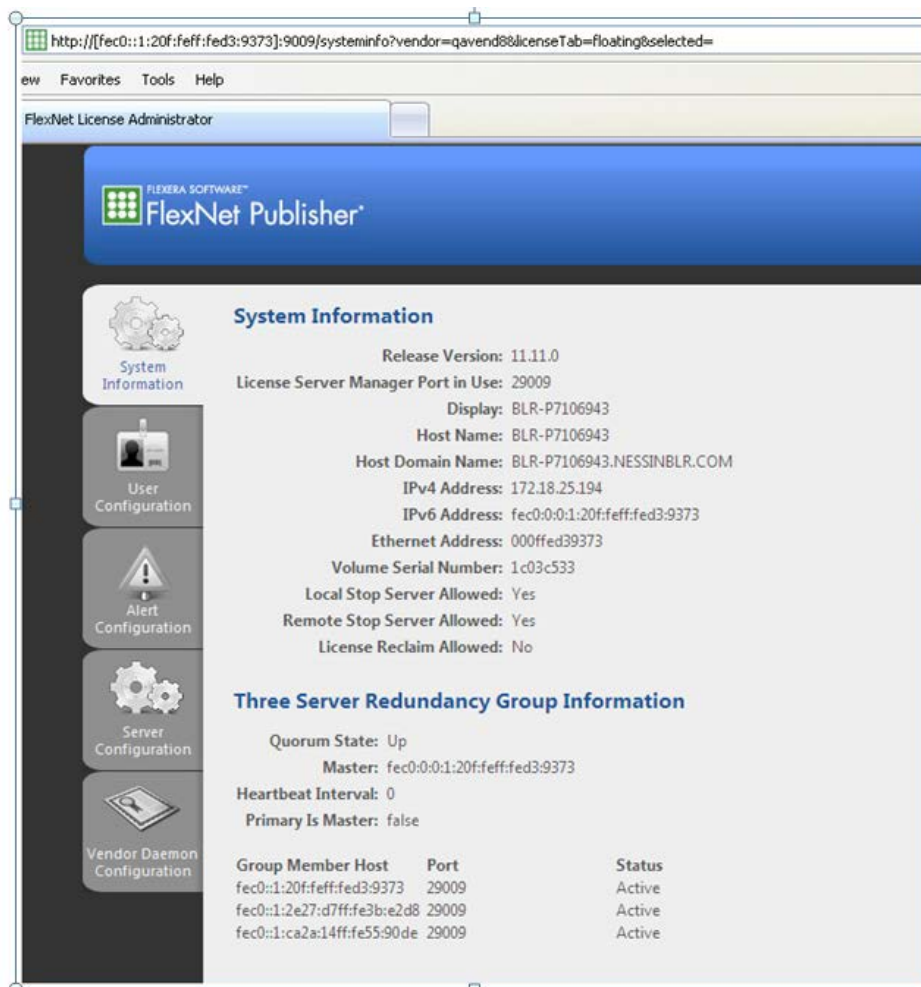


Figure 9-2: lmadmin Three-Server Redundancy

Installing lmadmin License Server Manager as an Operating System Service

While it is possible to manually start and stop the lmadmin license server manager, it is recommended that you install it as a service on the operating system so that it will automatically start whenever the operating system restarts.



Important • For security reasons, Revenera recommends that the license server be run with non-elevated user privileges (which, in turn, the vendor daemon inherits). See the later section, [Running FlexNet Publisher License Server as a System Service With Non-Elevated Privileges](#), for details on enabling the lmadmin operating-system service to run under a non-Root or non-administrator user account.

Windows Systems

As part of the lmadmin installation process on Windows systems, the lmadmin installer might provide the option to configure the lmadmin license server manager as a system service. However, only users in the Windows Administrators group can successfully perform this action. The **Startup Type** is set to **Automatic** so that the service starts automatically when the system is restarted.

As an alternative, use the following lmadmin command-line arguments to install and uninstall the service (see Table 9-4).



Important ▪ After you execute the command to install the license server manager as a Windows service, the service is not started automatically. You must manually start the service for the first time using the Windows Services Console.

Table 9-4 ▪ lmadmin command-line arguments to configure lmadmin as a Windows service.

lmadmin Command-Line Argument	Description
-installService <i>service name</i>	Creates a Windows service (with the name you provide) to run the license server manager. The service will run under the <i>LocalService</i> account.
-removeService	Uninstalls the Windows service with the name you specified. Make sure you stop the service before removing it.
-delay <i>nn</i>	Sets the number of seconds (<i>nn</i>) to delay between the time you start the service and the time it actually begins running. This delay is helpful when a FlexNet ID dongle is used to lock the license server to a machine (that is, when the FLEXID is used on the SERVER line). The license server can sometimes fail to start when a system reboots because the license server loads before the dongle device driver has a chance to load properly.



Note ▪ Use this option with the `-installService` argument.

To run lmadmin with any of the command-line arguments used to configure lmadmin as a Windows service requires that the user has administrator privilege. (Windows versions enforce administrator privileges for installation or removal of a service.) Therefore, to use these arguments you must do the following:

- Sign in as an administrator before running lmadmin with these arguments.
- Start the command prompt using the option **Run as Administrator**.

Linux Systems

For supported Linux platforms, a subdirectory of <lmadmin_install_dir>/examples/ provides the script files that contain service control instructions, and details on where each script file should be installed.

Depending on your platform, you need to install either of these script files:

- <lmadmin_install_dir>/examples/service/lmadmin—for platforms using the old BSD startup script.

- `<lmadmin_install_dir>/examples/systemd/lmadmin.service`—for use on platforms using the new systemd service startup.

(Do not confuse the `lmadmin` file with the executable for the license server manager.)

Additionally, see [Running FlexNet Publisher License Server as a System Service With Non-Elevated Privileges](#) for instructions on modifying the required script to run the `lmadmin` system service with non-elevated privileges.



Note • The instructions in these sample scripts should work on other `lmadmin`-supported UNIX platforms with minor changes.

OS X Systems

On OS X Systems, administrators have to create their own startup script in a directory such as `/Library/StartupItems/lmadmin`. The installed `<lmadmin_install_dir>/examples/service` script is the same script as installed for Linux and Solaris systems and is provided *for reference only*; it will not work properly on OS X systems. For more information on installing an executable file as a system service (and running it with non-elevated privileges) on OS X systems, see any of the many publicly available references such as <http://www.oreilly.com/pub/a/mac/2003/10/21/startup.html>.



Note • Revenera is not responsible for the accuracy of information obtained from such reference sources or for the results of the startup script that you write.

Running FlexNet Publisher License Server as a System Service With Non-Elevated Privileges

`lmadmin` can be configured to run as a service under the `LocalService` account. This account is both necessary and sufficient for running the service. (The vendor daemon inherits these same privileges.) This practice ensures that any security vulnerabilities exploited by malicious users with access to the corporate network will have minimal impact beyond the licensing services.

The instructions below are examples for enabling the `lmadmin` operating-system service to run under a non-root or non-administrator user account. The instructions assume that you have `lmadmin` already installed.

Procedure Overview

Managing the license server as a system service typically requires three main phases:

- **Phase 1**—Install the license server as a system service. This step typically requires elevated privileges. See the previous section, [Installing lmadmin License Server Manager as an Operating System Service](#).
- **Phase 2**—Identify or create a user account with non-elevated privileges. You should create a user account (either a domain account or a local account) that is dedicated to running the license server system service. In the instructions that follow, this user account is referred to as the *Service User account*. Make sure that this Service User account has sufficient permissions to read the license files and write the log files in the designated folders.
- **Phase 3**—Configure the installed license server system service to use the Service User account. This step requires elevated privileges as well.

The following sections provide steps (based on the above procedure overview) for enabling the lmadmin license server to run with non-elevated user privileges on various operating-system platforms. Adjust these steps as needed for your specific platform or platform version.

Windows Systems

Use the following procedure to run the lmadmin system service with non-elevated privileges on Windows.



Task

To configure the lmadmin system service to run with non-elevated user privileges:

1. If lmadmin was *not* set up as a Windows service during the lmadmin installation process, install the service from the command line:

```
lmadmin configDir <conf_path> -installService <servicename>
```

where <servicename> is the name of the lmadmin Windows service.

This step requires elevated user privileges.

2. Create a Service User account either on the local system or on the domain.
3. Grant this lesser-privileged account the permissions to start and stop the service. Use any appropriate method, such as Security Descriptor Definition Language or a publicly available third-party tool such as SetACL.

The following steps use SetACL to manage the permissions for the service:

- a. Download and install the command-line SetACL utility.
- b. From the command-line, change to the directory where the SetACL utility is installed, and execute this command:

```
SetACL.exe -on "<servicename>" -ot srv -actn ace -ace "n:\<domain>\<serviceuser>;p:start_stop"
```

where the following elements are defined as follows:

- *servicename* is the name of the installed lmadmin service.
 - *domain* is the domain to which the user belongs.
 - *serviceuser* is the user under which the lmadmin service is intended to run.
4. To designate the Service User, navigate to the Windows Services console (for example, on some Windows platforms, you access this console from the Task Manager or from **Administrative Services** on the Control Panel).
 5. From the Services console, do the following:
 - a. Right-click the lmadmin service, and select the **Properties** option.
 - b. Under the **Log On** tab, select **This account**, and specify the Service User and its password.

Linux Systems

The procedure for setting up lmadmin as a system service and configuring it to run with non-elevated rights is platform dependent:

- **Linux platforms that use the old BSD startup script:** Use the sample script file `lmadmin` (located in the `<lmadmin_install_dir>/examples/service` directory). Follow the instructions under [Procedure for Linux Platforms Using the Old BSD Startup Script](#).
- **Linux platforms that use the new systemd startup script:** Use the sample script file `lmadmin.service` (located in the `<lmadmin_install_dir>/examples/systemd` directory). Follow the instructions under [Procedure for Linux Platforms Using the New systemd Service Startup](#).

Procedure for Linux Platforms Using the Old BSD Startup Script

If you have already configured `lmadmin` to start up as a system service, skip steps 1 and 2 in the following procedure. However, you must still access this script to make additional modifications, as described below.



Task

To configure the `lmadmin` system service to run with non-elevated user privileges on platforms using the old BSD startup script:

1. Copy the sample script file `lmadmin` to the `/etc/rc.d/init.d` directory.



Important ▪ Once the script is copied, make sure it has execution privilege.

2. To update the run-level information for the service, execute the following command:

```
chkconfig lmadmin on
```

3. Modify the following line in the script to identify the `lmadmin` path:

```
InstallDir="/opt/FNPlm/lmadmin"
```

where the `installDir` value is the directory location where the `lmadmin` binary is installed.

4. Modify the following line in the script to identify the user under which the `lmadmin` system service will run:

```
lmadminUser="lmadmin"
```

where the `lmadminUser` value is the name of the Service User account with the non-elevated privileges.

5. Remove `--pidfile=${pidFile}` from the following line under the `start()` section:

```
daemon --user $lmadminUser --pidfile=${pidFile} "$lmadmin" -root "$installDir"
```

6. Save the changes in the script.

7. To start the `lmadmin` system service under the user you specified for `lmadminUser`, execute the following command:

```
service lmadmin start
```

Procedure for Linux Platforms Using the New systemd Service Startup

If you have already configured `lmadmin` to start up as a system service, skip [Step 1](#) through [Step 6](#) in the following procedure. However, you must still access the appropriate script to make additional modifications, as described below.



Task

To configure the ladmin system service to run with non-elevated user privileges on platforms using the systemd startup script:

1. Copy the sample script file `ladmin.service` to the `/etc/systemd/system` directory.



Important • Once the script is copied, make sure it has execution privilege.

2. Modify the following line in the script to identify the ladmin path:

```
Environment="installDir=/opt/FNP1m/ladmin"
```

where the `installDir` value is the directory location where the ladmin binary is installed.

3. Modify the following line in the script to identify the user under which the ladmin system service will run:

```
User=ladmin
```

where the `ladmin` value is the name of the Service User account with the non-elevated privileges.

4. Modify the following line in the script to identify the group under which the ladmin system service will run:

```
Group=ladmin
```

where the `ladmin` value is the name of the group with the non-elevated privileges.

5. Modify the following line in the script to identify the absolute ladmin executable path:

```
ExecStart=/opt/FNP1m/ladmin/ladmin -root ${installDir}
```

6. Save the changes in the script.

7. To update the run-level information for the service, execute the following command:

```
systemctl daemon-reload
```

8. To start the ladmin systemd service on boot, execute the following command:

```
systemctl enable ladmin.service
```

9. To start the ladmin system service under the user you specified for `ladminUser`, execute the following command:

```
systemctl start ladmin.service
```

Other UNIX Systems

After configuring ladmin to start up as a system service, as described in [Installing ladmin License Server Manager as an Operating System Service](#), modify the startup script (`ladmin` or similar script) to run the ladmin service with non-elevated privileges. Use the instructions in the previous section, [Linux Systems](#), as a guide for editing the script. However, adjust the procedure as needed for your specific UNIX platform.

OS X Systems

See the previous section, [OS X Systems](#), under [Installing ladmin License Server Manager as an Operating System Service](#).

lmadmin Command-line Arguments

This section describes in outline each of the lmadmin command-line arguments. Arguments defined as persistent will remain set until they are reset.

Usage

```
lmadmin [-version]
[-config <configFile>]
[-configDir <configFileDirectory>]
[-cacheDir <cacheDirectory>]
[-defaultAdminUser <domain\user>] (only on Windows and Linux)
[-defaultAdminGroup <domain\group>] (only on Windows)
[-ldapHost <ActiveDirectoryServerHost>] (only on Linux)
[-ldapPort <ActiveDirectoryServerPort>] (only on Linux)
[-ldapUser <domain\user>] (only on Linux)
[-ldapPassword <ldapUserPassword>] (only on Linux)
[-ldapBaseDN <ActiveDirectoryServerBaseDN>] (only on Linux)
[-root <lmadmin_install_dir>] [-force] [-import <licenseFileList>]
[-importInstallation <oldInstallDirectory>[-config <configFileForImport>] [-configDir
<configDirForImport>]]
[-licPort <licenseServerPort>]
[-webSecurePort <httpsPort>]
[-allowStopServer <yes|no>]
[-allowRemoteStopServer <yes|no>]
[-allowLicenseReclaim <yes|no>]
[-webPort <httpPort>]
[-cacheTimeout <timeout>]
[-restartRetries <retrycount>]
[-alertTimeInterval <timeInterval>]
[-foreground] [-adminOnly <yes|no>]
[-noWeb]
[-installService <servicename> [-addDependencyFNLS] [-delay <seconds>]] (only on Windows)
[-removeService <serviceName>] (only on Windows)
[-uploadDir <uploadDirectory>]
```

```
[-<userid> -Pass <password> -Role <roleType> [-FirstName <name>] [-LastName <name>][-offlineupdate <1/0>]]

[-<userid> [-Pass <password>] [-Role <role_type>] [-FirstName <name>] [-LastName <name>][-offlineupdate <1/0>]]

[-userdel <userid> [-foreground] [-adminOnly <yes|no>] [-logDir <logDirectory>][-offlineupdate <1/0>]]

[-vdSearchPaths <vendorDaemonBinarySearchPathsList>

[-enableLmreporterLink <1/0>]
```

Table 9-5 ■ lmadmin Command-line Arguments

Argument Syntax	Description	Function
-addDependencyFNLS	Default - Do not add dependency on FlexNet Licensing Service Persistent - N/A Set from UI - No	(applicable only for Windows) Can be used with the -installService option. This results in addition of a dependency on FlexNet Licensing Service, (32-bit or 64-bit depending on the lmadmin executable type), for lmadmin service. The default is not to have this dependency.
-adminOnly <yes no>	Default—Yes Persistent —Yes Set from UI—No	Restricts usage of lmdown, lmreread, and lmremove—as well as lmswitch, lmswitchr, and lmnewlog. If you set -adminOnly no, command-line access to these utilities is unrestricted. (Access to related features in the lmadmin UI is governed separately by lmadmin login credentials.) By default, lmadmin restricts command-line access to these utilities. The default argument, -adminOnly yes, overrides other lmadmin command-line options, such as -allowLicenseReclaim, -allowStopServer, and -allowRemoteStopServer. Restrictions vary depending on the operating system on which lmadmin is running. On Windows, command-line access to these utilities is completely restricted. If -adminOnly yes is used when starting lmadmin, no user on Windows can shut down the license server with lmdown, nor can they use the lmswitch, lmswitchr, and lmnewlog command-line utilities. On UNIX, -adminOnly yes permits access by the root user only, by default. However, if you define a UNIX group called lmadmin, then access is permitted to members of that group only. (If root is not a member of this group, then root does not have permission to use any of the above utilities.)

Table 9-5 ▪ lmadmin Command-line Arguments

Argument Syntax	Description	Function
-allowLicenseReclaim <yes no>	Default —No Persistent—Yes Set from UI—No	Controls the operation of lmremove (in License File-Based Licensing) . If set to yes, licenses can be reclaimed from a user. If set to no, licenses cannot be reclaimed from a user.  Note ▪ Restrictions from the -adminOnly yes argument may prevent license reclamation even if you set -allowLicenseReclaim yes.
-allowStopServer <yes no>	Default—Yes value Persistent —Yes Set from UI —No	Configures how the license server can be stopped. If set to yes, local clients can stop the license server using either lmdown or the Stop Server button in the UI.  Note ▪ Restrictions from the -adminOnly yes argument may prohibit stopping the server from the command line even if you set -allowStopServer yes. If set to no, then the license server must be stopped by stopping the process. See Manually Stopping the License Server Manager .  Note ▪ Setting -allowStopServer no also sets -allowRemoteStopServer to no.
-allowRemoteStopServer <yes no>	Default—No value Persistent—Yes Set from UI—No	Configures whether the license server can be stopped from a remote location. If set to yes, then you can stop the license server from a remote location and local clients can stop the license server using either lmdown or the Stop Server button in the UI.  Note ▪ Restrictions from the -adminOnly yes argument may prohibit remotely stopping the server from the command line even if you set -allowRemoteStopServer yes. If set to no, then it must be stopped from a local client. See Online help for further details.  Note ▪ Setting -allowRemoteStopServer yes when -allowStopServer is not defined, forces -allowStopServer to be set to yes.

Table 9-5 ▪ lmadmin Command-line Arguments

Argument Syntax	Description	Function
-config <configFile>	Default—server.xml Persistent—Yes Set from UI—No	Defines the name of the license server configuration file to use when starting the license server manager. If all defaults are set, the path and name is the following: <lmadmin_runtimedata_dir>\conf\server.xml.
-configDir <configDir>	Default— <lmadmin_runtimedata_dir> \conf (Windows); <lmadmin_install_dir>/conf (non-Windows) Persistent—Yes Set from UI—No	Defines the directory where the license server configuration files are located.
-cacheDir <cacheDirectory>	Default— <lmadmin_runtimedata_dir> \cache (Windows); <lmadmin_install_dir>/cache (non-Windows) Persistent—Yes Set from UI—No	Defines the destination directory path for cache files.
-delay <seconds>	Default—0 Persistent—Yes Set from UI—No	(applicable only for Windows) Used when configuring the license server as a service on Windows. See Installing lmadmin License Server Manager as an Operating System Service .
-defaultAdminUser <domain\username>	Default—N/A Persistent—Yes Set from UI—Yes	(applicable only for Windows and Linux) Adds the specified Windows Active Directory user as an lmadmin Administrator. The license administrator can then log in to the lmadmin user interface initially using this user ID (and its Active Directory password) and proceed to perform administrative tasks. To identify this user, use the format <i>domain\username</i> , where <i>domain</i> is a valid Windows Active Directory domain to which the machine running lmadmin has a trusted relationship, and <i>username</i> identifies a valid account within that domain. This value is not case-sensitive and can include up to 64 characters.
		 Note ▪ Use this argument sparingly. Its main purpose is to provide initial access to the lmadmin interface.

Table 9-5 ▪ lmadmin Command-line Arguments

Argument Syntax	Description	Function
-defaultAdminGroup <domain\groupname>	Default—N/A Persistent—Yes Set from UI—Yes	(applicable only for Windows) Adds the specified Windows Active Directory group as an lmadmin Administrator. The license administrator can then log in to the lmadmin user interface initially using any Active Directory user ID (and its associated password) belonging to this group and proceed to perform administrative tasks. To identify this group, use the format <i>domain\groupname</i> , where <i>domain</i> is a valid Windows Active Directory domain to which the machine running lmadmin has a trusted relationship, and <i>groupname</i> identifies a valid account within that domain. This value is not case-sensitive and can include up to 64 characters. 
		Note ▪ Consider the following: ● Setting a domain group as the lmadmin Administrator is helpful in a large enterprise where you might have several license administrators who need access to the interface. For more information, see the Online help once you have opened the interface. ● Use this argument sparingly. Its main purpose is to provide initial access to the lmadmin interface.
-enableLmreporterLink <1/0>	Default—0 Persistent—Yes Set from UI—No	Enables/disables the appearance of the lmreporter icon/button and its link on the lmadmin UI. This setting is recorded in the license server configuration file and will apply to all subsequent executions until changed.
-force	Default—Do not overwrite settings Persistent—No Set from UI—No	Use with the -import argument to overwrite existing vendor daemon settings in the license server configuration file. The following settings are overwritten or reset to the default: ● License file location (overwritten). ● Vendor daemon location (overwritten). ● Vendor daemon port (reset to default). ● Restart retries (reset to default). ● Date-based versions (reset to default). ● Overwrite vendor daemon log (reset to default). ● Vendor daemon log location and name (reset to default).

Table 9-5 ■ lmadmin Command-line Arguments

Argument Syntax	Description	Function
-foreground	Default—Run in background Persistent—No Set from UI—N/A	Runs lmadmin in the foreground (output status and errors are sent to the command window).
-import <licenseFileList>	Default—N/A Persistent—N/A Set from UI — Import License button	Updates the license server configuration file with information extracted from the specified license files. See license_file_list for details of the format of <licenseFileList>. This option does not start lmadmin. The only arguments that can be combined with -import are: -config, -configDir, -cacheDir, and -force.
-importInstallation <oldInstallDirectory> [-config <configFileForImport>] [-configDir <configDirForImport>]	Default—N/A Persistent—N/A Set from installer—Specify in Import Files from Previous Installation in the lmadmin installer.	Imports configuration information from the specified existing lmadmin installation directory or from a specified location for the license server configuration files (optional use of -config or -configDir). This option does not start lmadmin. The only arguments that can be combined with -importInstallation are: -config, -configDir, and -root.
-installService <serviceName>	Default—Do not install lmadmin as a Windows service Persistent—N/A Set from UI—No	(applicable only for Windows) Used when configuring the license server as a service on Windows. See Installing lmadmin License Server Manager as an Operating System Service .
-licPort <licenseServerPort>	Default—First available in range 27000-27009 Persistent—Yes Set from UI—Specify for License Server Manager Port (located under the License Server Configuration heading on the Server Configuration tab on the Administration page)	Configures the license server manager port. To set a specific port, enter a positive integer for <licenseServerPort>. A valid number is any unused port number in the range 1 to 65535. On UNIX, choose a port >1024, since those <1024 are privileged port numbers. If you specify a port number greater than 65535, the client fails to establish a connection with lmadmin. If no port is specified, the license server will automatically use the next available port number in the range 27000 to 27009. Applications, when connecting to a server, try all numbers in the range 27000 to 27009.
		 Note ■ Those producers concerned about potential DoS attacks based on knowledge of common license server ports may want to consider specifying a server port outside the range 27000 to 27009.

Table 9-5 ▪ lmadmin Command-line Arguments

Argument Syntax	Description	Function
-logDir <logDirectory>	Default— <lmadmin_runtimedata_dir> \logs (Windows); <lmadmin_install_dir>/logs (non-Windows) Persistent—Yes Set from UI—No	Writes the lmadmin logs to the location you specify for <logDirectory>. Enter an absolute path for <logDirectory>. Assuming no vendor daemon logs have been set (in the options file) to write to a custom location, lmadmin writes all lmadmin logs to the location you set for <logDirectory>. When a custom vendor daemon log location is set, that setting overrides the -logDir setting for that particular vendor-daemon log.
-offlineupdate <1/0>	Default is online update Persistent - N/A Set from UI - No	Do offline/online update for user configuration
-removeService <serviceName>	Default—N/A Persistent—N/A Set from UI—No	(applicable only for Windows) Used when configuring the license server as a service on Windows. See Installing lmadmin License Server Manager as an Operating System Service .
-root <lmadmin_install_dir>	Default— root installation directory. Persistent—No Set from UI—No	Specifies the root directory for lmadmin. This enables you to issue an lmadmin command from somewhere other than the directory where lmadmin is installed. 
Note ▪ Any command-line arguments that specify relative paths define paths relative to the current directory and not the directory specified with -root.		
-uploadDir <uploadDirectory>	Default— <lmadmin_runtimedata_dir> \licenses\ <vendor> (Windows); <lmadmin_install_dir>/ licenses/<vendor> (non- Windows) Persistent—Yes Set from UI—No	Configures the directory where license files that are uploaded to the license server manager are stored. A directory with the name of the vendor daemon can be set by using the string “%v” as in the following example: -uploadDir flexlicenses%\%v

Table 9-5 ▪ lmadmin Command-line Arguments

Argument Syntax	Description	Function
<code>-<userid> -Pass <password></code> <code>-Role <roleType> [-</code> <code>FirstName <name>]</code> <code>[-LastName <name>]</code>	Persistent—Yes Set from UI—Yes	Adds a new lmadmin user-interface user. Provide the following parameters: • -<userid>—Provide either value: For a locally managed account (that is, defined and managed by lmadmin), a user-defined sign-in ID, up to 64 characters. The value is case-sensitive and can include spaces but no backslashes (\). <i>or</i> For a Windows Active Directory domain account, the account ID in the following format: <code><domain>\<username> <groupname></code> where <domain> is the active Active Directory domain to which the machine running lmadmin has a trusted relationship and <username> or <groupname> identifies a valid user or group account existing within that domain. This value is not case-sensitive and can include up to 64 characters. Separate multiple domains with a backslash (\).  Note ▪ When a Windows Active Directory group account is defined to lmadmin, only the group name is identified in the account. However, a user must sign in to lmadmin using a domain user name (not the group name) and its password. lmadmin then contacts the domain server to validate the user as a member of a group account defined to lmadmin. • -Pass <password>—(Locally managed accounts only) Specify the password for the specified user ID. The password must be at least 8 characters.

Table 9-5 ■ lmadmin Command-line Arguments

Argument Syntax	Description	Function
-<userid> -Pass <password> -Role <roleType> [- FirstName <name>] [-LastName <name>] (continued)		-Role <roleType>—Specify one of these roles to assign lmadmin user-interface privileges to the account: user—The user is limited to lmadmin Dashboard functionality only.* Assign this role only when you are password-protecting the Dashboard.*** Otherwise, the user has automatic access to the Dashboard. admin—The user (locally managed or in Windows Active Directory) has administrator rights.** userGroup—(Windows Active Directory group account only) The domain group is limited to lmadmin Dashboard functionality only.* Assign this role only when you are password-protecting the Dashboard.*** Otherwise, the group has automatic access to the Dashboard. adminGroup—(Windows Active Directory group account only) The domain group has administrator rights.** -FirstName <name>—(Locally managed accounts only; optional) The user’s first name. This value can include up to 32 characters. -LastName <name>—(Locally managed accounts only; optional) The user’s last name. This value can include up to 32 characters.
-<userid> [-Pass <password>] [-Role <role_type>] [-FirstName <name>] [-LastName <name>]	Persistent—Yes Set from UI—Yes	Enables you to modify the lmadmin user-interface user account identified by the account ID (<userid>). You can modify the account's role and, for locally managed accounts, the password and first and last names of the user. See the -useradd argument described previously in this table for descriptions of these parameters.
-userdel <userid>	Persistent—Yes Set from UI—Yes	Removes the lmadmin user-interface user account identified by the account ID (<userid>).
-webPort <httpPort>	Default—8090 Persistent—Yes Set from UI—Specify for HTTP Port (located under the Web Server Configuration heading on the Server Configuration tab on the Administration page)	Configures the TCP/IP port that the Web server uses to listen for communication with clients connecting to the license server management interface. See License Server Manager Not Starting for an example of how to use this argument.

Table 9-5 ▪ ladmin Command-line Arguments

Argument Syntax	Description	Function
-webSecurePort <httpsPort>	Default—0 Persistent—Yes Set from UI—Specify for HTTPS Port (located under the Secure Web Server Configuration heading on the Server Configuration tab on the Administration page)	Configures the TCP/IP port (five characters maximum) that the ladmin web server uses to listen for HTTPS (HTTP-over-SSL) communication. If you change the port, you must stop and restart the license server.  Important ▪ Specify the -webSecurePort option only if you are configuring HTTPS communication with the ladmin web server interface.
-version	N/A	Outputs details of the ladmin version to the command prompt.
-vdSearchPaths <vendorDaemonBinarySearchPathsList>	Default—N/A Persistent—Yes Set from UI—No	Specify a list of absolute paths, from which vendor daemon binary or executable can be allowed to be picked up, separated by the Path separator character. This setting is recorded in the license server configuration file and will apply to all subsequent executions until altered.  Note ▪ Path separator character on the following platforms: • On non-Windows platforms, the Path separator character is ":". • On Windows platform, the Path separator character is ",".
* Dashboard privileges provide access to all the ladmin user-interface Dashboard functionality. This includes viewing license activity and alerts, changing the user password (locally managed accounts only), and selecting a locale when multiple locales are configured on the system. ** Administrator privileges include all Dashboard privileges plus the ability to access the Administration page of the ladmin user interface. From this page, the administrator can perform license-server configuration and management tasks, and set up, edit, or delete other ladmin user accounts. *** To password-protect the Dashboard, use the Password Protect Dashboard option on the Server Configuration tab located on the Administration page in the ladmin user interface.		

Obtaining ladmin Command-Line Help

For a handy reference to ladmin command-line arguments, use this procedure.



Task To obtain a handy reference of ladmin command-line arguments:

1. Open a command-prompt window.
2. Change to the directory where the ladmin file is located.

3. Enter the following command to view a list of available arguments with a description of each:

```
lmadmin -help
```

The help descriptions identify the default arguments and which arguments are *persistent*, arguments that will remain in effect for later instances of lmadmin.

Extending lmadmin License Server Capability

lmadmin can be customized. These customizations require some programming. The lmadmin installation package includes some example applications and files that demonstrate simple customizations.

Using the lmadmin Web Service Interface

lmadmin provides a Web service interface that exposes certain APIs that can be called from a custom-built utility. These services enable you to extend the core license server capabilities. The WSDL file needed to generate the client proxy can be found in the <lmadmin_install_dir>\wsdl directory.

The lmadmin installation package includes a set of examples in the <lmadmin_install_dir>\examples directory that demonstrate how to implement certain capabilities using the Web service interface.

Creating an lmadmin Alerter Service

The lmadmin license server installation includes an example of how to implement an email alerter service. This service will poll for alerts and then send a user an email when an alert has been triggered.

Using the Alerter Service Email Alerts

The sample Alerter service utility runs on the license server and enables a user to receive alert notifications by email. To use the Alerter service, Java Runtime Environment (JRE) 1.6 or later (for OS X: JRE 1.7 or later) must be installed on the license server.

To start the Alerter service, there are two files in the <lmadmin_install_dir>\examples\alerter directory:

- For Windows systems, the runalerter.bat file.
- For UNIX systems, the runalerter file.

When starting this service, you must configure certain command-line arguments to define the mail server, sender, receiver, and so on. To see the list of available command-line arguments for the runalerter script, type the following command:

```
runalerter -help
```

The source code for this utility is in the <lmadmin_install_dir>\examples\alerter\src directory.

License Server Manager “lmgrd”

The *license server manager* is one of the components that make up a license server (the other being the vendor daemon). It handles the initial contact with FlexEnabled applications, passing the connection on to the appropriate vendor daemon. The purposes of the license server manager are to:

- Start and maintain all the vendor daemons listed in the VENDOR lines of the license file used to start lmgrd.
- Refer application checkout (or other) requests to the correct vendor daemon.

lmgrd is an application-based version of the license server manager. On most platforms it is controlled from a command-line. On Windows, lmtools can be used to manage lmgrd.

A newer lmgrd can be used with an older vendor daemon or FlexEnabled application, but a newer vendor daemon or FlexEnabled application might not work properly with an older lmgrd. Always use the latest version of lmgrd. See [Version Compatibility Between Components](#) for detailed information.

Ensure that the local host name resolution (in /etc/hosts) is handled before the following scenarios:

- To have regular heartbeat interval when you communicate between lmgrd and vendor daemon, and
- To successfully start the license server.

Neither lmgrd nor the vendor daemon retain any state information on restart. That means, for example, the license server will have any recollection of any clients that had an outstanding checkout when they previously shut down.

The advantage of this design is that the server always starts in a known state. For example, there are no concerns about a potentially corrupt database of clients to worry about. Producers who wish to introduce their own state can use the various callback routine hooks provided in lsvendor.c to implement their own persistence mechanism.

lmgrd Command-Line Syntax

When you invoke lmgrd, it looks for a license file that contains information about vendors and features.

Usage

```
lmgrd [-c license_file_list] [-l [+]debug_log_path]
      [-2 -p] [-local] [-x lmdown] [-x lmremove] [-z] [-v] [-datestamp] [-m value] [-help] [-servTransferred]
```

The following table describes the syntax elements:

Table 10-1 ■ lmgrd Command-Line Syntax

Term	Description
-c license_file_list	Uses the specified license files.
-l [+]debug_log_path	Writes debugging information to file <i>debug_log_path</i>. This option uses the letter <i>l</i>, not the numeral 1. Prepending <i>debug_log_path</i> with the + character appends logging entries. Use the -l option before other options to log all debugging information to <i>debug_log_path</i>. See Debug Log File for more information on this file.  Note ■ On Windows, it is best practice to set the location of <i>debug_log_path</i> to a <i>ProgramData</i> sub-folder, since this location has user-write permission by default, which is needed when the license server runs as a service with <i>LocalService</i> permission.
-2 -p	All platforms: -2 -p is effective, and supported, only when used together with -local. On UNIX systems, -2 -p restricts usage of <i>lmdown</i>, <i>lmreread</i>, and <i>lmremove</i>—as well as <i>lmswitch</i>, <i>lmswitchr</i>, and <i>lmnewlog</i>—to a license administrator who is by default root. If there is a UNIX group called <i>lmadmin</i>, then use is restricted to only members of that group. If root is not a member of this group, then root does not have permission to use any of the above utilities. On Windows systems, if <i>lmgrd</i> is started with -2 -p -local, <i>lmgrd</i> and the vendor daemon can only interact with the command-line utilities (<i>lmreread</i>, <i>lmnewlog</i>, <i>lmdown</i>, <i>lmremove</i>, and <i>lmswitch</i>) if these are located on the same machine and if they are run with <i>LOCALSYSTEM</i> privileges.  Note ■ It is recommended to start your <i>lmgrd</i> with -2 -p -local. This protects <i>FlexNet Publisher</i> from attack that involve remote, unauthenticated access to server commands.

Table 10-1 • lmgrd Command-Line Syntax

Term	Description
-local	On UNIX systems, <code>-local</code> restricts the <code>lmdown</code> and <code>lmreread</code> commands to be run only from the same system where <code>lmgrd</code> is running. On Windows systems, if <code>-local</code> is used on its own, <code>lmutil</code> (and variations) can be run by any Local User as long as they are running <code>lmutil</code> from the same host as the license server. If <code>lmgrd</code> is started with <code>-2 -p -local</code>, <code>lmgrd</code> and the vendor daemon can only interact with the command-line utilities (<code>lmreread</code>, <code>lmnewlog</code>, <code>lmdown</code>, <code>lmremove</code>, and <code>lmswitch</code>) if these are located on the same machine and if they are run with <code>LOCALSYSTEM</code> privileges. All platforms: In the case of a three-server setup, any node in the triad is considered to be local.
-x lmdown	Disables the <code>lmdown</code> command (no user can run <code>lmdown</code>). If <code>lmdown</code> is disabled, stop <code>lmgrd</code> using <code>kill pid</code> (UNIX), or stop the <code>lmgrd</code> and vendor daemon processes through the Windows Task Manager or Windows service. On UNIX, be sure the <code>kill</code> command does not have a <code>-9</code> argument.
-x lmremove	Disables the <code>lmremove</code> command (no user can run <code>lmremove</code>).
-z	Runs in foreground. The default behavior is to run in the background. If <code>-l debug_log_path</code> is present, then no windows are used, but if no <code>-l</code> argument specified, separate windows are used for <code>lmgrd</code> and each vendor daemon.
-v	Displays the <code>lmgrd</code> version number and copyright and exits.
-d	(OS X/macOS only) Makes <code>lmgrd</code> and the vendor daemon OS X/macOS <code>launchd</code> compliant. Use the reference file <code>demo.plist</code> (included in the OS X/macOS toolkit) for information on how to instruct <code>launchd</code>.
-reuseaddr	[optional] Allows the server to explicitly bind to a same port, which remains in <code>TIME_WAIT</code> state after the server restart or crash  Note • It is recommended to use the <code>-reuseaddr</code> option only on non-Windows systems to avoid undefined behavior.
-datestamp	[optional] Enables writing the date, timestamp, and time zone offset in the debug log files.

Table 10-1 • lmgrd Command-Line Syntax

Term	Description
-m <i>value</i>	Enforces obfuscation of communication between client and server. Valid values: ● disabled: Message obfuscation is disabled. The license server ignores the obfuscation mechanism for the communication. ● permissive: Message obfuscation is enforced only if it is supported by the client. ● targeted: Message obfuscation is enforced only for the targeted messages that share the license file information with the clients. Requests sent by clients without message obfuscation are rejected. Communication with utilities must be obfuscated. ● enforcing: Message obfuscation is enforced on all new style messages.  Important ▪ If the argument <i>targeted</i> or <i>enforcing</i> is set, FlexNet Publisher clients or utilities prior to version 11.19.2 (2022 R3) can not communicate with the license server because they don't support the obfuscation mechanism.
-help	Displays usage information and exits.
-servTransferred	Enables the license server to serve transferred licenses by reading the transfer cache.



Note ▪ The license search path can also be specified by setting the environment variable `LM_LICENSE_FILE` to the file's path name. The `-c` path specification will override the setting of `LM_LICENSE_FILE`.

Starting the License Server Manager on UNIX Platforms

If any licenses in the license file are counted (license count > 0), the license server manager, and hence the license server, must be started before the FlexEnabled application can be used.

The license server manager, `lmgrd`, is started either manually on the command line or automatically at system startup. Both methods are discussed in the following sections.



Note ▪ Start `lmgrd` only on the system specified on the `SERVER` line in the license file.

If you are running license servers configured for three-server redundancy, maintain an identical copy of the license file (as well as the `lmgrd` and the vendor daemons binaries) locally on each system rather than on a file

server. If you do not do this, you lose all the advantages of having redundant servers, as the file server holding these files becomes a single point of failure.



Important • In addition, ensure that the http/https ports and the licensing ports (required by lmgrd) and the vendor daemon ports are opened on the firewall, so that remote clients can checkout licenses and the HTTP clients/browsers can connect to the lmgrd.

Manual Start



Task To start lmgrd from the command line

Start lmgrd from the UNIX command line using the following syntax:

```
lmgrd -c license_file_list -L [+]debug_log_path
```

where

- *license_file_list* is one or more of the following:
 - the full path to a single license file
 - a directory, where all files named *.lic in that directory are used

If the *license_file_list* value contains more than one license file or directory, they must be separated by colons.

- *debug_log_path* is the full path to the debug log file.

Prepending *debug_log_path* with the + character appends logging entries.

Start lmgrd by a user other than root since processes started by root can introduce security risks. If lmgrd must be started by the root user, use the su command to run lmgrd as a non-privileged user:

```
su username -c "lmgrd -c license_file_list -l debug_log_path"
```

where *username* is a non-privileged user. You must ensure that the vendor daemons listed in the license file have execute permissions for *username*. The paths to all the vendor daemons in the license file are listed on each VENDOR line.

Automatic Start

lmgrd can be automatically started using either systemd or SystemV.

Systemd Config File

The Linux toolkits contains a sample:

- shellscript called demo.sh
- systemd configuration file called demo.service.

Configure demo.sh

Configure demo.sh (located in <platform_dir>) as required for your system.



Task

To configure the file descriptor ulimit for the license server daemon on platforms from the systemd startup script:

1. Modify the following line in the script to identify the toolkit path:

```
Toolkit_Path=</opt/FNPlm/lmgrd_lic_log_Path>
```

where the Toolkit_Path value is the directory location where the license file and lmgrd is installed. The debug.log will be created at the same location as the license file.

2. Save the changes in the script.

Configure demo.service

Configure demo.service (located in <platform_dir>) as required for your system.



Task

To configure the demo system service to run with non-elevated user privileges on platforms using the systemd startup script:

1. Copy the sample script file demo.service to the /etc/systemd/system directory.



Important - Once the script is copied, make sure it has execution privilege.

2. Modify the following line in the script to identify the absolute demo.sh script path:

```
ExecStart=/opt/FNPlm/lmgrd_path/demo.sh
```

3. Save the changes in the script.
4. To update the run-level information for the service, execute the following command:

```
systemctl daemon-reload
```

5. To start the demo systemd service on boot, execute the following command:

```
systemctl enable demo.service
```

6. To start the demo system service under the user you specified for <user_name>, execute the following command:

```
systemctl start demo.service
```

SystemV Init Script

On UNIX, edit the appropriate boot script, which may be /etc/rc.boot, /etc/rc.local, /etc/rc2.d/Sxxx, /sbin/rc2.d/Sxxxx. Include commands similar to the following. See the following notes for a full explanation.

```
/bin/su daniel -c 'echo starting lmgrd > \  
/home/flexlm/v11/hp700_u9/boot.log'
```

```
/bin/nohup /bin/su daniel -c 'umask 022; \  
/home/flexlm/v11/hp700_u9/lmgrd -c \  
/home/flexlm/v11/hp700_u9/license.dat >> \  
/home/flexlm/v11/hp700_u9/boot.log'  
  
/bin/su daniel -c 'echo sleep 5 >> \  
/home/flexlm/v11/hp700_u9/boot.log'  
  
/bin/sleep 5  
  
/bin/su daniel -c 'echo lmdiag >>\  
/home/flexlm/v11/hp700_u9/boot.log'  
  
/bin/su daniel -c '/home/flexlm/v11/hp700_u9/lmdiag -n -c\  
/home/flexlm/v11/hp700_u9/license.dat >> \  
/home/flexlm/v11/hp700_u9/boot.log'  
  
/bin/su daniel -c 'echo exiting >>\  
/home/flexlm/v11/hp700_u9/boot.log'
```

Please note the following about how this script was written:

- All paths are specified in full because no paths are assumed at boot time.
- Because no paths are assumed, the vendor daemon must be in the same directory as lmgrd, or the VENDOR lines in the license file must be edited to include the full path to the vendor daemon.
- The su command is used to run lmgrd as a non-root user, **daniel**. It is recommended that lmgrd not be run as root since it is a security risk to run any program as root that does not require root permissions. lmgrd does not require root permissions.
- **daniel** has a csh login, so all commands executed as **daniel** must be in csh syntax. All commands not executed as **daniel** must be in /bin/sh syntax since that is what is used by the boot scripts.
- The use of nohup and sleep are required on some operating systems, notably HP-UX. These are not needed on Solaris and some other operating systems, but are safe to use on all.
- lmdiag is used as a diagnostic tool to verify that the server is running and serving licenses.



Note • This does not start the vendor daemon until you reboot the system.

Starting the License Server Manager on Windows

This section provides procedural information on manual starts from the command line and how to configure the License Server Manager (lmgrd) as a service.

Manual Start from the Command Line



Task To start lmgrd from the command line

Start `lmgrd` as an application from a Windows command shell using the following syntax:

```
C:\fnp> lmgrd -c license_file_list -L [+]debug_log_path
```

where

- `license_file_list` is one or more of the following:
 - the full path to a single license file
 - a directory, where all files named `*.lic` in that directory are used

If the `license_file_list` value contains more than one license file or directory, they must be separated by semicolons.

- `debug_log_path` is the full path to the debug log file

Prepending `debug_log_path` with the `+` character appends logging entries.

Spaces in pathnames require double quotes around the path.



Note ▪ On Windows platforms, it is best practice to set the location of `debug_log_path` to a sub-folder of `ProgramData`. This location has user-write permissions by default, which are needed when the license server runs as a service with `LocalService` permission.

Configuring the License Server Manager as a Windows Service

To configure a license server manager (`lmgrd`) as a service, you must have Administrator privileges. The service will run under the `LocalService` account. This type of account (`LocalService` account) is required to run this utility as a service.

Best practice is to install the vendor daemon and `lmgrd` in a subfolder of `Program Files` for a 64-bit license server or `Program Files (x86)` for a 32-bit license server. Data that is written by executables at runtime (the debug log and report log) should be written to a subdirectory of `ProgramData`.



Task To configure a license server as a service:

1. Run the `lmtools` utility as an administrator.
2. On the **Service/License File** tab, select **Configuration using Services**.
3. Click the **Config Services** tab. In the **Service Name** field, type the name of the service that you want to define, for example, **DEMO License Manager**. If you leave this field blank, the service will be named *FlexNet Publisher Service*.

4. In the **Path to the lmgrd.exe file** field, enter or browse to lmgrd.exe for this license server.
5. In the **Path to the license file** field, enter or browse to the license file for this license server.
6. In the **Path to the debug log file** field, enter or browse to the debug log file that this license server writes. Prepending the debug log file name with the + character appends logging entries. The default location for the debug log file is C:\ProgramData\FLEXlm\lmgrd\debug.log.

It is best practice to set the location of debug_log_path to a subfolder of ProgramData. This location has user-write permission by default, which is needed when the license server runs as a service with LocalService permission. If you specify a different location, make sure you specify a fully qualified path that can be accessed with LocalService permissions.

7. To save the new **DEMO License Manager** service, click **Save Service**.

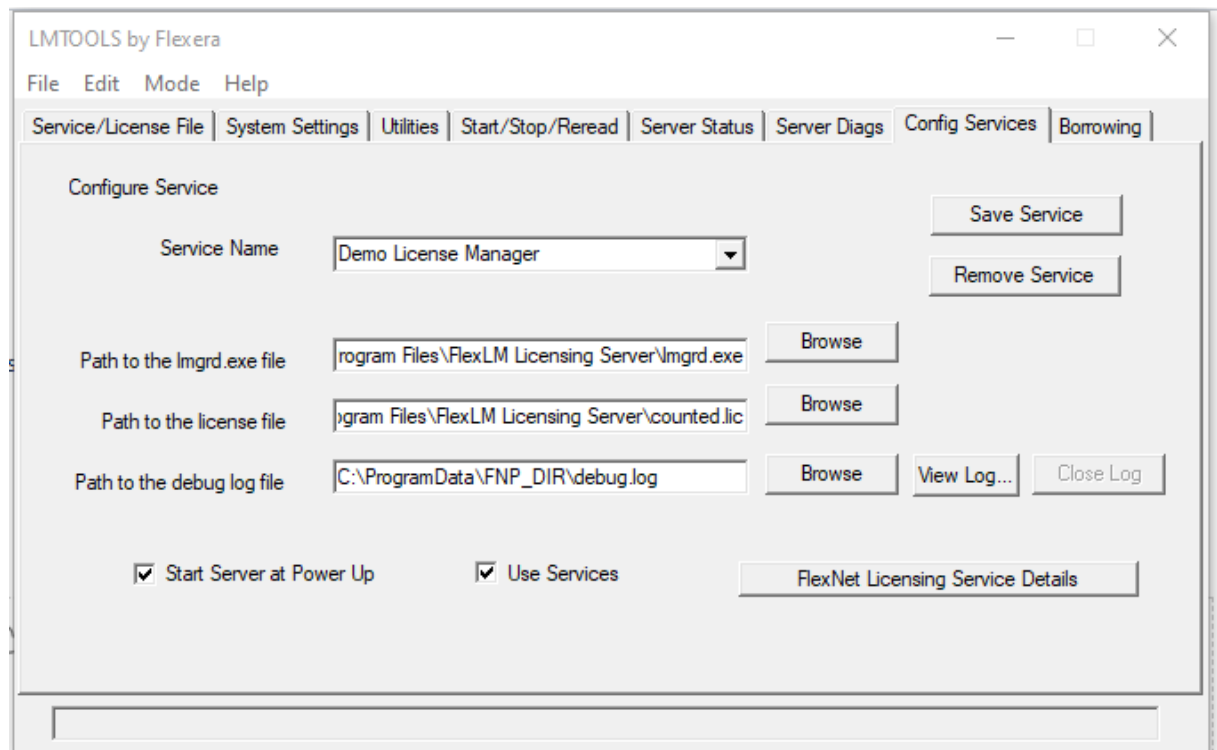


Figure 10-1: Completed Config Services Tab

Configuring the License Server Manager Service for a Delayed Start

In situations where the license server needs to wait for other drivers or services to start before it starts, you can configure a delay before the license server service starts. A typical scenario where a delay is needed is when a FlexNet ID dongle is used to lock the license server to a machine (the FLEXID is used on the SERVER line). In this scenario the license server will sometimes fail to start upon reboot of the system because the license server is loaded before the dongle device driver has loaded properly.



Task

To configure a delayed start for the license server manager service:

1. Configure the license server manager as a service ([Configuring the License Server Manager as a Windows Service](#)).

2. Locate the registry entry for your license server manager service at:

HKEY_LOCAL_MACHINE\SOFTWARE\FLEXlm License Manager\service_name

where service_name is the name of the license server manager service.

3. Optionally, to configure a delay longer than 20 seconds, add a string value to the registry entry and set the fields in this entry as follows:

Name—unlimitedServiceDelay

Type—REG_SZ (set automatically when a string value is created)

Data—no value set

4. Add a string value to the registry entry and set the fields in this entry as follows:

Name—serviceDelay

Type—REG_SZ (set automatically when a string value is created)

Data—the service delay in seconds. This value is limited to the range 1-20 seconds unless unlimitedServiceDelay has previously been defined (see Step 3).

Manually Starting the License Server Using the lmtools Utility

A graphical user interface to the license server manager tools is provided called lmtools. Some of the functions lmtools performs include:

- Starting, stopping, and configuring license servers
- Retrieving system information, including hostids
- Retrieving server status

In order to control the operation of lmgrd from the lmtools user interface, you first must configure it as a license server manager service. Follow the procedure in [Configuring the License Server Manager as a Windows Service](#) before proceeding.

Once the license server manager service is configured, lmgrd is started by starting the service from the lmtools interface.



Task

To start the service from the lmtools interface:

1. Start lmtools and click the **Service/License File** tab.
2. Click the **Configuration using Services** option.

3. Select the service name from the list presented in the selection box. In this example, the service name is **DEMO License Manager**.

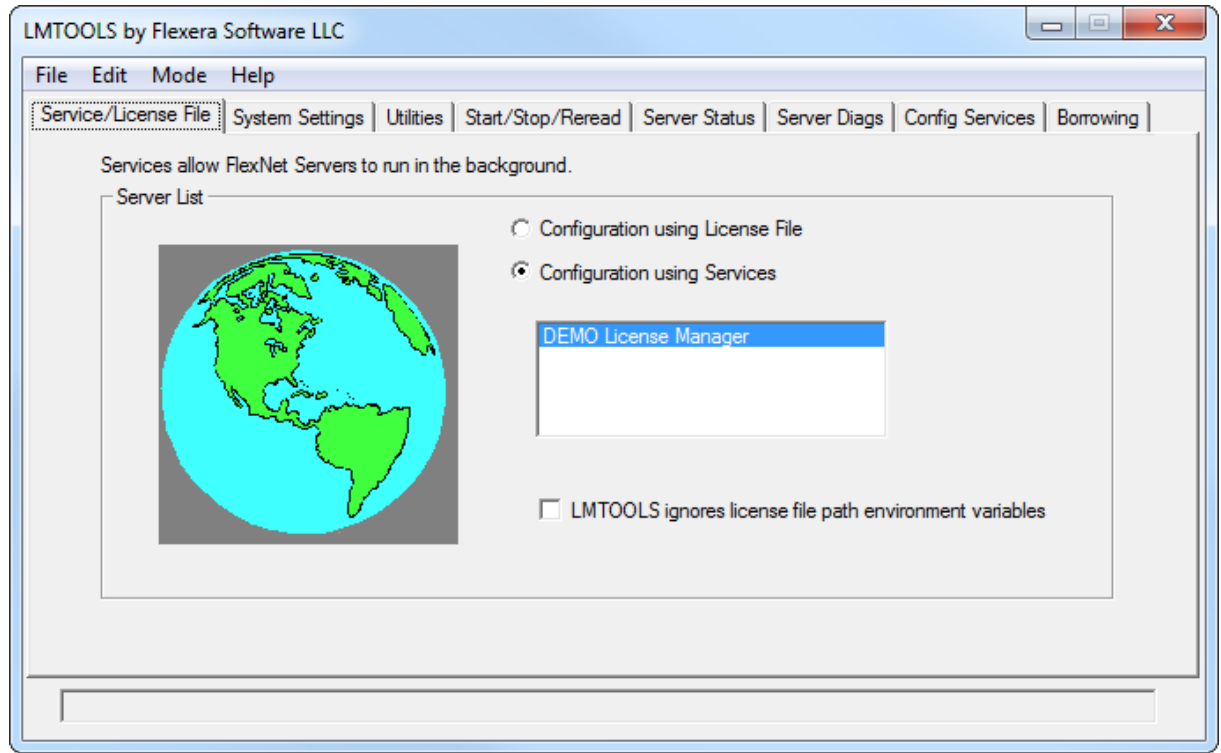


Figure 10-2: Service/License File Tab

4. Click the **Start/Stop/Reread** tab.
5. Start DEMO License Manager by clicking the **Start Server** button. DEMO License Manager license server starts and writes its debug log output to C:\ProgramData\FNP_DIR\debuglog.



Note ▪ The “Stop Server” button is disabled when the option “Disable lmdown utility, use task manager” option is selected under “Edit Advanced Settings”.

6. Click the **Server Status** tab.



Note ▪ The server name or host name is case sensitive. Use the same name as provided in your license file.

7. On the Stop/Reread page, use the **Remote Server List** button to refresh the active server list. The server details are listed in the format Vendor_name: port@host.

Automatically Starting the License Server at System Startup

In order for `lmgrd` to start up automatically at system start-up time, you first must configure it as a service. Follow the procedure in [Configuring the License Server Manager as a Windows Service](#) before proceeding, and then continue with the steps below.



Task

To configure `lmgrd` as a service:

1. With `lmtools` started and the desired service name selected, click the **Config Services** tab.

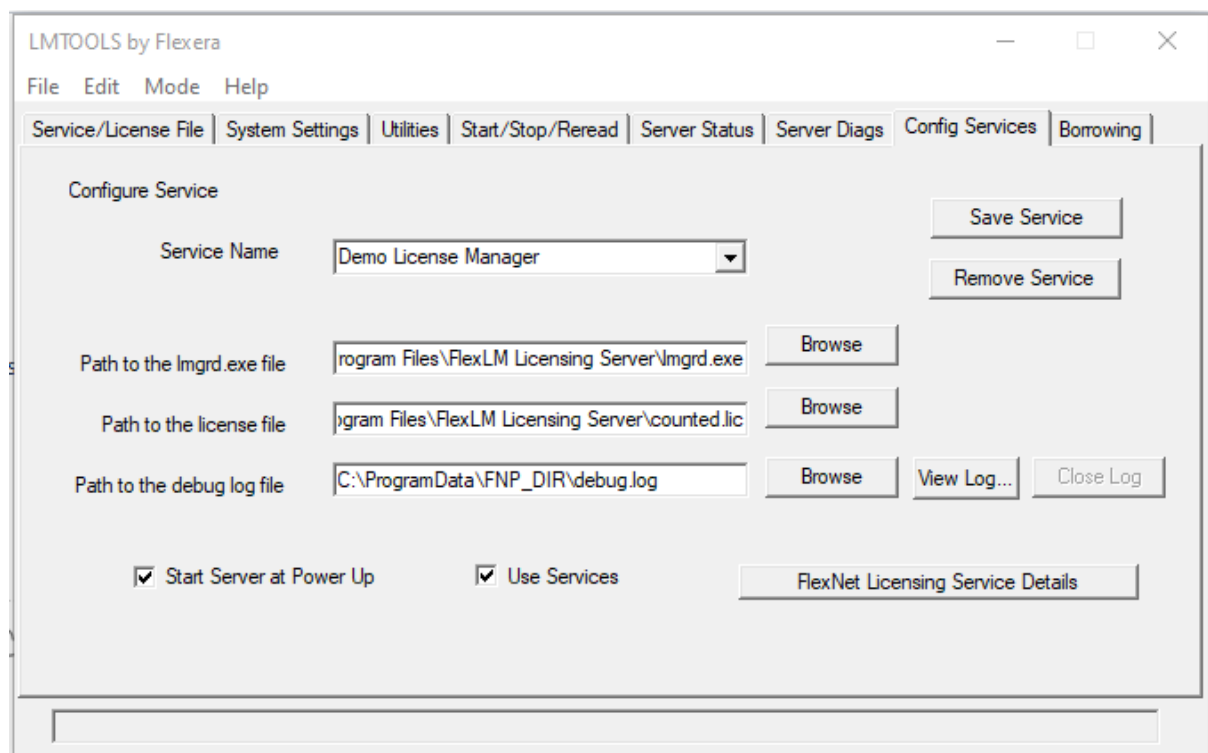


Figure 10-3: Config Services Tab

2. Make this license server manager a Windows service by selecting the **Use Services** check box.
3. Configure it to start at system startup time by selecting the **Start Server at Power Up** check box.



Note • On Windows platforms, it is best practice to set the location of `debug_log_path` to a sub-folder of `ProgramData`. This location has user-write permissions by default, which are needed when the license server runs as a service with `LocalService` permission.

From now on, when the system is rebooted, this license server manager starts automatically as a Windows service.

Three-Server Setup in Imtools



Task

To start the service from the Imtools interface:

1. Start Imtools and click the **Service/License File** tab.
2. Click the **Configuration using Services** option.
3. Select the service name from the list presented in the selection box. In this example, the service name is **DEMO License Manager**.

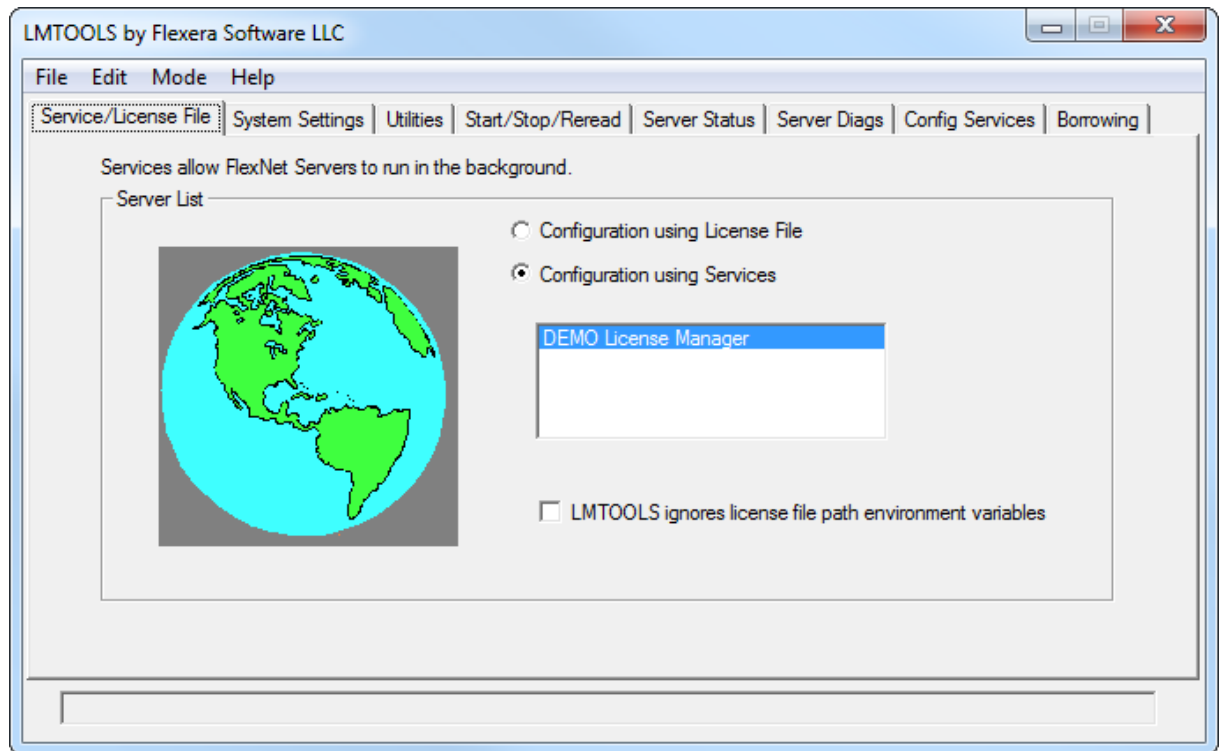


Figure 10-4: Service/License File Tab

4. Click the **Start/Stop/Reread** tab.
5. Start DEMO License Manager by clicking the **Start Server** button. DEMO License Manager license server starts and writes its debug log output to c:\ProgramData\FNP_DIR\debuglog.



Note ▪ The “Stop Server” button is disabled when the option “Disable Imdown utility, use task manager” option is selected under “Edit Advanced Settings”.

6. Click the **Server Status** tab.



Note ▪ The server name or host name is case sensitive. Use the same name as provided in your license file.

7. `lmtools` now displays the triad details in the familiar format, where the top node represents the complete triad address (port@host1,port@host2,port@host3) and the leaf nodes are the primary, secondary, and tertiary server.

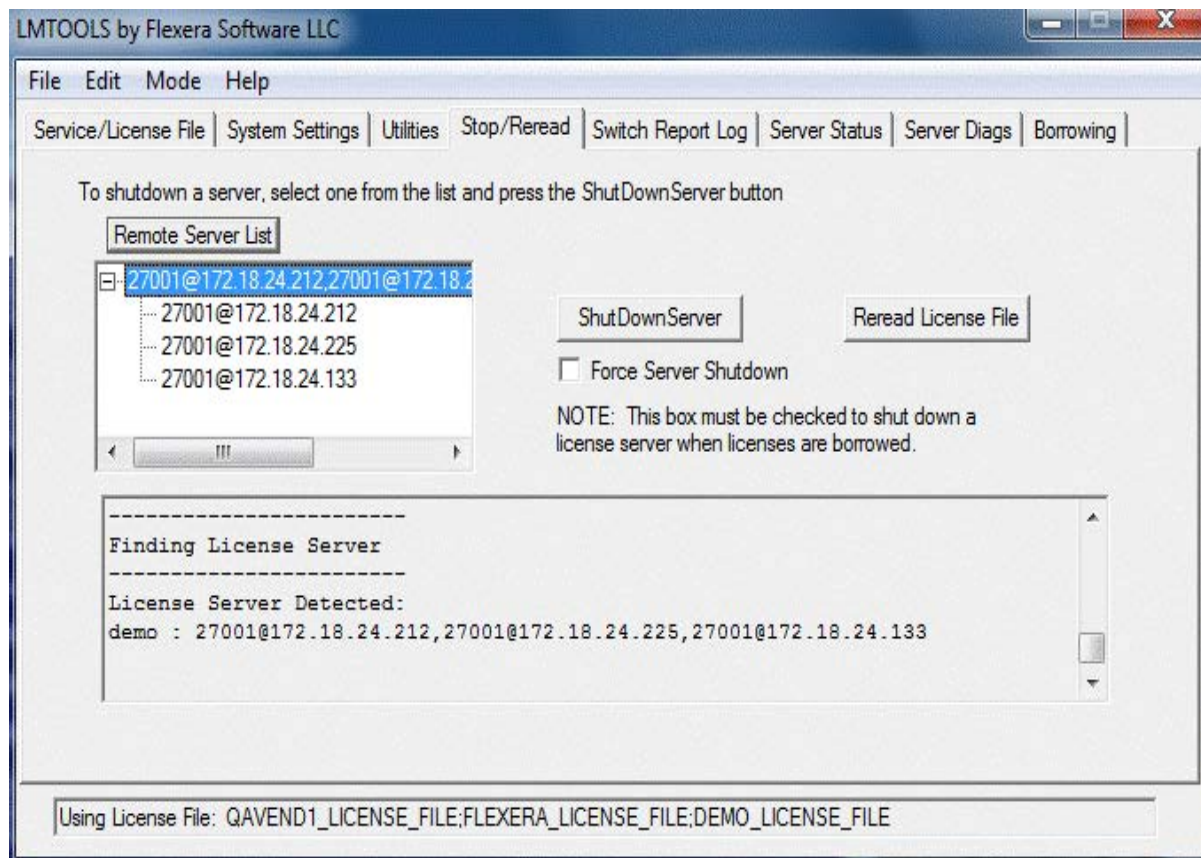


Figure 10-5: Stop/Reread Three server list and server details.

8. To shut down the triad, select the top node and click **ShutDown Server**.
9. To shut down a particular server in the triad, select the respective server and click **ShutDown**.

Maximum Client Connections to License Server

As already discussed, the license server consists of a license server manager (either `lmgrd` or `lmadmin`) and the vendor daemon. The license server manager is the gateway to the vendor daemon—it hands off client connections to the vendor daemon. The license server manager therefore has a smaller connection limit than the vendor daemon. `lmgrd` has a default simultaneous connection limit of approximately 1000 clients, which can be limited with the `LM_SERVER_HIGHEST_FD` environment variable.

The vendor daemon has a maximum connection limit approaching 30,000 jobs, although the best practice is to limit total connections to no more than 10,000 jobs in order to ensure sufficient license server performance. However, when using simple license models and powerful hardware for the license server, more than 10,000 jobs can be considered provided this is stress-tested by producers in advance.

For Linux and Unix systems, the ulimit for file descriptors should be set to be at least as large as the maximum number of job connections expected for the vendor daemon. For example, on Linux VMs, the ulimit for file descriptors is typically 1024, so this will need to be adjusted up, possibly as part of a license server startup script, or systemd or launchd configuration.

A typical client connection error when the maximum number of jobs to the vendor daemon has occurred is LM_CANTCONNECT (-15), and the debug log may contain this entry: “This license server system can handle no more concurrent clients since it is out of file descriptors”.

When the number of simultaneous client connections to lmgrd exceeds the limit (which is an indication that lmgrd is being overwhelmed with a spike of client connection requests), the client error is typically LM_CANTREAD (-16) and the debug log of the license server may contain this entry: “Warning: Maximum connections to the server has reached. Please disconnect some clients from the server”. Another indicator that lmgrd is being overwhelmed is a buildup of sockets in the ESTABLISHED and CLOSED_WAIT states on the license server—when client activity decreases, these socket states fall away as lmgrd stabilizes.

Migrating from lmgrd to lmadmin

A Fundamental Mode Change

The `lmadmin` license server manager combines all the functionality of the `lmgrd` license server manager with a Web-based, administrative interface. However, the `lmadmin` license server manager operates in some fundamentally different ways than the `lmgrd` license server manager.

The obvious change is that previous versions of the license server manager (`lmgrd`) used a command-line interface and the new license server manager (`lmadmin`) supports a browser-based client connection over HTTP. A more fundamental change in operation is that configuration options are now persistent—if you change settings and relaunch the tool, the previously set options stay in effect.

With `lmgrd`, the primary mode of operation is to run one instance of `lmgrd` for each vendor daemon where `lmgrd` obtains its configuration information from the command-line options used when the program is started, including the required specifying of a license file. To change settings you typically stop the license server, edit the license file and/or the script containing your command-line options, and relaunch the tool.

In contrast, the `lmadmin` license server manager is designed to:

- Support multiple vendor daemons with one `lmadmin` process.
- Launch without requiring any configuration options.
- Perform all server configuration and administration functions from the browser. (For special circumstances, the `lmutil` package provides additional functions.)
- Import existing license files (the new `lmadmin` license server manager is compatible with license files and vendor daemons produced using FlexNet Publisher 9.2 and later).
- Keep configuration options persistent.

Persistent configuration options is a significant change. Once set, settings remain in effect until changed. For example, if two vendor daemons are specified to use the same TCP port, only one will run. With `lmgrd`, this requires making changes to at least one of the license files as well as stopping and restarting the server. With `lmadmin`, you can change the TCP port for a vendor daemon while the license server is running. The manually specified port is then persistent and will remain as it was manually set the next time the license server is started, even if the license file is changed. The changes set in the license server manager override the license files.

Command Changes

Because of the changes in the fundamental operation of the system, many features have been redesigned. The following Imgrd command-line options are not supported by Imadmin.

Table 11-1 ■ Command-line options to Imgrd that are not supported by Imadmin.

Imgrd option	Imadmin notes
-2 -p	The replacement option is <code>-adminOnly yes</code> , which is the default for Imadmin.
-z	The replacement option is <code>-foreground</code> .
-c license_file_list	License files are now managed using either the license server management interface or the new <code>-import</code> option.
-v	Version information is now displayed in the license server management interface. The equivalent is <code>-version</code> option.
-l [+]debug_log_path	The <code>-logDir</code> option defines the path to the debug log file (Imadmin.log) written by the Imadmin license server manager. The default location is <code><Imadmin_install_dir>/logs</code>. The default location for the vendor daemon debug log files is <code><Imadmin_install_dir>/logs/<vendor>.log</code>. The path to vendor daemon debug logs can be changed via either the Imadmin license management interface or the options file.  Note ■ On Windows platforms, it is best practice to set the location of <code>debug_log_path</code> to a sub-folder of <code>ProgramData</code>. This location has user-write permissions by default, which are needed when the license server runs as a service with <code>LocalService</code> permission.
-local	The replacement is to use the following argument settings: <code>-allowStopServer Yes -allowRemoteStopServer No</code>
-x Imdown	The replacement option is <code>-allowStopServer</code> . Note that the logical direction of this option has been reversed.
-x Imremove	The replacement option is <code>-allowLicenseReclaim</code> . Note that the logical direction of this option has been reversed.

Imadmin License Administration Functions

Imadmin provides some of the license administration functions previously provided by the command-line based license administration utilities or Imtools on the Windows platform. The following table lists functions provided within Imadmin that replace those provided by the license administration utilities.

Table 11-2 • Imadmin License Administration Functions

Imadmin Function	Description	Replaces Utility
Dashboard - Licenses	Displays details of licenses rights available and in use.	Imstat
Vendor Daemon Configuration - Administer - Stop	Stops the vendor daemon.	Imdown (in some usage cases)
Vendor Daemon Configuration - Administer - Reread License Files	Rereads license rights from license files included in the Imadmin configuration. Required only when the content of a license file is updated.	Imreread (see also Changes in Imreread Behavior when Using Imadmin)
Vendor Daemon Configuration - Administer - Rotate Report Logs	Switches the report log to a new file name.	Imswitchr
Server Configuration - Stop Server	Stops the license server. Note that Imadmin's default setting enables this button. To disable the button, start Imadmin with the -allowStopServer No argument.	Imdown (in some usage cases)

The following table details which command-line utilities may no longer be required and which utilities are required when using Imadmin.

Table 11-3 • Imadmin Use of License Administration Utilities

Utility	Required when using Imadmin
Imborrow	Yes, if using license rights in license files and borrow capability.
Imdiag	Yes, to diagnose license checkout problems.
Imdown	Not normally required.
Imhostid	Not normally required for Imadmin as it displays information about the system it is running on that includes the various identities normally used as hostids. Required for determining the hostids of client systems.
Iminstall	Yes, converts license files between different formats.
Imlicvalidator	Not normally required.

Table 11-3 • Imadmin Use of License Administration Utilities

Utility	Required when using Imadmin
Imnewlog	Yes, if you use this function instead of <code>lmswitchr</code> to change to a new report log because you do not want to edit the report log file name in the options file.
Impath	Yes, allows users direct control over license file path settings.
Imremove (in License File-Based Licensing)	Yes, releases a hung license to the pool of free licenses. Note that <code>Imadmin</code> 's default setting disables the operation of <code>Imremove</code> . To enable <code>Imremove</code> , start <code>Imadmin</code> with the <code>-allowLicenseReclaim</code> argument.
Imreread	Not normally required. See also Changes in Imreread Behavior when Using Imadmin .
Imstat	Only required to show additional information (such as borrow or reservations).
Imswitch	Yes, controls debug log location and size.
Imswitchr	Not required.
Imver	Yes, reports the version of a library or binary file. Note that you can determine the version of <code>Imadmin</code> by starting it with the <code>-version</code> argument.

Changes in Imreread Behavior when Using Imadmin

Normally `Imreread` is not required when using `Imadmin`, however if you use `Imreread` with `Imadmin` the following use cases are not supported:

- **Using Imreread to restart a vendor daemon**—When using `Imgrd` you can shut down a vendor daemon using `lmdown` and then use the `Imreread` command to restart the vendor daemon. The following sequence of commands will result in an error when using `Imadmin`:


```
lmdown -vendor demo
Imreread -vendor demo
```
- **Using Imreread to load and start a new vendor daemon**—You can start `Imgrd` with a license file that specifies a vendor daemon and then replace this license file with one that includes information about a second vendor daemon. When `Imreread` is run, this second vendor daemon will be started. Using `Imreread` in this way with `Imadmin` will not load or start the vendor daemon. When using `Imadmin`, load and start a new vendor daemon as follows:
 1. To import a license file for the vendor daemon, go to the Administration page, open the Vendor Daemon Configuration tab, and click **Import License**.
 2. To start the vendor daemon, go to the Administration page, open the Vendor Daemon Configuration tab, and click **Administer** for the specific vendor daemon. Click **Start**.

Using License Administration Tools

License administration tools help license administrators manage licenses and license servers. Always use the latest version of the utilities. If you are using `lmadmin` as your license server manager, then it provides functionality that replace some of these utilities. The table, [License Administration Utilities](#), lists these utilities and indicates when `lmadmin` provides an alternative.

Command-Line Utilities

All license server utilities are packaged as a single executable called `lmutil`. The `lmutil` is either installed as individual commands (either by creating links to the individual command names, or making copies of `lmutil` as the individual command names), or as a wrapper that runs the individual command as `lmutil` command. For example, `lmutil lmstat` or `lmutil lmdown`.

On Windows systems, the `lmutil` command form of the commands are available. There is also a graphical user interface available for these commands—see [lmtools \(Windows only\)](#).

Table 12-1 • License Administration Utilities

Utility	Description	lmadmin Function
lmborrow	Supports license borrowing.	None
lmborrowl	Supports license borrowing when a hyphen is part of the feature name.	None
lmdiag	Diagnoses license checkout problems.	None
lmdown	Gracefully shuts down selected vendor daemons on the license server (or on all three systems in the case of three-server redundancy).	Vendor Daemon Configuration - Administer - Stop
lmhostid	Reports the hostid of a system.	System Information displays hostids for the license server.

Table 12-1 • License Administration Utilities

Utility	Description	Imadmin Function
Iminstall	Converts license files between different formats.	None
Imlicvalidator	Allows user to pre-validate a certificate license file (from a remote client) against a license server.	None
Imnewlog	Moves existing report log information to a new file name and starts a new report log file with existing file name.	None
Impath	Allows users direct control over license file path settings.	None
Imremove (in License File-Based Licensing)	Releases a hung license to the pool of free licenses.	None. Note Imadmin default setting disables Imremove.
Imremove (in Trusted Storage-Based Licensing)	Releases a hung license to the pool of free licenses in trusted storage.	None
Imreread	Causes the license daemon to reread the license file and start any new vendor daemons.	Vendor Daemon Configuration - Administer - Reread License Files
Imstat	Displays the status of a license server.	Dashboard - Licenses
Imswitch	Controls debug log location and size.	None
Imswitchr	Switches the report log to a new file name.	Vendor Daemon Configuration - Administer - Rotate Report Logs
Imtpminfo	Returns the status of the TPM (Trusted Platform Module). (Windows only)	None
Imtransfer	Supports license transferring	None
Imver	Reports the version of a library or binary file.	None
Imvminfo	Returns the environment of the system used as to virtual or not.	None

History

- The Imlicvalidator utility is introduced in the version 11.17.0 utilities.
- The Imborrowl utility was introduced in the version 11.15.1 utilities.

- The `lmtpminfo` utility was introduced in the version 11.15.0 utilities.
- The `lmvminfo` utility was introduced in the version 11.13.0 utilities.
- The `lmpath` utility was introduced in the version 7.0 utilities.
- The `lmborrow` utility was introduced in the version 8.0 utilities.
- The `lmswitch` utility was introduced in version 8.0 vendor daemon.
- The `lmswitchr` utility was introduced in version 5.0 vendor daemon.

Other Important Utilities

Other important utility is listed in the following table:

Table 12-2 • Other Important Utilities

Table 12-3 •

Utility	Description	ladmin Function
lmobfslog	Processes the debug log file to generate the obfuscated usernames.	None.

Common Arguments for lmutil

The following are valid arguments for most `lmutil` utilities:

Table 12-4 • lmutil Valid Arguments

Argument	Description
-c <i>license_file_path</i>	Most <code>lmutil</code> utilities need to know the path to the license file. This is specified with a <code>-c license_file_path</code> argument, or by setting the <code>LM_LICENSE_FILE</code> environment variable. Otherwise, the default location is used. The utilities also honor all <code>VENDOR_LICENSE_FILE</code> environment variables. Some utilities take more than one license file path in a license search path separated by colons on UNIX and semicolons on Windows. Pathnames that include spaces must be enclosed in double quotes.
-help	Displays usage information and exits.
-v	Displays the version of the utility and exits.
-verbose	Displays longer description for all errors found.



Note • The following information might be helpful:

- `VENDOR_LICENSE_FILE` environment variable is honored in utilities starting with version 7.0.
- The `-verbose` option was introduced in version 6.0 of the utilities.

Imborrow

The `lmborrow` utility supports the borrowing of those licenses that contain the `BORROW` attribute. Use `lmborrow` when clients have `LM_A_BORROW_RETURN_BY_VERSION=1` set (the default value).

This utility runs on the FlexEnabled client machine that borrows the licenses. The FlexEnabled application end users can use the utility to do the following:

- Initiate borrowing by setting the borrow period
- Clear the borrow period
- Purge expired licenses
- Determine borrow status
- Return a borrowed license early

Initiating Borrowing

To initiate borrowing, the end user of the FlexEnabled application sets the borrow period by running `lmborrow` from the command line or through `lmtools` on the FlexEnabled client machine.



Task

To initiate borrowing

From the FlexEnabled client machine, enter the following command:

```
lmborrow {vendor | all} enddate [time]
```

where:

Table 12-5 ■ `lmborrow` Arguments for Initiating Borrowing

Argument	Description
<code>vendor</code>	The vendor daemon name that serves the licenses to be borrowed, or <code>all</code> specifies all vendor daemons in that license server.
<code>enddate</code> [time]	Date the license is to be returned in <code>dd-mmm-yyyy</code> format. <code>time</code> is optional and is specified in 24-hour format (<code>hh:mm</code>) in the FlexEnabled application's local time. If <code>time</code> is unspecified, the checkout lasts until the end of the given end date.

For example, you might enter the following:

```
lmborrow sampled 20-aug-2017 13:00
```

This has the effect of setting `LM_BORROW` with the borrow period in either the registry (Windows) or in `$HOME/.flexlmborrow` (UNIX).

To borrow licenses for the desired vendor name, *on the same day and the same system* that the user runs `lmborrow`, run the applications to check out the licenses. If you run the applications more than once that day, no duplicate licenses are borrowed. No licenses are borrowed if the application is run on a day different than the date borrowing is initiated.

Aside from the `lmborrow` utility, you can use these other methods to initiate borrowing:

- Use the borrowing interface included in the FlexEnabled application, if such an interface is provided.
- Set the `LM_BORROW` environment variable directly.

See [Initiating License Borrowing](#) for more information on these other ways.

Clearing the Borrowed License Setting

End users of the FlexEnabled application can clear the `LM_BORROW` setting to stop licenses from being borrowed until borrowing is initiated again. For example, users might run `lmborrow -clear` after they have borrowed licenses for features that are used offline if—before disconnecting from the network—they want to run an application that checks out additional features, served by that *vendor name*, that are not meant to be borrowed. Clearing `LM_BORROW` does *not* change the status for already borrowed licenses.



Task To clear the `LM_BORROW` setting in the Windows registry or in `$HOME/.flexlmborrow`

From the FlexEnabled client machine, enter the following command:

```
lmborrow -clear
```

Purging Expired Licenses

End users can remove expired borrowed licenses from their FlexEnabled client machine.



Task To remove expired borrowed licenses

From the FlexEnabled client machine, enter the following command:

```
lmborrow -purge
```



Tip ▪ Combine this option with the `-status` option both to purge expired borrowed licenses from the client machine and to display information about the still valid borrowed licenses. See the next section [Determining Borrowed-License Status](#).

Determining Borrowed-License Status

End users can use the following commands to display the status of licenses (vendor, feature name, and expiration date) currently borrowed by their FlexEnabled application.



Note ▪ The current status of borrowed licenses is read from information stored on the client machine itself. Therefore, the client machine does not need to be connected to the network to obtain this information.



Task **To display the status of all currently borrowed features, expired or not**

From the FlexEnabled client machine, enter the following command:

```
lmborrow -status
```



Task **To purge expired borrowed licenses and display the status of non-expired borrowed licenses**

From the FlexEnabled client machine, enter the following command:

```
lmborrow -purge -status
```



Note ▪ You can specify the `-purge` and `-status` options in any order. The result is always the same--expired borrowed licenses are removed and the status of the remaining borrowed licenses is displayed.

Returning a Borrowed License Early



Task **To return a borrowed license early**

1. Reconnect the FlexEnabled client to the network.
2. From the FlexEnabled client machine, issue the following command:

```
lmborrow -return [-c licfile] [-d display] [-u username] [-h hostname] [-fqdn] [-vendor name]  
feature [-bv version]
```

where:

Table 12-6 ▪ lmborrow Arguments for Returning a Borrowed License Early

Argument	Description
-fqdn	Directs lmborrow to access the borrowing system using its fully qualified host name. Use this option if the license was borrowed based on the fully qualified host name, rather than the relative distinguished name. Use lmstat to determine the format of the host name used when the license was borrowed.
-c licfile	Use the specified license files. In some configurations, the license file needs to be specified in order to return the borrowed license early.

Table 12-6 ▪ `lmborrow` Arguments for Returning a Borrowed License Early

Argument	Description
-d <i>display</i>	Used to specify the display from which the borrow was initiated. Required if your current display is different than what was used to initiate the borrow. On Windows, it is the system name or, in the case of a terminal server environment, the terminal server client name. On UNIX, it is in the form <code>/dev/ttyxx</code> or the X-Display name.
-u <i>username</i>	Used to specify the user name from which the borrow was initiated. In case of overridden user name use -u option to return the borrowed licenses early with the specified user.
-h <i>hostname</i>	Used to specify the host name from which the borrow was initiated. In case of overridden host name use -h option to return the borrowed licenses early with the specified hostname.
<i>feature</i>	The name of the borrowed feature to be returned early. Use <code>lmborrow -status</code> to get a list of borrowed feature names. <code>lmborrow</code> will return an error if the name of the feature contains a hyphen. Instead, use <code>lmborrowl</code> to return borrowed features with a hyphen in their name.
-bv <i>version</i>	Used to specify the version of the borrowed feature to return early. Use <code>lmborrow -status</code> to get the list of borrowed feature versions.

If the borrowing system is not placed back on the network before attempting the early return, the license is not returned and `LM_BORROW` is kept intact. Additionally, an error message is issued to the user with notification that the system needs to be connected to the network.



Note ▪ Early borrowed license return was introduced in version 8.3 utilities.

lmborrowl

`lmborrowl` should be used instead of `lmborrow` when borrowing clients specify `LM_A_BORROW_RETURN_BY_VERSION=0`. This non-default behaviour is needed for legacy licensing models where a hyphen was used in the feature name.

`lmborrowl` does not allow the version to be specified when returning a feature, unlike `lmborrow`.

Only deploy this utility to end-users if your feature names historically contain hyphens. Revenera recommends against using hyphens in feature names.

See Also

[lmborrow](#)

`LM_A_BORROW_RETURN_BY_VERSION` in the *C/C++ Function Reference*

lmdiag

lmdiag enables you to diagnose problems when you cannot check out a license.

Usage

lmdiag [-c *license_file_list*] [-n] [*feature[:keyword=value]*]

where:

Table 12-7 • lmdiag Argument Usage

Argument	Description
-c <i>license_file_list</i>	Diagnose the specified files.
-n	Run in non-interactive mode; lmdiag does not prompt for any input in this mode. In this mode, extended connection diagnostics are not available.
<i>feature</i>	Diagnose this feature only.
<i>keyword=value</i>	If a license file contains multiple lines for a particular feature, select a particular line for lmdiag to report on. For example: lmdiag f1:HOSTID=12345678 attempts a checkout on the line with the hostid “12345678.” <i>keyword</i> is one of the following: VERSION, HOSTID, EXPDATE, KEY, VENDOR_STRING, ISSUER

If no feature is specified, lmdiag operates on all features in the license files in your list. lmdiag first prints information about the license, then attempts to check out each license. If the checkout succeeds, lmdiag indicates this. If the checkout fails, lmdiag gives you the reason for the failure. If the checkout fails because lmdiag cannot connect to the license server, then you have the option of running extended connection diagnostics.

These extended diagnostics attempt to connect to each TCP/IP port on the license server, and detects if the port number in the license file is incorrect. lmdiag indicates each TCP/IP port number that is listening, and if it is an lmadmin or lmgrd process, lmdiag indicates this as well. If lmdiag finds the vendor daemon for the feature being tested, then it indicates the correct port number for the license file to correct the problem.

See Also
[FLEXLM_DIAGNOSTICS](#)

lmdown

The lmdown utility allows for the graceful shutdown of selected license daemons on all systems.

Usage

```
lmdown -c license_file_list [-vendor vendor_daemon] [-q] [-all] [-force]
```

where:

Table 12-8 ■ lmdown Argument Usage

Argument	Description
-c <i>license_file_list</i>	Use the specified license files. Note that specifying -c license_file_list is always recommended with lmdown .
-vendor <i>vendor_daemon</i>	Shut down only this vendor daemon. The license server continues running.
-q	Don't prompt. Otherwise, lmdown provides a confirmation prompt.
-all	If multiple servers are specified, automatically shuts down all of them. -q is implied with -all .
-force	If licenses are borrowed, lmdown runs only from the system where the license server is running, and then only if the user adds -force .

If **lmdown** encounters more than one server (for example if **-c** specifies a directory with many *.lic files) and **-all** is not specified, a choice of license servers to shut down is presented.



Note ■ On UNIX, do not use **kill -9** to shut down license servers. On Windows, if you must use the Task Manager to kill the FlexNet Licensing Service, be sure to end the **lmadmin** (or **lmgrd**) process first, then all the vendor daemon processes.

When using the **lmdown** utility to shut down license servers configured for three-server redundancy, there is a one-minute delay. You can protect the unauthorized execution of **lmdown** when you start up the license server manager.

See Also

[Installing lmadmin](#)

[lmgrd Command-Line Syntax](#) for details about securing access to **lmdown**

lmhostid

The **lmhostid** utility returns the hostid of the current platform. Invoked without any arguments, **lmhostid** displays the default hostid type for the current platform. Otherwise, the hostid corresponding to the requested *type* is displayed, if supported on the current platform.

Obtaining any TPM, virtualization, or cloud-licensing hostid requires that the FlexNet Licensing Service be installed.

Usage

```
lmhostid [-n] [-utf8] [-ptype argument] [hostid_type] [-container_id] [-flexid -dtype]
```

Where:

Table 12-9 • lmhostid Argument Usage

Argument	Description
-n	Only the hostid, itself, is returned as a string, which is appropriate to use with HOSTID= in the license file. Header text is suppressed.
hostid_type	One of the following hostid types. If not specified, the default hostid for the current platform is displayed. See Hostids for Supported Platforms for a list of the default types. PLATFORM-DEPENDENT HOSTIDS -ether—Ethernet address.  Note • In Windows, Linux, and OS X platforms, this command returns all available addresses (active and inactive). For Windows and Linux platforms, these addresses include both teamed and non-teamed Ethernet interfaces. In general for teamed interfaces, the unique address for the team-bonding virtual adapter is returned. On Linux platforms, a separate (but same) virtual-adapter address is returned for each interface in the team. On Windows platforms, a single instance of the virtual-adapter address is returned for all teamed interfaces. -string—String id. -vsn—Volume serial number (Windows platforms only). -flexid— USB FLEXID identification. This is applicable only for those platforms that support FlexNet ID dongles. See Obtaining System Hostids for a complete list. -long—32-bit hostid. -uuid—UUID binding value. Used with the -ptype command line option, shown under -ptype below. Only permitted with platform types (ptype) VM or AMZN. -tpm_id1—TPM (Trusted Platform Module) ID. Requirements and limitations: - Windows platforms only. - Supported version: TPM 2.0. - FlexNet Licensing Service must be running to obtain this hostid. - Only supported on the SERVER line. - Supported as a served node-locked hostid from FlexNet Publisher version 11.15.0 onwards.

Table 12-9 • lmhostid Argument Usage

Argument	Description
hostid_type (continued)	PLATFORM-INDEPENDENT HOSTIDS • -user—Current user name. Note that user names that contain spaces, for example ‘test user’ cannot be used in the Options file. Use the first word of the user name, for example ‘test’, in the Options file. • -display—Current display name. On Windows, it is the system name or, in the case of a terminal server environment, the terminal server client name. On UNIX, it is in the form /dev/ttyxx or the X-Display name. • -hostname—Current host name. • -hostdomain—Current host domain. • -internet—IP address of the machine in the format of the machine’s protocol (IPv4 or IPv6). In a dual-stack environment, you can use the -v4 or -v6 switch to obtain the version-specific IP address (see the next bulleted items). • -v4—Use with -internet to obtain the IPv4 address. • -v6—Use with -internet to obtain the IPv6 address.  Tip ▪ Both -v4 and -v6 are optional; using just -internet fetches the default IP address (usually IPv4).  Tip ▪ (Windows only) If a Windows machine is connected to a VPN, lmhostid -internet returns the virtual adapter IP address. To ensure that this command returns the physical adapter IP address during checkouts, perform this workaround: Run regedit, navigate to HKEY_LOCAL_MACHINE\SYSTEM\CurrentControlSet\services\Tcpip\Linkage, locate the Bind parameter, and edit the adapter order so that the physical adapter is listed first. Run ipconfig/registerdns to put the edit into effect.
-utf8	The hostid is output as a UTF-8 encoded string rather than an ASCII string. If your hostid contains characters other than ASCII A through Z, a through z, or 0 through 9, use this option with lmhostid. To view a correct representation of the resulting hostid, use a utility, such as Notepad, that can display UTF-8 encoded strings.

Table 12-9 • Imhostid Argument Usage

Argument	Description
-ptype	(License-server binding in virtual environments only) Indicates the platform type. This option must be used in conjunction with a valid hostid binding type (see hostid_type (continued)). -ptype requires one of the following arguments, which then prefixes the generated hostid appropriately (as described): - VM—Supported for Xen, VMware, Oracle VirtualBox, Hyper-V, Parallels, QEMU-KVM, Google Compute, Microsoft Azure, everRun, Nutanix, and Amazon EC2. Imhostid -ptype VM -uuid is now the only supported usage of this option, generating VM_UUID. - AMZN—For the Elastic IP address, AMI instance ID, or AMI template ID in the Amazon EC2 environment. Prefixes hostid with AMZN_. VM_UUID is recommended instead of AMZN_IID. Both represent the AMI instance ID in the Amazon EC2 environment. - PHY—Physical machine only. Prefixes hostid with PHY_.
-container_id	Extracts Docker ContainerID.
-flexid -dtype	Displays the flexid and type information of the dongle. The -flexid fetches the flexid and -dtype the type information. -dtype is an optional parameter. The following is an example for output from Imhostid: Imhostid - Copyright (c) 1989-2021 Flexera. All Rights Reserved. The FlexNet host ID of this machine is "FLEXID=9-3ebc66c0:Type=HASPHL,Pro,Pro" Here in the above output, "FLEXID=9-3ebc66c0:Type=HASPHL,Pro,Pro": - FLEXID=9-3ebc66c0 represents the flexid of the dongle. - Type=HASPHL,Pro,Pro represents the dongle type information. - Type=Value consists of three parts. They are configuration, key_model, and key_type information, such as Type=configuration,key_model,key_type. - The delimiter between the flexid and type is colon(:) and between Type parameters is comma(,). Limitation: - SafeNet dongle only. - Not tested for hasp4 dongles since FlexNet Publisher 11.17.2.

The following are examples of output from Imhostid:

lmhostid - Copyright (c) 1989-2021 Flexera Software LLC. All Rights Reserved.

The FlexNet host ID of this machine is ""00ff5018c189 0019d244e9fc 0016cdfaf65d 001558809422 005056c00001 005056c00008""

Only use ONE from the list of hostids.

lmhostid -ptype VM -uuid

lmhostid - Copyright (c) 1989-2021 Flexera Software LLC. All Rights Reserved.

The FlexNet host ID of this machine is "VM_UUID=0011223344556677889911bbccddeeff"

To get Docker container ID:

lmhostid.exe -container_id

lmhostid - Copyright (c) 1989-2021 Flexera. All Rights Reserved.

The FlexNet host ID of this machine is "CONTAINER_ID=FF93BE61850C"



Note - Some limitations exist with hostid reporting on virtual devices:

- **Ethernet hostids on Windows platforms**—From lmutil version 11.6.1 onwards, only the hostids of physical ethernet adapters are reported. Devices identified as virtual ethernet adapters are not reported as these identities are not permanent.
- **Physical (bare metal) hostids on virtual machines**—When run from a virtual machine, lmhostid cannot return hostids for the physical machine that hosts the virtual machine. To obtain hostids for the physical machine, lmhostid must be run from the host OS.

See Also

[Hostids for Supported Platforms](#)

lminstall

The lminstall utility is designed primarily for typing in decimal format licenses to generate a readable format license file.

Usage

```
lminstall [-i in_lic_file] [-maxlen n] [-e err_file] [-o out_lic_file] \
          [-overfmt {2 | 3 | 4 | 5 | 5.1 | 6 | 7 | 7.1 | 8}] [-odecimal]
```

Normally, to convert from decimal to readable format, lminstall is used with no arguments; you are prompted for the name of the output license file. The default file name is today's date in `yyyymmdd.lic` format. Move this file to the application's default license file directory, if specified by the software publisher. Otherwise, use the `LM_LICENSE_FILE` or `VENDOR_LICENSE_FILE` environment variables to specify the directory where the `*.lic` files are located.

To finish entering, type `q` on a line by itself or enter two blank lines.

When an input file is specified with no output file specified, output goes to stdout; if neither input nor output file is specified, lminstall assumes that input comes from stdin and prompts the user for an output file name.

lminstall is also used to convert licenses from readable to decimal format and between different license versions.

To convert from readable to decimal:

```
lminstall -i in_lic_file -o out_lic_file -odecimal
```

To convert to v5.1 format:

```
lminstall -i in_lic_file -o out_lic_file -overfmt 5.1
```

To enforce a maximum line length of, for example, 50 characters:

```
lminstall -maxlen 50
```

Conversion errors are reported as necessary and can be written to a file by specifying `-e err_file`. `lminstall` has a limit of 1,000 lines of input.



Note - Decimal format is deprecated and is not supported.

Imlicvalidator

The `Imlicvalidator` helps you to pre-validate a certificate license file (from a remote client) against a license server. It allows you to know in advance whether the license certificate works or not when it is applied on the license server. The utility requires a license server and a vendor daemon against which the certificate license file is to be validated. The utility performs the range of validations of the certificate presented starting with basic encoding checks, extension checks, keyword validations and syntactic, and correctness of the license certificate. It reports errors if found or print a validation complete message when the validation is complete.

Usage

```
lmlcvalidator [-licfile license_file] [-licserv port@host] -repAllErr [-t timeout_value] -help
```

where:

Table 12-10 - Imlicvalidator Argument Usage

Argument	Description
<code>-licfile license_file</code>	Path to license certificate file that needs to be validated.
<code>-licserv port@host</code>	port@host where port and host are the TCP/IP port number and host name of license server against which the certificate needs to be validated.
<code>-repAllErr</code>	[Optional] By default utility will print only first 10 errors in certificate. When this argument is set it will display all the errors in the certificate.
<code>-t timeout_value</code>	[Optional] By default it's set to 4 seconds. When this argument can be followed by a timeout value in seconds.
<code>-help</code>	To print usage information.



Note ▪ The following information might be helpful:

- The *Imlicvalidator* utility is only available as part of platform directories *x64_n6*, *i86_n3*, *x64_linux* and *i86_linux*.
- The *Imlicvalidator* utility is so far tested and certified to work against license server running on, Windows, Suse, RHEL and CentOS.

The following table shows the keywords and syntax validation that *Imlicvalidator* supports and does not support:

Table 12-11 ▪ List of keywords and syntax

Table 12-12 ▪

Keyword / Syntax	Yes No	Notes
# Comments	Yes	
Common Vendor Daemon	No	The common vendor daemon related checks are not present in the license validation utility.
Encoding checks	Yes	Client side validation.
Expired Lines	Yes	
Feature Line	Yes	
File size restriction (1 MB)	Yes	Client and Server side validation.
Future dated LinesHost validation in server line	Yes	
Host-Id validation in server line	Yes	
Increment Line	Yes	
License file extension	Yes	Client side validation.
License file presence	Yes	Client side validation.
License Server status checks	Yes	
Line continuation	Yes	
Line start with valid keyword	Yes	

Table 12-12 •

Keyword / Syntax	Yes No	Notes
<i>Lines with permanent keyword</i>	Yes	
<i>Lines with signature mismatch</i>	Yes	
<i>Lines with uncounted keyword</i>	Yes	
<i>PACKAGE keyword</i>	No	The package line validation is not present in the license validation utility.
<i>Server line presence</i>	Yes	
<i>SERVER line with, [HEARTBEAT_INTERVAL=seconds]</i>	Yes	
<i>SERVER line with, [port]</i>	Yes	
<i>SERVER line with, [PRIMARY_IS_MASTER]</i>	Yes	
<i>Upgrade line</i>	Yes	
<i>USE_SERVER keyword</i>	Yes	
<i>Vendor daemon available checks</i>	Yes	
<i>Vendor line presence</i>	Yes	
<i>Vendor port range validation</i>	Yes	
<i>VENDOR/DAEMON line with, options_file_path</i>	Yes	
<i>VENDOR/DAEMON line with, port</i>	Yes	
<i>VENDOR/DAEMON line with, vendor_daemon_path</i>	Yes	

Imnewlog

The `lmnewlog` utility switches the report log file by moving the existing report log information to a new file, then starting a new report log with the original report log file name. If you rotate report logs with `lmnewlog` instead of `lmswitchr`, you do not have to change the file name in the `REPORTLOG` line of the vendor daemon's option file. Requires a version 7.1 or later vendor daemon.

Usage

```
lmnewlog [-c license_file_list] feature renamed_report_log [-secondary]
```

or:

```
lmnewlog [-c license_file_list] vendor renamed_report_log [-secondary]
```

where:

Table 12-13 • Imnewlog Argument Usage

Argument	Description
<code>-c license_file_list</code>	Use the specified license files.
<code>feature</code>	Any feature in this license file.
<code>vendor</code>	Name of the vendor daemon in this license file.
<code>renamed_report_log</code>	New file path where existing report log information is to be moved.
<code>-secondary</code>	A new report log file is generated on a secondary server. The <code>-secondary</code> option switches the report log file by moving the existing report log information to a new file, then starting a new report log with the original report log file name at the secondary server for 3-server setup.

Impath

The `lmpath` utility allows direct control over license search path settings. It is used to add to, override, or get the current license search path settings.

Usage

```
lmpath {-add | -override | -status} {vendor | all} license_file_list
```

where:

Table 12-14 • Impath Argument Usage

Argument	Description
<code>-add</code>	Prepends <code>license_file_list</code> to the current license search path or creates the license search path, if it doesn't exist, initializing it to <code>license_file_list</code> . Duplicates are discarded.

Table 12-14 • Impath Argument Usage

Argument	Description
-override	Overrides the existing license search path with <code>license_file_list</code> . If <code>license_file_list</code> is the null string, "", the specified list is deleted. ● <code>Impath -override all " "</code>—Deletes the value of <code>LM_LICENSE_FILE</code>. ● <code>Impath -override vendor " "</code>—Deletes the value of <code>VENDOR_LICENSE_FILE</code>.
-status	Displays current license search path settings.
<i>vendor</i>	A vendor daemon name. Affects the value of <code>VENDOR_LICENSE_FILE</code> .
all	Refers to all vendor daemons. Affects the value of <code>LM_LICENSE_FILE</code> .
<i>license_file_list</i>	On UNIX, separate values with a colon. On Windows, separate values with a semicolon. If <code>license_file_list</code> is the null string, " ", then the specified entry is deleted.

Impath accesses the license search path from the following location:

- Windows registry: `HKEY_CURRENT_USER\Software\FLEXlm License Manager`
- UNIX file path: `$HOME/.flexlmrc`



Task **To display the current license search path settings:**

Use the following command:

`Impath -status`

The following is displayed:

Impath - Copyright (c) 1989-2021 Flexera Software LLC.
Known Vendors:

demo: ./counted.lic:./uncounted.lic

Other Vendors:

/usr/local/flexlm/licenses/license.lic

Note that where the path is set to a directory, all the *.lic files are listed separately.

Imremove (in License File–Based Licensing)

The `lmremove` utility enables you to remove a single user's license for a specified feature. If the application is active, it rechecks out the license shortly after it is freed by `lmremove`. Note that `lmadmin`'s default setting disables `lmremove`. To enable `lmremove`, start `lmadmin` with the `-allowLicenseReclaim` argument.

Usage

```
lmremove [-c license_file_list] feature user user_host display
```

or

```
lmremove [-c license_file_list] -h feature server_host port handle
```

Table 12-15 ■ lmremove in License File-Based Licensing Argument Usage

Argument	Description
-c <i>license_file_list</i>	Specify license files.
-h	Specify the license by its handle. Include <i>feature</i> , <i>server_host</i> , <i>port</i> , and <i>handle</i> with this argument.
<i>feature</i>	Name of the feature or component checked out by the user whose license is to be removed.
	
	Important ■ Do specify a package name instead of a feature name.
<i>user</i>	Name of the user whose license you are removing, as reported by <code>lmstat -a</code> .
<i>user_host</i>	Name of the host the user is logged into, as reported by <code>lmstat -a</code> .
<i>display</i>	Name of the display where the user is working, as reported by <code>lmstat -a</code> .
<i>server_host</i>	Name of the host on which the license server is running.
<i>port</i>	TCP/IP port number where the license server is running, as reported by <code>lmstat -a</code> .
<i>handle</i>	License handle, as reported by <code>lmstat -a</code> .

The *user*, *user_host*, *display*, *server_host*, *port*, and *handle* information must be obtained from the output of `lmstat -a`.

`lmremove` removes all instances of *user* on *user_host* and *display* from usage of *feature*. If the optional `-c license_file_list` is specified, the indicated files are used as the license file.

The `-h` variation uses the *server_host*, *port*, and license *handle*, as reported by `lmstat -a`. Consider this example `lmstat -a` output:

```
joe nirvana /dev/tty5 (v1.000) (cloud9/7654 102), start Fri 10/29 18:40
```

In this example, the user is **joe**, the user host is **nirvana**, the display is **/dev/tty5**, the server host is **cloud9**, the TCP/IP port is **7654**, and the license handle is **102**.

To remove this license, issue one of the following commands:

```
lmremove f1 joe nirvana /dev/tty5
```

or

```
lmremove -h f1 cloud9 7654 102
```

When removing by handle, if licenses are grouped as duplicates, all duplicate licenses are also removed. If license lingering is set and `lmremove` is used to reclaim the license, `lmremove` starts, but does not override, the license's linger time.

You can protect the unauthorized execution of `lmremove` when you start up `lmgrd`. The default for `lmadmin` is to disable `lmremove` because removing a user's license is disruptive.

See Also

[Installing lmadmin](#)
[lmgrd Command-Line Syntax](#) for details about securing access to `lmremove`

lmremove (in Trusted Storage–Based Licensing)

The license administrator can use the `lmremove` utility to perform trusted storage borrow reclaim actions when these have been enabled by publisher. Note that `lmadmin`'s default setting disables `lmremove`. To enable `lmremove`, start `lmadmin` with the `-allowLicenseReclaim` argument.

Usage

Concurrent

```
lmremove [-c license_file_list] feature user user_host display
```

Activate or Borrow

```
lmremove <-c licfile> [-tsborrow <client_hostname>] | [-tsborrowstat]
```

where:

Table 12-16 ▪ `lmremove` in Trusted Storage Argument Usage

Argument	Description
<code>-c</code>	Specifies both license file and port@host.
<code>-tsborrow</code>	Specifies trusted storage borrow reclaim, using this option along with client host name is mandatory.
<i>client host name</i>	The deduction record(s) associated with the hostname that could be reclaimed.
<i>tsborrowstat</i>	Specifies trusted storage borrow reclaim status. When this option is used, no borrow reclaim operation would be performed and borrow reclaim status would be displayed with the following information: ● Number of licenses borrowed currently ● Percentage borrow reclaim level ● Count of reclaimed licenses ● Number of borrow reclaims currently available

The `lmremove` utility allows the license administrator to perform borrow reclaim action by removing the deduction records from server trusted storage for the specified client host.

The publisher can enable this feature by setting appropriate values for the borrow reclaim percentage. By default, the reclaim percentage is set to 0 so the reclaim operation is not allowed.

The licenses activated or borrowed from the activatable and hybrid license group can be reclaimed.

The Concurrent feature served out from a TS location can also be reclaimed.

To be able to use the borrow reclaim feature, the `lmremove` utility should run on the same system as license server.

For example,

- borrow reclaim percentage is 10%
- total license count is 25
- client "nirvana" borrowed a license from server trusted storage
- client "cloud9" activated 2 licenses from server trusted storage

To view the borrow reclaim status on the license server, issue one of the following commands:

```
lmremove -c port@host -tsborrowstat
```

or

```
lmremove -c counted.lic -tsborrowstat
```

Output similar to following is displayed:

```
=== ACTIVATION BORROW RECLAIM STATUS ===
Number of licenses borrowed currently: 24
Percentage borrow reclaim level: 10%
Count of reclaimed licenses:0
Number of borrow reclaims currently available: 2
```

Number of borrow reclaims currently available = ((total number of deduction records * borrow reclaim percentage) – (count of reclaimed licenses) = (24 * 10%) – (0) = 2.4 rounded to 2.

Assume, the client host "nirvana" is lost or damaged.

To reclaim the borrow licenses issued for client host "nirvana", issue one of the following commands:

```
lmremove -c port@host -tsborrow "nirvana"
```

or

```
lmremove -c counted.lic -tsborrow "nirvana"
```

Output similar to the following is displayed:

```
Lmremove: Sucessfully reclaimed (1) borrowed licenses(s) for the client "nirvana".
```

Now, view the borrow reclaim status, issue one of the following the commands:

```
lmremove -c port@host -tsborrowstat
```

or

```
lmremove -c counted.lic -tsborrowstat
```

Output similar to following is displayed:

```
=== ACTIVATION BORROW RECLAIM STATUS ===  
Number of licenses borrowed currently: 23  
Percentage borrow reclaim level: 10%  
Count of reclaimed licenses:1  
Number of borrow reclaims currently available: 1
```

Notice that, the count of reclaimed licenses is incremented by the number of deduction records that are reclaimed.



Note - When a reclaimed deduction record expires, the “Count of reclaimed licenses” is decremented by the one.

Assume, the client host “cloud9” is lost or damaged. To reclaim the borrow licenses issued for client host “cloud9”, issue one of the following commands,

```
lmremove -c port@host -tsborrow "cloud9"
```

or

```
lmremove -c counted.lic -tsborrow "cloud9"
```

Output similar to following is displayed:

```
lmremove: Failed to perform activation borrow reclaim operation. (-205, 14100)  
Failure reason: The requested activation borrow reclaim count (2) for client "cloud9" is greater than  
available reclaim count (1).
```

Assume, two more licenses are activated or borrowed. So, the number of deducted records is 25. Now, to reclaim the licenses issued for client host “cloud9” issue one of the following commands,

```
lmremove -c port@host -tsborrowstat
```

or

```
lmremove -c counted.lic -tsborrowstat
```

Output similar to following is displayed:

```
Lmremove: Successfully reclaimed (2) borrowed licenses for the client "cloud9".
```

Now, to view the borrow reclaim status, issue one of the following commands,

```
lmremove -c port@host -tsborrowstat
```

or

```
lmremove -c counted.lic -tsborrowstat
```

Output similar to following is displayed:

```
=== ACTIVATION BORROW RECLAIM STATUS ===  
Number of licenses borrowed currently: 23  
Percentage borrow reclaim level: 10%
```


Count of reclaimed licenses: 3

Number of borrow reclaims currently available: 0

Now the borrowed reclaim level of 10% is reached so no more reclaims are allowed. To reclaim any more licenses, the license administrator creates an offline request to reset the reclaimed license count.

To reset the reclaimed license count, issue the following command and send the request.xml to publisher.

```
servercomptranutil -new request.xml reference=BorrowClaimReclaim
```

To process the reset response from publisher, issue the following command,

```
servercomptranutil -process response.xml
```

Now, view the borrow reclaim status, issue one of the following commands,

```
lmremove -c port@host -tsborrowstat
```

or

```
lmremove -c counted.lic -tsborrowstat
```

Output similar to following is displayed:

```
=== ACTIVATION BORROW RECLAIM STATUS ===
Number of licenses borrowed currently: 23
Percentage borrow reclaim level: 10%
Count of reclaimed licenses: 0
Number of borrow reclaims currently available: 2
```

Notice that, the count of reclaimed licenses is reset to 0 and the number of borrow reclaims currently available are two.

See Also

[Installing lmadmin](#)

[lmgrd Command-Line Syntax](#) for details about securing access to lmread

lmreread

The following description refers to the operation of lmreread with lmgrd.



Note • lmadmin includes functionality to read license files and start vendor daemons instead of using lmreread.

The lmreread utility causes the license server manager to reread the license file and start any new vendor daemons that have been added. In addition, all currently running vendor daemons are signaled to reread the license file and their options files for changes. If report logging is enabled, any report log data still in the vendor daemon's internal data buffer is flushed. lmreread recognizes changes to system host names, but cannot be used to change server TCP/IP port numbers.

If the optional vendor daemon name is specified, only the named daemon rereads the license file and its options file (in this case, lmgrd does not reread the license file).

Usage

```
lmreread [-c license_file_list] [-vendor vendor] [-all]
```

where:

Table 12-17 • lmreread Argument Usage

Argument	Description
-c license_file_list	Use the specified license files.
-vendor vendor	Only the vendor daemon, specified by the vendor option, rereads the license file and the options file. Additionally, lmgrd restarts vendor if necessary.
-all	If more than one lmgrd is specified, instructs all instances of lmgrd to reread.



Note • If you use the -c license_file_list option, the license files specified are read by lmreread, not by lmgrd; lmgrd rereads the file it read originally.

You can protect the unauthorized execution of lmreread when you start up the license server manager, lmgrd.



Note • Ability for vendor daemon to participate in rereading of its option file introduced in version 8.0 vendor daemon

See Also

- Installing lmadmin
- Changes in lmreread Behavior when Using lmadmin
- lmgrd Command-Line Syntax for details about securing access to lmreread

lmstat

The lmstat utility helps you monitor the status of all network licensing activities, including:

- Daemons that are running
- License files
- Users of individual features
- Users of features served by a specific vendor daemon
- BORROW licenses borrowed

The lmstat utility prints information that it receives from the license server; therefore, it does not report on unserved licenses such as uncounted licenses. To report on an uncounted license, the license must be added to a served license file and the application must be directed to use the license server for that license file (via @host, port@host, or USE_SERVER). Queued users and licenses shared due to duplicate grouping are also not returned by lmstat.

Usage

```
lmstat [-a [--no-user-info]] [-asec] [-c license_file_list] [-f [feature] [--no-user-info]] [-i [feature]] [-s[server]] [-S [vendor]] [-t timeout_value] [-proj]
```

where:

Table 12-18 • lmstat Argument Usage

Argument	Description
-a	Displays all information. This option is a potentially expensive command. With many active users, this command option generates a lot of network activity.
-A	Lists all licenses that are currently checked out.
-asec	Displays information in <i>hh:mm:ss</i> format.
-c <i>license_file_list</i>	Uses the specified license files.
-f [<i>feature</i>]	Displays users of <i>feature</i> . If <i>feature</i> is not specified, usage information for all features is displayed.
-i [<i>feature</i>]	Displays information from the feature definition line for the specified <i>feature</i> , or all features if <i>feature</i> is not specified.
-lm	Displays the status of the license manager.
-proj	Appends project details along with usage information when the feature has been checked out via LM_PROJECT. It is an optional argument.
-s [<i>server</i>]	Displays the status of all license files listed in \$VENDOR_LICENSE_FILE or \$LM_LICENSE_FILE on <i>server</i> , or on all servers if <i>server</i> is not specified.
	 Note • Server name or host name is case sensitive; use the same name as provided in your license file.
-S [<i>vendor</i>]	Lists all users of <i>vendor</i> 's features.
-t <i>timeout_value</i>	Sets connection timeout to <i>timeout_value</i> . This limits the amount of time lmstat spends attempting to connect to <i>server</i> .
-v	Displays the lmstat version, revision, and patch.
-vd	Displays the status of the vendor daemon.
-old	Allows communications with an old server that uses communications version 1.2 or earlier.
-a --no-user-info	Displays user information for each feature except individual user details.

Table 12-18 • lmstat Argument Usage

Argument	Description
-f feature_name --no-user-info	Displays usage information for a specified feature except individual user details.

The output of `lmstat -a` without `-f` & without `--no-user-info` looks similar to the following:

```
lmstat - Copyright (c) 1989-2021 Flexera. All Rights Reserved.  
Flexible License Manager status on Mon 7/15/2019 16:37
```

```
License server status: 27000@localhost  
Vendor daemon status:  
    demo: UP v11.16.99  
Feature usage info:  
Users of f1: (Total of 8 licenses issued; Total of 2 licenses in use)  
    "f1" v3.0, vendor: demo, expiry: permanent(no expiration date)  
    platforms: x64_linux , vendor_string: mnc  
    floating license
```

```
Users of f2: (Total of 7000 licenses issued; Total of 0 licenses in use)
```

```
Users of f3: (Total of 9000 licenses issued; Total of 2100 licenses in use)  
    "f3" v3.0, vendor: demo, expiry: permanent(no expiration date)  
    floating license
```

The output of `lmstat -a --no-user-info` looks similar to the following:

```
lmstat - Copyright (c) 1989-2021 Flexera. All Rights Reserved.  
Flexible License Manager status on Mon 7/15/2019 16:37
```

```
Feature "f1" v2.0, vendor: demo, expiry: permanent(no expiration date) (Total of 4 licenses issued;  
Total of 0 floating non-reserved licenses in use)  
    (Total of 0 licenses queued; Total of 0 licenses reserved)  
    "f1" v2.0, vendor: demo, expiry: permanent(no expiration date)  
    platforms: i86_linux , vendor_string: abcd  
    floating license
```

where:

Table 12-19 • lmstat Output

Output	Argument	Description
daniel	<i>user</i>	User name.
myhost2	<i>user_host</i>	Host where user is running.
19.36.18.26	<i>display</i>	Display where user is running.
v1.0	<i>version</i>	Version of feature.
myhost1	<i>server_host</i>	Host where license server is running.

Table 12-19 • Imstat Output

Output	Argument	Description
40000	<i>port</i>	TCP/IP port on <i>server_host</i> where license server is running.
102	<i>handle</i>	License handle.
start Fri 5/3 7:29	<i>checkout_time</i>	Time that this license was checked out.

The *user*, *user_host*, *display*, *server_host*, *port*, and *handle* information is used when removing licenses with `lmremove`.

Imswitch

The `lmswitch` utility when used without `rollover` argument switches the debug log file written by a particular vendor daemon by closing the existing debug log for that vendor daemon and starting a new debug log for that vendor daemon with a new file name. It also starts a new debug log file written by that vendor daemon if one does not already exist.

The `lmswitch` utility when used with `rollover` argument rolls over the debug log by moving the existing debug log content to a new file, then starting the new debug log with original debug log file name. If the file name is not specified, debug log would be rolled-over to a file with a predefined file name format.



Note • The `lmswitch` utility always writes in a new debug log file. If the file already existed, an error will be shown.

Usage

```
lmswitch [-c license_file_list] vendor new_debug_log -rollover
```

where:

Table 12-20 • Imswitch Argument Usage

Argument	Description
-c <i>license_file_list</i>	Use the specified license files.
<i>vendor</i>	Vendor daemon in this license file.
<i>new_debug_log</i>	Path to new debug log file.
-rollover	Rolls over the debug log by moving the existing debug log content to a new file, then starting the new debug log with original debug log file name. If the file name is not specified, debug log would be rolled-over to a file with name format as <vendor daemon name><hostname><date>_T<time>.log.

By default, debug log output from the license server and all vendor daemons started by the server get written into the same debug file. `lmswitch` allows companies to keep separate log files for different vendor daemons and control the size of their debug log file.

If debug log output is not already directed to a separate file for this vendor daemon, `lmswitch` tells the vendor daemon to start writing its debug log output to a file, `new_debug_Log`. If this vendor daemon is already writing to its own debug log, `lmswitch` tells the vendor daemon to close its current debug log file and start writing its debug log output to `new_debug_Log`.



Note ▪ The effect of `lmswitch` continues only until the vendor daemon is shut down or its options file is reread via `lmreread`. When the vendor daemon is restarted or its options file is reread, it looks for a `DEBUGLOG` line in the options file to determine whether or not to write its debug log output into its own file and, if so, what file to write. It is best practice to set the location of `new_debug_Log` to a `ProgramData` sub-folder, since this location has user-write permission by default, which is needed when the license server runs as a service with `LocalService` permission.

See Also:
[Installing Imadmin](#) for information on `Imadmin` display
[DEBUGLOG](#)
[Debug Log File](#)

lmswitchr

The `lmswitchr` utility switches the report log file by closing the existing report log and starting a new report log with a new file name. It also starts a new report log file if one does not already exist.

Usage

```
lmswitchr [-c license_file_list] feature new_report_Log
```

With version 5.0 or later vendor daemon, the command looks like this:

```
lmswitchr [-c license_file_list] vendor new_report_Log
```

where:

Table 12-21 ▪ `lmswitchr` Argument Usage

Argument	Description
<code>-c license_file_list</code>	Use the specified license files.
<code>feature</code>	Any feature in this license file.
<code>vendor</code>	Vendor daemon in this license file.
<code>new_report_Log</code>	Path to new report log file.

If report logging is not enabled for the vendor daemon, `lmswitchr` tells it to start writing its report log output to `new_report_Log`. If report logging is already enabled for the vendor daemon, `lmswitchr` tells the vendor daemon to close its report log file and start writing its new report log output to `new_report_Log`.



Note ▪ The effect of `lmswitchr` continues only until the vendor daemon is shut down or its options file is reread via `lmreread`. When the vendor daemon is restarted or its options file is reread, it looks for a `REPORTLOG` line in the options file to determine whether or not to write report log output to a file and, if so, what file to write. It is best practice to set the location of `new_report_Log` to a `ProgramData` sub-folder, since this location has user-write permission by default, which is needed when the license server runs as a service with `LocalService` permission

See Also:

[REPORTLOG](#)

[lmlicvalidator](#)

[Report Log File](#)

lmtpminfo

The `lmtpminfo` utility returns the status of the TPM (Trusted Platform Module).

As a prerequisite to obtaining and using the TPM hostid, a TPM version 2.0 device must be available and enabled. In addition, the FlexNet Licensing Service must be installed. Contact your software publisher for information on how to install the FlexNet Licensing Service.



Note ▪ The `lmtpminfo` utility is only available for Windows platforms.

Usage

`lmtpminfo [-long]`

Where `-long` returns more detailed TPM information.

The following TPM information is available:

Table 12-22 ▪ TPM information from `lmtpminfo`

Output	Description
Trusted Platform Module (TPM) detected.	TPM 2.0 is available. A list of relevant TPM details is displayed.
Trusted Platform Module (TPM) status cannot be determined. ERROR: Could not obtain Trusted Platform Module (TPM) information. ERROR CODE: -213. The FlexNet Licensing Service is not installed.	TPM 2.0 could not be detected. The FlexNet Licensing Service must be installed to obtain the TPM hostid.

Table 12-22 • TPM information from `lmtpminfo`

Output	Description
Trusted Platform Module (TPM) status cannot be determined. ERROR: Could not obtain Trusted Platform Module (TPM) information. ERROR CODE: -214. The FlexNet Licensing Service version is not as expected.	TPM 2.0 could not be detected. Install the latest version of the FlexNet Licensing Service.
Trusted Platform Module (TPM) status cannot be determined. ERROR: Could not obtain Trusted Platform Module (TPM) information. ERROR CODE: -223. FlexNet Licensing Service requires new install.	TPM 2.0 could not be detected. Reinstall the latest version of the FlexNet Licensing Service.
Trusted Platform Module (TPM) status cannot be determined. ERROR: Could not obtain Trusted Platform Module (TPM) information. ERROR CODE: -227. TPM version is not supported.	The TPM status cannot be obtained. Possible reason: An unsupported TPM version was found. (Only TPM 2.0 is supported.)
Trusted Platform Module (TPM) status cannot be determined. ERROR: Could not obtain Trusted Platform Module (TPM) information. ERROR CODE: -229. TPM properties are not available.	The TPM status cannot be obtained. Possible reason: TPM 2.0 is available, but the TPM is currently disabled or turned off in the BIOS. To use the TPM, it must be turned on and enabled in the machine's BIOS (Security settings).

Imtransfer

The `lmtransfer` utility supports the transferring of licenses from a parent licensing server to a remote licensing server.

This utility runs on the FlexEnabled remote machine that stores transferred licenses. The FlexEnabled application users can use the utility to do the following:

- [Initiating License Transferring](#)
- [Clearing the Transferred License Setting](#)
- [Purging Expired Licenses](#)
- [Determining Transferred-License Status](#)
- [Returning a Transferred License Early](#)
- [Display Usage Information](#)

Initiating License Transferring

To initiate transferring, user of the FlexEnabled application sets the expiry period by running the `lmtransfer` from the command line or through `lmtools` on the FlexEnabled remote machine.



Task

To initiate transferring

From the FlexEnabled remote machine, enter the following command:

```
lmtransfer vendorname -set dd-mmm-yyyy [hh:mm]
```

where:

Table 12-23 • `lmtransfer` Arguments for Initiating Transferring

Argument	Description
vendorname	The vendor daemon name that serves the licenses to be transferred.
-set dd-mmm-yyyy [hh:mm]	Date the license is to be expired in dd-mmm-yyyy format. <i>hh:mm</i> is the time which is optional and is specified in 24-hour format in the FlexEnabled application's local time. If time is unspecified, the checkout lasts until the end of the given end date.

For example, you might enter the following:

```
lmtransfer sampled -set 20-aug-2023 [14:00]
```

To transfer licenses for the desired vendor name, on the same day and the same system that the user runs `lmtransfer`, run the applications to check out the licenses. If you run the applications more than once that day, no duplicate licenses are transferred. No licenses are transferred if the application is run on a day different than the date transferring is initiated.

Aside from the `lmtransfer` utility, you can initiate transferring by using the transferring interface included in the FlexEnabled application, if such an interface is provided.

Clearing the Transferred License Setting

Users of the FlexEnabled application can clear the transferred licenses setting to stop licenses from being transferred until transferring is initiated again. For example, users might run `lmtransfer vendorname -clear` after they have transferred licenses for features that are used offline if—before disconnecting from the network—they want to run an application that checks out additional features, served by that `vendorname`, that are not meant to be transferred. Clearing transferred licenses setting does not change the status for already transferred licenses.



Task

To clear the transferred license setting in the Windows registry

From the FlexEnabled remote machine, enter the following command:

```
lmtransfer vendorname -clear
```

Purging Expired Licenses

Users can remove expired transferred licenses from their FlexEnabled remote machine.



Task To remove expired transferred licenses

From the FlexEnabled remote machine, enter the following command:

```
lmtransfer.exe vendorname -purge
```



Tip ▪ Combine this option with the `-status` option, both to purge expired transferred licenses from the remote machine and to display information about the still valid transferred licenses. For more detail, see [Determining Transferred-License Status](#).

Determining Transferred-License Status

Users can use the following commands to display the status of licenses which are currently being transferred to a FlexEnabled remote machine.



Task To display the status of all currently transferred features, expired or not

From the FlexEnabled remote machine, enter the following command:

```
lmtransfer vendorname -status
```



Task To purge expired transferred licenses and display the status of non-expired transferred licenses

From the FlexEnabled remote machine, enter the following command:

```
lmtransfer vendorname -purge -status
```



Note ▪ You can specify the `-purge` and `-status` options in any order. The result is always the same—expired transferred licenses are removed and the status of the remaining transferred licenses are displayed.



Note ▪ The current status of transferred licenses is read from information stored on the FlexEnabled remote machine itself. Therefore, the FlexEnabled remote machine does not need to be connected to the network to obtain this information.

Returning a Transferred License Early

Users can early return a transferred license.

**Task** *To return a transferred license early*

From the FlexEnabled remote machine, enter the following command:

```
lmtransfer -return [-vendor name] feature
```

where:

feature used to specify the name of the transferred feature to be returned early. Use `lmtransfer vendorname -status` to get a list of transferred feature names.



Note ▪ To return a transferred license early, parent licensing server should have the `Ls_borrow_return_early` vendor variable enabled.

Display Usage Information

Users can display the `lmtransfer` usage information.

**Task** *To display lmtransfer usage information*

From the FlexEnabled remote machine, enter the following command:

```
lmtransfer -help
```

lmver

The `lmver` utility reports the version of a FlexNet Publisher library or binary file.

Usage

```
lmver filename
```

where *filename* is one of the following:

- The name of an executable file built with FlexNet Publisher
- The license server manager
- A license administration tool
- A vendor daemon

For example, if you have an application called **spell**, type `lmver spell`.

lmvminfo

The `lmvminfo` utility returns the environment as virtual or not.

Usage

`lmvminfo -Long`



Note ▪ Flexnet Licensing Service needs to be installed for 'lmvminfo -long' to fetch the required details.

Where `-Long` returns the following:

- Family of the virtual machine. The Family generally denotes the provider of the detected hypervisor software that the virtual machine (VM) is running on.

The virtual environments for which Family names are generated are:

Table 12-24 ▪ List of VM_family and VM_name and hypervisor supported

Hypervisor	Family	Name
Hyper-V	MICROSOFT	HYPERV
VMware Workstation	VMWARE	DESKTOP, previously VMWARE
VMware ESXi	VMWARE	SERVER, previously VMWARE
Oracle VirtualBox	VIRTUALBOX	VIRTUALBOX
Citrix XenServer	XEN	XEN
Amazon EC2	AMAZON	EC2
Parallels	PARALLELS	PARALLELS
QEMU	QEMU	QEMU
QEMU-KVM*	QEMU	QEMU-KVM
Google Compute	GOOGLE	COMPUTE
Microsoft Azure	MICROSOFT	AZURE, previously HYPER-V
everRun	STRATUS	EVERRUN
Nutanix	NUTANIX	NUTANIX

* The KVM suffix is only appended to the QEMU hypervisor 'name' attribute (not the family attribute) when the presence of the KVM virtualization infrastructure has been detected.

- Name of the virtual machine. The Name specifies the detected hypervisor product that the virtual machine (VM) is running on. The Name is for future use and is intended to be a subset of family.
- UUID of the machine
- GENID of the machine

Output example for the command `lmvminfo -long`:

```
C:\Users\Administrator\Desktop\64bit\x64_n6>lmvminfo.exe -long

lmvminfo - Copyright (c) 1989-2021 Flexera Software LLC. All Rights Reserved.

Running on Virtual Platform

FAMILY=VMWARE

NAME=VMWARE

UUID=0F064D56-465C-1286-6243-81C70C2766FF

GENID=433646c3cd09838f:39d695b09d953cb4
```



Note ▪ If the GENID is not available then “GENID: ERROR - UnavailaBle” is displayed.

Other Important Utilities

Imobfslog

The `Imobfslog` utility processes the debug log file to generate the obfuscated usernames. It is used for debug log processing.



Note ▪ If the debug log has been produced with `OBf_ADDMARK` option, only then the `Imobfslog` utility can generate obfuscated usernames in the debug log.

Usage

```
Imobfslog [-i debug_log] [-o new_debug_log] <-rmarker>
```

Where:

Table 12-25 ▪ Imobfslog Argument Usage

Argument	Description
-help	To print usage information.
-i debug_log	Path to existing debug log file with markers that is passed to obfuscate the usernames.
-o new_debug_log	Path to new debug log file that is generated after usernames are obfuscated.
-rmarker	Removes the markers from the debug log file.

Imreporter

The Imreporter is an optional user interface that allows you to view feature usage, peaks, and denials. The Imreporter utility uses the debug logs, where the server events are captured, created by the vendor daemon. This functionality must be explicitly enabled by the vendor.

The following sections describe how to use the Imreporter:

- [Setting Up the Imreporter Without Imadmin Installer](#)
- [Accessing the Imreporter GUI](#)
- [Log Analysis Page](#)
- [Usage Over Time Page](#)
- [Peak Usage Page](#)
- [Denials Page](#)
- [Download Reports](#)

Setting Up the Imreporter Without Imadmin Installer

This section guides you through the installation of the Imreporter without using a Imadmin installer. The Imreporter is provided as Docker images contained in a .zip file. Therefore, you must install Docker to be able to run the Imreporter.

The Imreporter runs on a NGINX web server. The NGINX web server is included in the Imreporter-<version>.zip package, therefore, you do not need to install NGINX.



Note - Installing Docker is not covered in this section. For information about downloading and installing Docker, see the [Docker documentation](#).



Task

To start Docker:

1. Start Docker Desktop.
2. In Docker Desktop, click the **Start** button.

You can verify if Docker is up and running by using the following command:

```
docker -info
```

You will see output similar to the following:

```
C:\Users\fnpauto>docker -info
Client:
Context:    default
Debug Mode: false
Plugins:
  buildx: Docker Buildx (Docker Inc., v0.8.1)
  compose: Docker Compose (Docker Inc., v2.3.3)
  scan: Docker Scan (Docker Inc., v0.17.0)
```

```

Server:
Containers: 0
  Running: 0
  Paused: 0
  Stopped: 0
Images: 8
Server Version: 20.10.13
Storage Driver: overlay2
  Backing Filesystem: extfs
  Supports d_type: true
  Native Overlay Diff: true
  userxattr: false
Logging Driver: json-file
Cgroup Driver: cgroupfs
Cgroup Version: 1
Plugins:
  Volume: local
  Network: bridge host ipvlan macvlan null overlay
  Log: awslogs fluentd gcplogs gelf journald json-file local logentries splunk syslog
Swarm: inactive
Runtimes: io.containerd.runc.v2 io.containerd.runtime.v1.linux runc
Default Runtime: runc
Init Binary: docker-init
containerd version: 2a1d4dbdb2a1030dc5b01e96fb110a9d9f150ecc
runc version: v1.0.3-0-gf46b6ba
init version: de40ad0
Security Options:
  seccomp
  Profile: default
Kernel Version: 5.10.102.1-microsoft-standard-WSL2
Operating System: Docker Desktop
OSType: linux
Architecture: x86_64
CPUs: 12
Total Memory: 24.77GiB
Name: docker-desktop
ID: 2H3I:EJDK:PLLS:PPFZ:7RLG:NFNS:IJ4G:TWGH:HJJU:Q3JQ:GMLM:RY6V
Docker Root Dir: /var/lib/docker
Debug Mode: false
HTTP Proxy: http.docker.internal:3128
HTTPS Proxy: http.docker.internal:3128
No Proxy: hubproxy.docker.internal
Registry: https://index.docker.io/v1/
Labels:
Experimental: false
Insecure Registries:
  hubproxy.docker.internal:5000
  127.0.0.0/8
Live Restore Enabled: false

WARNING: No blkio throttle.read_bps_device support
WARNING: No blkio throttle.write_bps_device support
WARNING: No blkio throttle.read_iops_device support
WARNING: No blkio throttle.write_iops_device support

```



Task

To Install the lmreporter:

1. Extract the lmreporter-<version>.zip file.
2. Run the following command to load the Docker image file:

```
docker load -i <location of .tar file>
```

You will see output similar to the following:

```
C:\lmreporter>docker load -i lmReporterBackend.tar
cc2447e1835a: Loading layer [=====>] 7.626MB/7.626MB
fced6d5a2299: Loading layer [=====>] 118.7MB/118.7MB
ce3e1bf7cab7: Loading layer [=====>] 7.8MB/7.8MB
643ccf548ae4: Loading layer [=====>] 3.584kB/3.584kB
a8b35d4fdf6c: Loading layer [=====>] 3.584kB/3.584kB
ab3b5baef649: Loading layer [=====>] 301.1kB/301.1kB
c5b1800f7df1: Loading layer [=====>] 46.64MB/46.64MB
768f1b3b1d45: Loading layer [=====>] 14.31MB/14.31MB
b8e6fd9022c7: Loading layer [=====>] 69.71MB/69.71MB
Loaded image: lmreporter-backend: latest
C:\lmreporter>docker load -i lmReporterFrontend.tar
81fdcc81a9d0: Loading layer [=====>] 10.5MB/10.5MB
854101110f63: Loading layer [=====>] 3.584kB/3.584kB
38067ed663bf: Loading layer [=====>] 4.608kB/4.608kB
f126bda54112: Loading layer [=====>] 2.56kB/2.56kB
901e6ddcc99: Loading layer [=====>] 5.12kB/5.12kB
01e36c0e0b84: Loading layer [=====>] 7.168kB/7.168kB
4b701b99fec7: Loading layer [=====>] 31.37MB/31.37MB
5f70bf18a086: Loading layer [=====>] 1.024kB/1.024kB
3b90ec4959af: Loading layer [=====>] 4.608kB/4.608kB
89dc7efb93f6: Loading layer [=====>] 3.584kB/3.584kB
baf6836f3739: Loading layer [=====>] 1.968MB/1.968MB
Loaded image: lmreporter-frontend: latest
```

3. Use the following command to run the lmreporter service:

```
docker compose up --detach
```

You will see output similar to the following:

```
C:\lmreporter>docker compose up --detach
Building 0.0s (0/0)
docker:default
Running 3/3
Network lmreporter_default    Created
0.1s
Container lmreporter-backend-1 Started
0.1s
Container lmreporter-frontend-1 Started
```

4. Use the following command to verify the running Docker containers:

```
docker ps
```

You will see the following output:

```
C:\lmreporter>docker ps
```


REPOSITORY	TAG	IMAGE ID	CREATED	SIZE
lmreporter-frontend	latest	5e2a7df9f80c	2 weeks ago	49.6MB
lmreporter-backend	latest	987dc4b0c387	2 weeks ago	254MB

Accessing the Imreporter GUI

The Imreporter GUI can be accessed in the following two ways:

- [Imreporter via Imadmin Installer](#)
- [Imreporter via Docker Images](#)

Imreporter via Imadmin Installer

If Imadmin is installed with the **Install Imreporter tool** option selected, the **Imreporter** button will be displayed in the Home page of Imadmin. Click on the **Imreporter** button to access the Imreporter GUI. The web browser opens with the URL `http://<LmadminHostName>:27009`. You can now view the **Log Analysis** page of the Imreporter.

The following displays Imadmin UI with Imreporter button:

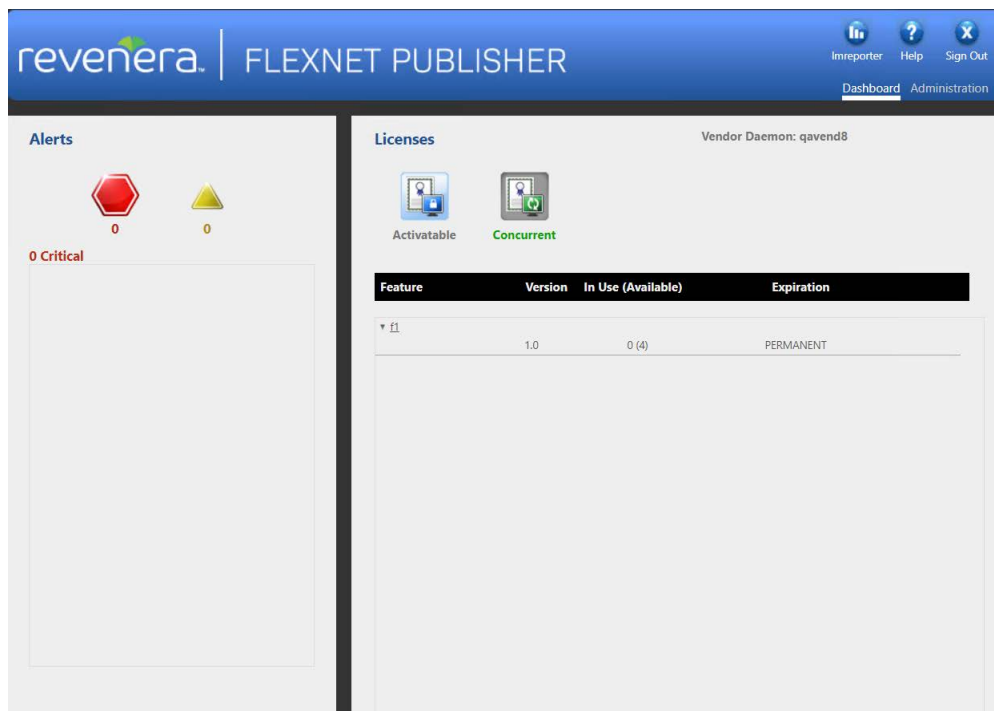


Figure 12-1: Imadmin UI with Imreporter Button

Imreporter via Docker Images

This section describes how to access the Imreporter GUI through the Docker images.



Note ■ Installing Docker is not covered in this section. For information about downloading and installing Docker, see the [Docker documentation](#).

To access the Imreporter GUI, point a web browser to `http://<LmadminHostName>:27009`. The web browser displays the **Log Analysis** page of the Imreporter.

Log Analysis Page

The **Log Analysis** page in the Imreporter is designed to analyze the debug log files and present insightful graphs based on those uploaded debug log files. Follow the steps below to leverage this feature:

- **Upload debug log files**—To initiate graph generation, you need to specify the location of the debug logs folder. In the **Import Log Files** area of the **Create Report** page, specify or browse to the **Log File Location**, and then click **Import**.

Create Report

Import Log Files

Enter the directory containing the log files you want to use for this report and then click **Import**.

Log File Location

... **IMPORT**

Figure 12-2: Upload Debug Log Files

- **Import and view server information**—After you click **Import**, the server information is displayed. The following server information is displayed:
 - **First Start Date** and **Last End Date** of the log
 - **Vendor(s)** (Vendor Daemon) name
 - **License Server Host(s)** name

revenera. | FLEXNET PUBLISHER Imreporter

Create Report

Import Log Files

Enter the directory containing the log files you want to use for this report and then click **Import**.

Log File Location

debug.log, Debug_All_Handled_Msgs.log, myDebug (2).log

...

IMPORT

Import Log Files

First Start Date: 01-09-2023 10:40:21

Last End Date: 13-12-2023 04:47:49

Vendor(s): qavend8,demo

License Server Host(s): everrun-RHEL8.6,BLR-LT-104276

Figure 12-3: Server Information

- **Generate Reports**—After importing the debug log files, select a specific feature and (if more than one is specified) a vendor daemon from the drop-down menus. The following displays selecting a specific feature from the **Select Feature With Usage** drop-down:

revenera | FLEXNET PUBLISHER Imreporter

Create Report

Import Log Files

Enter the directory containing the log files you want to use for this report and then click **Import**.

Log File Location

debug.log, Debug_All_Handled_Msgs.log, myDebug (2).log

...

IMPORT

Import Log Files

First Start Date: 01-09-2023 10:40:21
Last End Date: 13-12-2023 04:47:49
Vendor(s): qavend8,demo
License Server Host(s): everrun-RHEL8.6,BLR-LT-104276

Select Feature With Usage

Select...

GENERATE REPORT

f1
ADD
SUBTRACT
MULTIPLY
DIVIDE
AVG
POWER
ff1

Figure 12-4: Select Feature With Usage Drop-Down

The following displays selecting a specific vendor daemon from the **Select Daemon** drop-down:

revenera.

FLEXNET PUBLISHER Imreporter

Create Report

Import Log Files

Enter the directory containing the log files you want to use for this report and then click **Import**.

Log File Location

debug.log, Debug_All_Handled_Msgs.log, myDebug (2).log

...

IMPORT

Import Log Files

First Start Date: 01-09-2023 10:40:21

Last End Date: 13-12-2023 04:47:49

Vendor(s): qavend8,demo

License Server Host(s): everrun-RHEL8.6,BLR-LT-104276

Select Feature With Usage

Select Daemon

f1

qavend8

qavend8

demo

GENERATE REPORT

Figure 12-5: Select Daemon Drop-Down

Click the **Generate Report** button to generate the report based on the selections.

Usage Over Time Page

The **Usage Over Time** page provides a visual representation of the usage trends for a selected feature associated with a selected vendor daemon. It offers insights into how the usage of the feature has evolved over a specified period. The following displays the graphical representation of a selected feature usage over specific interval on the **Usage Over Time** page:

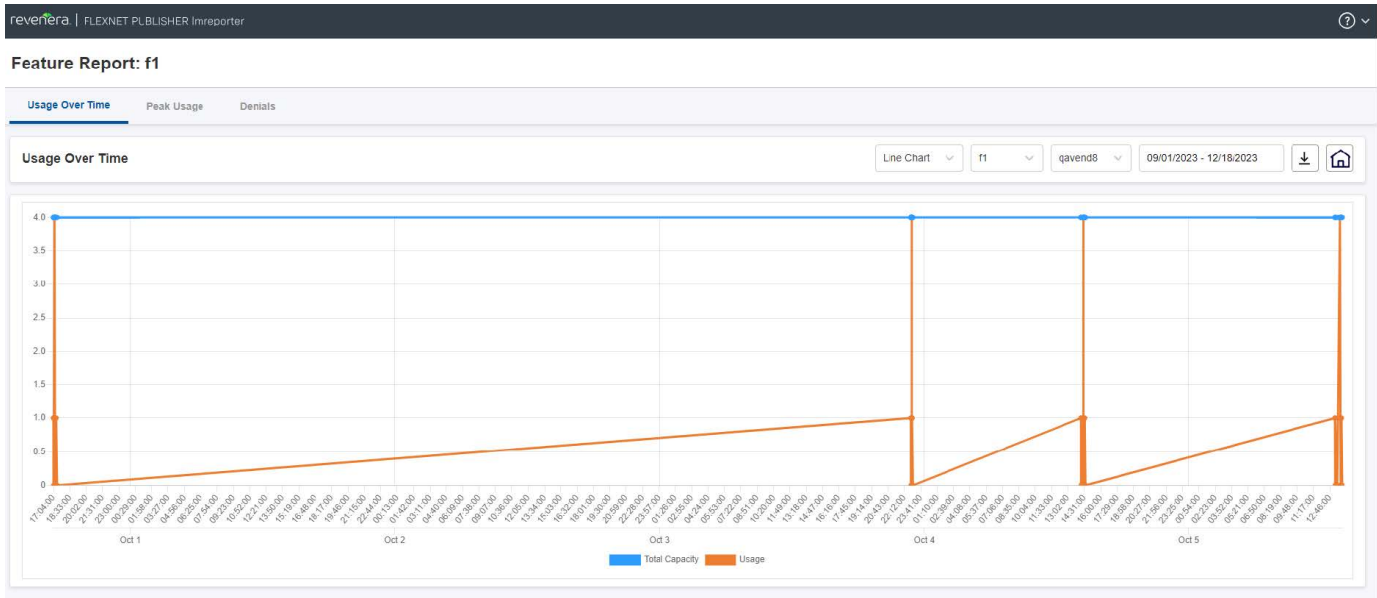


Figure 12-6: Usage Over Time Page

Key Features

The following are the key features of this page:

- **Time-based visualization**—This page presents a graphical representation of usage trends, allowing users to observe the patterns and fluctuations over the regular interval.
- **Feature-centric view**—Users can track the usage of a specific feature and gain valuable information about its popularity, growth, or potential decline.

You can use the drop-down menus above the chart to select a different feature, vendor daemon, or date range:

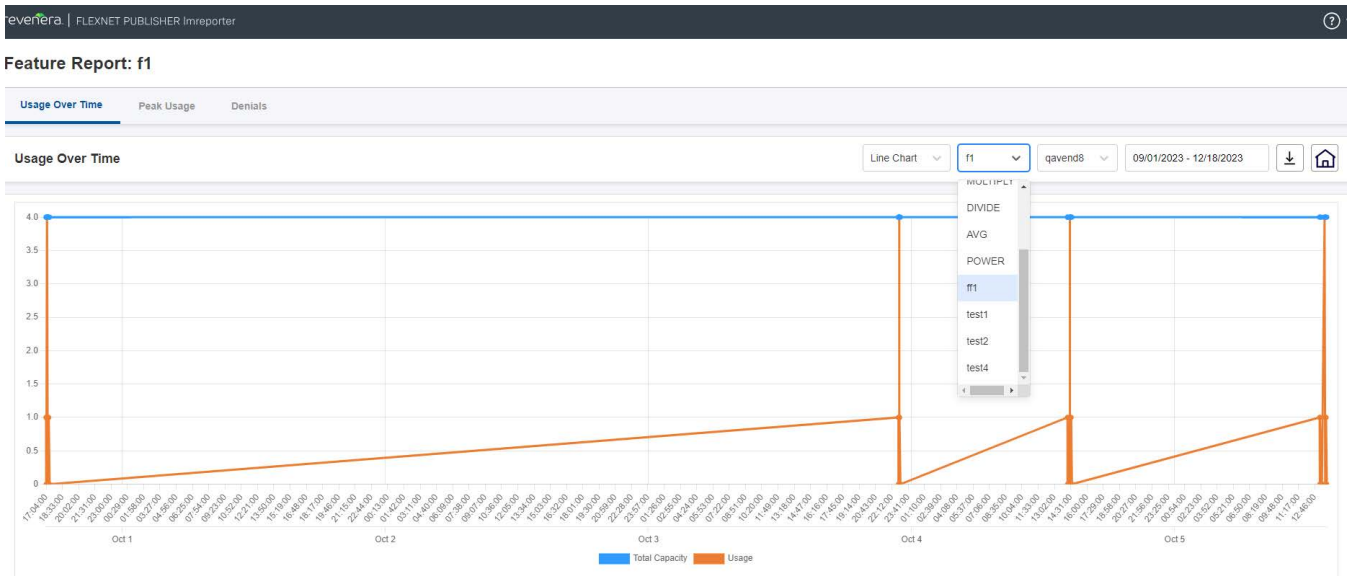


Figure 12-7: Usage Over Time Page with Feature Drop-Down

The following page shows selecting a specific vendor daemon:

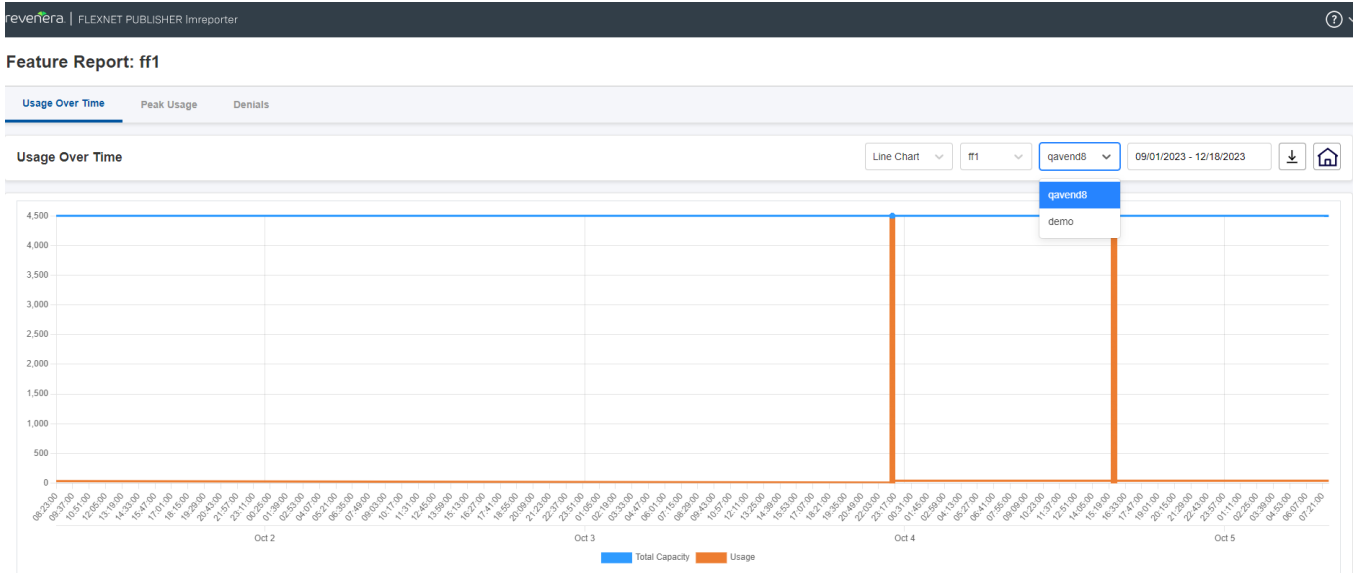


Figure 12-8: Usage Over Time Page with Vendor Daemon Drop-Down

The end user has the flexibility to choose the desired date range for the report creation.

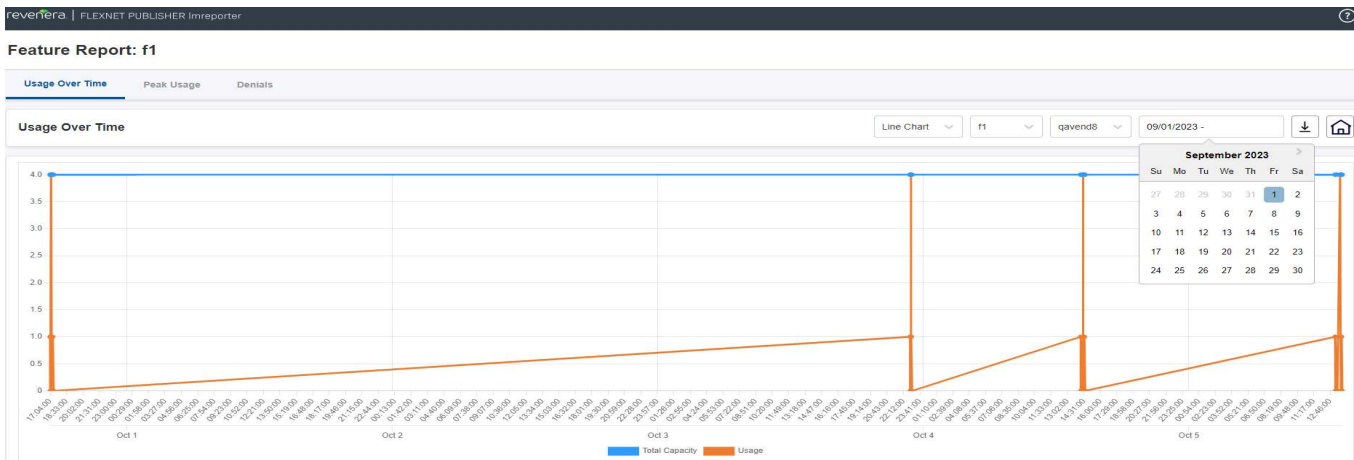


Figure 12-9: Usage Over Time Page with Date Range Selection

Below all graphs, a table containing comprehensive data related to the presented information is displayed. This table offers a more granular breakdown of the data points shown in graph:

Client Version	Client User Name	Client IP	Usage Count	Total Count	In Use Time	Date Of Event	Time Of Event
1.0	fnpauto	10.78.130.187	1	4	0:0:0	2023-09-30	17:3:45
1.0	fnpauto	10.78.130.187	0	4	0:0:28	2023-09-30	17:4:13
1.0	fnpauto	10.78.130.187	1	4	0:0:0	2023-09-30	17:4:17
1.0	fnpauto	10.78.130.187	0	4	0:2:9	2023-09-30	17:6:26
1.0	fnpauto	10.78.130.187	4	4	0:0:0	2023-09-30	17:9:27

Figure 12-10: Data Beneath Graph

The following table describes the data as shown above:

Table 12-26 • Data Beneath Graph

Field	Description
Client Version	Provides the version of client.
Client User Name	Indicates the user name of client.
Client IP	Provides the IP (Internet Protocol) of client.
Usage Count	Indicates number of licenses used in a particular interval.
Total Count	Indicates number of licences served by the license server.
In Use Time	Indicates time intervals during which the licenses were in checked-out status.
Date Of Event	Provides the date intervals at which events occurred.
Time Of Event	Provides the time intervals at which events occurred.

Peak Usage Page

The **Peak Usage** page provides a focused view of the highest activity or utilization of a selected feature on a selected day. It offers insight into the maximum usage reached for the chosen feature during that particular interval. The following displays the bar graph representation of a selected feature peak usage over a specific interval on the **Peak Usage** page:

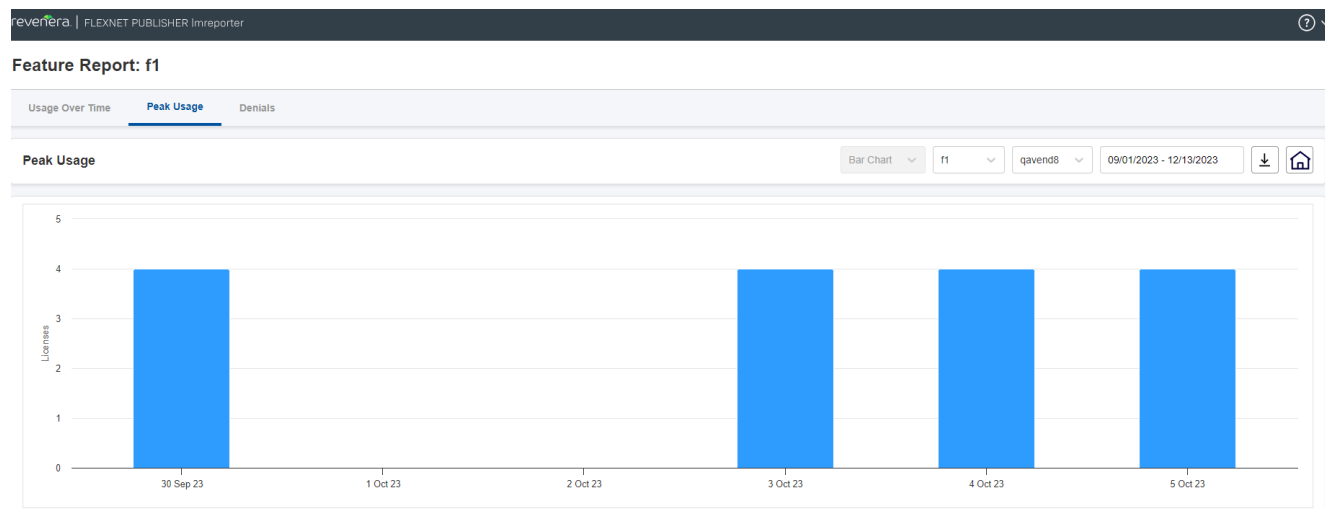


Figure 12-11: Peak Usage Page

Key Features

The following are the key features of this page:

- **Day-specific focus**—This page concentrates on the peak usage of a selected feature for the selected day.
- **Feature-centric display**—Users can quickly identify and analyze the highest point of activity for a selected feature.

You can use the drop-down menus above the chart to select a different feature, vendor daemon, or date range:

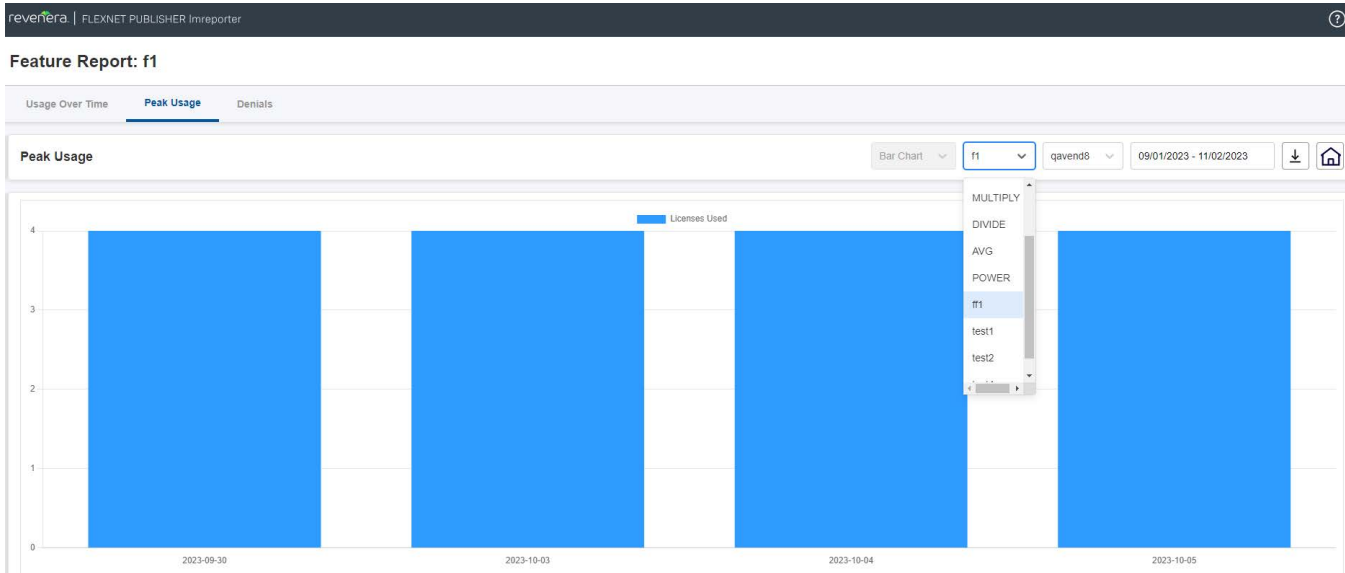


Figure 12-12: Peak Usage Page with Feature Drop-Down

The following page shows selecting a specific vendor daemon:

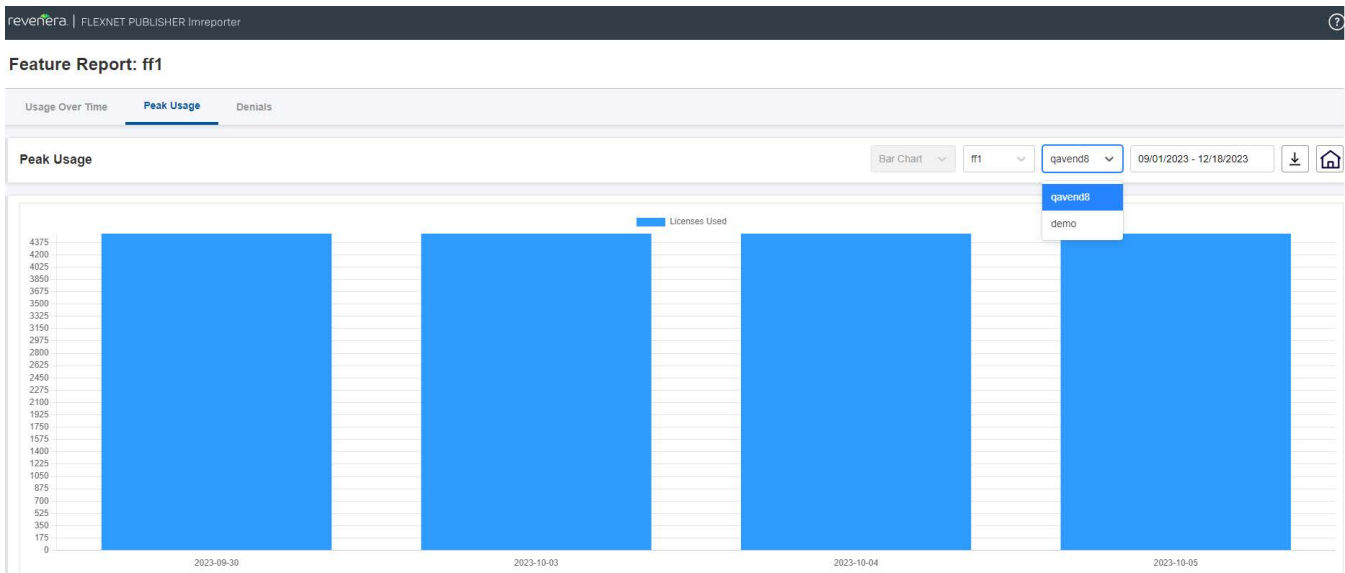


Figure 12-13: Peak Usage Page with Vendor Daemon Drop-Down

The end user has the flexibility to choose the desired date range for the report creation.

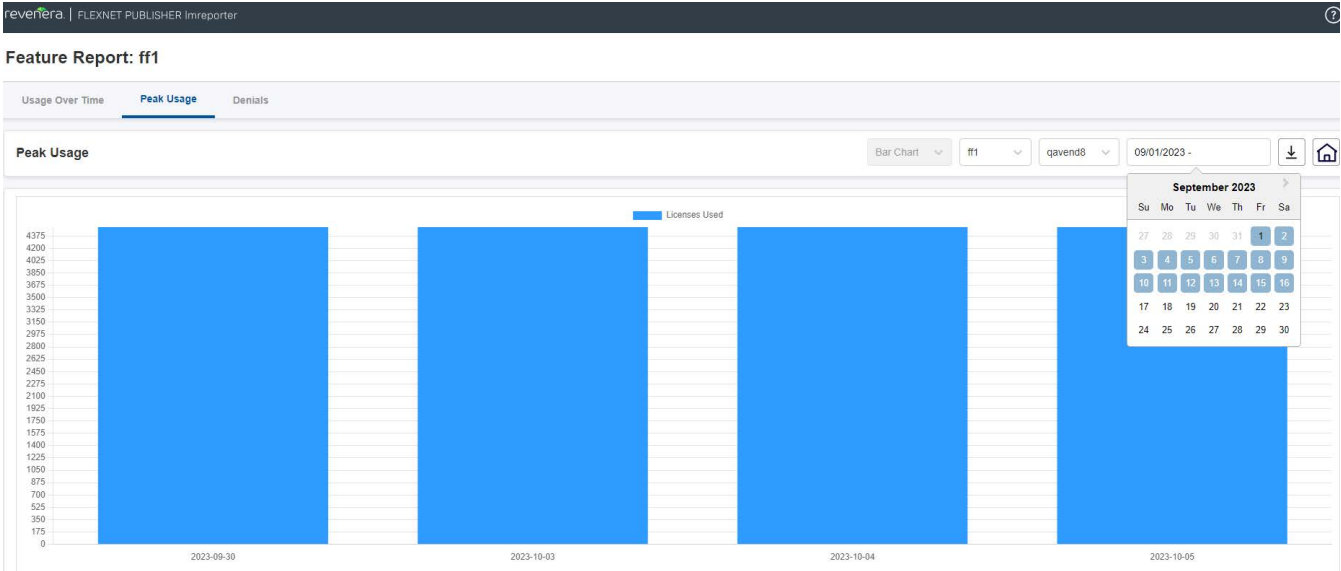


Figure 12-14: Peak Usage Page with Date Range Selection

Denials Page

The **Denials** page offers a detailed view of a selected feature that was denied on a specific day. It provides a number of denied features for each specific feature, giving users insights into the denial activity for that particular interval. The following displays the bar graph representation of a selected feature denied over a specific interval on the **Denials** page:

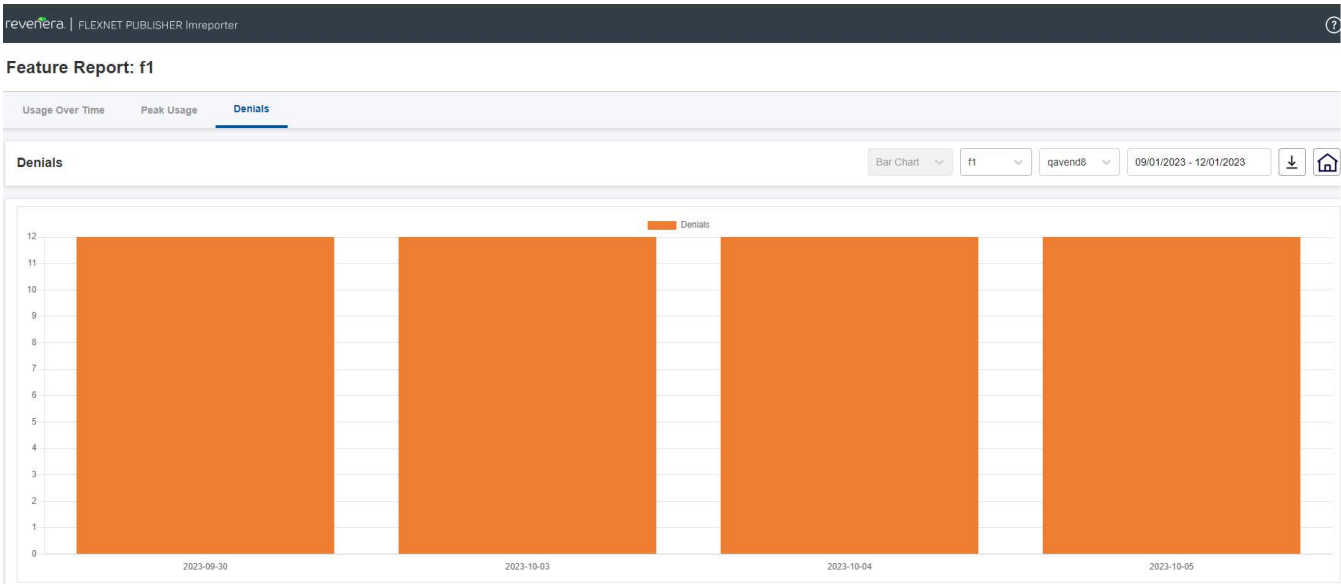


Figure 12-15: Denials Page

Key Features

The following are the key features of this page:

- **Denied features display**—This page displays a list of features that are denied for a selected day.

- **Dynamic graph update**—Users can update the displayed graph by selecting a specific feature and vendor daemon from the drop-down menu.

You can use the drop-down menus above the chart to select a different feature, vendor daemon, or date range:

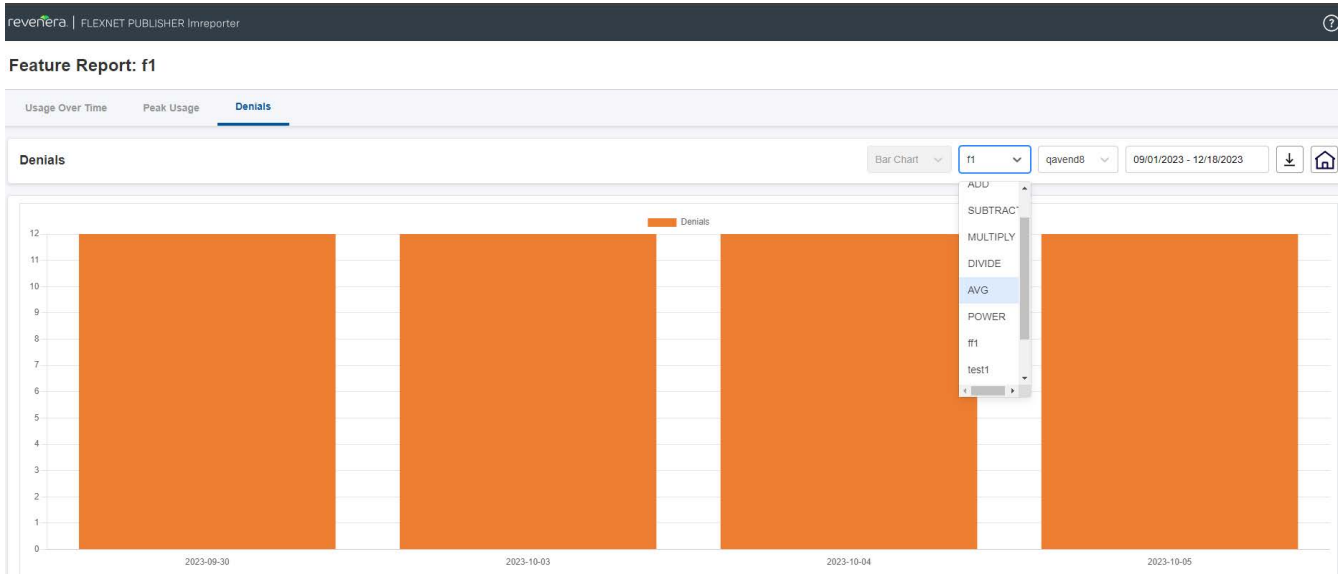


Figure 12-16: Denials Page with Feature Drop-Down

The following page shows selecting a specific vendor daemon:

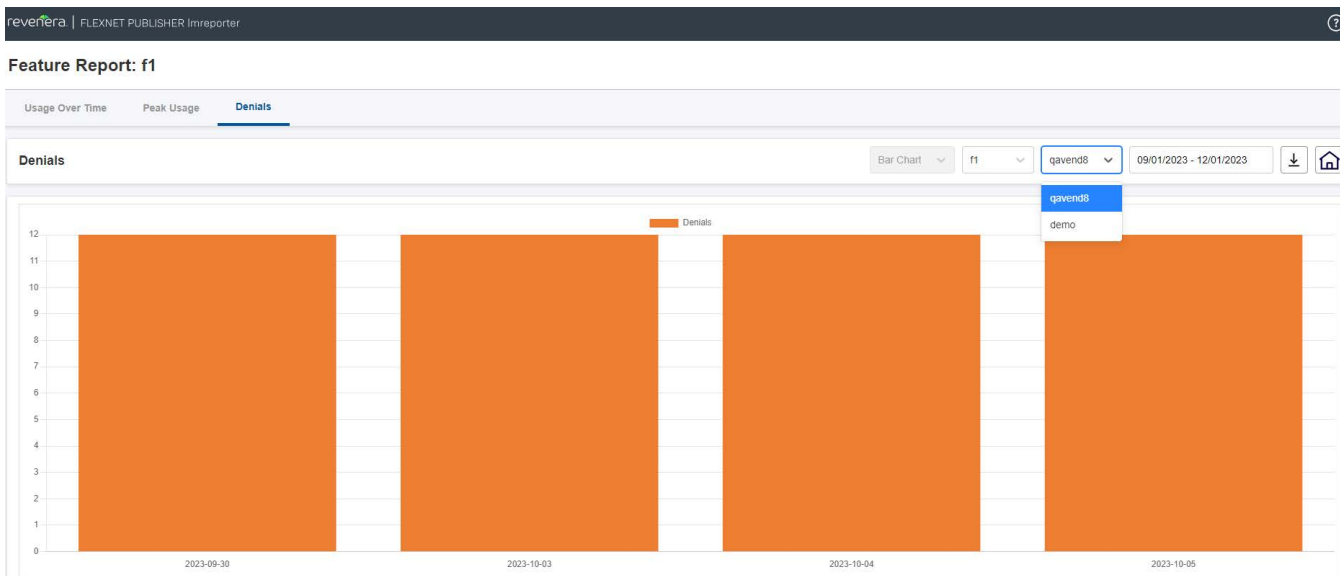


Figure 12-17: Denials Page with Vendor Daemon Drop-Down

The end user has the flexibility to choose the desired date range for the report creation:

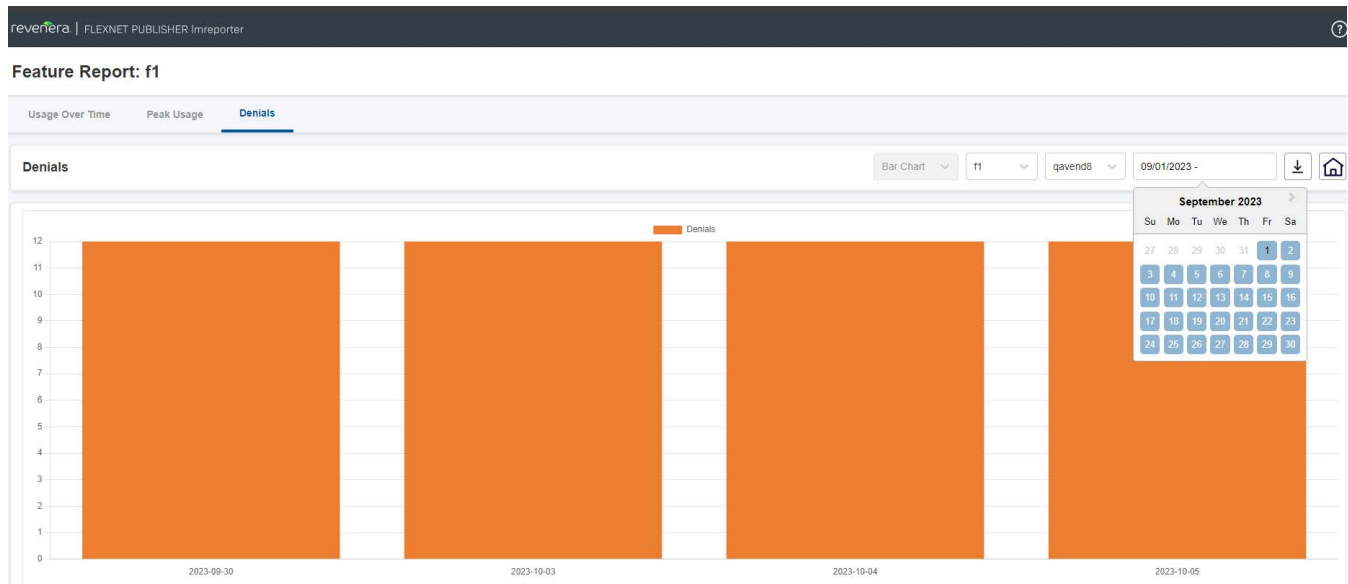


Figure 12-18: Denials Page with Date Range Selection

Limitation of Denials Page

All the denied features are added. Features are being served later are not deducted from the denied list shown in the graph. This will be taken care in the future release

Download Reports

The Imreporter enables users to download various reports in different formats for comprehensive data analysis. Users can choose one of the following download option in accordance with their preferences and requirements:

- **CSV (Comma-Separated Values)**—A table containing all the information specific to the server events written in the debug log in CSV format.
- **JSON (JavaScript Object Notation)**—A table containing all the information specific to the server events written in the debug log in JSON format.
- **JPG (Joint Photographic Experts Group)**—Download a visual snapshot of the specific graph as it appeared during the selected timeframe in JPG format.
- **PNG (Portable Network Graphics)**—Download a visual snapshot of the specific graph as it appeared during the selected timeframe in PNG format.

Imtools (Windows only)

The Imtools utility is a graphical user interface that enables you to administer the license server. This executable is available in the 32-bit and 64-bit Windows packages. Always use the newest version possible.

Some of the functions this utility performs include:

- Starting, stopping, and configuring license servers
- Getting system information, including hostids

- Getting server status

The lmtools utility has two modes in which to configure a license server:

- Configuration using a license file
- Configuration using services

You must run the lmtools utility as an administrator. If you do not run this executable as an administrator, the User Account Control (UAC) dialog will display as soon as it is started (as long as the UAC prompt is not disabled on the system).

Configuration Using License File

Operations are performed on a particular license file. The file can be either local or remote. In this mode, you cannot start the lmadmin or lmgrd process, but you can do everything else.



Task

To configure this mode:

1. Run the lmtools utility.
2. Click the **Configuration using License File** button.
3. Enter one or more the license file names or *port@host* specifications.

Configuration Using Services

Operations are performed on a service, which enables starting lmadmin or lmgrd processes local to the system on which lmtools is running. For details on configuring services, see [Configuring the License Server Manager as a Windows Service](#).

Limitation on File Path Lengths

The following file paths, used when configuring lmtools, are limited to 255 characters:

- Path to the lmadmin.exe or lmgrd.exe file
- Path to the license file
- Path to the debug log file

Ethernet hostids on Windows platforms

From version 11.6.1 onwards lmtools reports only the hostids of physical ethernet adapters. Devices identified as virtual ethernet adapters are not reported as these identities are not permanent.

Physical (Bare Metal) hostids on Virtual Machines

When run from a virtual machine, lmtools cannot return hostids for the physical machine that hosts the virtual machine. To obtain hostids for the physical machine, [lmhostid](#) must be run from the host OS.

Japanese User Identities

lmtools, when running on a system where native Microsoft shift-jis user identities are used, does not correctly display the user identity using non-ASCII, multi-byte (such as Japanese) characters. To display user identities correctly in multi-byte characters, use **lmstat** instead.

Managing the Options File

The options file enables the license administrator to control various operating parameters within the constraints of the license model. Users are identified by their user name, host name, display, IP address, or PROJECT (which is set with the LM_PROJECT environment variable).

For concurrent (floating) licenses, the license administrator can:

- Allow the use of features
- Deny the use of features
- Reserve licenses

The concurrent licenses can be held either in license files or in fulfillment records within trusted storage.

For activatable licenses, the license administrator can:

- Allow activation of licenses in a specific fulfillment record
- Deny activation of licenses in a specific fulfillment record

For all licenses, the license administrator can:

- Restrict the number of licenses available
- Control the amount of information logged about license usage
- Enable a report log file
- Control the automatic rereading of licenses

Options files enable you, as the license administrator, to be as secure or open with licenses as you like.

Lines in the options file are limited to 4000 characters. The \ character is the line-continuation character.



Note • The following lists changes in the options file for FlexNet Publisher versions:

- PROJECT identification (set by LM_PROJECT) in options file was introduced in the version 7.0 vendor daemon.

- Option file control for licenses held in fulfillment records in trusted storage was introduced in the 11.3 vendor daemon.
- `AUTOMATIC_REREAD` keyword introduced in the version 11.7 vendor daemon.
- Keywords `EXCLUDEALL_ENTITLEMENT` and `INCLUDEALL_ENTITLEMENT` were introduced in FlexNet Publisher version 11.15.0.

Creating an Options File



Task

To create an options file:

1. Use the appropriate options listed in [Options File Syntax](#) to create the options file for a vendor daemon using any text editor.
2. Locate the options file anywhere; however, it is recommended that the options file be placed in the same directory as the license file.
3. Add the path to the options file in the license file as the fourth field on the `VENDOR` line for the application's vendor daemon. For example:

```
VENDOR sampled /etc/sampled \  
      [OPTIONS=]/sample_app/sampled/licenses/sampled.opt
```

enables the `sampled` vendor daemon to look at the specified options file.

If the path is omitted, the vendor daemon automatically looks for a file according to the following criteria:

- The name of the file is `vendor.opt`, where `vendor` is the vendor daemon name.
- The directory that contains the license file used by the license server manager.



Note • The default options file name, `vendor.opt`, was introduced in the version 6 vendor daemon.

Options File Syntax

The following is an overview of the available options. See [Options File Examples](#) for examples and additional information. Each line of the file controls one option.

Table 13-1 • Option Keywords

Option Keyword	Description
<code>AUTOMATIC_REREAD</code>	Turn off automatic reread of licenses at midnight.
<code>ACTIVATION_LOWWATER</code>	Controls the number of licenses that cannot be borrowed or transferred.
<code>ACTIVATION_EXPIRY_DAYS</code>	Controls the activation request based on the expiration dates mentioned in options file during activation.

Table 13-1 • Option Keywords

Option Keyword	Description
BORROW_LOWWATER	Set the number of BORROW licenses that cannot be borrowed.
DAEMON_SELECT_TIMEOUT	Define the vendor daemon's timeout limit.
DEBUGLOG	Write debug log information for this vendor daemon to the specified file (version 8.0 or later vendor daemon).
EXCLUDE	Deny a user access to a feature.
EXCLUDE_BORROW	Deny a user the ability to borrow BORROW licenses.
EXCLUDE_ENTITLEMENT	Deny a user the ability to activate licenses held in a fulfillment record in trusted storage.
EXCLUDEALL	Deny a user access to all features served by this vendor daemon.
EXCLUDEALL_ENTITLEMENT	Deny a user the ability to activate all licenses held in a fulfillment record in trusted storage.
FQDN_MATCHING	Set the level of host name matching.
GROUP	Define a group of users for use with any options.
GROUPCASEINSENSITIVE	Set case sensitivity for user lists specified in GROUP keywords.
HOST_GROUP	Define a group of hosts for use with any options (version 4.0 or later).
INCLUDE	Allow a user to use a feature.
INCLUDE_BORROW	Allow a user to borrow BORROW licenses.
INCLUDE_ENTITLEMENT	Allow a user to activate licenses held in a fulfillment record in trusted storage.
INCLUDEALL	Allow a user to use <i>all</i> features served by this vendor daemon.
INCLUDEALL_ENTITLEMENT	Allow a user to activate <i>all</i> licenses held in a fulfillment record in trusted storage.
LINGER	Allow a user to extend the linger time for a feature beyond its check in.
MAX	Limit usage for a particular user/group—prioritizes usage among users.
MAX_BORROW_HOURS	Change the maximum borrow period for the specified feature.
MAX_CONNECTIONS	Configures the maximum number of connections to the vendor daemon at any point in time.

Table 13-1 • Option Keywords

Option Keyword	Description
MAX_OVERDRAFT	Limit overdraft usage to less than the amount specified in the license.
NOLOG	Turn off logging of certain items in the debug log file.
REPORTLOG	Specify that a report log file suitable for use by the FlexNet Manager license usage reporting tool be written.
RESERVE	Reserve licenses for a user or group of users/hosts.
TIMEOUT	Specify idle timeout for a feature, returning it to the free pool for use by another user.
TIMEOUTALL	Set timeout on all features.

Comments

Include comments in your options file by starting each comment line with the hash symbol, #.

Specifying Features

When used within an options file entry, the feature name can be modified with an optional keyword-value pair to fully qualify it. This notation is used for distinguishing a particular group of licenses when there are multiple FEATURE lines for a single feature. The following syntax is used:

feature:keyword=value

For example:

f1:VERSION=2.0

specifies the version 2.0 pool of licenses for feature f1.

The following option keywords are used as feature name modifiers to denote a specific group of licenses:

- VERSION=
- HOSTID=
- EXPDATE=
- ENTITLEMENT= (to be used only with the keywords **INCLUDE** or **EXCLUDE**)
- KEY=
- SIGN=
- ISSUER=
- NOTICE=
- VENDOR_STRING= (if configured by the publisher as a pooling component)
- dist_info=

- user_info=
- asset_info=

If the USER_BASED or HOST_BASED keywords appear in a FEATURE line, this feature specification syntax must be used to qualify the feature.

Using a package name in place of a feature name applies the option to all of the components in the package.



Note • A colon (:) is a valid feature name character. If colons are in your feature names, specify a group of licenses with the following alternative syntax using quotation marks and spaces:

`"feature keyword=value"`

Specifying License Restrictions Using Type

Some option keywords restrict who may use licenses or where licenses may be used. These options take a type argument that specifies what the restriction is based on.

When using the option keywords EXCLUDE, EXCLUDE_ENTITLEMENT, EXCLUDEALL, EXCLUDEALL_ENTITLEMENT, EXCLUDE_BORROW, INCLUDE, INCLUDE_ENTITLEMENT, INCLUDEALL, INCLUDEALL_ENTITLEMENT, INCLUDE_BORROW, MAX, and RESERVE, the following values can be used for type:

- **USER**—User name of the user executing the FlexEnabled application. User names are case sensitive and cannot contain spaces.

For the MAX option, you can also specify ALL_USERS as a value for the USER type. See **MAX** for details.



Note • For all the option keywords except MAX, The ALL_USERS value for the USER type is not supported.

- **GROUP**—Name of the group of users executing the FlexEnabled application. (Before you can restrict an option by a specific group, the options file must also include a **GROUP** option that defines the group.)

For the MAX option, you can also specify ALL_GROUPS as a value for the GROUP type. See **MAX** for details.

- **HOST**—System host name or IP address where the application is executing. Host names are not case sensitive. The IP address can contain wildcard characters.

When using the option keywords EXCLUDE, EXCLUDEALL, EXCLUDE_BORROW, INCLUDE, INCLUDEALL, INCLUDE_BORROW, MAX, and RESERVE, the following values can be used for type:

- **DISPLAY**—Display where the application is displayed. On UNIX, DISPLAY is `/dev/ttyxx` (which is always `/dev/tty` when an application is run in the background) or the X-Display name. On Windows, it is the system name or, in the case of a terminal server environment, the terminal server client name. Display names are case sensitive.
- **INTERNET**—IP address of the system where the application is executing.
- **PROJECT**—LM_PROJECT environment variable set by the user who is executing the FlexEnabled application. Project names are case sensitive. See **Environment Variables**, for more information about this variable.

On Windows (without terminal server), the HOST and DISPLAY names are both set to the system name. For licenses that allow checkouts from a terminal server (TS_OK keyword in the FEATURE line), the USER, HOST, and DISPLAY names can be different from one another.

The types listed above take a single member. For example:

```
EXCLUDE coolsoft USER joe
```

To specify a list of users or hosts, first define the list using the `GROUP` or `HOST_GROUP` option lines, then use the `GROUP` or `HOST_GROUP` type to specify the group name. For example:

```
GROUP stars joe barbara susan  
EXCLUDE coolsoft GROUP stars
```



Note - Changes in the Options file:

- IP address as a *HOST* specification introduced in version 8 vendor daemon.
- Colons in feature names introduced in version 8 vendor daemon.

AUTOMATIC_REREAD

This option applies to all concurrent licenses held in license files or trusted storage.

```
AUTOMATIC_REREAD OFF|ON
```

Controls the automatic rereading of license files and trusted storage when any features are found to have expired. The default when this option is not set is that at midnight each day a check of each license is made to determine if it has expired. When any license is found to have expired, all license files and trusted storage are reread.

To turn off this automatic reread at midnight, enter `AUTOMATIC_REREAD OFF` in the options file.

ACTIVATION_LOWWATER

This option enables license administrators to control the number of activatable licenses that cannot be borrowed by a FlexEnabled client or transferred to another license server.

```
ACTIVATION_LOWWATER entitlementID count  
ACTIVATION_LOWWATER entitlementID:FID=fulfillmentID count
```

For example, the server has an entitlement `AAAA` with 10 activatable licenses. The following line in the options file specifies that 6 licenses from entitlement `AAAA` cannot be borrowed or transferred:

```
ACTIVATION_LOWWATER AAAA 6
```

Table 13-2 - ACTIVATION_LOWWATER Terms

Term	Description
entitlementID	The entitlement ID originally used when requesting license activation.
count	Number of licenses that cannot be borrowed or transferred.
fulfillmentID (optional)	A unique identity for this fulfillment record. Used to specify the fulfillment record to be returned.



Note ▪ If more than one `ACTIVATION_LOWWATER` option for the same `entitlementID` and `fulfillmentID` is specified in the options file, only the first one is considered and subsequent entries are discarded.

ACTIVATION_EXPIRY_DAYS

This option enables license administrators to control the activation request based on the expiration dates mentioned in the options file during activation. The publisher controls the activations based on the number of expiration days. If the expiration date is greater than the date set in the options file, the functionality is not successful.

`ACTIVATION_EXPIRY_DAYS entitlementID days`

`ACTIVATION_EXPIRY_DAYS entitlementID:FID=fulfillmentID days`

Table 13-3 ▪ `ACTIVATION_EXPIRY_DAYS`

Term	Description
entitlementID	The entitlement ID originally used when requesting license activation.
days	Number of days until license will be returned.
fulfillmentID (optional)	A unique identity for this fulfillment record. Used to specify the fulfillment record to be returned.

For example, if there is more than one entry in the options file, the functionality works fine with respect to the first line given in the options file.

`ACTIVATION_EXPIRY_DAYS ENTL-EZCALC 3`

As per the example, if the current date is 11-December-2016 then the client can activate the license until 13-dec-2016. In case of normal activation without the options file, the client can use the license beyond 13-dec-2016.



Note ▪ If more than one `ACTIVATION_EXPIRY_DAYS` option for the same `entitlementID` and `fulfillmentID` with different count of days is specified in the options file, then the functionality always considers the first line and ignores subsequent lines.

BORROW_LOWWATER

This option is used for licenses held in license files. When licenses are available in trusted storage, activation is normally provided instead of BORROW.

`BORROW_LOWWATER feature[:keyword=value] n`

Sets the number of licenses for a BORROW feature that cannot be borrowed.

Table 13-4 • BORROW_LOWWATER Terms

Term	Description
feature	Name of feature being affected.
keyword=value	Feature name modifier to denote a group of licenses. See Also Specifying Features for details.
n	Number of licenses that cannot be borrowed via license borrowing.

For example, a feature f1 has a count of 10 and borrowing is enabled in the application and on the FEATURE line:

```
FEATURE f1 ... 10 ... BORROW SIGN=...
```

The following line in the options file allows only 7 licenses for this feature to be borrowed:

```
BORROW_LOWWATER f1 3
```

DAEMON_SELECT_TIMEOUT

Specifies the threshold for the amount of time (in seconds) that can elapse before a vendor-daemon connection experiences a timeout. The maximum value is **60**; the minimum is **1**.

DAEMON_SELECT_TIMEOUT *value_in_seconds*



Important • You can define the DAEMON_SELECT_TIMEOUT option only when the publisher has enabled it (that is, set the vendor variable `ls_support_custom_daemon_select_timeout` to **1**). Best practice is for publishers to enable DAEMON_SELECT_TIMEOUT only for environments that have limited bandwidth. If this option is not enabled, the timeout value for the vendor-daemon connection is 1 second, by default.

The following rules apply to DAEMON_SELECT_TIMEOUT:

- A value over 60 is always considered **60**.
- A value under 1 is always considered **1**.
- The default value is **1**.

DEBUGLOG

Specifies a location for the debug log output from the vendor daemon associated with this options file. Preceding the *debug_log_path* with a + character appends logging entries; otherwise, the file is overwritten each time the daemon is started. Note that this affects output from only the vendor daemon associated with this options file. The debug log output of the license server manager and any other vendor daemons in the same license file is not captured in this file. Generating debug logs with DEBUGLOG keyword is better alternative than `lmgrd` command line option `-l` when debug log is only required for the vendor daemon. One of the advantage is that server detaches itself from previous debug log after server has been linked to new debug log through `lmswitch` or with server reread (option file DEBUGLOG keyword edited to new debug log).

DEBUGLOG *[+]debug_Log_path* [OBF_ADDMARK] [AUTO_ROLLOVER *size*]

On Windows, path names which include spaces have to be enclosed in double quotes. If `lmgrd` is started as a service, the debug log file is by default generated in the path specified in the options file. It is best practice to set the location of `debug_log_path` to a `ProgramData` sub-folder, since this location has user-write permission by default, which is needed when the license server runs as a service with `LocalService` permission. The `OBF_ADDMARK` enables the affixing of obfuscation markers (`OBF_STRT: username :END_OBF`) to username in debug log. The `AUTO_ROLLOVER` option enables the auto rollover of debug log file, which automatically performs the rollover functionality of debug log file when the debug log file size crosses the specified value. The default value would be 512MB. The automatic rollover will always be performed at midnight.

See Also:

[Configuring the License Server Manager as a Windows Service](#)

[lmswitch](#)

[Debug Log File](#)—Debug log output restricted to that of just the vendor daemon introduced in version 8 vendor daemon.

EXCLUDE

This option applies to concurrent licenses held in license files and trusted storage.

EXCLUDE *feature[:keyword=value]* *type* {*name* | *group_name*}

Excludes a user or predefined group of users from the list of who is allowed to use the feature. EXCLUDE supersedes INCLUDE; conflicts between the EXCLUDE list and the INCLUDE list are resolved by the EXCLUDE taking precedence.

Table 13-5 • EXCLUDE Terms

Term	Description
<i>feature</i>	Name of the feature or package being affected.
<i>keyword=value</i>	Feature name modifier to denote a group of licenses. See Specifying Features for details. For examples of using the modifiers <code>EXPDTE</code> and <code>ENTITLEMENT</code> , see Using the EXPDATE Modifier and Using the ENTITLEMENT Modifier , respectively.
<i>type</i>	One of <code>USER</code> , <code>HOST</code> , <code>DISPLAY</code> , <code>INTERNET</code> , <code>PROJECT</code> , <code>GROUP</code> , or <code>HOST_GROUP</code> . See Specifying License Restrictions Using Type for details.
<i>name</i>	Name of an item of type <i>type</i> for which license usage is excluded.
<i>group_name</i>	Name of the group to exclude. Group names are case sensitive.

For example, the following option syntax excludes the user `hank` from the list of users able to use feature `f1`:

```
EXCLUDE f1 USER hank
```

Using the EXPDATE Modifier

When using `EXPDTE` as a modifier for `INCLUDE` or `EXCLUDE` in the options file, add at least one pooling component keyword in the `FEATURE` or `INCREMENT` line.

Consider the following two INCREMENT lines

```
INCREMENT f1 demo 1.0 31-dec-2017 5 SIGN="<...>"
INCREMENT f1 demo 1.0 30-nov-2017 5 SIGN="<...>"
```

And the following options file line

```
EXCLUDE f1:EXPDATE=31-dec-2017 GROUP stars
```

Because both instances of f1 are in the same license pool, the above options file line excludes them both.

In order to exclude just the later expiring license, ensure each increment line is in a separate pool using a pooling keyword, such as `VENDOR_STRING`, for example:

```
INCREMENT f1 demo 1.0 31-dec-2017 5 VENDOR_STRING="31-dec-2017" SIGN="<...>"
INCREMENT f1 demo 1.0 30-nov-2017 5 VENDOR_STRING="30-nov-2017" SIGN="<...>"
```

With this example, the vendor daemon needs to be built with vendor variables `ls_compare_vendor_on_increment = 1` and `ls_compare_vendor_on_upgrade = 1` in order to turn on pooling with the vendor string. It should also be noted that feature version is a pooling keyword; the vendor string is needed here because these increment lines are for the same feature version.

For a list of pooling keywords, see [FEATURE and INCREMENT Lines](#) on page 34.

Using the ENTITLEMENT Modifier

The ENTITLEMENT modifier enables the license administrator to differentiate between features that have identical names but belong to different products. In combination with the EXCLUDE option, the license administrator can restrict the concurrent checkout of a feature when the license originates from trusted storage. (For allowing/restricting the checkout of activatable features, use the option `INCLUDE_ENTITLEMENT` or `EXCLUDE_ENTITLEMENT`.)

To use the ENTITLEMENT modifier, you must set the variable `ls_entitlement_based_pooling` in `lsvendor.c` to 1 (default). This enables pooling based on the entitlement ID. For more information, see the *Programming Reference for License File-Based Licensing*, chapter *Customizing the Vendor Daemon*.

Example

Consider the following scenario. Licenses are available for the following features—which have identical names—of two products:

- Product EZCALC-N:
 - Entitlement ID: ENTL-ADD-1
 - Hybrid licenses: 3
 - Feature(s):

```
INCREMENT ADD demo 1.0 permanent 1 SIGN="XXX"
INCREMENT SUBTRACT demo 1.0 permanent 1 SIGN="XXX"
INCREMENT MULTIPLY demo 1.0 permanent 1 SIGN="XXX"
```
- Product EZCALC-S:
 - Entitlement ID: ENTL-ADD-2
 - Hybrid licenses: 3
 - Feature(s):

```
INCREMENT ADD demo 1.0 permanent 1 SIGN="XXX"
INCREMENT SUBTRACT demo 1.0 permanent 1 SIGN="XXX"
INCREMENT MULTIPLY demo 1.0 permanent 1 SIGN="XXX"
```


The license administrator wants to deny user Joanna access to feature ADD from EZCALC-N (entitlement ID ENTL-ADD-1). The administrator therefore adds the following line to the options file:

```
EXCLUDE ADD:ENTITLEMENT=ENTL-ADD-1 USER Joanna
```

EXCLUDE_BORROW

This option is used for licenses held in license files. When licenses are available in trusted storage, activation is normally provided instead of BORROW.

```
EXCLUDE_BORROW feature[:keyword=value] type \  
                {name | group_name}
```

Excludes a user or predefined group of users from the list of who is allowed to borrow licenses for this BORROW feature. EXCLUDE_BORROW supersedes INCLUDE_BORROW; conflicts between the EXCLUDE_BORROW list and the INCLUDE_BORROW list are resolved by the EXCLUDE_BORROW taking precedence.

Table 13-6 ■ EXCLUDE_BORROW Terms

Term	Description
<i>feature</i>	Name of the feature being affected.
<i>keyword=value</i>	Feature name modifier to denote a group of licenses. See Specifying Features for details.
<i>type</i>	One of USER, HOST, DISPLAY, INTERNET, PROJECT, GROUP, or HOST_GROUP. See Specifying License Restrictions Using Type for details.
<i>name</i>	Name of an item of type <i>type</i> for which license borrowing is excluded.
<i>group_name</i>	Name of the group to exclude from borrowing. Group names are case sensitive.

For example, the following option syntax excludes the user fred from the list of users able to borrow feature f1 (assuming the feature has the BORROW attribute):

```
EXCLUDE_BORROW f1 USER fred
```

EXCLUDE_ENTITLEMENT

This option only applies to licenses held in trusted storage and supplied using activation.

```
EXCLUDE_ENTITLEMENT entitlementId type {name | group_name}
```

Excludes a user or pre-defined group of users, etc., from the list of who is allowed to activate the licenses contained in a fulfillment record held in trusted storage. EXCLUDE_ENTITLEMENT supersedes INCLUDE_ENTITLEMENT; conflicts between the EXCLUDE_ENTITLEMENT list and the INCLUDE_ENTITLEMENT list are resolved by the EXCLUDE_ENTITLEMENT taking precedence.

Table 13-7 • EXCLUDE_ENTITLEMENT Terms

Term	Description
<i>entitlementId</i>	The entitlement Id used when requesting a license activation.
<i>type</i>	One of USER, HOST, GROUP, or HOST_GROUP. See Specifying License Restrictions Using Type for details.
<i>name</i>	Name of an item of type <i>type</i> for which license usage is excluded.
<i>group_name</i>	Name of the group to exclude. Group names are case sensitive.

For example, the following option syntax excludes the user pete from the list of users able to activate licenses provided in the fulfillment record specified by the entitlement Id AB456:

```
EXCLUDE_ENTITLEMENT AB456 USER pete
```

EXCLUDEALL

This option applies to concurrent licenses held in license files and trusted storage.

```
EXCLUDEALL type {name | group_name}
```

Excludes a user or predefined group of users from the list of who is allowed to use all features served by this vendor daemon.

Table 13-8 • EXCLUDEALL Terms

Term	Description
<i>type</i>	One of USER, HOST, DISPLAY, INTERNET, PROJECT, GROUP, or HOST_GROUP. See Specifying License Restrictions Using Type for details.
<i>name</i>	Name of an item of type <i>type</i> for which license usage is excluded.
<i>group_name</i>	Name of the group to exclude. Group names are case sensitive.

For example, the following option syntax excludes any user on the system called chaos using all features served by this vendor daemon:

EXCLUDEALL HOST chaos

EXCLUDEALL_ENTITLEMENT

This option applies to activatable and hybrid licenses held in trusted storage.

EXCLUDEALL_ENTITLEMENT *type* {*name* | *group_name*}

Excludes a user or predefined group of users from the list of who is allowed to activate the licenses contained in all fulfillment records held in trusted storage.

EXCLUDEALL_ENTITLEMENT supersedes INCLUDEALL_ENTITLEMENT; conflicts between the EXCLUDEALL_ENTITLEMENT list and the INCLUDEALL_ENTITLEMENT list are resolved by the EXCLUDEALL_ENTITLEMENT taking precedence. If a user, host, group, or host group is specified neither on the EXCLUDEALL_ENTITLEMENT list nor on the INCLUDEALL_ENTITLEMENT list, activation is denied.

Table 13-9 • EXCLUDEALL_ENTITLEMENT Terms

Term	Description
<i>type</i>	One of USER, HOST, GROUP, or HOST_GROUP. See Specifying License Restrictions Using Type for details.
<i>name</i>	Name of an item of type <i>type</i> for which license usage is excluded.
<i>group_name</i>	Name of the group to exclude. Group names are case sensitive.

For example, the following option syntax excludes the user Chris from the list of users able to activate the licenses provided in all fulfillment record held in trusted storage:

EXCLUDEALL_ENTITLEMENT USER Chris



Note • It is recommended that you do not combine EXCLUDEALL_ENTITLEMENT and EXCLUDE_ENTITLEMENT, INCLUDE_ENTITLEMENT, or INCLUDEALL_ENTITLEMENT for the same user.

FQDN_MATCHING

This option applies to all licenses held in license files or trusted storage.

FQDN_MATCHING exact | lenient

Sets the level to which host names used in HOST type-specifiers must match the host name sent by the FlexEnabled application. The application is configured to send either its host name or its fully qualified domain name (FQDN) to the vendor daemon for validation with HOST type-specifiers. Check with your software publisher to determine fully qualified domain name support.

Table 13-10 • FQDN_MATCHING Terms

Term	Description
exact	The host name in the HOST type specifier must match in content and format to that sent by the application. This is the default setting.
lenient	The host name sent by the application needs to match to the extent supplied in the HOST type specifier or by the application, which ever is less restrictive.

Only the last FQDN_MATCHING keyword in the options file has effect; all others are ignored.

The following table shows the outcome of matching attempts between HOST type-specifiers in the options file and host names sent by the application.

Table 13-11 • Host Name Matching Matrix

Options File Settings		Application configured for FQDN—sends myhost.abc.com	Application not configured for FQDN—sends myhost
FQDN_MATCHING exact	INCLUDE <i>feature</i> HOST myhost	no	yes
	INCLUDE <i>feature</i> HOST myhost.abc.com	yes	no
FQDN_MATCHING lenient	INCLUDE <i>feature</i> HOST myhost	yes	yes
	INCLUDE <i>feature</i> HOST myhost.abc.com	yes	yes

Examples

Consider the following example that demonstrates restrictive host name matching:

```
INCLUDE f1 HOST myhost.abc.com
```

```
FQDN_MATCHING exact
```

This includes myhost.abc.com on the list of hosts able to use feature f1. Furthermore, the host name sent by the application must be a fully qualified domain name that matches myhost.abc.com exactly.

In contrast, consider this example, which is less restrictive:

```
INCLUDE f2 HOST myhost.abc.com
```

```
FQDN_MATCHING lenient
```

This includes myhost.abc.com on the list of hosts able to use feature f2. The license rights are authenticated and a checkout allowed if any of the following match:

- The FQDN—myhost.abc.com
- The host name—myhost
- The domain name—.abc.com

The example below is even more lenient:

```
INCLUDE f2 HOST myhost
```

```
FQDN_MATCHING lenient
```

This includes the host name, myhost, on the list of hosts for feature f3. Since lenient matching is specified, host names such as myhost, myhost.abc.com, and myhost.xyz.com match, whereas yourhost or yourhost.abc.com do not match.

See Also

[Specifying License Restrictions Using Type](#)

FQDN_MATCHING introduced in version 9.3 client library and vendor daemon.

GROUP

Defines a group of users for use in INCLUDE, INCLUDEALL, INCLUDE_ENTITLEMENT, INCLUDEALL_ENTITLEMENT, EXCLUDE, EXCLUDEALL, EXCLUDE_ENTITLEMENT, EXCLUDEALL_ENTITLEMENT, and RESERVE option lines.

```
GROUP group_name user_list
```

Table 13-12 • GROUP Terms

Term	Description
<i>group_name</i>	Name of the group being defined. Group names are case sensitive.
<i>user_list</i>	List of user names in that group. Names are case sensitive and cannot contain spaces. Set the GROUPCASEINSENSITIVE options file keyword to turn on case insensitivity. See GROUPCASEINSENSITIVE .

To create a large user group, define several GROUP lines each containing up to the maximum of 4,000 characters. All the users will be placed in a single group: Multiple GROUP lines for the same group name add all the specified users into the group.

For example, the following option syntax defines the group Hackers consisting of bob, howard, and james:

```
GROUP Hackers bob howard james
```



Note • USER_GROUP is an alias for GROUP.

GROUPCASEINSENSITIVE

Sets case sensitivity for user lists.

GROUPCASEINSENSITIVE OFF|ON

If set to **ON**, user names specified with the **GROUP** keyword are treated as case-insensitive.

By default, **GROUPCASEINSENSITIVE** is **OFF**, and user names are treated as case-sensitive.

HOST_GROUP

Defines a group of hosts for use in **INCLUDE**, **INCLUDEALL**, **INCLUDE_ENTITLEMENT**, **INCLUDEALL_ENTITLEMENT**, **EXCLUDE**, **EXCLUDEALL**, **EXCLUDE_ENTITLEMENT**, **EXCLUDEALL_ENTITLEMENT**, and **RESERVE** option lines.

HOST_GROUP *group_name host_list*

Multiple **HOST_GROUP** lines add all the specified hosts into the group.

Table 13-13 • HOST_GROUP Terms

Term	Definition
group_name	Name of the group being defined. Host group names are case sensitive.
host_list	List of host names in that group. Host names are case sensitive.

For example, the following option syntax defines the host group **Pacific** consisting of **tokyo**, **seattle**, and **auckland**:

```
HOST_GROUP Pacific tokyo seattle auckland
```

Anywhere a host name can be used in an options file, an IP address can be used instead.

INCLUDE

This option applies to concurrent licenses held in license files and trusted storage.

INCLUDE *feature[:keyword=value] type {name | group_name}*

Includes a user or predefined group of users in the list of who is allowed to use licenses for this feature. Any user who is not in an **INCLUDE** or **INCLUDEALL** statement is not allowed to use that feature. **EXCLUDE** supersedes **INCLUDE**; conflicts between the **EXCLUDE** list and the **INCLUDE** list are resolved by the **EXCLUDE** taking precedence.

Table 13-14 • INCLUDE Terms

Term	Definition
feature	Name of the feature or package being affected.

Table 13-14 ■ INCLUDE Terms

Term	Definition
keyword=value	Feature name modifier to denote a group of licenses. See Specifying Features for details. For examples of using the modifiers EXPDATE and ENTITLEMENT, see Using the EXPDATE Modifier and Using the ENTITLEMENT Modifier , respectively.
type	One of USER, HOST, DISPLAY, INTERNET, PROJECT, GROUP, or HOST_GROUP. See Specifying License Restrictions Using Type for details.
name	Name of an item of type <i>type</i> for which license usage is included.
group_name	Name of the group for which license usage is included. Group names are case sensitive.

To include user bob in the list of users able to use feature f1:

```
INCLUDE f1 USER bob
```

The include list is created from all the INCLUDEALL and INCLUDE lines in the options file.



Note ■ INCLUDE is required for USER_BASED or HOST_BASED features. The license administrator specifies which users are allowed to use the product, via INCLUDE, and the license limits the number of users that are INCLUDED. In a USER_BASED or HOST_BASED license model, users (or predefined groups of users) who are not listed with the INCLUDE keyword cannot check out a license.

Using the EXPDATE Modifier

When using EXPDATE as a modifier for INCLUDE or EXCLUDE in the options file, add at least one pooling component keyword in the FEATURE or INCREMENT line.

Consider the following two INCREMENT lines

```
INCREMENT f1 demo 1.0 31-dec-2017 5 SIGN="<...>"
INCREMENT f1 demo 1.0 30-nov-2017 5 SIGN="<...>"
```

And the following options file line

```
INCLUDE f1:EXPDATE=30-nov-2017 USER bob
```

Because both instances of f1 are in the same license pool, the above options file line includes them both.

In order to include just the earlier expiring license, ensure each increment line is in a separate pool using a pooling keyword, such as VENDOR_STRING, for example:

```
INCREMENT f1 demo 1.0 31-dec-2017 5 VENDOR_STRING="31-dec-2017" SIGN="<...>"
INCREMENT f1 demo 1.0 30-nov-2017 5 VENDOR_STRING="30-nov-2017" SIGN="<...>"
```

With this example, the vendor daemon needs to be built with vendor variables `ls_compare_vendor_on_increment = 1` and `ls_compare_vendor_on_upgrade = 1` in order to turn on pooling with the vendor string. It should also be noted that feature version is a pooling keyword; the vendor string is needed here because these increment lines are for the same feature version.

For a list of pooling keywords, see [FEATURE and INCREMENT Lines](#) on page 34.

Using the ENTITLEMENT Modifier

The ENTITLEMENT modifier enables the license administrator to differentiate between features that have identical names but belong to different products. In combination with the INCLUDE option, the license administrator can allow or restrict the concurrent checkout of a feature when the license originates from trusted storage. (For allowing/restricting the checkout of activatable features, use the option INCLUDE_ENTITLEMENT or EXCLUDE_ENTITLEMENT.)

To use the ENTITLEMENT modifier, you must set the variable `ls_entitlement_based_pooling` in `lsvendor.c` to 1 (default). This enables pooling based on the entitlement ID. For more information, see the *Programming Reference for License File-Based Licensing*, chapter *Customizing the Vendor Daemon*.

Example

Consider the following scenario. Licenses are available for the following features—which have identical names—of two products:

- Product EZCALC-N:
 - Entitlement ID: ENTL-ADD-1
 - Hybrid licenses: 3
 - Feature(s): INCREMENT ADD demo 1.0 permanent 1 SIGN="XXX"
INCREMENT SUBTRACT demo 1.0 permanent 1 SIGN="XXX"
INCREMENT MULTIPLY demo 1.0 permanent 1 SIGN="XXX"
- Product EZCALC-S:
 - Entitlement ID: ENTL-ADD-2
 - Hybrid licenses: 3
 - Feature(s): INCREMENT ADD demo 1.0 permanent 1 SIGN="XXX"
INCREMENT SUBTRACT demo 1.0 permanent 1 SIGN="XXX"
INCREMENT MULTIPLY demo 1.0 permanent 1 SIGN="XXX"

The license administrator wants to allow/restrict access as follows:

- Only the user Joanna should be allowed to check out feature ADD from EZCALC-N (entitlement ID ENTL-ADD-1).
- Other users should be allowed to check out features SUBTRACT and MULTIPLY from EZCALC-N (entitlement ID ENTL-ADD-1), but not feature ADD.
- There should be no restrictions to checking out features from EZCALC-S.

To restrict access as outlined, the license administrator adds the following line to the options file:

```
INCLUDE ADD:ENTITLEMENT=ENTL-ADD-1 USER Joanna
```

INCLUDE_BORROW

This option is used for licenses held in license files. When licenses are available in trusted storage, normally activation is provided instead of BORROW.


```
INCLUDE_BORROW feature[:keyword=value] type {name | group_name}
```

Includes a user or predefined group of users in the list of who is allowed to borrow the BORROW feature. Anyone not in an INCLUDE_BORROW statement is not allowed to borrow licenses. EXCLUDE_BORROW supersedes INCLUDE_BORROW; conflicts between the EXCLUDE_BORROW list and the INCLUDE_BORROW list are resolved by the EXCLUDE_BORROW taking precedence.

Table 13-15 • INCLUDE_BORROW Terms

Term	Definition
<i>feature</i>	Name of the feature and package being affected. Specify the package name and the feature name or specify only the package name.
<i>keyword=value</i>	Feature name modifier to denote a group of licenses. See Specifying Features for details.
<i>type</i>	One of USER, HOST, DISPLAY, INTERNET, PROJECT, GROUP, or HOST_GROUP. See Specifying License Restrictions Using Type for details.
<i>name</i>	Name of an item of type <i>type</i> for which license borrowing is included.
<i>group_name</i>	Name of the group for which license borrowing is included. Group names are case sensitive.

For example, the following option syntax includes user tom in the list of users able to borrow feature f1:

```
INCLUDE_BORROW f1 USER tom
```



Note • For USER_BASED or HOST_BASED features a user or predefined group of users must be on both an INCLUDE list and an INCLUDE_BORROW list to borrow a feature.

INCLUDE_ENTITLEMENT

This option only applies to licenses held in trusted storage.

```
INCLUDE_ENTITLEMENT entitlementId type {name | group_name}
```

Includes a user or predefined group of users in the list of who is allowed to activate the licenses contained in a fulfillment record held in trusted storage. EXCLUDE_ENTITLEMENT supersedes INCLUDE_ENTITLEMENT; conflicts between the EXCLUDE_ENTITLEMENT list and the INCLUDE_ENTITLEMENT list are resolved by the EXCLUDE_ENTITLEMENT taking precedence.

Table 13-16 • INCLUDE_ENTITLEMENT Terms

Term	Definition
<i>entitlementId</i>	The entitlement Id originally used when requesting a license activation.

Table 13-16 • INCLUDE_ENTITLEMENT Terms

Term	Definition
type	One of USER, HOST, GROUP, or HOST_GROUP. See Specifying License Restrictions Using Type for details.
name	Name of an item of type <i>type</i> for which license usage is included.
group_name	Name of the group to include. Group names are case sensitive.

For example, the following option syntax includes the user `claire` in the list of users able to activate licenses provided in the fulfillment record specified by the entitlement Id `AB456`:

```
INCLUDE_ENTITLEMENT AB456 USER claire
```

Example for Include variations with hybrid trusted storage:

This example demonstrates the behavior of INCLUDE, INCLUDE_BORROW and INCLUDE_ENTITLEMENT on *feature F1*. *F1* originates from a served trusted storage hybrid license with *EntitlementID ENTLID1*.

Table 13-17 • Example for INCLUDE, INCLUDE_BORROW, INCLUDE_ENTITLEMENT

Option file Contains	Behavior
INCLUDE F1 USER John	concurrent checkout limited to John? Yes certificate borrow limited to John? Yes activation borrow limited to John? No
INCLUDE_BORROW F1 USER John	concurrent checkout limited to John? No certificate borrow limited to John? Yes activation borrow limited to John? No
INCLUDE F1 USER John INCLUDE_ENTITLEMENT ENTLID1 USER John	concurrent checkout limited to John? Yes certificate borrow limited to John? Yes activation borrow limited to John? Yes
INCLUDE_ENTITLEMENT ENTLID1 USER John	concurrent checkout limited to John? No certificate borrow limited to John? No activation borrow limited to John? Yes

Another example: *ENTLID12* contains hybrid features *F1* and *F2*. The following options file allows only John to perform concurrent checkout, certificate borrow or activation borrow on all *ENTLID12*'s features:

```
INCLUDE F1 USER John
```

```
INCLUDE F2 USER John
```

INCLUDE_ENTITLEMENT ENTID12 USER John



Note • Activation borrow always activates all features from an entitlement.

INCLUDEALL

This option applies to concurrent licenses held in license files and trusted storage.

INCLUDEALL *type* {*name* | *group_name*}

Includes a user or predefined group of users in the list of who is allowed to use all features served by this vendor daemon.

Table 13-18 • INCLUDEALL Terms

Term	Definition
type	One of USER, HOST, DISPLAY, INTERNET, PROJECT, GROUP, or HOST_GROUP. See Specifying License Restrictions Using Type for details.
name	Name of an item of type <i>type</i> for which license usage is included.
group_name	Name of the group to include. Group names are case sensitive.

For example, the following option syntax allows the user jane to use all features served by this vendor daemon:

INCLUDEALL USER jane

The include list is created from all the INCLUDEALL and INCLUDE lines in the options file.

INCLUDEALL_ENTITLEMENT

This option applies to activatable and hybrid licenses held in trusted storage.

INCLUDEALL_ENTITLEMENT *type* {*name* | *group_name*}

Includes a user or predefined group of users in the list of who is allowed to activate the licenses contained in all fulfillment records held in client-side trusted storage.

EXCLUDEALL_ENTITLEMENT supersedes INCLUDEALL_ENTITLEMENT; conflicts between the EXCLUDEALL_ENTITLEMENT list and the INCLUDEALL_ENTITLEMENT list are resolved by the EXCLUDEALL_ENTITLEMENT taking precedence. If a user, host, group, or host group is specified neither on the EXCLUDEALL_ENTITLEMENT list nor on the INCLUDEALL_ENTITLEMENT list, activation is denied.

Table 13-19 • INCLUDEALL_ENTITLEMENT Terms

Term	Description
type	One of USER, HOST, GROUP, or HOST_GROUP. See Specifying License Restrictions Using Type for details.

Table 13-19 • INCLUDEALL_ENTITLEMENT Terms

Term	Description
name	Name of an item of type <i>type</i> for which license usage is included.
group_name	Name of the group to include. Group names are case sensitive.

For example, the following option syntax includes the user Susan in the list of users allowed to activate the licenses provided in all fulfillment records held in trusted storage:

```
INCLUDEALL_ENTITLEMENT USER Susan
```



Note • It is recommended that you do not combine INCLUDEALL_ENTITLEMENT and EXCLUDE_ENTITLEMENT, INCLUDE_ENTITLEMENT, or EXCLUDEALL_ENTITLEMENT for the same user.

LINGER

This option applies to concurrent licenses held in license files and trusted storage.

```
LINGER feature[:keyword=value] seconds
```

A lingering license stays checked out for a specified period of time beyond its checkin or FlexEnabled application exit, whichever comes first. The linger time may have been configured by the software publisher in the FlexEnabled application. When this is the case, then the longer linger time is applied. Thus you can set a longer linger time than configured by the software publisher but not shorten the linger time.

In case, if the set linger time of a user is more than the feature expiration time, a warning will be displayed stating "Linger time overlaps expiration time", as the user can not use a feature after its expiration time. Hence the linger time is automatically reset to end on the expiration time of the feature.

Table 13-20 • LINGER Terms

Term	Definition
feature	Name of the feature.
keyword=value	Feature name modifier to denote a group of licenses. See Specifying Features for details.
seconds	Number of seconds the license lingers. The software publisher sets a minimum value. If you specify a value for <i>seconds</i> that is smaller than the minimum, the minimum is used.

For example, the following option syntax sets the linger value for feature f1 to one hour (3600 seconds):

LINGER f1 3600

The actual linger time varies somewhat since the vendor daemon checks all lingering licenses just once per minute. Also if a new license request is made that would otherwise be denied, a check of the lingering licenses is made immediately to attempt to satisfy the new request.

MAX

This option applies to concurrent licenses held in license files and trusted storage.

MAX *num_lic* *feature[:keyword=value]* *type* {*name* | *group_name*}

Limits usage for a user or group.



Note - When this option is used for a group, it limits the usage for a entire group, not just individual users within that group.

Table 13-21 • MAX Terms

Term	Description
<i>num_lic</i>	Usage limit for this user or group.
<i>feature</i>	Feature or package this limit applies to.
<i>keyword=value</i>	Feature name modifier to denote a group of licenses. See Specifying Features for details.
<i>type</i>	One of USER, HOST, DISPLAY, INTERNET, PROJECT, GROUP, or HOST_GROUP. See Specifying License Restrictions Using Type for details.
<i>name</i>	Name of an item of type <i>type</i> for which usage is limited. To avoid having to repeat the MAX line multiple times to restrict all users, you can use one MAX line that specifies ALL_USERS as the value for the type USER (for example, USER ALL_USERS). See for USER and GROUP Examples more information.
<i>group_name</i>	Name of the group to limit. Group names are case sensitive. (The options file must include a GROUP line that defines this group.) To avoid having to repeat the MAX line multiple times to restrict all groups defined in the options file, you can use one MAX line that specifies ALL_GROUPS as the value for the type GROUP (for example, GROUP ALL_GROUPS). See USER and GROUP Examples for more information.

USER and GROUP Examples

The following are examples of using the USER and GROUP types for the MAX option.



Note - Never position a MAX line that restricts USER *user_name* for a specific feature after a MAX line that restricts USER ALL_USERS for the same feature. Likewise, never position a MAX line that restricts GROUP *group_name* for a specific feature after a MAX line that restricts GROUP ALL_GROUPS for the same feature. The positioning invalidates the restriction in the later line.

- To limit the user jan to five licenses for feature f1, include the following line in the options file:

```
MAX 5 f1 USER jan
```

- To limit all users to five licenses for feature f1, include the following line:

```
MAX 5 f1 USER ALL_USERS
```

- To limit the user jan to five licenses for feature f1 and all other users to four licenses for the same feature, include the following lines:

```
MAX 5 f1 USER jan  
MAX 4 f1 USER ALL_USERS
```

- To limit group DEV to three license for feature f2 and all other groups to one license for the same feature, include these lines:

```
MAX 3 f2 GROUP DEV  
MAX 1 f2 GROUP ALL_GROUPS
```

- If you attempt to position a MAX line that specifies ALL_USERS (or ALL_GROUPS) before a MAX line that specifies a single user (or single group), FlexNet Publisher cannot enforce the restriction in the second line because the user or group in the second line (in this case, the DEV group) is a part of ALL_USERS or ALL_GROUPS in the first line.

For example, the following line positioning is *invalid*:

```
MAX 1 f2 GROUP ALL_GROUPS  
MAX 3 f2 GROUP DEV
```

Queueing Behavior When Requested Licenses Exceed MAX Limit

For current version vendor daemons, if queueing is allowed by the application, requests for licenses that exceed the limit set by the MAX keyword will be denied. From the above example, if the user jan requests six licenses for feature f1, the request will be denied. Requests from users or groups within the MAX limit that exceed the number of available licenses will be queued. For example, if the license file includes ten licenses for feature f1 and nine of those licenses are already checked out, a request from the user jan for two licenses will be queued.

MAX_BORROW_HOURS

This option is used for licenses held in license files. When licenses are available in trusted storage, normally activation is provided instead of BORROW.

`MAX_BORROW_HOURS feature[:keyword=value] num_hours`

Changes the maximum period a license can be borrowed from that specified in the license file for *feature*. The new period must be less than that in the license file. If multiple `MAX_BORROW_HOURS` keywords appear in the options file, only the last one is applied to *feature*.

Table 13-22 • `MAX_BORROW_HOURS` Terms

Term	Description
<i>feature</i>	Feature this borrow period applies to. The <i>feature</i> must have <code>BORROW</code> enabled.
<i>keyword=value</i>	Feature name modifier to denote a group of licenses. See Specifying Features for details.
<i>num_hours</i>	Number of hours in the new borrow period. This value must be less than that specified in the license file for <i>feature</i> (the default, if not specified, is 168 hours).

MAX_CONNECTIONS

This option limits the maximum number of connections with vendor daemon.

Table 13-23 • `MAX_CONNECTIONS` Terms

Term	Description
<i>num_conn</i>	Connection limit for the vendor daemon. The range is (31 - 10,00,000). The default value is 10,00,000 (if not specified).

On non-Windows (POSIX-compliant) platforms, the vendor daemon will consider the minimum value out of the following and `(systemLimit - certainReserve)` will be used as the limit:

- 'getrlimit()' call will be made once at vendor daemon startup to calculate the system limit for the number of open file descriptors
- the value set for '`MAX_CONNECTIONS`'.

Once connection limit has reached, the vendor daemon disconnects further connections with error code as `-237(LM_VD_MAX_CLIENTS_REACHED)`. Vendor daemon holds buffer of 30 connections for interaction with important utilities (for example: `lmreread`, `lmstat`), in case connection limit is reaching towards threshold.



Important • The older clients (version < 11.16.3) may receive error code as `-140(LM_BADCOMMAND)` while interacting with vendor daemon and `MAX_CONNECTIONS` limit is already reached.

MAX_OVERDRAFT

This option applies to concurrent licenses held in license files and trusted storage.

MAX_OVERDRAFT *feature[:keyword=value] num_lic*

Limits OVERDRAFT license usage below the OVERDRAFT allowed by the license file.

Table 13-24 • MAX_OVERDRAFT Terms

Term	Description
<i>feature</i>	Feature this limit applies to.
<i>keyword=value</i>	Feature name modifier to denote a group of licenses. See Specifying Features for details.
<i>num_lic</i>	Usage limit for this user or group.

NOLOG

Suppresses logging the selected type of event in the debug log file.

NOLOG {IN | OUT | DENIED | QUEUED | UNSUPPORTED}

Table 13-25 • NOLOG Terms

Entry	Description
NOLOG IN	Turns off logging of checkins. Two separate NOLOG lines are required to turn off logging of checkouts (including COAVAIL requests and queued requests).
NOLOG OUT	Turns off logging of checkouts.
NOLOG DENIED NOLOG QUEUED	Turns off logging of checkouts and queued requests. License administrators use this option to reduce the size of the debug log file. However, it can reduce the usefulness of the debug log when debugging license server problems.
NOLOG UNSUPPORTED	Suppresses UNSUPPORTED messages in the debug log. This suppresses error messages in the debug log that report a failure due to the feature being unsupported.

See Also

[lmswitch](#)

Programming Reference for License File-Based Licensing for details about the LM_CO_AVAIL_NOWAIT flag

REPORTLOG

REPORTLOG specifies the report log file for this vendor daemon. It is recommended preceding the `report_log_path` with a + character to append logging entries; otherwise, the file is overwritten each time the daemon is started.

REPORTLOG *[+]report_Log_path* *[HIDE_User]*

On Windows, path names that include spaces have to be enclosed in double quotes. If `lmgrd` is started as a service, the default location for the report log file is the current working directory unless a fully qualified path is specified.

The optional parameter `[HIDE_USER]` can be specified so that any user names will be SHA2 hashed before being written to the encrypted `REPORTLOG`.



Note ▪ *FlexNet Manager is a separate product available from Revenera, it is used to process report log files. FlexNet Manager processes only report log files, not debug log files. On Windows, It is best practice to set the location of `report_log_path` to a `ProgramData` sub-folder, since this location has user-write permission by default, which is needed when the license server runs as a service with `LocalService` permission. If the option `HIDE_USER` is used, then user names in the processed report log file obtained from FlexNet Manager will remain obfuscated.*

Reporting on Projects with `LM_PROJECT`

The FlexNet Manager report writer reports on projects. A project is set up by having all users working on the same project set their `LM_PROJECT` environment variable (or registry on Windows) to a string that describes the project. FlexNet Manager groups usage by project, as defined by what `LM_PROJECT` was set to when the application was run.

See Also

[Configuring the License Server Manager as a Windows Service](#)
[Environment Variables](#)
[Report Log File](#)

RESERVE

This option applies to concurrent licenses held in license files and trusted storage.

`RESERVE num_lic feature[:keyword=value] type {name | group_name}`

Reserves licenses for a specific user.

Table 13-26 ▪ RESERVE Terms

Term	Description
<i>num_lic</i>	Number of licenses to reserve for this user or group.
<i>feature</i>	Feature or package this reservation applies to.
<i>keyword=value</i>	Feature name modifier to denote a group of licenses. See Specifying Features for details.
<i>type</i>	One of USER, HOST, DISPLAY, INTERNET, PROJECT, GROUP, or HOST_GROUP. See Specifying License Restrictions Using Type for details.
<i>name</i>	Name of an item of type <i>type</i> for which license usage is reserved.

Table 13-26 • RESERVE Terms

Term	Description
group_name	Name of group for which license usage is reserved. Group names are case sensitive.

For example, the following option syntax reserves one license of feature f1 for user mel:

```
RESERVE 1 f1 USER mel
```

If you want to reserve a license for *each* of several users or groups, you must use a separate RESERVE line for each user or group. If a package name is specified, all components that comprise the package are reserved.

The RESERVE keyword should not be used on packages and package suites that also use the SUITE_RESERVED option. The RESERVE keyword in the options file includes static information about the reservation policy. The SUITE_RESERVED option reserves a set of package components. Once one package component is checked out, all the other components are reserved for that same user. When the license keys are checked out by a user, the SUITE_RESERVED option within the Package will dictate additional reservation policy which can dynamically change depending on the product usage pattern. Because these options present conflicting reserve parameters, they can not be used together. The total number of reservations supported per options file is 10000.



Note • Any licenses reserved for a user are dedicated to that user. Even when that user is not actively using the license, it is unavailable to other users. However, a RESERVED license does not cause usage to be reported by FlexNet Manager if the license is not actually in use.

TIMEOUT

This option applies to concurrent licenses held in license files and trusted storage.

```
TIMEOUT feature[:keyword=value] seconds
```

Sets the time after which an inactive license is freed and reclaimed by the vendor daemon.



Note • The software publisher must have enabled this feature in the FlexEnabled application for it to work. Contact your software publisher to find out if this feature is implemented.

Table 13-27 • TIMEOUT Terms

Term	Description
feature	Name of the feature.
keyword=value	Feature name modifier to denote a group of licenses. See Specifying Features for details.

Table 13-27 • TIMEOUT Terms

Term	Description
<i>seconds</i>	Number of seconds after which inactive license is reclaimed. The software publisher sets a minimum value. If you specify a value for <i>seconds</i> that is smaller than the minimum or greater than the minimum, the greater value is used.

For example, the following option syntax sets the timeout for feature f1 to one hour (3600 seconds):

```
TIMEOUT f1 3600
```

TIMEOUT checks in the licenses if the FlexEnabled application has been inactive for a period longer than the specified time period. The daemon declares a process inactive when it has not received heartbeats from it whereas an active FlexEnabled application sends heartbeats. A TIMEOUT line must be present in the options file in order to take advantage of this feature.

See Also

Programming Reference for License File–Based Licensing for the vendor variable **ls_prefer_feature_timeout_over_tcp**.

TIMEOUTALL

This option applies to concurrent licenses held in license files and trusted storage.

```
TIMEOUTALL seconds
```

Same as TIMEOUT, but applies to all features.

See Also

Programming Reference for License File–Based Licensing for the vendor variable **ls_prefer_feature_timeout_over_tcp**.

How the Vendor Daemon Uses the Options File

When the vendor daemon is started by `lmadmin` or `lmgrd`, the vendor daemon reads its options file. There is only one options file per vendor daemon and each vendor daemon needs its own options file. For any changes in an options file to take effect, the vendor daemon must read its options file. The `lmreread` utility causes the vendor daemon to reread its options file.

The `lmreread` utility enhanced in version 8.0 vendor daemon so that it causes the vendor daemon to reread the options file. If you are using earlier versions, the vendor daemon must be stopped and restarted in order for the options file to be reread.

Rules of Precedence in Options Files

Rules of precedence take effect when INCLUDE and EXCLUDE statements are combined in the same options file and control access to the same feature (in license files) or fulfillment record (in trusted storage). The following define the precedence when both types of statements appear together:

- If there is only an EXCLUDE list, everyone who is not on the list is allowed to use the feature.
- If there is only an INCLUDE list, only those users on the list are allowed to use the feature.
- If neither list exists, everyone is allowed to use the feature.
- The EXCLUDE list is checked before the INCLUDE list; someone who is on both lists is not allowed to use the feature.

Once you create an INCLUDE or EXCLUDE list, everyone else is *implicitly* outside the group. This feature allows you, as a license administrator, the ability to control licenses without having to *explicitly* list each user that you wish to allow or deny access to. In other words, there are two approaches; you either:

- Give most users access and list only the exceptions, or
- Severely limit access and list only the those users that have access privileges

Options File Examples

The following information gives some examples of options files intended to illustrate ways to effectively control access to your licenses.

Simple Options File Example

```
RESERVE 1 compile USER robert  
RESERVE 3 compile HOST mainline  
EXCLUDE compile USER lori  
NOLOG QUEUED
```

This options file restricts the use of concurrent licenses as follows:

- Reserves one license for the feature `compile` for the user `robert`.
- Reserves three licenses for the feature `compile` for anyone on the system with the host name `mainline`.
- Prevents the user `lori` from using the `compile` feature on any system on the network.
- Causes `QUEUED` messages to be omitted from the debug log file.

The sum total of the licenses reserved must be less than or equal to the number of licenses specified in the `FEATURE` line. In the example above, there must be a minimum of four licenses on the `compile` `FEATURE` line. If fewer licenses are available, only the first set of reservations (up to the license limit) is used.

If this data were in file `/a/b/sampled/licenses/sampled.opt`, then modify the license file `VENDOR` line as follows:

```
VENDOR sampled /etc/sampled /sample_app/sampled/licenses/sampled.opt
```

Limiting Access for Multiple Users

Each INCLUDE, INCLUDEALL, INCLUDE_BORROW, INCLUDE_ENTITLEMENT, INCLUDEALL_ENTITLEMENT, EXCLUDE, EXCLUDEALL, EXCLUDE_BORROW, EXCLUDE_ENTITLEMENT, EXCLUDEALL_ENTITLEMENT, MAX, and RESERVE line must have a single user name (or group) listed. To affect more than one user name create a GROUP. For example to exclude bob, howard, and james from using the feature called toothbrush, create the following options file:

```
EXCLUDE toothbrush USER bob
EXCLUDE toothbrush USER howard
EXCLUDE toothbrush USER james
```

However, there is an easier way. Create a GROUP and exclude the list of users from using the feature. Like the previous example, the following options file excludes bob, howard, and james from using the feature called toothbrush:

```
# First define the group "Hackers"
GROUP Hackers bob howard james
# Then exclude the group
EXCLUDE toothbrush GROUP Hackers
```

Now when you want to allow or deny access to any feature to that group, you have an alias list to make it simple.

Use HOST_GROUP to allow, deny, or reserve licenses for multiple hosts. For example, to exclude all users logged in on the hosts fred and barney from using a feature called f1, add these lines to your options file:

```
HOST_GROUP writers fred barney
EXCLUDE f1 HOST_GROUP writers
```

See Also

[HOST_GROUP](#) for more information about defining groups

EXCLUDE Example

```
#First Define the group "painters"
GROUP painters picasso mondrian klee
EXCLUDE spell GROUP painters
EXCLUDE spell USER bob
EXCLUDE spell INTERNET 123.123.123.*
```

This options file:

- Prevents the users picasso, mondrian, and klee from using the feature spell on any system on the network.
- Prevents the user bob from using the feature spell on any system on the network.
- Prevents any user logged into a host with an IP address in the range 123.123.123.0 through 123.123.123.255 from using the feature spell.
- Allows any other user, as long as they are not on the excluded IP addresses, *and* they are not a member of the painters GROUP, *and* they are not bob, to use feature spell (by implication).

Note that bob could have been added to the group painters. However, painters might be used for some other purpose in the future so the license administrator chose to handle bob as a special case here. In this case, the two EXCLUDE statements concatenate to create a list of four users.

EXCLUDE_ENTITLEMENT Example

```
#First Define the group "admin"
GROUP admin johns adrianp maryt
EXCLUDE_ENTITLEMENT qf573k GROUP admin
EXCLUDE_ENTITLEMENT qf573k USER bob
EXCLUDE_ENTITLEMENT qf573k HOST cordelia
```

This options file:

- Prevents the users johns, adrianp, and maryt from activating any licenses contained in the fulfillment record obtained using the entitlement Id qf573k on any system on the network.
- Prevents the user bob from activating any licenses contained in the fulfillment record obtained using the entitlement Id qf573k on any system on the network.
- Prevents any user on the system called cordelia from activating any licenses contained in the fulfillment record obtained using the entitlement Id qf573k.
- By implication allows any other users on any system other than cordelia to activate the licenses contained in the fulfillment record obtained using the entitlement Id qf573k.

INCLUDE Example

```
INCLUDE paint USER picasso
INCLUDE paint USER mondrian
INCLUDE paint HOST bigbrush
```

This options file:

- Allows the user picasso to use the feature paint on any system on the network.
- Allows the user mondrian to use the feature paint on any system on the network.
- Allows any user, as long as they are on the host bigbrush, to use feature paint.
- Denies access to the feature paint to anyone except picasso, mondrian, or anyone from the host bigbrush (by implication).

INCLUDE_ENTITLEMENT Example

```
INCLUDE_ENTITLEMENT gy7210 USER tom
INCLUDE_ENTITLEMENT gy7210 USER anthony
INCLUDE_ENTITLEMENT gy7210 HOST jupiter
```

This options file does the following:

- Allows the user tom to activate any licenses contained in the fulfillment record obtained using the entitlement Id gy7210 on any system on the network.
- Allows the user anthony to activate any licenses contained in the fulfillment record obtained using the entitlement Id gy7210 on any system on the network.
- Allows any user, as long as they are on the host jupiter to activate any licenses contained in the fulfillment record obtained using the entitlement Id gy7210.

- By implication, denies the activation of any licenses contained in the fulfillment record obtained using the entitlement Id gy7210 to anyone except tom, anthony, or someone on the host jupiter.

Efficient Reservation

RESERVATION on features can also be applied at runtime from remote machine. Once request reaches to the license server, it moves the licenses from the floating pool to reserve pool. There is no requirement for running whole re-read process on the license server for it. Once the purpose of reservation is completed, the licenses can be moved back to the floating pool. Producer has to provide the tool, which is implemented as wrapper on the FNP provided API. To get the tool, contact your software producer. The functionality is based out of RESERVE line in the Options file reserves license for a specific type.

The lifetime of these reservations on license server can be any of the following:

- Lifetime of the vendor daemon process on which efficient reservation is applied.
- Intentionally removed with the producer provided tool.
- The reservation will be removed automatically once duration has expired.

Efficient reservation does not have any dependency on Options file and can be applied without it. The only exception is if reservation is applied for type GROUP. In that case, the Options file is required to enable that GROUP in vendor daemon.

Single efficient reservation request can have multiple RESERVE lines for different features. The request will be considered successful if all the RESERVE lines are successfully reserve licenses on the license server. In case any of the RESERVE lines fails , the license server will reject whole reservation request.

Ensuring License Availability

You can configure multiple license servers to allow FlexEnabled applications to continue to check out licenses if one of the license servers goes down. This failover protection for license servers can be provided using either of the following methods:

- **Redundancy using the license search path**—Configure and maintain multiple independent license servers, each with a subset of the total licenses available to the enterprise. Configure the FlexEnabled client with the license servers in the license search path. This provides load balancing capabilities and limited failover protection. You must manage different versions of the license rights on each license server. This configuration option is available when licenses are held in license files and in trusted storage.
- **Three-server redundancy**—Configure and maintain a set of three license server systems configured specifically for three-server redundancy. This provides failover protection only. You manage only one version of the license file and vendor daemon on all three license servers. This configuration option is only available when licenses are held in license files.

Do not store your license files on a single network file server (separate from the license servers) if you are using either of these methods of failover protection: The failure of the file server will cause all the license servers to fail.

Redundancy Using the License Search Path

In this configuration you install multiple license servers that each use a subset of the available licenses. Network machines are configured with a license search path that contains details of each license server. A FlexEnabled application tries each license server on the license search path in order until it succeeds or gets to the end of the list.

Example of Redundancy Using the License Search Path

This example demonstrates the use of two license servers, **chicago** and **tokyo**, that serve five licenses each for the features **f1** and **f2**. The publisher supplies the following license files:

- **For chicago**

```
SERVER chicago 17007ea8 1700
VENDOR sampled /etc/mydaemon
```

```
FEATURE f1 sampled 1.000 31-dec-2020 5 SIGN=.....  
FEATURE f2 sampled 1.000 31-dec-2020 5 SIGN=.....
```

- **For tokyo**

```
SERVER tokyo 17007ea8 1700  
VENDOR sampled /etc/mydaemon  
FEATURE f1 sampled 1.000 31-dec-2020 5 SIGN=.....  
FEATURE f2 sampled 1.000 31-dec-2020 5 SIGN=.....
```

The license search path is set on the network machines using the LM_LICENSE_FILE environment variable so that machines in the US request licenses first from the license server **chicago** and machines in Japan request licenses first from the license server **tokyo**.

- **US machines** set LM_LICENSE_FILE to - 1700@chicago:1700@tokyo
- **Japanese machines** set LM_LICENSE_FILE to - 1700@tokyo:1700@chicago

This example uses UNIX syntax (:) for separating entries on the license search path. See [Setting the License Search Path Using an Environment Variable](#) for full details of the license search path syntax.

Limitations of Redundancy Using the License Search Path

The main limitation is that this method only provides limited protection: When a license server fails, the licenses it serves are no longer available.

All licenses must be checked out from a single server

By default, once a FlexEnabled application has successfully checked out a license from a license server, all subsequent license requests from that application must be served by the same license server. When an application makes subsequent license requests and no more licenses are available from that license server, the license request is denied even though licenses may exist on another server. However, this behavior is configurable by software publishers. Contact your software publisher to determine whether or not each new license request scans all the license servers.

Licenses are queued from a single server

If the application supports license queueing, all licenses are queued from the first license server on the list rather than the request moving to another license server.

Overview of Three-Server Redundancy

Using the three-server redundancy capability in FlexNet Publisher, all three license servers operate to form a triad. The license servers send periodic messages to each other to make sure that at least two servers are running and communicating. A quorum is formed when at least two of the three license servers are running and communicating with each other.

The license servers are identified as either primary, secondary, or tertiary. One license server is also designated as the master [m] and is responsible for:

- Serving licenses to FlexEnabled applications
- Recording information into the debug log.

- Recording information into the report log.

If the master fails, then another license server becomes the master.

In the following figure, the primary license server is the master [m]. When a FlexEnabled application sends a checkout request for a license, the master responds and then serves the license to the FlexEnabled application.

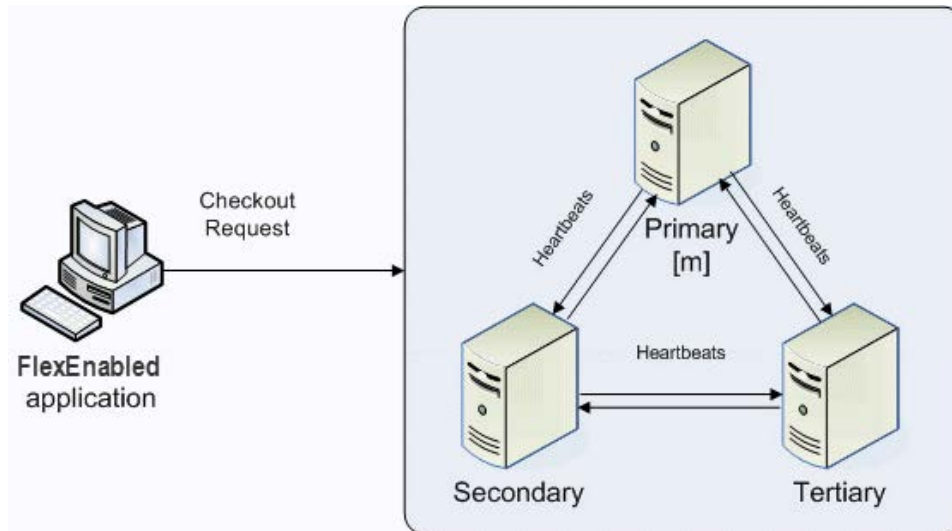


Figure 14-1: Three-Server Redundancy Overview

If the master fails, then the secondary license server becomes the master (see the following figure) and will serve licenses to FlexEnabled applications. The tertiary license server can never be the master. If both the primary and secondary license servers go down, licenses are no longer served to FlexEnabled applications. The master will not serve licenses unless there are at least two license servers in the triad running and communicating.

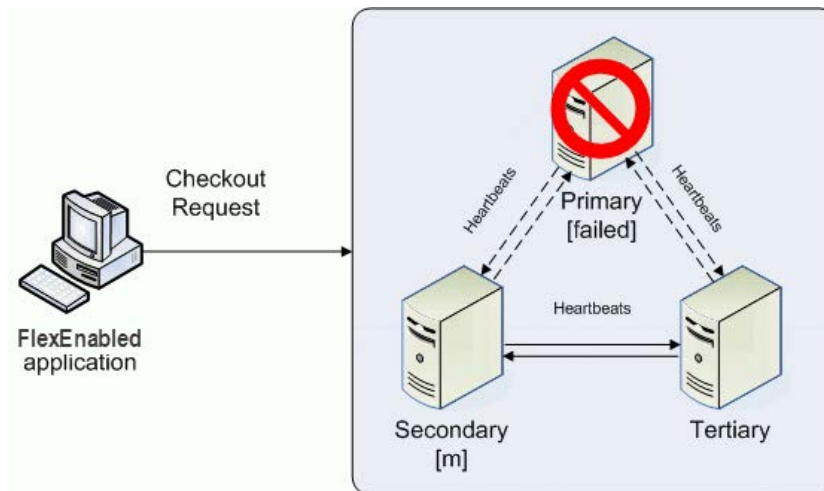


Figure 14-2: Three-Server Redundancy Backup Failover

Understanding How License Servers Communicate

When started, each license server reads the license file and checks that it can communicate with the other license servers. Until each license server establishes this first connection with the others, it will continue to send messages periodically.

Once the initial communication has been established, each license server periodically sends a heartbeat to the others. Heartbeats are messages sent over TCP/IP. Each license server sends a heartbeat and waits for a response from the other license servers. If a license server does not receive a response, it shuts down the vendor daemon so that it cannot serve licenses. A publisher or license administrator can configure the amount of time a license server waits to receive a heartbeat using the HEARTBEAT_INTERVAL property.

Poor network communication causes system performance to slow. Slow network communication can also cause a delay in the transmission of heartbeats between license servers.

Configuring License Servers for Three-Server Redundancy

Perform the following steps to configure three-server redundancy:

1. Before obtaining the license server software package from the publisher, identify and set up the three systems. When selecting systems, make sure they are stable. Do not use systems that are frequently rebooted or shut down.
2. Send the publisher the hostname and hostid values for these systems. Ask the publisher what system identifier they need for the hostid. This could be an Ethernet address, disk serial number, etc. The publisher will create license server components specifically for these systems.



Note - Best practice is to obtain a hostid of the same type for each of the three machines. Issues can arise if different hostid types are used in the configuration. If you plan to obtain the hostid using the default hostid type for a given machine, be aware that the default type can vary between operating-system platforms.

3. After receiving the license server package from the publisher, change the following SERVER line properties in the license file if necessary:
 - port number the license servers use to listen for communication
 - PRIMARY_IS_MASTER keyword
 - HEARTBEAT_INTERVAL property

Do not change the hostid values. If the hostid changes at any time, the license administrator must work with the software publisher to obtain a new license file.

4. Perform any additional configuration as required by the software publisher.
5. Copy or install the license server software package to each of the three systems.
6. Start the license servers in the following order: primary, secondary, and then tertiary.

An Example License File

The following is an example of a license file that is configured for three-server redundancy.

```
SERVER pat 17003456 2837 PRIMARY_IS_MASTER
SERVER lee 17004355 2837
SERVER terry 17007ea8 2837
VENDOR demo
FEATURE f1 demo 1.0 31-dec-2020 10 SIGN="<...>"
```

```
FEATURE f2 demo 1.0 31-dec-2020 10 SIGN="<...>"
```

The following portions of the license file directly affect the three-server redundant configuration:

- **SERVER lines**—Each line contains this information for the given server:
 - The **host name** value: **pat**, **lee**, or **terry**.
 - The **hostid** value: **17003456**, **17004355**, or **17007ea8**. This example uses the value returned by the `lmhostid` utility default hostid type for each server.



Note ▪ Keep in mind that the default hostid type can vary between operating-system platforms. Best practice is to use the same hostid type for all three servers.

- The TCP/IP **port**—In this example, **2837**. All servers use the same port to listen for communication.

The following properties of the license file do not affect the three-server redundant configuration directly, but are used to define license rights or configure the license server.

- **VENDOR line**—This required element references the publisher's vendor daemon.
- **FEATURE lines**—The two features, **f1** and **f2**, define the license rights. The **SIGN** value for each **FEATURE** line encodes the license server hostid values.

Managing License Servers in a Three-Server Redundant Configuration

Using the `lmstat` Utility

The output message generated by the `lmstat` utility identifies which license server is the master. In the following example `lmstat` output, the secondary license server is the master.

```
[Detecting lmgrd processes...]  
License server status: 30000@RMD-PRIMARY,30000@RMD-SECONDARY,  
30000@RMD-TERTIARY  
License file(s) on RMD-PRIMARY: C:\server\3.lic:  
RMD-PRIMARY: license server UP v11.4  
RMD-SECONDARY: license server UP (MASTER) v11.4  
RMD-TERTIARY: license server UP v11.4
```

Starting and Stopping License Servers

To start the entire system, you must start each license server manager (`lmadmin` or `lmgrd`). Generally, it is good practice to start the primary license server before the secondary or tertiary license server. This allows the primary license server to become the master before the others start. If you start the secondary and tertiary before the primary, then the secondary will establish itself as master.

If you do not set the `PRIMARY_IS_MASTER` keyword for the primary license server, then the order in which you start the license servers is important. If you do not set this property, when you start the primary license server after the secondary license server control will not transfer to the primary license server. By setting the `PRIMARY_IS_MASTER` keyword, you ensure that when the primary license server is running, it is always the master.

In three-server scenarios, a system administrator may wish to shut down one of the servers to perform maintenance on it, while the other two servers maintain a quorum and continue to serve licenses. Previously, shutting down a license server in a triad through `lmdown` (or `lmttools`) would shut down the entire triad. Now, a system admin can run `lmdown -c 3server-license.lic` from one of the machines in the triad. `lmdown` will read the 3-server license file and provide a prompt asking which of the three listed servers to shut down.

Running the License Server Manager as a Service on Windows

There are no dependencies or known issues related to running the license server manager as a service in this configuration.

Logging and the Debug Log

When using three-server redundancy, the master records information to its local debug log and report log (and the Windows event log if this is configured). If this system fails, another license server becomes the master and records information to its local debug log and report log. Subsequently, there may be different versions of the debug log and report log on the primary and secondary license server which each contains different information.

Using Other Capabilities with Three-Server Redundancy

The following section describe other capabilities available in FlexNet Publisher and how they interact with three-server redundancy.

Configuring the License Search Path

This configuration can be performed by either the software publisher or the license administrator. Before a FlexEnabled application can check out a license, it must know where to locate the license rights. The license search path identifies the location of license rights.

When connecting to a license server configured for three-server redundancy, the FlexEnabled application must use the `<port>@<host>` convention (and not a license file location) in the license search path.

The license search path should list the license servers in the same order that they appear in the license file. This helps shorten the amount of time it takes to identify the master server and respond to the checkout request. Although the configuration will work if you include only one of the license servers in the license search path, this may lengthen the amount of time it takes for the license server to respond to the checkout request. This is because the license server must identify all other license servers and designate a master.

Separate each `<port>@<host>` entry with a comma. Using the previous license file as an example, the license search path should be the following:

```
2837@pat,2837@lee,2837@terry
```

The FlexEnabled application will try to connect to each of the license servers in the list, in the order listed, until it either successfully connects to a license server or reaches the end of the list. This helps ensure that the FlexEnabled application can connect to the quorum.

Specifying Three-Server Redundancy in the License Finder

When the license search path has not been configured, the FLEXlm License Finder dialog is displayed on Windows platforms when a FlexEnabled application is run.



Task

To specify a triad of license servers in the License Finder dialog:

1. Select **Specify the License File**.
2. Click **Next**.
3. Type the path name or use the browse button to specify your three-server redundant license file. [An Example License File](#) shows a typical three-server redundant license file.
4. Click **Next**.

Note that the License Finder dialog option, **Specify the License Server System**, allows you to only specify a single license server and not a triad of license servers.

Using License File Keywords

The following keywords and properties for the SERVER line enable you to modify the configuration. You can change their values after the license file has been signed (that is, without invalidating the signature).

- **host**—The hostname of the system. The publisher should know this information when generating the license file.
- **port**—The port number that the license server uses to listen for communication. Unlike with single license servers, each SERVER line must include a port number. This can be any number in the range 1 to 65535 that is not used by another process running on the system. On UNIX, choose a port >1024, since those <1024 are privileged port numbers. If not specified, the license server will automatically use the next available port number in the range 27000 to 27009.

If you specify a port number greater than 65535, the client fails to establish a connection with the license server manager (lmgrd or lmadm).



Note ▪ Those producers concerned about potential DoS attacks based on knowledge of common license server ports may want to consider specifying a server port outside the range 27000 to 27009.

If you are using lmadm, you do not need to edit the license file; you can configure the port number using the interface. The port number specified in lmadm takes precedence over any port set in the license file, either before or after the port is specified using lmadm. (See the online help for more details.) Editing the port number using lmadm is only recommended if no port number is specified in the license file. If a port number is already specified in the license file and the port is subsequently changed in lmadm, during the next restart the vendor daemon will exit with a “port mismatch” error.

To make it easier to administer the license server, it is strongly recommended that you define the same port number for each SERVER line.

- **PRIMARY_IS_MASTER**—This keyword ensures that the primary server is the master whenever it is running and communicating with one of the other license servers.

- If this is set and the primary server goes down, when the primary server comes back up again, it will always become the master.
- If this is not set and the primary server goes down, the secondary server becomes the master and remains the master even when the primary server comes back up. The primary can only become the master again when the secondary license server fails.

This parameter is optional and should be placed on the first SERVER line. The license server must be running a version 10.8 or later vendor daemon to use this keyword.

- `HEARTBEAT_INTERVAL=seconds`—This indicates how long the license servers wait to receive a heartbeat from another license server before shutting down the vendor daemon. This value is used in the following equation to calculate the actual timeout value:

$$\text{timeout} = (3 * \text{seconds}) + (\text{seconds} - 1)$$

The default value is 20, which equates to an actual timeout of 79 seconds. Valid values are 0 through 120. This parameter is optional and should be placed on the first SERVER line in the license file. The license server must be running a version 10.8 or later vendor daemon to use this keyword.

Using Options File Keywords

None of the keywords in the options file affect three-server redundancy.

Troubleshooting Tips and Limitations for Three-Server Redundancy

Three-server redundancy configurations require all three servers use the same platform type. You can use any of the following platform types in the configuration, but each server must use the same platform type: `VM_UUID` or `PHY_*`.

Separating the Contents of a License File

Because the hostid values in the SERVER lines are computed into the signature of each feature definition line, make sure you keep SERVER lines together with any feature definition lines as they were generated. This means that if you move a feature definition line to another file, you must also move the respective SERVER lines and VENDOR line.

Putting the License File on a Network File Server

Do not put the license file on a network file server. If you do this, you lose the advantages of having failover protection because the file server becomes a possible single point of failure.

Using License Servers in Heavy Network Traffic

On a network with excessive traffic, the license servers may miss heartbeats which causes them to shut down the vendor daemon. The master may then stop serving licenses. If you find that heavy network traffic causes this to occur, you should set the `HEARTBEAT_INTERVAL` to a larger value. Enterprises can experience a performance issue when there is slow network communication or if FlexEnabled clients are using a dial-up link to connect to the network.

Using Multiple Vendor Daemons

The license server manager (`lmadmin` or `lmgrd`) can not start vendor daemons from multiple software publishers when configured for three-server redundancy. The license server manager can only manage one vendor daemon. If one of the systems runs more than one vendor daemon, then the license administrator must run separate instances of the license server on that system to support the other vendor daemons. Make sure that the port numbers do not clash.

Switching Between Three-Server Redundancy and Single-Server Configuration

While running, the license server manager (`lmadmin` or `lmgrd`) is not designed for switching from three-server redundancy to a single-server configuration (and vice-versa). To switch configurations, you need to do the following:

1. Shut down the license server manager in the single-server configuration or, in a three-server redundancy, all the license server managers currently running.
2. Do one of the following:

- For `lmadmin`:

- a. From the `lmadmin` installation directory on a given machine, import the required license files for the new configuration to which you are switching (either the three-server-redundant or single-server configuration):

```
lmadmin -import <new_license_file_list> -force
```

- b. Restart `lmadmin`.

- For `lmgrd`:

To restart `lmgrd` on a given machine in the configuration to which you are switching (either the three-server-redundant or single-server configuration), enter the following from the directory in which `lmgrd` is located:

```
lmgrd -c <new_license_file_list>
```

Avoiding Undefined `lmdown` Behavior

If any two license servers in a three-server redundancy group are started with the `-allowStopServer` no option (`lmadmin`) or the `-x lmdown` option (`lmgrd`), then the behavior of `lmdown` is undefined for that system.

Managing Virtualized License Servers for File-Based Licensing

Virtualization software allows you to run multiple instances of a license server on a single machine. You can use virtualized license servers to take advantage of the high availability and fault tolerance that virtual machines offer.



Note ▪ This chapter deals with managing license servers configured for license file-based licensing in a virtual environment. For information about trusted storage-based licensing in a virtual environment, see [Trusted Storage](#).

Binding Solutions in a Virtual Environment

For license file-based licensing, a license server can be bound to one of:

- the UUID of the virtual machine (refer VM_UUID)
- the MAC address of the virtual machine (ETHER)
- A FLEXID, where the hypervisor concerned supports USB passthrough of the dongle

The FlexNet Licensing Service must be installed on machines that use virtualization features.

Setting Up a Virtual License Server on Microsoft Hyper-V

The process of setting up a license server on a virtual machine hosted by a Microsoft Hyper-V hypervisor depends on the hostid chosen to bind the license file to the license server. This hostid can be either the virtual machine's UUID (Universally Unique ID) or the hostid of a physical machine—either the hypervisor or a remote physical machine that communicates with the virtual machine. The following sections demonstrate these two binding methods.

Using the UUID Hostid

In general, the process for binding to the UUID of the virtual machine hosting the license server follows this outline. The process requires the `lmhostid` utility.



Task

To bind to the UUID of the virtual machine hosting the license server

1. Identify the virtual machine on which the license server will be run.
2. Run the following command from the command line of the virtual machine:

`lmhostid -ptype VM -uuid`
3. Send the output of this command to the software publisher. The software publisher sends back a license certificate in which the license server is bound to the UUID value.
4. Modify the license certificate to configure server parameters (like TCP port number, options file, or other parameters).
5. Launch the license server on the virtual machine by pointing to the license certificate.



Note - If the license server has to be moved to another physical machine, simply copy the virtual-machine image and move it to the new physical host machine.

Setting Up a Virtual License Server on VMware ESXi or XenServer

The process of setting up a license server on a virtual machine hosted by a VMware ESXi or XenServer hypervisor depends on the hostid chosen to bind the license file to the license server. This hostid can be either the virtual machine's UUID (Universally Unique ID) or the hostid of a remote physical machine that communicates with the virtual machine. The following sections demonstrate these two binding methods.

If your license-server virtual machine is being hosted by a VMware ESX hypervisor.

Using the UUID Hostid

In general, the process for binding to the UUID of the virtual machine hosting the license server follows this outline. The process requires the `lmhostid` utility.



Task

To bind to the UUID of the virtual machine hosting the license server

1. Identify the virtual machine on which the license server will be run.
2. Run the following command from the command line of the virtual machine:

```
lmhostid -ptype VM -uuid
```

3. Send the output of this command to the software publisher. The software publisher sends back a license certificate in which the license server is bound to the UUID value.
4. Modify the license certificate to configure server parameters (like TCP port number, options file, or other parameters).
5. Launch the license server on the virtual machine by pointing to the license certificate.



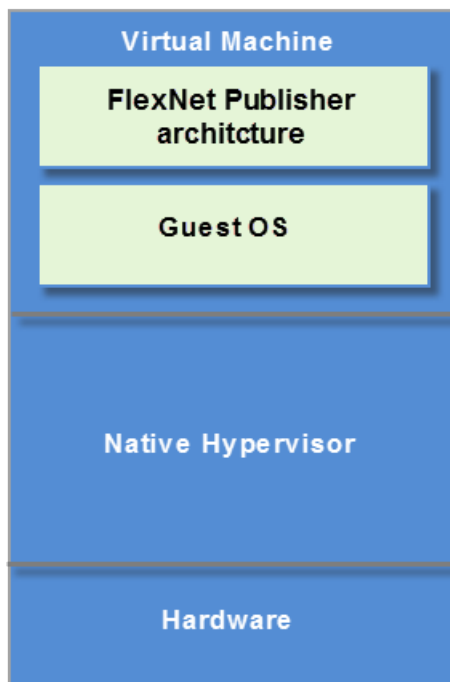
Note ▪ If the license server has to be moved to another physical machine, simply copy the virtual-machine image and move it to the new physical host machine.

Hypervisor COS Use

Running a license server or a FlexEnabled client application on the COS of the hypervisor is not supported. The license server and FlexEnabled client application must run either on a physical machine or on a virtual machine.

Virtualization Support

The following picture illustrates how the FlexNet licensing server or a FlexEnabled application operates within a Virtualization stack. The table below the picture lists the Virtualization stacks that FlexNet Publisher supports.



Use the table available in Release notes to determine the supported Virtualization stacks.

Licensing in a Cloud-Computing Environment

This chapter provides the following information to help the license administrator implement secure software licensing in a cloud-computing:

- [Licensing Challenges in a Cloud Environment](#)
- [Scope of Support for Cloud Licensing](#)
- [Hostids for Binding](#)



Note ■ For simplification, the remainder of this chapter refers to the cloud-computing environment as the “cloud environment”.

Licensing Challenges in a Cloud Environment

The fundamental concept of software licensing involves binding the license to characteristics of the physical machine on which the software resides and, for served licenses, on which the license server resides. These characteristics, called *binding elements*, include machine identities such as the Ethernet address or the host name. However, a cloud environment runs on virtual machines that are instantiated and brought down frequently. Consequently, the traditional hardware-based binding elements are not reliable in a cloud environment. Additionally, the FlexNet Publisher bare-metal-bindings (BMB) feature available in prior releases for on-premises virtual machines is unusable in a cloud environment; the license administrator does not have access to the physical hardware on which the virtual instances are running. This access is a prerequisite for the BMB feature.

A public cloud provider charges the customer based on the number of hours a machine instance is used. To optimize billing charges, a user typically stops or terminates an instance when it is not in use. This type of stop-restart usage is contrary to the on-premises machine usage, which is typically continuous. Therefore, the ideal binding element in the cloud environment is one that has these attributes:

- Globally unique within the cloud infrastructure to prevent over-licensing by cloned images
- Unchanging between normal start/stop, suspend/resume, live-process migration, and reboots of machine instances

- Not easily changed by the user
- Not usable when the license is outside the cloud

FlexNet Publisher provides licensing solutions that meet these binding-element requirements. These solutions are based on typical licensing use cases in a cloud environment and are within the scope of cloud-licensing support for this release.

Scope of Support for Cloud Licensing

FlexNet Publisher limits its support for cloud licensing to 32-bit Windows, 64-bit Windows, 32-bit Linux, or 64-bit Linux machine instances (all in IPv4 format) for running FlexEnabled applications and license servers.

Additionally, FlexNet Publisher does *not* support the following for cloud licensing:

- Cross-version signatures on served, node-locked license clients
- Composite hostids using Amazon machine bindings
- Three-server redundancy



Note - Cloud environments have other methods that compensate for three-server redundancy to ensure high availability of the license server.

Hostids for Binding

The binding element used to bind a license to the specific AMI instance on which the license client or the license server resides is called the *hostid* of the AMI instance. This section discusses the types of hostids that FlexNet Publisher supports for licensing in the Amazon EC2 environment.

Supported Hostid Types

The following table describes the hostids that FlexNet Publisher supports in the Amazon EC2 environment. For instructions on how to retrieve each hostid, see the next section, [Retrieving and Specifying Hostids](#).

Obtaining any TPM, virtualization, or cloud-licensing hostid requires that the FlexNet Licensing Service be installed.

Table 16-1 • Hostid Types Supported in Amazon EC2

Hostid Type	Binding Applicability	Characteristics
Elastic IP (EIP) address (AMZN_EIP)	License Server only	• IPv4 format only. • For use on the SERVER line in the license file. • Manually assigned to any running AMI instance. • Static identity—that is, can be reassigned to another AMI instance if the current instance crashes. For example, if the instance containing the license server crashes, the license administrator can copy the server to another instance and reassign the IP address. • Cross-version signatures not required.
AMI template ID (AMZN_AMI)	Client only	AMZN_AMI uniquely identifies the AMI template from which an instance is created. It does not uniquely identify AMI instances.
VM_UUID (AMI instance IID)	License Server only	• Unique ID automatically assigned to an AMI instance. • Remains intact when the instance is suspended, resumed, or rebooted, but disappears if the instance terminates accidentally or crashes. • Cross-version signatures not supported. Therefore, legacy FlexEnabled applications cannot obtain licenses that are bound to this hostid.
Elastic Network Interface (ENI)	Server or client	Extracted through the ETHER keyword.

Retrieving and Specifying Hostids

The following table lists the methods used by the software user or publisher (depending on the use case) to retrieve hostids in the Amazon EC2 environment. The table also shows how a given hostid is used in the license file.

Obtaining any TPM, virtualization, or cloud-licensing hostid requires that the FlexNet Licensing Service be installed.



Note ▪ The System Info page in the Imadmin user interface does not show these hostids specific to the Amazon EC2 environment. To obtain these hostids, you must use the methods described the following table

Table 16-2 ▪ Retrieving and Specifying Hostids

Hostid Type	Method to Obtain ID	License File Syntax
Elastic IP (EIP) address (AMZN_EIP)	Use either method: - On the AMI instance containing the server, run the following command: <code>lmhostid -ptype AMZN -eip</code> - Obtain the EIP from the Amazon infrastructure. <i>This lmhostid command actually returns the public IP address for the AMI instance, and this value might not always be the EIP address. Consider the following:</i> - If the AMI instance is associated with an EIP address, the EIP address is the public IP address. Consequently, <i>lmhostid</i> returns the EIP address. - If the AMI instance has no EIP address associated with it, EC2 assigns a default public IP address, which is the value that <i>lmhostid</i> returns. <i>FlexNet Publisher strongly recommends obtaining the EIP address. To ensure lmhostid returns this value, associate the instance with an EIP address before running the command. If necessary, consult Amazon for help with this process.</i>	Syntax: <code>AMZN_EIP=IPv4 address</code> Example: <code>SERVER this_host AMZN_EIP=184.72.45.35</code>
AMI template ID (AMZN_AMI)	As software publisher, use either method: - On the AMI instance used for the VA, run the following command: <code>lmhostid -ptype AMZN -ami</code> - Obtain the template ID from the Amazon infrastructure.	Syntax: <code>AMZN_AMI=AMI template ID</code> Example: <code>INCREMENT F1 demo...</code> <code>HOSTID=AMZN_AMI=ami-6a807503...SIGN=xxx</code>

Table 16-2 • Retrieving and Specifying Hostids

Hostid Type	Method to Obtain ID	License File Syntax
VM_UUID (AMI instance IID)	Use either method: - On the AMI instance containing the license server, run the following command: <code>lmhostid -ptype VM -uuid</code> - Obtain the instance ID from the Amazon infrastructure.	Syntax: <i>VM_UUID=universal unique identifier for AMI Instance ID</i> Example: <code>HOSTID=VM_UUID=i-7d409db1</code>
Elastic Network Interface (ENI)	On the AMI instance containing the license server, run the following command: <code>lmhostid -ether</code>	Syntax: <i>lmhostid -ether"</i> Example: <code>Host ID=""06007e2c60c7 06835b200fe7""</code>  Note ▪ <i>ifconfig</i> returns <i>eth0</i> and <i>eth1</i>. <i>eth0(primary network interface)=06835b200fe7 eth1(Elastic Network Interface)=06007e2c60c7</i>

IPv6 Support

Internet Protocol version 6 (IPv6) is the next generation IP protocol. This section contains information for software publishers who want to support IPv6 addresses in their license model. The information in this section assumes the reader has a familiarity with the IPv6 networking protocols.

The following sections of this chapter describe the FlexNet Publisher support for IPv6 and contains information related to both license file-based licensing and trusted storage-based licensing.

- [Capabilities that Support IPv6](#)
- [Testing Considerations](#)

Capabilities that Support IPv6

This section describes the capabilities in FlexNet Publisher that support IPv6. It includes information about both license file-based licensing and trusted storage-based licensing.

When working with a software publisher to obtain a software package that supports IPv6, you should collect and provide the IP addresses of systems (FlexEnabled clients and license servers) that will be used in the license file.



Note •

- *In the license and options files, FlexNet Publisher supports only the site-local form of the IPv6 address (those addresses prefixed with FEC0).*
- *While an IPv6 address can be used in license or options files, the best-practice recommendation is to use hostname or IPv4 address.*
- *A mix of IPv4 and IPv6 addresses in the license and/or options file is not supported.*

License File

In a license file, the SERVER line can define an IPv6 address as the hostid and host value.

In FEATURE, INCREMENT, and UPGRADE lines, an IPv6 address can be used as the HOSTID value when using the INTERNET type.

The following keywords (used with duplicate grouping) support IPv6 addresses:

- The DUP_GROUP keyword, using HOST as the type.
- The SUITE_DUP_GROUP keyword using HOST as the type.

Options File

An options file can contain an IPv6 address to specify host restrictions when using the:

- INTERNET type in these keywords: EXCLUDE, EXCLUDEALL, EXCLUDE_BORROW, INCLUDE, INCLUDEALL, INCLUDE_BORROW, MAX, and RESERVE.
- HOST type in these keywords: EXCLUDE, EXCLUDE_ENTITLEMENT, EXCLUDEALL, EXCLUDE_BORROW, INCLUDE, INCLUDE_ENTITLEMENT, INCLUDEALL, INCLUDE_BORROW, MAX, and RESERVE
- HOST_GROUP keyword (it takes IP addresses).

License Search Path

Entries in the license search path that use the 'port@host' convention to identify the license server, can specify an IPv6 address as the 'host' value.

FlexEnabled Application

Entries in the license search path that use the *port@host* convention to identify the license server, can specify an IPv6 address as the *host* value.

The following activation functions take either a URL (when connecting to FLEXnet Operations) or "port@host" value (when connecting to a license server) argument. These functions allow you to use an IPv6 address in the URL or as the 'host' value.

- **flxActCommonHandleSetRemoteServer(...)**
- **flxActBorrowReturn(...)**
- **flxActBorrowActivate(...)**

The following attributes, set using the **lc_set_attr** function, support IPv6 addresses:

- LM_A_HOST_OVERRIDE
- LM_A_PROMPT_FOR_FILE (Windows Only) - the dialog that appears takes an IPv6 address.
- LM_A_DISPLAY_OVERRIDE
- LM_A_INTERNET_OVERRIDE

You can use an IPv6 address when setting the LM_DUP_HOST flag in the **lc_checkout** function.

Trusted Storage

The activation server chain property in a fulfillment record will store the IPv6 address.

You can use an IPv6 address as a binding identity in the trusted configuration.

License Server

The **lmhostid** utility returns an IPv6 address.

Environment Variables

Listed below are the environment variables that handle performance and hostname resolutions.

FNP_IP_ENV

This client-side environment variable determines how the client's IP address is presented to the server.

- If the variable is set to a value of 0, 4, or 6, the IP address sent to the server is resolved on the client, from the client's hostname, as follows:
 - If the variable is set to 0, then the client resolves its hostname to both an IPv4 and an IPv6 address.
 - If the variable is set to 4, then the client resolves its hostname only to an IPv4 address.
 - If the variable is set to 6, then the client resolves its hostname only to an IPv6 address.
- If the variable is set to 1 (default value), the client-side hostname resolution is bypassed. Instead, the client's IP address is determined from the socket connection at the server, which means that a NAT-translated IP address for the client can be obtained.

FNP_IP_PRIORITY

The environment variable FNP_IP_PRIORITY decides the IP priority for hostname resolution, and can apply to the server, the client, or utilities like `lmhostid`. Values are:

- **4** (default): hostname resolution is attempted to IPv4 address first; if that fails, resolution to IPv6 address is attempted.
- **6**: hostname resolution is attempted to IPv6 address first; if that fails, resolution to IPv4 address is attempted. Examples of when to set to this value:
 - on the machine where running `lmhostid`, if IPv6 address is required from `lmhostid -internet`
 - on the client if node-locked IPv6 INTERNET HostID is used
 - on the server if IPv6 address is used in the options file or as a server HostID.

Note: IPv4 and IPv6 addresses cannot be mixed on the server.

Support Pure IPv6 for the Client

In order to support the pure IPv6 mode, FNP_IP_PRIORITY is enhanced with a new value of '6not4'.

On setting FNP_IP_PRIORITY to '6not4', the Server comes up with a pure IPv6 mode and accepts only IPv6 client connections. DNS lookup happens only in the IPv6 mode.

Client IP Address Received by the Server

Clients send both IPv4 and IPv6 addresses in the checkout message to the server.

In some licensing models, it may be desirable for the server to operate on a NAT-translated IP address instead of the client's actual IP address—for example, if using the EXCLUDE keyword in an options file to exclude all clients originating from behind a specific firewall. To enable such use cases, set FNP_IP_ENV=1 on the client. Setting this environment variable prevents the client resolving its own hostname to an IP address, which means the server instead obtains an IP address for the client from the socket connection.

Using Wildcards in an IPv6 Address

The wildcard character, “*,” may be used in place of an entire field or on a byte-by-byte basis to specify a range of addresses without having to list them all. For example, in this example feature definition line is locked to four specific addresses:

```
FEATURE f1 myvendor 1.0 31-dec-2020 uncounted \  
  HOSTID="INTERNET=127.17.0.1, \  
    INTERNET=fec0::1:e947:a213:1378:253c, \  
    INTERNET=127.17.0.4, \  
    INTERNET=fec0::1:e947:a213:1378:253c" \  
  SIGN=0
```

In the following example feature definition line specifies an entire range of addresses, including the four specific ones from the line above:

```
FEATURE f1 myvendor 1.0 31-dec-2020 uncounted \  
  HOSTID="INTERNET=127.17.0.*, \  
    INTERNET=fec0::1:e947:a213:1378:253c" \  
  SIGN=0
```

Testing Considerations

When testing IPv6-compatible FlexEnabled applications and license server, remember to accommodate for the behavior of 4to6 routers, 6to4 routers, and IPv6 firewalls configured on your system.

Managing Licenses from Multiple Software Publishers

Overview of Multiple License Management Strategies

When you are running FlexEnabled applications from multiple software publishers, you might need to take steps to prevent conflicts during installation. There are several strategies to accomplish this, three of which are presented here:

- Multiple systems, each running one license server manager, one vendor daemon, and using one or more license files.
- One system running multiple license server managers, each managing one vendor daemon and one or more license files.
- One system running one license server manager, that manages multiple vendor daemons, each using its own license files. License files share a common directory.

Each of these three strategies is described in detail in the following sections. Variations are mentioned in [Additional Considerations](#).

Multiple Systems

In this scenario, each license server instance (ladmin or lmgrd, vendor daemon, license file, and other files) is located on a separate system. Each system serves licenses just for its vendor daemon and runs its own local copy of the license server manager. [Figure 18-1](#) shows this arrangement.

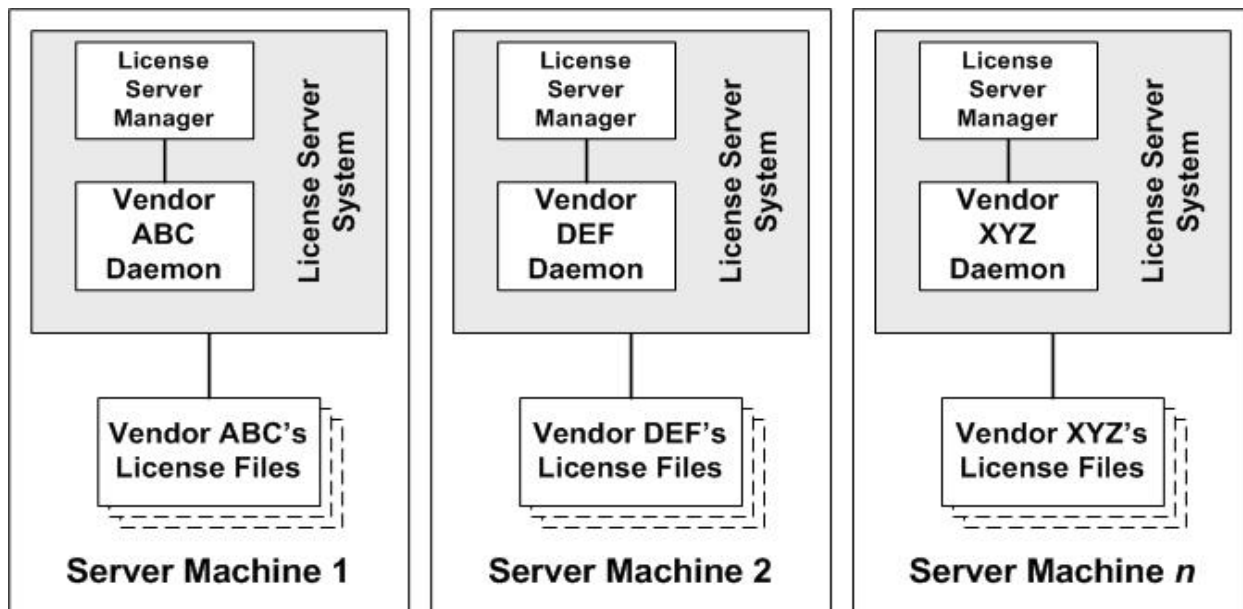


Figure 18-1: Multiple license server systems

Advantages

- The license files for each software publisher are independent from one another.
- Systems are maintained separately. If one system goes down, the other systems continue to serve licenses for their software publishers.
- Each server has its own debug log.
- The license requests are distributed.

Disadvantages

- Administrative overhead is the highest.

Starting the License Servers

In this configuration, you start the license server on each machine separately. Each server runs a single vendor daemon. Use ladmin or lmgrd as the license server manager.

Using ladmin

When using ladmin, you can start each license server using the ladmin user interface or a command line. Repeat the procedure on each machine to start the license server on that machine.

Starting the License Servers Using the User Interface



Task

To use the *lmadmin* user interface to start the license servers:

1. At a command prompt from the *lmadmin* installation directory on the machine, enter *lmadmin*. No special command-line options are required.
2. Open the *lmadmin* user interface. For instructions, see [Accessing the License Server Management Interface](#).
3. From the *lmadmin* user interface's main window, click **Administration** to open the Administration page, and sign in with your administrator credentials.
4. Go to the Vendor Daemon Configuration tab.
5. Click **Import License File**.
6. Select and import the license file.



Note ■ A status message displayed on the *lmadmin* user interface lists any follow-up steps you need to perform after the import. If the message requests that you start (or restart) the vendor daemon, click **Administer** for the specific vendor daemon on the Vendor Daemon Configuration tab; and then click **Stop** or **Start**.

7. Repeat the import process for each license file you want to import for this server.
8. Close down the *lmadmin* user interface.
9. Repeat the previous steps to start the license server on the next machine.

Starting the License Servers Using the Command Line



Task

To use the command line to start the license servers:

1. Open a command window on the machine.
2. At a command prompt from the *lmadmin* installation directory, enter the following to import the license files:

```
lmadmin -import <server_system_n_license_list>
```

where *<server_system_n_license_list>* is the list of license files for the specific vendor daemon on the license server, as described in [Managing Multiple License Files](#).



Note ■ Use the *-force* option along with *-import* if you wish to reset configuration in *server.conf* to the current vendor daemon being imported.

3. Enter *lmadmin* (and any *lmadmin* options you might need) to start the license server. For a description of options, see [lmadmin Command-line Arguments](#) in [Chapter 9, License Server Manager "lmadmin"](#).
4. Repeat the previous steps to start the license server on the next machine.

Using Imgrd



Task To start the license server on each machine:

On each machine, enter the following command:

```
lmgrd -c <server_system_n_license_list>
```

where <server_system_n_license_list> is the list of license files for the specific vendor daemon on the license server, as described in [Managing Multiple License Files](#). In turn, the `lmgrd` instance on the machine starts the vendor daemon referred to in its license files.

One System with Multiple License Server Instances

In this model, each vendor daemon and its associated license file or files is served by its own license server manager, and everything is contained in one system. [Figure 18-2](#) depicts this scheme.

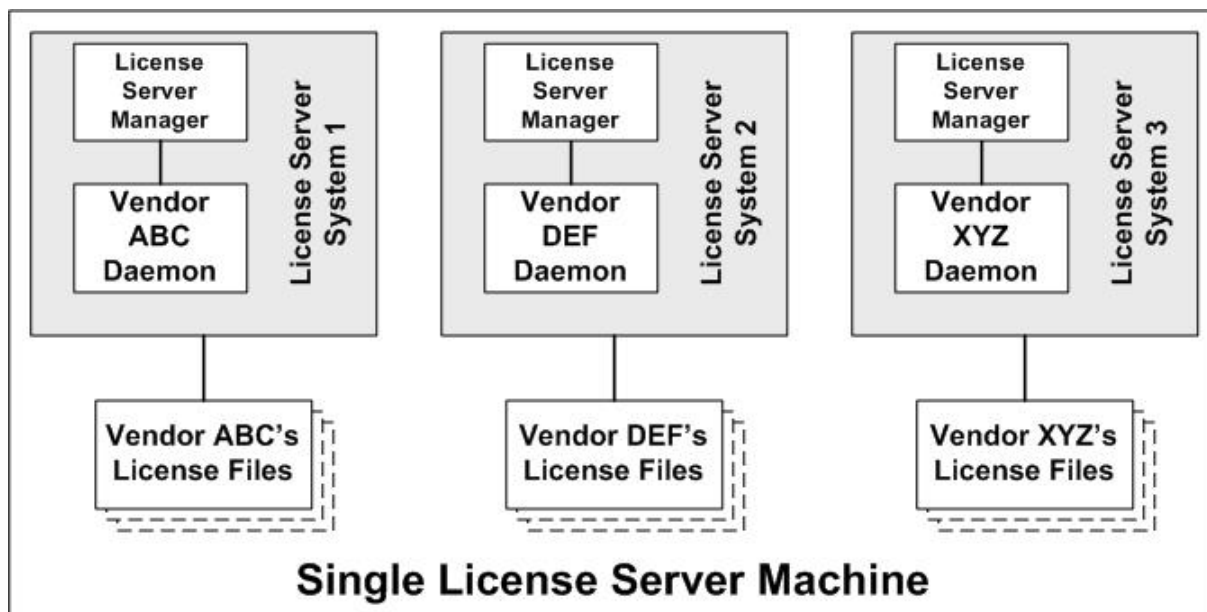


Figure 18-2: Multiple license server managers, multiple license files

When maintaining separate license servers on the same system, keep in mind:

- If the TCP/IP port number is specified on the `SERVER` line, it must be different for each license server instance. Use a standard text editor to change the TCP/IP port number in each license file so that they are all different. For information on the `SERVER` format, see [SERVER Lines](#) in [Chapter 4, Reading a License File](#).



Note ▪ For security purposes, best practice is not to let the license server manager assign a default port for each license server. Instead, for each server, specify a unique port number outside of the range 27000 through 27009.

- You must make sure that you are using a compatible version of `lmadmin` (or `lmgrd`) for each particular license file. This is done by using an explicit path. See [Version Component Compatibility](#).
- The number of license server instances is limited only by the CPU, available memory, and networking of the system.

Advantages

- The license files for each software publisher are independent from one another.
- License servers are maintained separately. If one server goes down, the other servers continue to serve licenses.
- Each server has its own debug log.

Disadvantages

- Administrative overhead is high.
- If the system goes down, all licenses are disabled.
- License request load is concentrated to one system.

Starting the License Servers

In this configuration, you start multiple license servers on one machine, each server running a single vendor daemon. Use `lmadmin` or `lmgrd` as the license server manager.

Using lmadmin

To start multiple `lmadmin` instances on the same machine, use the command line.



Important • While you can run multiple `lmadmin` instances on a single machine, best practice is to run multiple vendor daemons with a single instance of `lmadmin`. See [One System with One License Server and Multiple License Files](#).



Task

To start the license server instances on one machine:

1. Open a command window on the machine.
2. At a command prompt from the directory where the first `lmadmin` is installed, enter the following to import the license files:

```
lmadmin -import <server_system_n_license_list>
```

where `<server_system_n_license_list>` is the list of license files for the specific vendor daemon on the license server, as described in [Managing Multiple License Files](#).

3. To start the `lmadmin` instance, enter the following:

```
lmadmin -webPort <httpPort> -webSecurePort <httpsPort>
```

where:

- <httpPort> is the TCP/IP port number that the web server uses to listen for communication with clients connecting to the lmadm user interface. While 8090 is the default port number for the web server, you must provide a different port number for each lmadm instance you are configuring.
- <httpsPort> is the TCP/IP port number for the Secure Web Server. You must provide a different port number for each lmadm instance you are configuring.



Important • Specify the `-webSecurePort` option only if you are configuring an HTTP-over-SSL (HTTPS) communication with the lmadm web server interface. To define additional certificate information for the Secure Web Server, access the lmadm user interface using the correct URL based on <httpPort>. (On the Administration page, go to the **Secure Web Server** section of the Server Configuration tab.) For more instructions, see [Accessing the License Server Management Interface in Chapter 9, License Server Manager “lmadm”](#).

You can also specify any other lmadm options you might need to start the license server. For a description of options, see [lmadm Command-line Arguments in Chapter 9, License Server Manager “lmadm”](#).

4. Repeat the previous steps to start each lmadm instance on the machine.

Using lmgrd



Task

To start the license servers:

On the machine, invoke each license server:

- For Server 1: `lmgrd -c vendor_ABC_license_dir_list`
- For Server 2: `lmgrd -c vendor_DEF_license_dir_list`
- For Server 3: `lmgrd -c vendor_XYZ_license_dir_list`

where `vendor_nnn_license_list` is a list of license files as described in [Managing Multiple License Files](#). Each lmgrd starts the vendor daemon referred to in its license files.

One System with One License Server and Multiple License Files

In this scenario, one license server manager runs on the system and serves one or more vendor daemons, each with one or more license files. If you are using lmadm, you can maintain license files from different publishers in separate directories. If you are using lmgrd, all the license files are usually held in the same directory. The standard filename extension for license files is `.lic`. The number of vendor daemons is not limited by FlexNet Publisher. [Figure 18-3](#) illustrates this scenario.

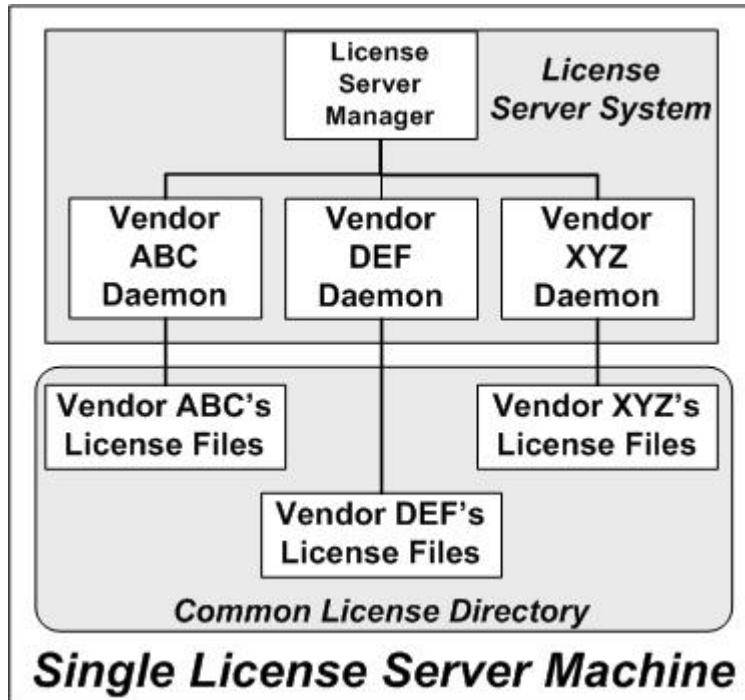


Figure 18-3: One license server manager, multiple license files

Advantages

- The license files can be maintained separately.
- Reduced administrative overhead.

Disadvantages

- One license server manager serves all vendor daemons. If the license server manager goes down, all licenses are unavailable.
- If the system goes down, all licenses are unavailable.
- Output from all vendor daemons goes into one common debug log unless separate debug logs are specified with `DEBUGLOG` in each vendor daemon's options file. Having one common debug log makes it harder to debug a single vendor daemon's problem.
- Maximizes licensing load to one system and one license server manager.

Starting the License Server

In this configuration, you start one license server on a single machine. The license server uses multiple license files (and hence possibly multiple vendor daemons). Use `lmadmin` or `lmgrd` as the license server manager.

Using `lmadmin`

When using `lmadmin`, you can start the license server using the `lmadmin` user interface or a command line.

Starting the License Server Using the User Interface



Task

To use the *lmadmin* user interface to start the license server:

1. At a command prompt from the *lmadmin* installation directory on the machine, enter **lmadmin**. No special command-line options are required.
2. Open the *lmadmin* user interface. For instructions, see [Accessing the License Server Management Interface in Chapter 9, License Server Manager “lmadmin”](#).
3. From the *lmadmin* user interface’s main window, click **Administration** to open the Administration page, and sign in with your administrator credentials.
4. Go to the Vendor Daemon Configuration tab.
5. Click **Import License File**.
6. Select and import the license file.



Note ▪ A status message displayed on the *lmadmin* user interface lists any follow-up steps you need to perform after the import. If the message requests that you start (or restart) the vendor daemon, click **Administer** for the specific vendor daemon on the Vendor Daemon Configuration tab; and then click **Stop** or **Start**.

7. Repeat the import process for each license file you want to import.
8. Close down the *lmadmin* user interface.

Starting the License Server Using the Command Line



Task

To use the command line to start the license server:

1. Open a command window on the machine.
2. At a command prompt from the *lmadmin* installation directory, enter the following to import the license files:

```
lmadmin -import <license_file_list>
```

where <license_file_list> is the list of license files to import, as described in [Managing Multiple License Files](#).



Note ▪ Use the `-force` option with `-import` only if you want to overwrite `server.xml` settings previously defined for the vendor daemons specified in the license files.

3. Enter **lmadmin** (and any *lmadmin* options you might need) to start the license server. For a description of options, see [lmadmin Command-line Arguments](#) in [Chapter 9, License Server Manager “lmadmin”](#).

Using Imgrd



Task

To start the license server:

Enter the following command:

```
lmgrd -c <common_license_directory>
```

where *<common_license_directory>* is the directory containing the license files, as described in [Managing Multiple License Files](#). (By specifying the directory, you do not need to enumerate each license file name on the `lmgrd` command line.)

`lmgrd` processes all files with the `.lic` extension in the directory and then starts all vendor daemons to which the files refer.

See Also

[Managing Multiple License Files](#)

[Capturing Debug Log Output for a Particular Vendor Daemon](#)

Managing Multiple License Files

As described in the previous sections, both `lmadmin` and `lmgrd` handle multiple license files. This section briefly describes how the license server managers handle multiple files and how you set up a multiple-license-file list.

Managing Multiple File in Imadmin

When using `lmadmin` as your license server manager, you can use the `lmadmin` user interface to import the license files one at a time or use the command-line `-import` option (and the `license_file_list`) to specify the files. See the previous sections for details.

Managing Multiple Files in Imgrd

When using `lmgrd` as the license server manager, you can manage multiple license files that are on the same system via a license search path. A license search path is specified two ways:

- By using the `-c` option to `lmgrd`:

```
lmgrd -c license_file_list [other lmgrd options]
```
- By defining the `LM_LICENSE_FILE` environment variable within the scope of the `lmgrd` process's environment.

`lmgrd` builds up an internal license search path when it starts up by parsing each entry in the order listed.

Defining the License File List

Install the license files in convenient locations on the system, and then define the `license_file_list`.

Wherever `license_file_list` is specified, it consists of a list of one or more of the following components. (Use a colon `:` to separate the license file names on UNIX; on Windows, use a semicolon `;`.)

- The full path to the license file
- A directory containing one or more license files with a `.lic` extension
- The license server port. Use one of the following:
 - A `port@host` setting, where `port` and `host` are the TCP/IP port number and host name from the SERVER line in the license file.
 - The shortcut specification, `@host`, if the license file SERVER line uses a default TCP/IP port or specifies a port in the default port range (27000–27009).
 - A comma separated list of three `port@host` specifiers denoting a license servers configured for three-server redundancy (for example, `port1@host1,port2@host2,port3@host3`).

Some scenarios where a license search path is used include those described in [Multiple Systems, One System with Multiple License Server Instances](#), or [One System with One License Server and Multiple License Files](#).

See Also

[Setting the License Search Path Using an Environment Variable](#)
[Ensuring License Availability](#)
[Environment Variables](#)

Additional Considerations

CVD support

If you desire a common vendor daemon (CVD) to support the virtualization hostid types (such as VM_UUID or PHY_*), both the primary and all the secondary vendor daemons that constitute the CVD must have the appropriate vendor keys that support virtualization. If only some of the secondary vendor daemons have the requisite vendor keys that support virtualization, the behavior of the CVD is unpredictable with respect to the support for virtualization hostid types.

Combining License Files

If you have two or more products whose licenses are intended for the same system, as specified by their SERVER lines, you may be able to combine the license files into a single license file. This has advantages if you are using `lmgrd` as your license server manager.



Note • `lmadmin` supports the use of a combined license file. However, use of such a file is redundant when you can simply use the `lmadmin` user interface to add (import) any number of license files. Once a license file is imported, `lmadmin` manages it.

The license files for the models described in [One System with Multiple License Server Instances](#) and [One System with One License Server and Multiple License Files](#) could be combined if they met certain criteria. See [Criteria for Combining License Files](#). Figure 18-4 shows one possible scenario using a combined license file.

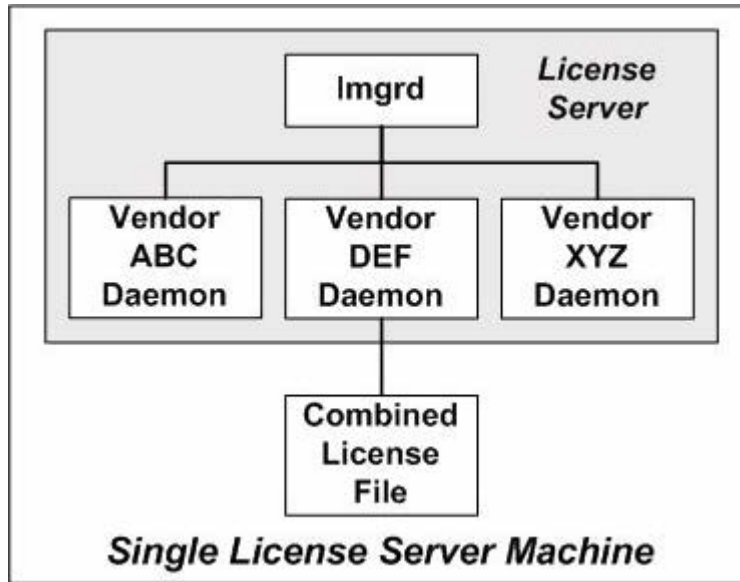


Figure 18-4: One license server manager, one license file

Advantages

- A single license file to administer.
- Once the files are combined, there is low administrative overhead.

Disadvantage

- Careful planning must be given in combining license lines from multiple software publishers into one file, initially and over time.

Criteria for Combining License Files

Your product's license files define the license server systems by host name and hostid in the SERVER lines in the license file. License files are candidates for combining under the following conditions:

- The number of SERVER lines in each file is the same.
- The hostid field of each SERVER line in one file *exactly* matches the hostid field of each SERVER line in the other file.

Some possible reasons license files may not be compatible are:

- License files are set up to run on different server systems, so hostids are different.
- One file is set up for a single license server (has only one SERVER line), the other is set up for a three-server redundancy (has three SERVER lines).
- Hostids for the same system use different hostid types. For example, the SERVER line in one license file uses INTERNET= for its hostid type and the other file uses the ethernet MAC address for its hostid type.

If your license files are compatible as described above, then you have the option of combining license files as summarized in [Figure 18-4](#) and below in [How to Combine License Files](#). Note that you are not required to combine compatible license files. There is no performance or system-load penalty for not combining the files.

How to Combine License Files

If your license files are compatible, use any text editor to combine them. To combine license files, read all of the compatible license files into one file, then edit out the extra SERVER lines so that only one set of SERVER lines remains. Save the resulting data, and you have your combined license file. Figure 18-5 shows an example of combining license files.

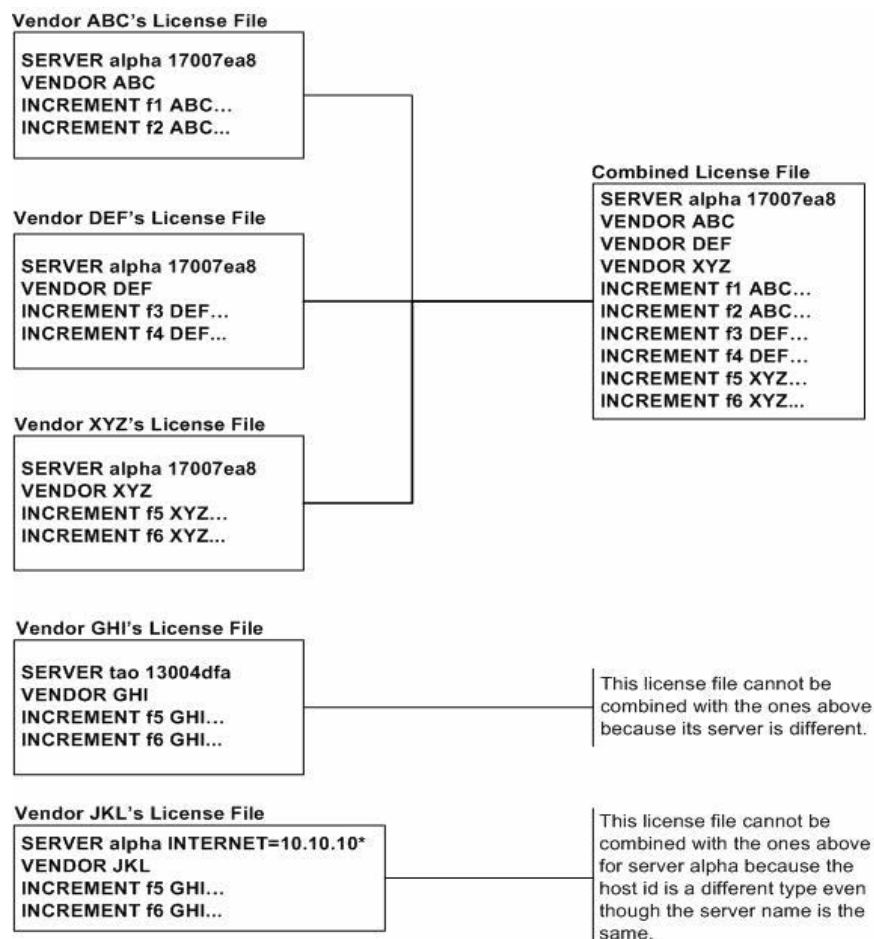


Figure 18-5: Combining license information

Starting the License Server



Task *To start the license server with a combined license file:*

Invoke the license server manager once on the system.

```
lmgrd -c combined_license_file
```

Version Component Compatibility

When one license server manager manages multiple vendor daemons, it may be the case that those vendor daemons do not use the same version of FlexNet Publisher. By observing the FlexNet Publisher version compatibility rules described in [Version Compatibility Between Components](#) you are assured that all of your FlexNet Publisher components are compatible.

You can maintain multiple versions of FlexEnabled applications in the enterprise. The vendor daemon for an application must be at least the same version as the FlexNet Publisher version used in the FlexEnabled application.

Troubleshooting

This section documents areas of the license server that have given customers difficulty in the past.

General Troubleshooting Hints

This list provides some general debugging information:

- When you start the license server be sure that you direct the output into a local log file where you can examine it. The log file often contains useful information. Examine it when you have a problem, and be prepared to answer questions about it when you talk to a support person.
- If the license server appears to have started correctly (which you can determine from the log file), try running `lmstat -a` and `lmdiag` to see if that program has the same problem as your application.
- If your application is version 4.1 or later (version 5 or later on Windows), you can use the `FLEXLM_DIAGNOSTICS` environment variable. Set `FLEXLM_DIAGNOSTICS` to 1, 2, or 3. A setting of 3 gives more information than 2, 2 gives more information than 1 (in particular, the feature name that was denied). See [FLEXLM_DIAGNOSTICS](#) for more information.

- When you talk to a support person, be prepared with answers to the following questions:

- What kind of system is your license server running on?
- What version of the operating system?
- What system and operating system is the application running on?
- What version of FlexNet Publisher does the FlexEnabled application use?

Use the `lmver` script, or, on UNIX, execute the following command on your license server manager, vendor daemon, and application:

```
strings binary_name | grep Copy
```

Alternatives are: for `lmadmin`, use the command `lmadmin -version`; for `lmgrd` and the vendor daemon use the `-v` argument, for example `lmgrd -v`.

- What error or warning messages appear in the log file?

- Did the server start correctly? Look for a message such as:
`server xyz started for: feature1 feature2.`
- What is the output from running `lmstat -a`?
- Are you running other FlexEnabled products?
- Are you using a combined license file or separate license files?
- Are you using three-server redundancy (i.e. there are multiple SERVER lines in your license file)?

FLEXLM_DIAGNOSTICS



Note • The ability for FlexNet Publisher to produce diagnostic output is controlled by your software publisher.

FLEXLM_DIAGNOSTICS is an environment variable that causes the application to produce diagnostic information when a checkout is denied. The format of the diagnostic information may change over time.

On UNIX, the diagnostic output goes to stderr.

On Windows, the output is a file called `flexpid.log` (where `pid` is the application's process ID).

See Also

FLEXLM_DIAGNOSTICS_PATH in [Environment Variables](#)

Level 1 Content

If FLEXLM_DIAGNOSTICS is set to 1, then the standard FlexNet Publisher error message is presented, plus a complete list of license files that the application tried to use. For example:

```
setenv FLEXLM_DIAGNOSTICS 1
FlexNet checkout error: Cannot find license file (-1,73:2) No such file or directory
license file(s): /usr/myproduct/licenses/testing.lic license.lic
```

Level 2 Content

If FLEXLM_DIAGNOSTICS is set to 2, then, in addition to level 1 output, the checkout arguments are presented. For example:

```
setenv FLEXLM_DIAGNOSTICS 2
FlexNet checkout error: No such feature exists (-5,116:2) No such file or directory
license file(s): /usr/myproduct/licenses/testing.lic license.lic
lm_checkout("f1", 1.0, 1, 0x0, ..., 0x4000)
```

Note that the error message actually contains two separate problems, which both occurred during the checkout:

- There is no such feature in the license it did find.
- It was unable to find the other license file, which is what produces the message `No such file or directory`.

This is a description of the arguments to **lm_checkout**:


```
lm_checkout(feature, version, num_lic, queue_flag, ..., dupgroup_mask)
```

where:

Table 19-1 • lm_checkout Arguments

Argument	Description
feature	The requested feature.
version	The requested version. The license file must contain a version $\geq$ the requested version.
num_lic	Number of licenses requested. Usually 1.
queue_flag	If 0, no queueing If 1, queue for license (“blocking” queue) If 2, queue for licenses, but return to application (“non-blocking” queue)
dupgroup_mask	Indicates duplicate grouping, also called license sharing. User, host, and display are as shown by <code>lmstat -a</code> .

Level 3 Content (Version 6.0 or Later Only)

If FLEXLM_DIAGNOSTICS is set to 3, then, in addition to level 1 and 2 output, if a checkout is successful, information is printed explaining how the license was granted:

```
setenv FLEXLM_DIAGNOSTICS 3
app
Checkout succeeded: f0/14263EAEA8E0
License file: ./servtest.lic
No server used
app2
Checkout succeeded: f1/BC64A7B120AE
License file: @localhost
License Server Machine: @localhost
app3
Checkout succeeded: f1/BC64A7B120AE
License file: servtest.lic
License Server Machine: @speedy
```

Note that the feature name and license key are printed, along with the license file location (or host name if `@host` were used) and host name of the server, where applicable.

Error Codes

This section documents FlexNet Publisher error messages, including general format and error message descriptions.

Error Message Format

FlexNet Publisher error messages presented by applications have the multiple components, which are described in the following table. An error message may also contain other optional supporting information.

Table 20-1 • FlexNet Publisher Error Message Components

Component	Description	Required
Error Number	A positive or negative integer that identifies the error.	
Error Text	Sentence that summarizes the issue.	
Error Explanation	Paragraph that explains the problem and provides possible solutions or workarounds.	
Minor Error Number	A positive integer. These numbers are unique error identifiers and are used by software publishers for more advanced support assistance. Their meaning is not documented.	
System Error Number	Error code last set by the operating system.	
System Error Explanation	Sentence that explains the system error.	

These error messages may occur in two formats available with FlexNet Publisher, or they may appear in a format customized by the application.

Format 1 (short)

FlexNet error text (*Lm_errno*, *minor_num*[:*sys_errno*]) [*sys_error_text*]

The error information may be missing.

Example

Can't connect to license server machine (-15,12:61) Connection refused

Format 2 (long)

FlexNet error text

FlexNet error explanation

[Optional Supporting information]

FlexNet error: *Lm_errno*, *minor_num*. [System Error: *sys_errno*] [*system_error_text*]

Example

Cannot connect to license server system

The server (lmgrd) has not been started yet, or
the wrong port@host or license file is being used, or the
port or hostname in the license file has been changed.

Feature: f1

Server name: localhost

License path: @localhost:license.dat:/*.lic

FlexNet error: -15,12. System Error: 61 "Connection refused"

Error Code Descriptions

The following table lists the most common errors produced by FlexEnabled applications.

Table 20-2 • Error Codes

Error Code	Description
24	Insufficient privilege to complete initialization.
23	Attempt to access server trusted storage from client app, or vice-versa.
22	Service configuration or dependency issue. See event log for Windows error code.
21	lc_flexinit failed because there were insufficient rights to start the FlexNet Licensing Service. Resolve this by setting the service to start automatically.
20	FlexNet Licensing Service is not installed.
13	Computed path to required file is too long for OS X operating system.
12	Invalid bundle ID on OS X operating system.

Table 20-2 • Error Codes

Error Code	Description
11	Framework specified by bundle ID was not loaded.
10	Error creating path from URL.
9	Error creating URL.
8	Path string not specified in UTF-8 format.
7	A call to <code>lc_flexinit</code> is not allowed after a call to <code>lc_flexinit_cleanup</code> .
6	Activation utility has not been processed using the <code>preptool</code> , or the activation library for the activation utility cannot be found.
5	Unable to allocate resources.
4	Initialization failed.
3	Unsupported version of the operating system.
2	Unable to load activation library.
1	Unable to find activation library.
-1	Cannot find license file.
-2	Invalid license file syntax.
-3	No license server system for this feature.
-4	Licensed number of users already reached.
-5	No such feature exists.
-6	No TCP/IP port number in license file, and FlexNet Licensing Service does not exist. (pre-v6 only)
-7	No socket connection to license server manager service.
-8	Invalid (inconsistent) license key or signature. The license key/signature and data for the feature do not match. This usually happens when a license file has been altered.
-9	Invalid host. The <code>hostid</code> of this system does not match the <code>hostid</code> specified in the license file.
-10	Feature has expired.

Table 20-2 • Error Codes

Error Code	Description
-11	Invalid date format in license file.
-12	Invalid returned data from license server system.
-13	No SERVER lines in license file.
-14	Cannot find SERVER host name in network database. The lookup for the host name on the SERVER line in the license file failed. This often happens when NIS or DNS or the hosts file is incorrect. Work around: Use IP address (for example, 123.456.789.123) instead of host name.
-15	Cannot connect to license server system. The server (lmadmin or lmgrd) has not been started yet, or the wrong <i>port@host</i> or license file is being used, or the TCP/IP port or host name in the license file has been changed. Windows XP SP2 platforms have a limit on the number of TCP/IP connection attempts per second that can be made, which your application may have exceeded. Refer to the manufacturer's documentation on how to change this limit.
-16	Cannot read data from license server system.
-17	Cannot write data to license server system.
-18	License server system does not support this feature.
-19	Error in select system call.
-20	License server system busy (no majority).
-21	License file does not support this version.
-22	Feature checkin failure detected at license server system.
-23	License server system temporarily busy (new server connecting).
-24	Users are queued for this feature.
-25	License server system does not support this version of this feature.
-26	Request for more licenses than this feature supports.
-29	Cannot find ethernet device.
-30	Cannot read license file.
-31	Feature start date is in the future.

Table 20-2 • Error Codes

Error Code	Description
-32	No such attribute.
-33	Bad encryption handshake with vendor daemon.
-34	Clock difference too large between client and license server system.
-35	In the queue for this feature.
-36	Feature database corrupted in vendor daemon.
-37	Duplicate selection mismatch for this feature. Obsolete with version 8.0 or later vendor daemon.
-38	User/host on EXCLUDE list for feature.
-39	User/host not on INCLUDE list for feature.
-40	Cannot allocate dynamic memory.
-41	Feature was never checked out.
-42	Invalid parameter.
-43	No key data supplied in call to lc_new_job or lc_init .
-44	Invalid key data supplied.
-45	Function not available in this version.
-47	Clock setting check not available in vendor daemon.
-48	Platform not enabled.
-49	Date invalid for binary format.
-50	Key data has expired.
-51	Not initialized.
-52	Vendor daemon did not respond within timeout interval.
-53	Checkout request rejected by vendor-defined checkout filter.
-54	No FEATURESET line in license file.
-55	Incorrect FEATURESET line in license file.

Table 20-2 • Error Codes

Error Code	Description
-56	Cannot compute FEATURESET data from license file.
-57	socket call failed.
-58	setsockopt() failed.
-59	Message checksum failure.
-60	License server system message checksum failure.
-61	Cannot read license file data from license server system.
-62	Network software (TCP/IP) not available.
-63	You are not a license administrator.
-64	Imremove request before the minimum Imremove interval.
-65	Unknown VENDORCODE struct type passed to lc_new_job or lc_init .
-66	Include file/library mismatch.
-67	No licenses available to borrow.
-68	License BORROW support not enabled.
-69	FLOAT_OK can't run standalone on license server system.
-71	Invalid TZ environment variable.
-72	Old VENDORCODE(3-word) struct type passed to lc_new_job() or lc_init() .
-73	Local checkout filter rejected request.
-74	Attempt to read beyond end of license file path.
-75	SYS\$SETIMR call failed (VMS). Indicates an error due to an operating system failure.
-76	Internal FlexNet Licensing error. Please report error to Revenera.
-77	Bad version number must be floating-point number with no letters.
-78	FLEXadmin API functions not available.
-82	Invalid PACKAGE line in license file.
-83	FlexNet Licensing version of client newer than server.

Table 20-2 • Error Codes

Error Code	Description
-84	USER_BASED license has no specified users; see license server system log.
-85	License server system doesn't support this request.
-87	Checkout exceeds MAX specified in options file.
-88	System clock has been set back.
-89	This platform not authorized by license.
-90	Future license file format or misspelling in license file. The file was issued for a later version of FlexNet Licensing than this program understands.
-91	Encryption seeds are non-unique.
-92	Feature removed during lmrread, or wrong SERVER line hostid.
-93	This feature is available in a different license pool. This is a warning condition. The server has pooled one or more INCREMENT lines into a single pool, and the request was made on an INCREMENT line that has been pooled.
-94	Attempt to generate license with incompatible attributes.
-95	Network connect to THIS_HOST failed. Change this_host on the SERVER line in the license file to the actual host name.
-96	License server machine is down or not responding. See the system administrator about starting the server, or make sure that you're referring to the right host (see LM_LICENSE_FILE environment variable).
-97	The desired vendor daemon is down. 1) Check the lmadm or lmgrd log file, or 2) Try lmrread.
-98	This FEATURE line can't be converted to decimal format.
-99	The decimal format license is typed incorrectly.
-100	Cannot remove a linger license.
-101	All licenses are reserved for others. The system administrator has reserved all the licenses for others. Reservations are made in the options file. The server must be restarted for options file changes to take effect.
-102	A FLEXID borrow error occurred.

Table 20-2 • Error Codes

Error Code	Description
-103	Terminal Server remote client not allowed.
-104	Cannot borrow that long.
-105	Feature already returned to license server.
-106	License server system out of network connections. The vendor daemon can't handle any more users. See the debug log for further information.
-107	Can't borrow a PACKAGE component.
-110	Cannot read dongle: check dongle or driver. Either the dongle is unattached, or the necessary software driver for this dongle type is not installed.
-112	Missing dongle driver. In order to read the FLEXID hostid, the correct driver must be installed. These drivers are available from your software publisher.
-114	SIGN= keyword required, but missing from license certificate. You need to obtain a SIGN= version of this license from your publisher.
-115	Error in Public Key package.
-116	TRL not supported for this platform.
-117	BORROW failed.
-118	BORROW period expired.
-119	lmdown and lmreread must be run on license server.
-120	Cannot lmdown the server when licenses are borrowed.
-121	FLOAT_OK requires exactly one FLEXID hostid.
-122	Unable to delete local borrow info.
-123	Returning a borrowed license early is not supported. Contact the publisher for further details.
-124	Error returning borrowed license.
-125	A PACKAGE component must be specified.

Table 20-2 • Error Codes

Error Code	Description
-126	Composite hostid not initialized.
-127	A item needed for the composite hostid is missing or invalid.
-128	Error, borrowed license doesn't match any known server license.
-135	Error enabling the event log.
-136	Event logging is disabled.
-137	Error writing to the event log.
-139	Communications timeout.
-140	Bad message command.
-141	Error writing to socket. Peer has closed socket.
-142	Error, cannot generate version specific license tied to a single hostid, which is composite.
-143	Version-specific signatures are not supported for uncounted licenses.
-144	License template contains redundant signature specifiers.
-145	Bad V71_LK signature.
-146	Bad V71_SIGN signature.
-147	Bad V80_LK signature.
-148	Bad V80_SIGN signature.
-149	Bad V81_LK signature.
-150	Bad V81_SIGN signature.
-151	Bad V81_SIGN2 signature.
-152	Bad V84_LK signature.
-153	Bad V84_SIGN signature.
-154	Bad V84_SIGN2 signature.
-155	License key required but missing from the license certificate. The application requires a license key in the license certificate. You need to obtain a license key version of this certificate from your publisher.

Table 20-2 • Error Codes

Error Code	Description
-156	Invalid signature specified with the AUTH= keyword.
-157	Trusted storage has been compromised; repair needed. Contact your publisher for repair instructions.
-158	Trusted storage open failure. Contact your publisher for further information.
-159	Invalid fulfillment record. Contact your publisher for further information.
-160	Invalid activation request received. Contact your publisher for further information.
-161	No fulfillment exists in trusted storage which matches the request. Contact your publisher for further information.
-162	Invalid activation response received. Contact your publisher for further information.
-163	Cannot return the specified activation. Contact your publisher for further information.
-164	Return count(s) would exceed the maximum for the fulfillment. Contact your publisher for further information.
-165	No repair count left. Contact your publisher for further repair authorization.
-166	Specified operation not allowed. Contact your publisher for further information.
-167	The requested activation has been denied because the user or host is excluded from activating this entitlement by a specification in the options file.
-168	The options file contains include specifications for the entitlement, and this user or host is not included in these specifications.
-169	Activation error. Contact your publisher for further information.
-170	Invalid date format in trusted storage. Can be caused by setting your system clock to an earlier date. Check that your system clock is set to the current date and time.
-171	Message encryption failed. Internal error. Report issue to Revenera.
-172	Message decryption failed. Internal error. Report issue to Revenera.
-173	Bad filter context. Internal error. Report issue to Revenera.
-174	SUPERSEDE feature conflict. Contact your publisher for further information.
-175	Invalid SUPERSEDE_SIGN syntax. Contact your publisher for further information.

Table 20-2 • Error Codes

Error Code	Description
-176	SUPERSEDE_SIGN does not contain a feature name and license signature. Contact your publisher for further information.
-177	ONE_TS_OK is not supported in this Windows Platform.
-178	Internal error. Report issue to Revenera. When more than one remote desktop checkout is performed expect -178: Internal error if more than one terminal server remote client checkout occurred. If it is not more than one remote desktop checkout, then report the issue to Revenera.
-179	Only one terminal server remote client checkout is allowed for this feature.
-180	Internal error. Report issue to Revenera.
-181	Internal error. Report issue to Revenera.
-182	Internal error. Report issue to Revenera.
-183	More than one ethernet hostid not supported in composite hostid definition. Contact your publisher for further information.
-184	The number of characters in the license file paths exceeds the permissible limit. There is a limit on the number of license files that can be used by a license server manager. This limit is on the number of characters in the combined license file paths to the license files: • UNIX—40,960 characters • Windows—20,400 characters Reduce the number of license files, or relocate them so that the paths are shorter.
-185	Invalid TZ keyword syntax. Returned at license encryption time if the TZ keyword syntax is not valid. Ensure that, if specifying multiple time zones, the space delimiter is used and the values are enclosed in quotation marks. If using the SERVETZ value, it must be used by itself.
-186	Invalid time zone override specification in the client.
-187	The time zone information could not be obtained. A license that is time zone limited could not be checked out because time zone information could not be obtained for the machine on which the license is required. Contact your publisher for further information.

Table 20-2 • Error Codes

Error Code	Description
-188	License client time zone not authorized for license rights. A license that is time zone limited could not be checked out because the time zone of the machine on which the license is required does not match the time zone specified in the license.
-189	Invalid syntax for VM_PLATFORMS keyword. Returned at license encryption time if an invalid keyword is specified for use with VM_PLATFORMS.
-190	Feature can be checked out from Physical machine only. The license specifies that it cannot be used on a virtual machine: The FlexEnabled application is installed on a virtual machine so checkout has been denied. Install the FlexEnabled application on a physical machine.
-191	Feature can be checked out from Virtual machine only. The license specifies that it cannot be used on a physical machine. The FlexEnabled application is installed on a physical machine so checkout has been denied. Install the FlexEnabled application on a virtual machine.
-192	VM platform not authorized by license. Returned at run time when the feature definition line specifies a virtual environment but the client is running in another virtual environment. For example, VM_PLATFORMS=VMW but the client is checking out from a HYPER-V virtual environment.
-193	FNP vendor keys do not support Virtualization feature.
-194	Checkout request denied as it exceeds the MAX limit specified in the options file.
-198	Unsupported hostid provided in feature line.
-199	Failed to load ServerQuery request.
-200	Failed to generate ServerQuery response.
-201	Invalid IP address used while overriding. The IP address specified for the LM_A_INTERNET_OVERRIDE attribute, used to override the existing IP address, is invalid.
-202	Returning borrowed feature failed as same feature borrowed from different Vendor daemons.
-203	Failed to get the total feature count.
-204	Activation borrow reclaim operation is not allowed.
-205	Failed to perform activation borrow reclaim operation.

Table 20-2 • Error Codes

Error Code	Description
-206	No deduction record found for the requested client host.
-207	The license server does not support trusted storage.
-208	Trusted storage could not be saved. Used in vendor daemon for debug log purposes. In FlexNet utilities, the usage of this error code is limited to activation borrow reclaim and in that case, "Trusted storage could not be saved by the license server" is seen as a failure reason. This error code is not used in a FlexNet client or an activation client.
-209	Maximum number of servers reached.
-210	License service failed to return VM attributes.
-211	VM attributes not available on physical machine.
-212	FlexNet Licensing Service was found to be disabled.
-213	FlexNet Licensing Service is not installed.
-214	FlexNet Licensing Service version is not as expected.
-215	The VM Host ID is not available.
-216	Failed to get all requested license(s) in reconnection.
-217	The supplied checkout option flag is not supported.
-218	The checkout-avail flag is not supported with PACKAGE licenses.
-219	The checkout-avail (COAVAIL) checkout is not supported.
-220	Failed to get licenses from trusted storage.
-221	A transfer from a license server on the same machine is not allowed.
-222	An unsupported hostid.
-223	FlexNet Licensing Service requires new install.
-224	FlexNet Licensing Service is not available.
-225	Error in communication with the FlexNet Licensing Service.
-226	FlexNet Licensing Service failed to return TPM attributes.

Table 20-2 • Error Codes

Error Code	Description
-227	TPM version is not supported.
-228	TPM is disabled.
-229	TPM properties are not available.
-230	TPM hostid is not available.
-231	TPM is not supported on this platform.
-232	TPM attributes could not be obtained.
-233	AMZ Hostid not initialized.
-234	Not an Amazon EC2 Instance.
-235	Activation is not allowed in three-server failover configuration.
-236	Lmborrowl utility is required for returning features containing a hyphen in the name.
-237	Maximum number of connections reached for this vendor daemon.
-238	string size exceeds MAX_VENDOR_CHECKOUT_DATA
-239	Checkout Data String exceeds the limit
-240	The server does not support asynchronous lc_vsend processing
-241	There is an asynchronous message currently either queued or executing
-242	The lc_vsend message is invalid
-243	There is no message response either pending or completed on the server
-244	The server cannot currently accept asynchronous lc_vsend messages
-245	Invalid license file extension
-246	Unsupported license file encoding.
-247	Invalid Vendor/ Daemon name in the license file.
-248	No Vendor/Daemon line in license file.
-249	Vendor ports are not in range in license file.
-250	Vendor name mentioned is not served by the lmgrd.

Table 20-2 • Error Codes

Error Code	Description
-251	Feature Line license key is invalid.
-252	Feature line is not for the current vendor.
-253	Feature line format is invalid.
-254	Few keywords in the license file are invalid.
-255	The license file size has exceeded the maximum allowed limit of 1 MB for validation.
-256	File names passed should be different.
-257	Unable to open the file
-258	Incorrect RESERVE line format.
-259	Unsupported RESERVE line Keyword
-260	Invalid license count passed in RESERVE line
-261	RESERVATION is unsupported for feature with UNCOUNTED licenses
-262	Reservation Id does not exist on vendor daemon
-263	Invalid duration passed for reservation
-264	Cannot reserve BUNDLE packages with RESERVE lines
-265	No such file exists
-266	File is empty
-267	File doesn't have read permission
-268	Length of reserve line exceeded the maximum size
-269	Number of reserve lines exceeded the specified limit
-270	Markers misplaced, please check the debug log file.
-271	Not running under docker platform
-272	Docker ContainerID not found
-273	Transaction not found
-274	Transaction entry attribute is not set

Table 20-2 • Error Codes

Error Code	Description
-275	Local checks for checkout failed
-276	Duplicate checkout not possible under DUP_GROUP criteria
-277	Too many entry for batch checkout

Report Log File

The license server produces both report log files and debug log files. The focus of this section is report log files. For information on debug log files see [Debug Log File](#).

The report log file contains feature usage information and is generated by the vendor daemon. However, a vendor daemon does not write report logs by default; this action must be enabled. The data in report logs is compressed, authenticated, and organized into a repository.

Use Revenera's software license administration solution, FlexNet Manager, to gain exceptional visibility into license usage data and to create insightful reports on critical information like license availability and usage. FlexNet Manager can be fully automated to run these reports on schedule and can be used to track license servers and usage across a heterogeneous network of server including Windows NT, Linux and UNIX. Contact Revenera at www.revenera.com for more details on how to obtain an evaluation copy of FlexNet Manager for your enterprise.

Managing Report Log Output

As a vendor daemon runs for a period of time, the volume of report log output increases. If you have a lot of license activity, these log files grow very large. You need to consider where to put these files and how often to rotate and archive them. Therefore, it may be necessary to rotate or switch report log output into different files over time, each file containing license activity over a particular period of time.

Report log data is collected by the vendor daemon into an internal data buffer area before being flushed to the output file. The daemon's internal buffer is flushed once a minute or whenever it gets full, whichever occurs first. To ensure the freshest data possible in the report log file, flush the buffer on demand with the `lmmreread` command. Use standard file compression tools to reduce the size of a report log file when it is no longer being written.

To avoid corruption and for performance, it is suggested that the vendor daemon write its report log to a file on a disk local to the system running the vendor daemon. Each vendor daemon must write to its own report log file.

Enabling Report Log Output for a Vendor Daemon

There are two ways to enable report logging for a particular vendor daemon either before or after starting the license server.

- Add the REPORTLOG line to the options file for that vendor daemon. See [REPORTLOG](#) for more details.
- Invoke `lmswitchr` on the vendor daemon. See [lmswitchr](#) for more details.

Redirecting Report Log Output for a Vendor Daemon

The report log output for a particular vendor daemon can be moved into separate files, each file representing activity over a different period of time. There are three ways in which to do this whether the vendor daemon is running or not:

- Change the REPORTLOG line in the vendor daemon's options file and reread its options file by invoking `lmreread` (version 8.0 or later vendor daemon) or restart.
- Invoke `lmswitchr` on the vendor daemon. See [lmswitchr](#) for more details.
- Invoke `lmnewlog` on the vendor daemon. Requires a version 7.1 or later vendor daemon. See [lmlicvalidator](#) for more details.

Debug Log File

The license server produces both debug log files and report log files. For information on report log files, see [Report Log File](#).

A debug log file contains status and error messages useful for debugging the license server. The `lmgrd` debug log displays the server's system date, time, and time zone at the beginning of a log. A license server always generates debug log output. Some of the debug log output describes events specific to `lmadmin` or `lmgrd` and some of the debug log output describes events specific to each vendor daemon.

Managing Debug Log Output

As the license server manager and its vendor daemons run for a period of time, the volume of this output increases. As it gets older, the value of the debug log output decreases; therefore, it may be necessary for you to separate old debug log output from current output; either archive or delete the old output.

For performance, it is suggested that each debug log file be on a disk that is local to the system that is running the license server manager and its vendor daemons. However, if the debug log file must be on a remotely-mounted disk and you find that the license server is too slow, start `lmgrd` with the `-nfs_log` option to improve performance.

See [Debug Log Messages](#) for a description of the debug log output format.

Capturing Debug Log Output for a License Server

If you are using `lmadmin` as your license server manager, separate log files are created for `lmadmin` and each vendor daemon that it manages. The log files are written to the `logs` directory within the `lmadmin` install directory (default on non-Windows platforms) or the `<lmadmin_runtimedata_dir>\logs` folder (default on Windows platforms). However, you can use the `-logDir` option on the command line to change the location to which `lmadmin` writes `lmadmin.log`. See [lmadmin Command-line Arguments](#) for details.

By default, `lmgrd` and the vendor daemons it manages write debug log output to standard out. To put this debug log output in a file, either redirect the output of the license server to a file or start `lmgrd` with the `-l debug_log_path` option. See [lmgrd Command-Line Syntax](#) for more information.

Capturing Debug Log Output for a Particular Vendor Daemon

The debug log output from different vendor daemons controlled by the same license server can be written to their own files (version 8.0 and later vendor daemon). There are three ways to do this:

- If you are using `lmadmin` as your license server manager, you configure the location and file name from the Vendor Daemon Configuration screen. See on-line help for information on Vendor Daemon Log.
- Add the `DEBUGLOG` line to the options file for each vendor daemon. See [DEBUGLOG](#) for more details.
- Invoke `lmswitch` on the vendor daemon. See [lmswitch](#) for more details.

Note that `lmgrd` writes its own debug log output to standard out.

Redirecting Debug Log Output for a Running Vendor Daemon

It is possible to redirect the debug log output for a particular vendor daemon to a different file. There are two ways to do this:

- Change the `DEBUGLOG` line to the options file for the vendor daemon and reread its options file by invoking `lmreread`. See [DEBUGLOG](#) for more details.
- Invoke `lmswitch` on the vendor daemon. See [lmswitch](#) for more details.

Limiting Debug Log Output for a Vendor Daemon

By default, debug log output contains all events. To limit the events that are logged for a particular vendor daemon, add a `NOLOG` line to the options file of that vendor daemon. One of the reasons you may want to limit the events that are logged is to reduce the size of the debug log output.

See Also
[NOLOG](#)

License Server Diagnostics in the Debug Log

Enhanced license server diagnostics will now be emitted in the license server debug log. Additional diagnostics are prefixed by “`(@lmgrd-SLOG@)`” for the `lmgrd` and “`(@<vendor>-SLOG@)`” for the Vendor Daemon, facilitating easy `grep`-like extraction from the debug-log. The Vendor Daemon would emit the additional diagnostics when run with `lmgrd` or `lmadmin`. These diagnostics are intended to assist publishers and Revenera diagnose license server issues. These diagnostics include:

- For `lmgrd`:

Start timestamp, Version, ProcessID, Network Info (IPv4/IPv6 interface, listening port), command-line options, list of license files, whether the license server is running as a Windows service.
- For Vendor Daemon:

Start timestamp, Version, ProcessID, Network info (IPv4/IPv6 interface, listening port, select timeout of vendor daemon), Vendor daemon name, options file used, Trusted Storage accessed (yes/no). Host name, Virtual/physical environment, (limited) hypervisor info if virtual, #Restarts of vendor daemon since lmgrd startup, #Rereads since vendor daemon (reO start, Last reread time, Reread initiation type (automatic versus lmreread)

Every 4 hours (and at vendor daemon shutdown) the following performance data are logged:

Details of the 10 peak client transactions (including client hostname, user, transaction time, concurrent clients at time of transaction, memory usage (Windows only)), Peak concurrent clients served, High watermark memory usage (Windows only), Transaction time for last 10 transactions.

Debug Log Messages

FlexNet Publisher processes generate debug log files in the following format:

hh:mm:ss (daemon) message

where:

Table 22-1 ▪ Debug Log Messages

Message	Description
<i>hh:mm:ss</i>	Time that the message was logged.
daemon	Either lmadm, lmgrd or the vendor daemon name. In the case where a single copy of the daemon cannot handle all of the requested licenses, an optional “_” followed by a number indicates that this message comes from a forked daemon.
message	The text of the message.

To get the date and time zone offset along with the timestamp in the debug log files, use the command line option “-datestamp”. This optional format follows the ISO8601 standard, where the time is represented as:

yyyy-mm-ddThh:mm:ssZ

Where Z is the time zone offset.



Note ▪ This command line option is made available for lmgrd and not supported in lmadm.

The debug log files can be used to diagnose configuration problems and vendor-daemon software errors.



Note ▪ A debug log file cannot be used for usage reporting with FlexNet Manager.

Informational Messages

The following table lists the various informational messages used within FlexNet Publisher.

Table 22-2 ■ Information Messages

Message	Description
Connected to host	This daemon is connected to its peer on host.
CONNECTED, master is host	The license daemons log this message when a quorum is up and everyone has selected a master.
DENIED: “feature” @host (reason (error)) ^^^ ver user ip pid handle in_use out_of project_name flex_version flex_revision date time is_dup_group ^^^	User was denied access to licenses of feature when the <code>ls_enable_verbose_dlog</code> vendor variable is set to 1.
DENIED: num_lic feature to user	User was denied access to <code>num_lic</code> licenses of feature.
DEQUEUED: “feature” @host ^^^ ver user ip pid handle in_use out_of project_name flex_version flex_revision date time is_dup_group ^^^	User was exited from the queue upon receiving a free license when the <code>ls_enable_verbose_dlog</code> vendor variable is set to 1.
EXITING DUE TO SIGNAL nnn EXITING with code nnn	All daemons list the reason that the daemon has exited.
EXPIRED: feature	feature has passed its expiration date.
IN: “feature” @host (num_license) ^^^ ver user ip pid handle in_use out_of project_name flex_version flex_revision date time is_dup_group ^^^	user has checked in <code>num_lic</code> licenses of feature when the <code>ls_enable_verbose_dlog</code> vendor variable is set to 1.
IN: “feature” user (num_lic licenses)	User has checked in <code>num_lic</code> licenses of feature.
IN: feature user@host [cdstring] (num_lic licenses)	user has checked in <code>num_lic</code> licenses of feature from host with custom duplicate grouping using <code>cdstring</code> as the checkout data string.
Lost connection to host	A daemon can no longer communicate with its peer on node host, which can cause the clients to have to reconnect, or cause the number of daemons to go below the minimum number, in which case clients may start exiting. If the license daemons lose the connection to the master, they kill all the vendor daemons; vendor daemons shut themselves down.

Table 22-2 • Information Messages

Message	Description
Lost quorum	The daemon lost quorum, so it processes only connection requests from other daemons.
MULTIPLE vendor servers running. Kill and restart license daemon.	The license server manager has detected that multiple vendor daemons with the same vendor name are running. Shutdown <code>lmadmin</code> or <code>lmgrd</code> and all vendor daemons and then restart <code>lmadmin</code> or <code>lmgrd</code> .
OUT: “feature“ @host (num_license) ^^^ ver user ip pid handle in_use out_of project_name flex_version flex_revision date time is_dup_group ^^^	User has checked out <code>num_lic</code> licenses of feature when the <code>ls_enable_verbose_dlog_vendor</code> variable is set to 1.
OUT: feature user (num_lic licenses)	User has checked out <code>num_lic</code> licenses of feature.
OUT: feature user@host [cdstring] (num_lic licenses)	User has checked out <code>num_lic</code> licenses of feature from host with custom duplicate grouping using <code>cdstring</code> as the checkout data string.
QUEUED: “feature“ @host ^^^ ver user ip pid handle in_use out_of project_name flex_version flex_revision date time is_dup_group ^^^	User has been pushed into the queue to wait until a license gets free as maximum number of licenses usage has reached, when the <code>ls_enable_verbose_dlog_vendor</code> variable is set to 1.
RESERVE feature for USER user RESERVE feature for HOST host	A license of feature is reserved for either user or host.
REStarted vendor (internet port nnn)	Vendor daemon <code>vendor</code> was restarted at TCP/IP port <code>nnn</code> .
Retrying socket bind (address in use)	The license servers try to bind their sockets for approximately six minutes if they detect “address in use” errors.
Selected (EXISTING) master host.	This license daemon has selected an existing master <i>host</i> as the master.
SERVER shutdown requested.	A daemon was requested to shut down via a user-generated <code>kill</code> command.
Server started on host for: feature_list	A (possibly new) server was started for the features listed.
Shutting down vendor	The license server manager is shutting down the vendor daemon <code>vendor</code> .
SIGCHLD received. Killing child servers.	A vendor daemon logs this message when a shutdown was requested by the license daemon.

Table 22-2 • Information Messages

Message	Description
Started vendor	The license server manager logs this message whenever it starts a new vendor daemon.
TIMESTAMP	A vendor daemon logs this message at regular intervals. The default interval between vendor daemon timestamps is 6 hours 5 minutes. A license server manager (lmadmin or lmgrd) logs this message at regular intervals. The default interval between license server manager timestamps is 6 hours.
Trying to connect to host	The daemon is attempting a connection to host.
UPGRADE: “<feature>”(x->y licenses)	Identical checkout requests (using the LM_A_CHECKOUT_DATA attribute) has incremented the number of licenses checked out for a given feature within the same license job. The previous request checked out x licenses, while latest request has checked out y licenses, for a total of x + y licenses checked out for the feature.

Verbose Debug Log Messages

If a producer enables the verbose information to be displayed in debug logs, you would be observing extra info being printed in the logs.

Enabling the verbose information affects the following events :

- OUT
- IN
- QUEUED
- DEQUEUED
- DENIED

For instance, the following is the OUT debug log message format after enabling the verbose information:

```
“feature“ @host (num_license) ^^^ ver user ip pid handle in_use out_of project_name flex_version flex_revision  
date time is_dup_group ^^^
```

Table 22-3 • OUT Debug Log Message Format

Terminology	Description
feature	Indicates the feature name.
@host (num_license)	Indicates the client’s host name.
ver	Indicates the feature’s version as requested in the client application.

Table 22-3 • OUT Degug Log Message Format

Terminology	Description
user	Indicates the client's user name.
ip	Specifies the client's IP (Internet Protocol) address.
pid	Specifies the client's Process ID.
handle	Indicates the client handle.
in_use	Specifies the licenses in use.
out_of	specifies the total number of licenses including all the licences in use.
project_name	Indicates value of the LM project.
flex_version	Specifies the FlexNet Publisher version of client. For instance, flex_version for 11.19 version is 11.
flex_revision	Specifies the FlexNet Publisher Revision of client. For instance, flex_revision for 11.19 version is 19.
date	Indicates the date of operation.
time	Indicates the time of operation.
is_dup_group	Indicates the value of is_dup_group attribute.

For more informaton on the IN, OUT, QUEUED, DEQUEUED, and DENIED debug log messages, see the [Informational Messages](#).



Note • The following shows the difference in the OUT debug log messages without and with enabling the verbose information:

- By default, the vendor daemon generates the 8:41:54 (demo) OUT: "f1" ADas@BLR-LT-104276 debug log message.
- If your vendor has enabled the verbose information, the vendor daemon generates the OUT : 8:41:54 (demo) OUT: "f1" ADas@BLR-LT-104276 ^^^ 1.0 ADas 127.0.0.1 29865 0x12CE 1 3 N/A 11 19 8-8-2023 8:41:56 YES ^^^ debug log message.



Note • These verbose debug log messages can be used as inputs for the Imreporter tool. For more information, see [Imreporter](#).

Configuration Problem Messages

The following table lists configuration problem messages found in FlexNet Publisher.

Table 22-4 • Configuration Problem Messages

Message	Description
host: Not a valid server host, exiting	This daemon was run on an invalid host name.
host: Wrong hostid, exiting	The hostid is wrong for host.
BAD CODE for feature	The specified feature name has a bad license key or signature. It was probably typed in wrong, or modified by the end user.
CANNOT OPEN options file	The options file specified in the license file could not be opened.
Couldn't find a master	The daemons could not agree on a master.
License daemon: lost all connections	This message is logged when all the connections to a server are lost, which often indicates a network problem.
Lost lock, exiting Error closing lock file Unable to re-open lock file	The vendor daemon has a problem with its lock file, usually because of an attempt to run more than one copy of the daemon on a single node. Locate the other daemon that is running via a ps command, and kill it with kill -9.
No DAEMON line for vendor	The license file does not contain a DAEMON or VENDOR line for vendor.
No DAEMON lines, exiting	The license daemon logs this message if there are no DAEMON or VENDOR lines in the license file. Because there are no vendor daemons to start, there is nothing for the license daemon to do.
No features to serve!	A vendor daemon found no features to serve. This could be caused by a corrupted or incorrectly entered license file.
UNSUPPORTED FEATURE request: feature by user	The user has requested a feature that this vendor daemon does not support. This can happen for a number of reasons: the license file is bad, the feature has expired, or the daemon is accessing the wrong license file.
Unknown host: host	The host name specified on a SERVER line in the license file does not exist in the network database (probably /etc/hosts).

Daemon Software Error Messages

The following table lists various daemon software error messages:

Table 22-5 ▪ Daemon Software Error Messages

Message	Description
accept: message	An error was detected in the accept system call.
Can't allocate server table space	A malloc error. Check swap space.
Connection to <i>host</i> TIMED OUT	The daemon could not connect to host.
Illegal connection request to <i>vendor</i>	A connection request was made to vendor, but this vendor daemon is not vendor.
read: error message	An error in a “read” system call was detected.
select: message	An error in a “select” system call was detected. This is usually a sign of a system networking failure.
Server exiting	The server is exiting. This is normally due to an error.

Identifying FlexNet Publisher Versions

This section explains how to identify FlexNet Publisher versions.

- [Version Compatibility Between Components](#)
- [Determining the License File Version](#)

Version Compatibility Between Components

In general, always use the latest version of `lmadmin`, `lmgrd`, `lmutil`, and `lmttools`, all of which are available from www.revenera.com, to exploit the enhancements available in the most recent versions of FlexNet Licensing. However, some enhancements require a vendor daemon built with a newer version of FlexNet Publisher, and yet others require a FlexEnabled application built with a newer version of FlexNet Publisher. Contact your software publisher for the latest version of their vendor daemon.

The rules about FlexNet Licensing component version compatibility are summarized as:

- Version of `lmutil`/`lmttools` must be `>=`
- Version of `lmadmin` (or `lmgrd`), which must be `>=`



Note ▪ `lmadmin` can only be used with components with a version of 9.2 or later.

- Version of vendor daemon, which must be `>=`
- Version of the client library linked to the FlexEnabled application, which must be `>=`
- Activation utility, which must be `>=`
- Version of license file format

Except for the license file, use `lmver` to discover the version of all these components. For the vendor daemon, `lmgrd`, and `lmutil`, you can also use the `-v` argument to print the version.

Determining the License File Version

The following rules apply to individual FEATURE, INCREMENT or UPGRADE lines. It is possible to have a mix of versions in a single file. Only the features that a particular application checks out determine the version of the license for that feature.

Table 23-1 ■ Determining the License File Version

Version	Description
Version 2	Blank quotes or a quoted string at the end of the FEATURE line.
>= Version 3	INCREMENT or UPGRADE line.
>= Version 4	OVERDRAFT, DUP_GROUP, INTERNET, or PACKAGE appear.
>= Version 5	SUPERSEDE, ISSUED, USER_BASED, HOST_BASED, or SN appear.
>= Version 6	START appears.
>= Version 7.1	SIGN= keyword appears.
>= Version 8.0	BORROW, FLOAT_OK, and TS_OK appear.
>= Version 8.1	SUITE_RESERVED appears.
>= Version 8.4	COMPOSITE appears.
>= Version 11.5	ONE_TS_OK and SUPERSEDE_SIGN appear.
>=Version 11.7	VM_PLATFORMS and TZ appear.

Environment Variables

Environment variables are not required in order to use FlexEnabled applications. Environment variables are normally used for debugging or for changing license default location.

How to Set Environment Variables

FlexNet Publisher environment variables are set in two different ways:

- In the process environment.
- In the registry (Windows version 6.0 or earlier) or in `$HOME/.flexlmrc` (UNIX version 7.0 or earlier), which functions like the registry on UNIX.

Windows Registry

On Windows systems, the registry key location is `HKEY_CURRENT_USER\SOFTWARE\FLEXlm License Manager`.

On UNIX, the equivalent information is stored in `$HOME/.flexlmrc`. In this file, the syntax is *variable=value*.

Precedence

If the variable is `LM_LICENSE_FILE` or `VENDOR_LICENSE_FILE`, then both the environment and the registry are used, with the environment used first, and the registry appended to the path.

If it's a different variable, then if the environment is set, only that is used, otherwise the registry is used. That is, the registry is only used if the environment is not set.

Environment Variables

The table below provides various environment variables and their definitions:

Table 24-1 ■ Environment Variables

Variable	Definition
FLEXLM_BATCH	(Windows only) Prevents interactive pop-ups from appearing. Set to 1 if a batch application. (Version 7.0 and later clients)
FLEXLM_DIAGNOSTICS	Used for debugging where applications do not print error message text. Set to 1, 2, or 3, depending on the amount of diagnostic information desired. See FLEXLM_DIAGNOSTICS (Version 5.0 and later clients)
FLEXLM_DIAGNOSTICS_PATH	Provides the path for the flexlm diagnostics logs file. During initialization, the priority for the diagnostics log file creation is defined in this environment variable. If unsuccessful, the log file will be created in the current working directory. If log file creation on both the paths were unsuccessful then, • in Windows, it tries to create the log file again in "<system drive>\ProgramData\FLEXnet" else will be set as "stderr". • in Non Windows platform, the diagnostics logs file will be set as "stderr".
FLEXLM_TIMEOUT	(Windows only) Sets the timeout value a FlexEnabled application uses when attempting to connect to a license server. Values are in microseconds, within the range of 200,000 (0.2 seconds) through 20,000,000 (20 seconds). The default value is 3,000,000 microseconds (that is 3 seconds).
LM_BORROW	Used to initiate the borrowing and transferring of licenses, as well as set the borrow and transfer period for licenses. On UNIX platforms, the registry used for license borrowing is \$HOME/.flexlmborrow, instead of \$HOME/.flexlmrc, and the registry used for license transferring is \$HOME/.flexlmtransfer, instead of \$HOME/.flexlmrc. For more information, refer the below: • Initiating License Borrowing • Initiating License Transferring
LM_LICENSE_FILE or VENDOR_LICENSE_FILE	Reset path to license file. Can be a license search path, separated by ":" on UNIX and ";" on Windows. If VENDOR_LICENSE_FILE is used, <i>VENDOR</i> is the vendor daemon name used by this application. For example, Revenera products use MVS_N_LICENSE_FILE. Can be a file name, or <i>port@host</i>. See also Setting the License Search Path Using an Environment Variable (VENDOR_LICENSE_FILE requires version 6.0 and later clients).

Table 24-1 • Environment Variables

Variable	Definition
LM_PROJECT	LM_PROJECT's value (up to 255 characters) is logged in the report log file and later reported on by FlexNet Manager. It can also be used to RESERVE, INCLUDE, and so on licenses with PROJECT. For example: <pre>RESERVE 1 f1 PROJECT airplane</pre> The variable is read upon the license server's initial connection with the FlexEnabled client and is reread for each job, enabling you to update the variable value as needed on a job-to-job basis. However, the value with which a job initiates remains in effect for the duration of that job. You cannot modify this value mid-job.  Note • This feature requires that the FlexEnabled client be built with FlexNet Publisher 5.0 or later and that the vendor daemon be built with FlexNet Publisher 7.0 or later.
LM_SERVER_HIGHEST_FD	Used to limit the number of simultaneous client connections to lmgrd. The default is to allow the maximum number of simultaneous client connections (approximately 1000). On Linux, the maximum and default value is 1024. On Windows, because four file descriptors are used per lmgrd job connection, the maximum and default value is 4096. It should be noted that lmgrd hands off connections to the vendor daemon (see Maximum Client Connections to License Server) and that this environment variable does not affect the number of allowed client connections to the vendor daemon.
LM_UTIL_CASE_SENSITIVE	Used by the FlexLM utilities. If set to 1, the utilities process license file lines as case sensitive. By default, this variable is set to 0; license files are treated as case insensitive. This environment variable is applicable only when the license server, itself, has been configured by your software publisher to treat license files in a case sensitive manner.
TCP_NODELAY	Improves license server performance when processing license requests. Set to 1 to enable performance enhancements. Use with caution: when enabled it may cause an increase in network traffic. Set TCP_NODELAY as System variables when setting the service to start at power up using lmtools.
FNP_IP_ENV	Improves the performance while checking out a license. When this environment variable value is set to 1, the hostname resolution function calls are bypassed and the performance is improved.
FNP_IP_PRIORITY	Decides the IP priority for hostname resolution. By default IPv4 address resolution is used, if this resolution fails then it attempts for the IPv6 address resolution. Set FNP_IP_PRIORITY = '6' to use the IPv6 addresses resolution, if this resolution fails then it attempts for the default IPv4 address resolution.

Index

A

activatable license [20](#)
activation [19](#)
ANY hostid [54](#)
asset_info [39](#)
AUTH [35](#), [42](#)

B

BORROW_LOWWATER [189](#)
borrowing [64](#)

C

client connections, maximum [124](#)
cloud
 hostids [232](#)
 licensing environments [231](#)
 support [232](#)
COMPOSITE
 hostid [54](#)
concurrent license [16](#), [20](#), [59](#)
 See also floating license
converting license formats [143](#)
creating a large user group [197](#)
creating options file [184](#)

D

debug log [16](#)
debugging license server [255](#)
DEBUGLOG [190](#)
decimal format licenses [143](#)

- default license server ports 105
- DEMO hostid 54
- diagnosing checkout problems
 - troubleshooting checkouts 138
- disabling
 - lmdown 113, 128
 - lmremove 113, 128
- DISPLAY
 - hostid 54
 - type 187
- dist_info 39
- DUP_GROUP 36

E

- enabling report log 208
- ENTITLEMENT modifier
 - for EXCLUDE 192
 - for INCLUDE 200
- environment variables
 - FLEXLM_BATCH 290
 - FLEXLM_DIAGNOSTICS 290
 - FLEXLM_DIAGNOSTICS_PATH 290
 - FLEXLM_TIMEOUT 290
 - LM_BORROW 290
 - LM_LICENSE_FILE 290
 - LM_PROJECT 291
 - LM_SERVER_HIGHEST_FD 124, 291
 - setting 289
 - VENDOR_LICENSE_FILE 290
- eport
 - VENDOR line 33
- error code
 - descriptions 260
 - format 259
- EXCLUDE 191
- EXCLUDE_BORROW 193
- EXCLUDEALL 194
- EXCLUDEALL_ENTITLEMENT 195
- EXPDATE modifier
 - for EXCLUDE 191
 - for INCLUDE 199
- expiration date 35

F

- feature
 - version 35
- FEATURE line 34
 - asset_info 39
 - AUTH 35
 - dist_info 39
 - DUP_GROUP 36
 - expiration date 35

- feature version [35](#)
- FLOAT_OK [36](#)
- HOST_BASED [36](#)
- HOSTID [36](#)
- ISSUED [36](#)
- ISSUER [37](#)
- license count [35](#)
- ONE_TS_OK [37](#)
- order of precedence [40](#)
- OVERDRAFT [37](#)
- serial number [37](#)
- SIGN [35](#)
- signature [35](#)
- SN [37](#)
- sort [39](#)
- sorting order [40](#)
- START [37](#)
- SUITE_DUP_GROUP [37](#)
- SUPERSEDE [37](#)
- syntax [40](#)
- TS_OK [38](#)
- TZ [38](#)
- USER_BASED [38](#)
- user_info [39](#)
- vendor daemon name [35](#)
- vendor_info [39](#)
- VENDOR_STRING [38](#)
- VM_PLATFORMS [38](#)
- FlexEnabled application [15](#)
- FLEXLM License Finder [223](#)
- FLEXLM_BATCH [290](#)
- FLEXLM_DIAGNOSTICS [256](#)
 - level 1 [256](#)
 - level 2 [256](#)
 - level 3 [257](#)
- FLEXLM_TIMEOUT [290](#)
- FlexNet ID dongle with FLOAT_OK [62](#)
- FlexNet Manager [209](#)
- FLOAT_OK [36](#)
- floating license [16](#), [59](#)
 - See also concurrent license
- fulfillment record [15](#)
 - example [24](#)

G

- GROUP type [197](#)
- GROUPCASEINSENSITIVE [198](#)

H

- HOST type [187](#)
- HOST_BASED [36](#)
- HOST_GROUP type [198](#)

host, SERVER line 31
 HOSTID 36
 hostid 17

- ANY 54
- COMPOSITE 54
- DEMO 54
- DISPLAY 54
- HOSTNAME 54
- ID 54
- in cloud licensing 232
- INTERNET 55
- SERVER line 31
- special 54
- TPM 56
- USER 55

 HOSTNAME hostid 54

I

ID hostid 54
 INCLUDE 198
 INCLUDE_BORROW 200
 INCLUDEALL 203
 INCLUDEALL_ENTITLEMENT 203
 INCREMENT line 34
 INTERNET

- hostid 55
- type 187

 IPv6

- support overview 237

 ISSUED 36
 ISSUER 37

L

license

- borrowing 64
- concurrent 59
- contents 15
- definition 15
- floating 59
- mixed 60
- network license 59
- node-locked 59

 license count 35
 license directory 115, 118
 license file 15

- combining 250
- compatibility between different versions 252
- FEATURE line 34
- format 29
- how to combine 251
- INCREMENT line 34
- lminstall 143

- multiple [249](#)
- order of lines [43, 60](#)
- PACKAGE line [41](#)
- rereading after an update [153](#)
- SERVER lines [252](#)
- specifying for license server [112, 128](#)
- specifying location [45](#)
- types [59](#)
- UPGRADE line [43](#)
- USE_SERVER line [34](#)
- VENDOR line [32](#)
- with multiple servers [114](#)

License Finder [223](#)

license model [15](#)

license pool [34, 186](#)

license rehosting [62](#)

license search path [45](#)

- redundancy [217](#)
- setting using environment variable [46](#)

license server [15, 77, 111](#)

- alerts [77](#)
- debugging [255](#)
- default ports [105](#)
- disk space used [74](#)
- install as service [79, 94](#)
- install as Windows service [181](#)
- license rights [77](#)
- lmadmin [77](#)
 - run in foreground [128](#)
- lmgrd [111](#)
 - run in foreground [113](#)
- maximum client connections [124](#)
- sockets used [73](#)
- specifying license files [112, 128](#)
- starting
 - lmadmin [79, 94](#)
 - lmgrd [115](#)

license server debug log

- lmadmin [128](#)
- starting for lmgrd [112](#)

license server manager [16, 77, 111](#)

LINGER [204](#)

LM_BORROW [290](#)

LM_LICENSE_FILE [290](#)

LM_PROJECT [291](#)

- reporting on project [209](#)
- use in options file [187](#)

LM_SERVER_HIGHEST_FD [124, 291](#)

lmadmin [77](#)

- license server manager not starting [87](#)
- managing multiple license files [249](#)
- starting [79, 94](#)
 - manually [87](#)
- stopping [77, 88, 129](#)

- upgrading 83
- lmborrow 134
- lmdiag
 - syntax 138
 - troubleshooting 138
- lmdown
 - disabling 113, 128
 - enabling for use with lmadmim 129
 - restricting access 101, 112, 128
 - syntax 138
- lmgrd
 - and redundant servers 114
 - compatibility between versions 111
 - debug log file 279
 - managing multiple license files 249
 - maximum client connections 124
 - starting 111, 115
 - starting debug log 112
 - syntax 111
 - use latest 287
- lmhostid 139
- lminstall
 - license file format 143
 - syntax 143
- lmlicvalidator 144
- lmnewlog 147
- lmobfslog 165
- lmremove
 - disabling 113, 128
 - enabling 130
 - restricting access 101, 112, 128
 - syntax 148
- lmreread 153
 - behavior changes with lmadmim 130
 - restricting access 101, 112, 128
 - syntax 153
- lmstat
 - output for lmreread 154
 - syntax 154
- lmswitch 157
- lmswitchr 158
- lmtools 180
- lmtpminfo 159
- lmutil
 - arguments 133
 - lmborrow 134
 - lmdiag 138
 - lmdown 138
 - lmhostid 139
 - lminstall 143
 - lmnewlog 147
 - lmremove 148
 - lmreread 153
 - lmstat 154

- lmswitch [157](#)
- lmswitchr [158](#)
- lmtppminfo [159](#)
- lmver [163](#)
- single executable [131](#)
- lmver [163](#)

M

- MAX [205](#)
- MAX_BORROW_HOURS [206](#)
- MAX_CONNECTIONS [207](#)
- MAX_OVERDRAFT [207](#)
- memory usage, daemons [74](#)
- mixed licenses [60](#)
- mobile licensing
 - borrowing [64](#)
 - FlexNet ID dongle with FLOAT_OK [62](#)
 - node-locked to FlexNet ID dongle [62](#)
 - node-locked to laptop [62](#)
 - node-locked to user name [67](#)
 - prepaid license pool fulfillment [67](#)
- modifiers
 - ENTITLEMENT for EXCLUDE [192](#)
 - ENTITLEMENT for INCLUDE [200](#)
 - EXPDATE for EXCLUDE [191](#)
 - EXPDATE for INCLUDE [199](#)
 - for feature name [186](#)
- multiple licenses, managing [249](#)

N

- network bandwidth and FlexNet Publisher [74](#)
- network license [59](#)
- node-locked license [59](#)
- NOLOG [208](#)

O

- ONE_TS_OK [37](#)
- options file [16](#)
 - BORROW_LOWWATER [189](#)
 - creating [184](#)
 - creating a large user group [197](#)
 - DEBUGLOG [190](#)
 - DISPLAY type [187](#)
 - examples [212](#)
 - EXCLUDE [191](#)
 - EXCLUDE_BORROW [193](#)
 - EXCLUDEALL [194](#)
 - EXCLUDEALL_ENTITLEMENT [195](#)
 - GROUP type [197](#)
 - GROUPCASEINSENSITIVE [198](#)

- HOST type 187
- HOST_GROUP type 198
- INCLUDE 198
- INCLUDE_BORROW 200
- INCLUDEALL 203
- INCLUDEALL_ENTITLEMENT 203
- INTERNET type 187
- LINGER 204
- MAX 205
- MAX_BORROW_HOURS 206
- MAX_CONNECTIONS 207
- MAX_OVERDRAFT 207
- NOLOG 208
- PROJECT type 187
- read by vendor daemon 211
- REPORTLOG 208
- required for HOST_BASED 36
- required for USER_BASED 38
- RESERVE 209
- rules of precedence 212
- TIMEOUT 210
- TIMEOUTALL 211
- type argument 187
- USER type 187
- options file path 33
- OPTIONS=SUITE 42
- OPTIONS=SUITE_RESERVED 42
- order of lines in license file 43, 60
- OVERDRAFT 37

P

- PACKAGE line 41
 - AUTH 42
 - OPTIONS=SUITE 42
 - OPTIONS=SUITE_RESERVED 42
 - SIGN 42
 - signature 42
 - syntax 41
- package suite 42
- port number
 - server default range 31
 - SERVER line 31
 - VENDOR line 33
- ports
 - license server 105
- precedence or FEATURE lines 40
- PROJECT type 187

R

- rehosting, license 62
- remote disks, guidelines for using 75
- report log 17

- report log file [74](#)
- reporting on project [209](#)
- REPORTLOG [208](#)
- RESERVE [209](#)
- restricting access
 - lmdown [101](#), [112](#), [128](#)
 - lmremove [101](#), [112](#), [128](#)
 - lmreread [101](#), [112](#), [128](#)
- return [19](#)

S

- SERVER line [30](#)
 - combining license files [252](#)
 - default port numbers [31](#)
 - host [31](#)
 - hostid [31](#)
 - port number [31](#)
 - syntax [30](#)
 - three-server redundancy [30](#)
- setting environment variables [289](#)
- SIGN [35](#), [42](#)
- signature [35](#), [42](#)
- SN [37](#)
- sockets
 - number used by license server [73](#)
- sort [39](#)
- specifying location of license file [45](#)
- START [37](#)
- starting lmadmin [79](#), [94](#)
- starting lmgrd [115](#)
- status of license server [154](#)
- SUITE_DUP_GROUP [37](#)
- SUPERSEDE [37](#)
- switching debug log
 - lmswitch [157](#)
- switching report log
 - lmadmin [129](#)
 - lmnewlog [147](#)
 - lmswitchr [158](#)

T

- three-server redundancy
 - separate license files [114](#)
 - SERVER lines [30](#)
- TIMEOUT [210](#)
- TIMEOUTALL [211](#)
- TPM hostid [56](#)
- troubleshooting
 - with FLEXLM_DIAGNOSTICS [256](#)
 - with lmdiag [138](#)
- trusted storage [15](#), [19](#)
- TS_OK [38](#)

[TZ](#) [38](#)

U

[UPGRADE line, syntax](#) [43](#)

[USE_SERVER line](#) [34](#)

[USER hostid](#) [55](#)

[USER type](#) [187](#)

[USER_BASED](#) [38](#)

[user_info](#) [39](#)

utilities

[Imborrow](#) [134](#)

[Imdiag](#) [138](#)

[Imdown](#) [138](#)

[Imhostid](#) [139](#)

[Iminstall](#) [143](#)

[Imnewlog](#) [147](#)

[Imremove](#) [148](#)

[Imreread](#) [153](#)

[Imstat](#) [154](#)

[Imswitch](#) [157](#)

[Imswitchr](#) [158](#)

[Imtools](#) [180](#)

[Imtpminfo](#) [159](#)

[Imutil](#) [131](#)

[Imver](#) [163](#)

V

[vendor daemon](#) [16](#)

[and redundant servers](#) [114](#)

[debug log file](#) [279](#)

[Imnewlog](#) [147](#)

[Imreread](#) [153](#)

[Imswitchr](#) [158](#)

[memory usage](#) [74](#)

[options file](#) [184](#)

[report log](#) [129](#)

[VENDOR_LICENSE_FILE](#) [290](#)

[version compatibility](#) [111](#)

[vendor daemon name](#)

[FEATURE line](#) [35](#)

[VENDOR line](#) [33](#)

[vendor daemon path](#) [33](#)

[VENDOR line](#) [32](#)

[eport](#) [33](#)

[options file path](#) [33](#)

[port number](#) [33](#)

[vendor daemon name](#) [33](#)

[vendor daemon path](#) [33](#)

[vendor_info](#) [39](#)

[VENDOR_LICENSE_FILE](#) [47](#), [290](#)

[VENDOR_STRING](#) [38](#)

[vendor.opt](#) [33](#), [184](#)

virtualization [227](#)
Vista [289](#)
VM_PLATFORMS [38](#)

